Machine Learning Model Engineering

Exam-Ready Revision Notes

Approximately 100 pages of explanations, formulas, comparisons, traps, and answer frames

Every syllabus topic retained

224 exact topic IDs across 13 teaching weeks

Tier A = high-yield core concept; Tier B = supporting/application concept; Tier C = introduction, setup, or recap.

Build date: 8 August 2026

How to Use This Exam Edition

Read the definition first, then reproduce the core explanation without looking. For calculation or architecture questions, write the formula or flow before the prose. Use the red exam-trap line to protect marks, and finish each week with its answer frame. The exact topic badge is also the coverage key used by the audit report.

Coverage and priority. All 224 source lessons are present: Tier A 155, Tier B 58, Tier C 11. Condensation changes depth, not syllabus coverage. Definitions, mechanisms, applicable formulas, examples or contrasts, quick-revision facts, and pitfalls are selected from the edited course notes.

Mark-wise method. For 2 marks: definition + one distinction. For 5 marks: definition + mechanism + example + limitation. For 10 marks: add a diagram or formula, compare alternatives, state metrics, and justify the final decision.

Week Roadmap

Week	Focus	Coverage
Week 1	Foundations of ML Model Engineering	18 topics
Week 2	Inference Patterns and Performance Metrics	18 topics
Week 3	Serving, APIs, Containers, and Rollouts	17 topics
Week 4	Deployment Pipelines and MLOps Foundations	16 topics
Week 5	Monitoring, Observability, Drift, and Alerts	14 topics
Week 6	Continuous Training and Safe Promotion	17 topics
Week 7	Model Standardization and Optimization	19 topics
Week 8	Accuracy, Latency, Cost, and UX Trade-offs	17 topics
Week 9	Production Features and Feature Stores	18 topics
Week 10	Data Engineering and Real-Time ML Pipelines	18 topics
Week 11	Security, Compliance, Fairness, and Responsible AI	18 topics
Week 12	Multi-Model, Multi-Tenant, and Retrieval Systems	18 topics
Week 13	Large-Scale ML System Design	16 topics

Contents

Week 1: Foundations of ML Model Engineering

Week roadmap: Production ML and Model Engineering: Course Overview → What Is Machine Learning Model Engineering? → Why Models Die in the Notebook: The Integration Gap → Defining Model Engineering: From Artifact to Production Service → Four Core Responsibilities of Model Engineering → The Seven-Stage ML Lifecycle → 12 more topics.

Coverage: 18 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

MLME-W01-T01 Production ML and Model Engineering: Course Overview

Tier C

Definition and why it matters: Most ML education focuses on training models in notebooks and optimizing offline metrics like accuracy, F1, or AUC. In real products, the harder problem is everything that comes after: deploying the model, keeping it healthy, and operating it reliably as traffic, data, and requirements change

Core explanation: This course bridges that gap - moving from "I have a model in a notebook" to "we have a reliable ML service in production."

Must remember:

- Production ML = deploy + monitor + retrain, not just train
- Course bridges notebook models to reliable ML services
- Modules 1-3: serving basics; 4-6: MLOps backbone; 7-9: optimization and features; 10-12: scale; 13: capstone
- Labs build a realistic stack: API, Docker, MLflow, feature store, registry, ONNX
- Assessments test system design and trade-offs, not just definitions
- Code along, use the layered architecture diagram, keep a playground repo

Exam trap: Treating high offline accuracy as sufficient for production readiness - deployment, monitoring, and operations are separate skill sets Skipping labs because theory feels sufficient - production ML is learned by building and measuring

MLME-W01-T02 What Is Machine Learning Model Engineering?

Tier B

Definition and why it matters: Machine learning model engineering is the discipline of making trained models usable by other systems and teams - reliably under real-world conditions and scalable as traffic grows or shrinks

Core explanation: Training a model in a notebook is a well-understood skill. Running a system that real users depend on every day is a fundamentally different problem. There is a large gap between a "cool notebook" and a production system - and model engineering exists to close it A model artifact (.pkl, .pt, checkpoint) sitting on a disk is not a product feature. Model engineering transforms that artifact into:

Must remember:

- Model engineering closes the gap between notebook prototypes and production systems
- Data science answers what model; model engineering answers how to run it safely
- Output is services, not experiments - integrated, monitored, versioned
- Small teams may combine roles; the responsibilities remain distinct
- Foundation for the rest of the course: responsibilities, lifecycle, constraints, MLOps

Example or contrast: Dimension: Core question; Data Science / Research: What model should we use?; Model Engineering: How do we run this model safely in production every day?

Exam trap: Equating "trained a good model" with "shipped ML to production" - integration, monitoring, and operations are separate work Assuming data scientists automatically handle deployment - model engineering is a distinct specialization

MLME-W01-T03 Why Models Die in the Notebook: The Integration Gap

Tier B

Definition and why it matters: Think of model training as manufacturing a product in a factory. Model engineering is the logistics, retail, quality control, and customer support that gets the product into customers' hands and keeps it working A perfect product in a warehouse helps nobody

Core explanation: A team trains a model in a notebook. Metrics look great - accuracy is up, error is down. Everyone is excited. Then nothing happens. The model never reaches customers. Months later, the notebook is forgotten This scenario is extremely common. Understanding why it happens is the entry point to model engineering Concept flow: Trained Model: in Notebook - API / Service Layer - Infrastructure: Scale, reliability - Monitoring: Drift, errors, health - Product Feature: Users see value

Must remember:

- High notebook metrics often lead nowhere without engineering around the model
- Production needs APIs, infrastructure, and monitoring - not just weights
- Model engineering is the layer between "trained model" and "product feature"
- Missing this layer is why most notebook models never ship
- The discipline focuses on integration and operations, not algorithm novelty

Exam trap: Blaming the model when the real blocker is missing infrastructure - great metrics do not auto-deploy Assuming product teams will "figure out integration" - without APIs and services, integration stalls indefinitely

MLME-W01-T04 Defining Model Engineering: From Artifact to Production Service Tier B

Definition and why it matters: Machine learning model engineering is everything needed to make models usable by other systems and teams - reliable under real-world conditions and scalable as traffic grows or shrinks Practically, this means taking a trained model artifact (.pkl, .pt, saved checkpoint) and turning it into a production-grade service that is integrated, monitored, and trusted by the rest of the organization

Core explanation: Model engineering is not about inventing model ideas or proving new algorithms. It is about making existing models work in the real world Stage: Input; State: Trained artifact on disk

Must remember:

- Model engineering = making models usable, reliable, and scalable in production
- Takes artifacts (checkpoints, pickles) and produces integrated, monitored services
- Data science: what model; model engineering: how to run it
- Not about inventing algorithms - about real-world operation
- Roles overlap in small teams but responsibilities remain distinct

Example or contrast: Concept flow: Explore data - Build prototypes in notebooks - Compare approaches with experiments - Refactor into robust services - Own deployment and scaling - Monitor in production - Integrate into applications - DS - ME

Exam trap: Defining model engineering as "deployment only" - monitoring, versioning, and collaboration are equally core Assuming a saved .pkl file is a deliverable - artifacts need services around them

MLME-W01-T05 Four Core Responsibilities of Model Engineering Tier B

Definition and why it matters: Model engineering spans four interconnected responsibility areas. Each addresses a distinct production challenge

Core explanation: The first movement where a model becomes something other systems can reliably call Concept flow: Notebook code - Python modules/packages - Inference pipeline: preprocess - predict - postprocess - Service layer: REST / gRPC / batch / stream - Deployable artifact: Docker + env config Step: Refactor; Detail: Notebook code - proper Python modules with tests Step: Inference logic; Detail: Preprocessing, model call, postprocessing - explicit and reusable Step: Service wrapper; Detail: REST API, gRPC, batch job, or streaming consumer Step: Deployability; Detail: Containers (Docker), environment-specific config (dev/staging/prod) Constraint: Latency; What to Measure: P95, P99 tail latencies - not just average; behavior under traffic spikes

Must remember:

- Four responsibilities: services, constraints, change management, collaboration
- Notebook → modules → inference pipeline → API → Docker/deployable
- Balance accuracy with latency (P95/P99), uptime, and cost per request
- Version everything; use canary/shadow/A-B; enable fast rollback
- Collaborate with data, infra, and product teams - translation role is central

Exam trap: Focusing only on accuracy while ignoring P99 latency - users feel the slow tail, not the average Skipping versioning in regulated industries - auditability requires tracing predictions to exact model + data versions

MLME-W01-T06 The Seven-Stage ML Lifecycle Tier A

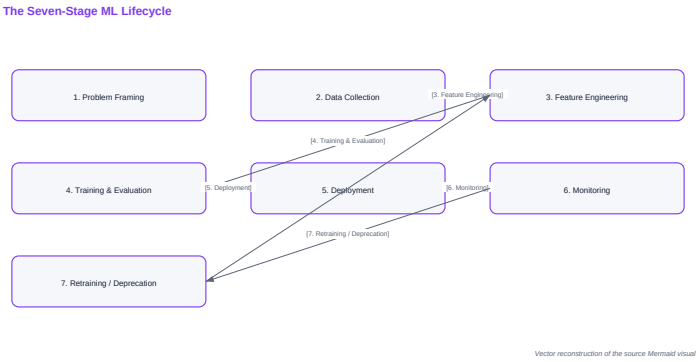
Definition and why it matters: Most ML projects move through seven key stages. Different companies may slice them differently, but these core stages appear almost everywhere. Understanding which stages are research-heavy vs production-heavy clarifies where model engineering focuses

Core explanation: Concept flow: 1. Problem Framing - 2. Data Collection - 3. Feature Engineering - 4. Training & Evaluation - 5. Deployment - 6. Monitoring - 7. Retraining / Deprecation Question: Where does data come from?; Why It Matters: Logs, databases, third-party, sensors, user behavior Question: Do we have labels?; Why It Matters: Ground truth vs manual labelling vs weak supervision Question: Is data representative?; Why It Matters: Training distribution must match production Before any models or pipelines, answer foundational questions: Question: Privacy/compliance constraints?; Why It Matters: GDPR, HIPAA, regional data residency What decision or workflow are we improving? What metrics are we trying to move? (CTR, revenue, fraud loss, time saved, error rate) Is ML even the right tool, or would a heuristic suffice? How does this fit into the product? (recommendations, rankings, alerts) Good problem framing saves enormous time later - it gives a clear target for every subsequent stage Without the right data, even the best architecture fails Turn raw data into signal the model can consume: Aggregations (counts, averages, time windows)

Must remember:

- Seven stages: framing → data → features → training → deployment → monitoring → retraining
- Stages 1-4 are research/exploration; 5-7 are production (model engineering focus)
- Problem framing defines success metrics before any code is written
- Feature consistency between training and serving prevents skew
- Offline metrics must connect to business/product outcomes

Exam trap: Skipping problem framing and jumping to model selection - optimizes the wrong metric Training on non-representative data - great offline metrics, poor production performance



Concept map: The Seven-Stage ML Lifecycle

MLME-W01-T07 Deployment, Monitoring, and Retraining: The Production Loop

Tier A

Definition and why it matters: Stages 5-7 of the ML lifecycle are where model engineering becomes central. These stages form a continuous loop, not a straight line

Core explanation: Deployment means exposing a trained model so other systems can call it and integrating predictions into the product experience Deployment Mode: Online API; Example Use Case: Real-time fraud score, search ranking Deployment Mode: Batch job; Example Use Case: Nightly churn predictions written to a warehouse Concept flow: Monitoring Layers - System Health: Latency, errors, uptime - Data Quality & Drift: Input distributions, missing values - Model Performance: Accuracy over time with delayed labels Layer: System health; What to Watch: Within latency targets? Error rates spiking? Service up? Once deployed, accuracy alone is insufficient. Engineers must also optimize: Layer: Data quality / drift; What to Watch: Input distributions changing? New categories? Unexpected patterns? Latency - response time under load Layer: Model performance; What to Watch: With ground truth or delayed labels, are predictions still accurate? Uptime and reliability - availability targets Cost - compute at scale Security - safe operation with sensitive data Action: Retrain; When: Performance drops due to data drift; fresh data available

Must remember:

- Deployment: expose model via API or batch; integrate into product; balance latency, uptime, cost, security
- Monitoring: system health + data drift + model performance - catch issues early
- Retraining/deprecation: respond to drift; retire models when cost exceeds benefit
- Lifecycle is a loop: deploy → monitor → retrain → deploy again
- Model engineering lives primarily in stages 5-7

Exam trap: Treating deployment as a one-time event - it is the start of an ongoing loop Monitoring only system health (200 OK, latency) while ignoring data drift - silent quality degradation

MLME-W01-T08 Where This Course Fits in the ML Lifecycle

Tier A

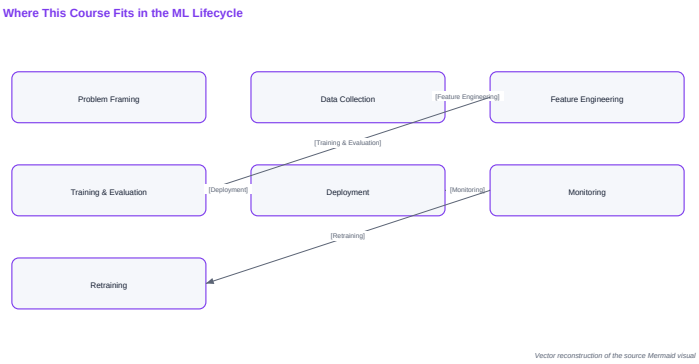
Definition and why it matters: This course primarily operates in the later stages of the seven-stage ML lifecycle:

Core explanation: Concept flow: Problem Framing - Data Collection - Feature Engineering - Training & Evaluation - Deployment - Monitoring - Retraining Stage: Problem framing, data, features; Course Coverage: Discussed as context for building production systems Stage: Deployment; Course Coverage: Core focus - turning models into services Stage: Monitoring; Course Coverage: Core focus - observing real-world behavior Stage: Retraining / deprecation; Course Coverage: Core focus - evolving models over time Connection: Monitoring - Retraining; Example: Drift alerts trigger retraining pipelines Connection: Deployment - Monitoring; Example: Cannot monitor what is not deployed Connection: Feature engineering - Deployment; Example: Training-serving consistency prevents skew When encountering architecture diagrams later in the course, mentally place concepts into these three production stages Each module is a deep dive into one or more parts of the lifecycle, especially the production-focused stages: ML in the real world is not about training a model once. It is about running a system over time with: Careful engineering discipline The next modules build the concrete skills and tools needed to operate in these later lifecycle stages as a model engineer

Must remember:

- Course focuses on deployment, monitoring, and retraining - not primary training/research
- Earlier stages (framing, data, features) appear as production context
- Use the lifecycle diagram as a mental map for every module
- Real-world ML = systems over time with feedback loops, not one-time training
- Each module deep-dives into production-focused lifecycle stages

Exam trap: Assuming this course teaches algorithm design - it teaches production operations Ignoring earlier lifecycle stages entirely - problem framing and features matter as context for serving



Concept map: Where This Course Fits in the ML Lifecycle

MLME-W01-T09 Production Constraints: Latency, Throughput, Cost, and Reliability Tier A

Definition and why it matters: Users do not experience average latency - they feel the slowest calls. That is why production systems focus on percentile latencies:

Core explanation: Metric: P95; Meaning: 95% of requests complete faster than this value Traffic Pattern: Daily cycles; Challenge: Morning/evening peaks, weekend dips Traffic Pattern: Event spikes; Challenge: Marketing campaigns, product launches, news events At scale, even 1% slow requests = a huge number of bad experiences (timeouts, spinning loaders, broken workflows) Budget reality: The model gets only a slice of total request latency. Network hops, databases, and upstream services consume the rest. Model engineers must design for strict latency budgets in user-facing real-time applications Cost Component: Compute; What Drives It: CPU/GPU time per request How many requests per second (RPS) can the system handle? Cost Component: Memory; What Drives It: RAM/VRAM to keep model loaded

Must remember:

- Offline metrics necessary but not sufficient for production
- Latency: focus on P95/P99; model gets only part of total budget
- Throughput: design for peak RPS and traffic spikes, not averages
- Cost: per-request compute + memory + storage compounds at scale
- Reliability: uptime targets, error budgets, graceful degradation, fallbacks
- Compliance: data residency, PII, auditability - design in from the start

Example or contrast: Offline metrics (accuracy, F1, AUC, loss curves) resemble Kaggle competitions. Shipping a real ML product operates under entirely different constraints Great offline performance is necessary but not sufficient for production readiness. A model with amazing test metrics can still be impossible to ship because it is too slow, too expensive, or too fragile

Exam trap: Optimizing average latency while ignoring P99 - tail latency drives user experience at scale Sizing for average RPS instead of peak - systems collapse during traffic spikes

MLME-W01-T10 Common Production Failure Modes Tier B

Definition and why it matters: Even with good architecture and strong offline metrics, production ML systems fail in predictable, recurring ways. These are engineering and operations problems - not solved by swapping one algorithm for another

Core explanation: Definition: Features computed during training differ from features computed during serving Divergence Source: Different preprocessing code; Example: Offline script vs online service Divergence Source: Different missing value handling; Example: Training fills with median; serving fills with zero Divergence Source: Different encoding/normalization; Example: Training uses one-hot; serving uses label encoding Concept flow: Training Pipeline: Feature logic A - Model - Serving Pipeline: Feature logic B - Poor online performance

Must remember:

- Four recurring failure modes: training-serving skew, data drift/staleness, silent failures, infra/dependency issues
- Training-serving skew: different feature logic offline vs online - fix with shared pipelines/feature store
- Data drift: world changes, model does not - monitor inputs, retrain proactively
- Silent failures: healthy infra metrics masking broken data/model pipelines
- Infra issues: timeouts, scaling failures, dependency changes - versioning, observability, collaboration
- These are engineering/operations problems, not algorithm problems

Exam trap: Debugging training-serving skew with more tuning - fix the feature pipeline, not hyperparameters Monitoring only HTTP status codes - silent failures hide behind 200 responses

MLME-W01-T11 Why ML Needs Engineering and Operations Tier B

Definition and why it matters: The production constraints (latency, throughput, cost, reliability, compliance) and the common failure modes (training-serving skew, data drift, silent failures, infrastructure issues) together explain why ML needs dedicated engineering and operations - not just clever modeling

Core explanation: What Looks Like a Model Problem: Poor online accuracy after great offline metrics; What It Actually Is: Training-serving skew (integration) What Looks Like a Model Problem: Gradual business metric decline; What It Actually

Is: Data drift + stale model (operations) What Looks Like a Model Problem: "Everything is fine" but users complain; What It Actually Is: Silent failure (monitoring gap)

Must remember:

- Constraints + failure modes = why ML needs engineering and operations
- Failures are integration, deployment, monitoring, maintenance - not algorithm choice
- Better models alone do not fix skew, drift, silent failures, or infra issues
- Model engineering + MLOps address complexity around the model
- Use constraints and failures as backdrop for every design decision in the course

Exam trap: Proposing "use a better model" as the fix for integration bugs - engineering discipline is required Treating MLOps as optional overhead - it prevents the silent degradation that kills production systems

MLME-W01-T12 MLOps: The Operating System for Production ML

Tier A

Definition and why it matters: MLOps = applying DevOps-style practices to ML systems, with focus on: Pillar: Automation; Goal: Training, testing, deployment, monitoring

Core explanation: Production ML is not "train once, push to prod, forget." Real systems must handle: Constantly changing data and user behavior Complex infrastructure (services, databases, queues, cloud resources) Strict constraints: latency, cost, reliability, compliance A disciplined approach to building, deploying, monitoring, and updating models safely and repeatedly is required. That discipline is MLOps Layer: System; What to Monitor: Latency, error rates, uptime, resource usage Layer: Data & model; What to Monitor: Input distributions, missing values, out-of-range features, prediction distribution shifts Layer: Performance; What to Monitor: Actual metrics once labels arrive (delayed ground truth) Monitoring must cover both system and model health: Alerts must fire when: Dashboards stop updating Without serious monitoring, ML systems suffer silent failures - infrastructure looks healthy while the model does the wrong thing. Good MLOps makes these issues visible and actionable

Must remember:

- MLOps = DevOps practices adapted for ML systems
- Handles changing data, model degradation, reproducibility, auditability
- MLOps = DevOps + Data + Models
- Monitor system health AND data/model health
- Alerts for drift, regressions, pipeline failures - prevent silent failures
- Goal: safe, repeatable path from training to production and back

Example or contrast: Concept flow: DevOps Foundation: CI/CD, monitoring, logging, tracing - MLOps = DevOps + Data + Models Shared with DevOps: Automated builds, tests, deployments; ML-Specific Twists: Data and labels change even when code stays the same Shared with DevOps: CI/CD pipelines; ML-Specific Twists: Models degrade over time due to data drift

Exam trap: Equating MLOps with "using Kubernetes" - it is a practice set, not a tool Applying DevOps CI/CD without ML-specific checks (data validation, model quality gates)

MLME-W01-T13 MLOps Core Practices: Versioning, Pipelines, and CI/CD

Tier A

Definition and why it matters: Three foundational MLOps practices turn ML from ad hoc experimentation into an accountable, repeatable system

Core explanation: In regular software, code is versioned. In MLOps, everything that affects model behavior must be versioned: Artifact: Data; Examples: Exact dataset or feature snapshot used for training Artifact: Model; Examples: Checkpoints, serialized artifacts Artifact: Config; Examples: Hyperparameters, thresholds, routing rules, feature flags Need: Reproducibility; Requirement: Recreate the exact model from 6 months ago with identical behavior Need: Auditability; Requirement: Trace any prediction back to specific model + dataset (regulated industries) Need: Safe rollback; Requirement: Quickly revert to a known-good version when a new model misbehaves Concept flow: Ingest & Validate Data - Feature Engineering - Train & Evaluate - Package & Deploy Without Pipelines: "Works on my laptop"; With Pipelines: Shared, documented process Why versioning matters: Without Pipelines: Debugging is person-dependent; With Pipelines: Everyone uses the same steps Without Pipelines: Collaboration is fragile; With Pipelines: Onboarding and handoffs are smooth Versioning everything is foundational to treating ML as a real, accountable system Replace ad hoc scripts and notebooks with an end-to-end pipeline describing: Pattern: Canary; Description: Route small percentage of traffic to new version

Must remember:

- Version data, model artifacts, and config - for reproducibility, auditability, rollback
- Reproducible pipelines: ingest → features → train → deploy; same input = same output for anyone
- CI: test transforms, training, inference, model quality gates
- CD: canary, shadow, A/B - small frequent safe releases
- These three practices form the MLOps backbone alongside monitoring

Exam trap: Versioning only the model artifact - data and config changes cause silent behavior shifts Pipelines that only one person can run - reproducibility requires team-wide execution

MLME-W01-T14 The ML Model Engineer Role

Tier B

Definition and why it matters: The model engineer sits at the intersection of three worlds:

Core explanation: Concept flow: Research / Data Science: Model ideas, prototypes - Model Engineer: Translation layer - Infrastructure / Platform: Underlying systems - Product / Business: Requirements, SLAs - Production Services Ownership Area: Serving path; Scope: How inputs become predictions in production Ownership Area: Deploy-monitor-retrain loop; Scope: Safe deployment, observation, evolution Ownership Area: Core principle; Scope: Models must be usable in the real world - not just clever Take research prototypes and turn them into robust, observable, scalable services Partner: Data engineers; Interaction: Define and maintain feature pipelines Partner: Infra/platform teams; Interaction: Deploy and scale services Partner: Product; Interaction: Understand SLAs, UX constraints, "good enough" thresholds

Must remember:

- Model engineer: intersection of research, infra, and product
- Mission: prototypes → robust, observable, scalable services
- Day-to-day: refactor code, build services, logging/metrics, CI/CD, versioning, collaboration
- Primary lifecycle stages: deployment, monitoring, retraining
- Anchors the deploy-monitor-retrain loop
- Combines ML knowledge + software engineering + operations

Exam trap: Treating model engineer as "junior data scientist who deploys" - it is a distinct engineering specialization Owning only deployment without monitoring - the loop requires all three stages

MLME-W01-T15 Lab 1: Building an ML Project from Scratch

Tier B

Definition and why it matters: This lab establishes the end-to-end workflow for building a real ML/AI application - focusing on understanding the flow before diving into implementation details

Core explanation: Component: Development environment; Purpose: Clean, isolated workspace Component: Project structure; Purpose: Organized files and dependencies Component: Experimentation; Purpose: Jupyter notebooks for exploration Component: Model sourcing; Purpose: Hugging Face Hub for pre-trained models Component: Prediction; Purpose: Run inference locally Component: Packaging; Purpose: Structured Python package from notebook code Concept flow: Setup Environment - Create Project - Jupyter Experimentation - Hugging Face Model - Run Prediction - Python Package Concept flow: System Python: Base installation - Project A venv: numpy 1.19.5 - Project B venv: numpy 1.21.0

Must remember:

- Lab flow: environment → project → notebook → Hugging Face → prediction → package
- Virtual environments isolate dependencies per project
- UV: fast package manager for venv creation and dependency install
- Jupyter in VS Code for experimentation with correct kernel selection
- Start with a lightweight use case (text assistant) to validate the full pipeline

Example or contrast: Install the Jupyter extension in VS Code

Exam trap: Skipping virtual environments - dependency conflicts cause "works locally, fails in CI" bugs Running notebooks without selecting the correct kernel - uses system Python instead of project venv

MLME-W01-T16 Lab 1: Virtual Environments, Jupyter, and Hugging Face Setup

Tier B

Definition and why it matters: Simple text assistant - generates short, helpful responses Selection criteria for the first model:

Core explanation: After creating and activating a virtual environment, all subsequent package installs and code execution run in isolation Platform: Linux / Mac; Activate Command: source .venv/bin/activate Platform: Windows CMD; Activate Command: venv\Scripts\activate Concept flow: Python 3.x - Global packages - Python - numpy 1.19.5, torch 2.x - Python - numpy 1.21.0, transformers 4.x - System - ProjectA - ProjectB Visual indicator: Prompt shows (.venv) prefix when active Component: Model Hub; Purpose: Thousands of pre-trained models (text, image, audio) Component: Dataset Hub; Purpose: Real-world datasets for training and evaluation Each project gets dedicated dependencies. No cross-contamination Component: Libraries; Purpose: Transformers, Datasets, Accelerate (used throughout the course)

Must remember:

- Activate venv before any installs: source .venv/bin/activate (Mac/Linux)
- Jupyter: install extension, use .ipynb, select venv as kernel
- Hugging Face = model hub + datasets + libraries + playground
- Gated models need account + license acceptance + access token
- First lab use case: lightweight text assistant on CPU - validate pipeline, not quality

Exam trap: Forgetting to activate venv before uv pip install - packages install globally Using .py extension instead of .ipynb for notebooks - file won't open in Jupyter

MLME-W01-T17 Lab 1: Hugging Face Ecosystem and Text Generation Pipeline

Tier A

Definition and why it matters: Hugging Face democratized AI development. Before centralized model hubs, building language models required massive in-house resources - data preparation, training pipelines, GPU infrastructure. Today, a few lines of code lets anyone download, experiment with, and deploy state-of-the-art models

Core explanation: Role: Model hub; What Hugging Face Provides: Thousands of community and enterprise models Role: Dataset hub; What Hugging Face Provides: Real-world data for fine-tuning Role: Open-source libraries; What Hugging Face Provides: Transformers, Datasets, Accelerate Role: Community; What Hugging Face Provides: Continuous model improvements and sharing Property: Type; Value: Compressed GPT-2 (distilled) Property: Size; Value: Small, fast download Property: Hardware; Value: Runs on CPU - no GPU required Property: Quality; Value: Simple/imperfect text - ideal for learning Concept flow: Hugging Face Hub - Download tokenizer + model - Load into memory - Tokenize prompt - model.generate - tokenizer.decode - Generated text Mastering this ecosystem means learning how modern AI is actually built in industry - not reinventing models from scratch For the first lab, DistilGPT-2 is the recommended starting model: Artifact: Tokenizer; Purpose: Converts text token IDs the model understands Artifact: Model weights; Purpose: The neural network parameters Selection principle: Pick a model you can run easily, not the best model in the world. Validate the pipeline first; improve the model later

Must remember:

- Hugging Face = model hub + datasets + libraries + community ecosystem
- DistilGPT-2: lightweight CPU-friendly model for pipeline learning
- Pipeline: download → load → tokenize → generate → decode
- Two artifacts: tokenizer (text<→tokens) and model (weights)
- Imperfect output is fine - validate that the pipeline works end-to-end
- Ecosystem mastery builds incrementally across the course

Exam trap: Judging lab success by output quality instead of pipeline correctness Skipping local model save - re-downloads on every run waste time and bandwidth

MLME-W01-T18

Module 1 Summary: Model Engineering Foundations

Tier B

Definition and why it matters: Machine learning model engineering is everything needed to make models usable by real systems and users - reliable under real-world conditions and scalable as usage grows

Core explanation: Concept flow: 1. Notebooks - Services: APIs, containers, deployable code - 2. Real-World Constraints: Latency, uptime, cost - 3. Change Management: Versioning, rollouts, rollback - 4. Collaboration: Data, infra, product teams - Production ML System : 1; Responsibility: Turn notebooks into services; Key Activities: Refactor code, build inference pipelines, wrap in APIs, containerize : 2; Responsibility: Meet constraints; Key Activities: P95/P99 latency, uptime SLOs, cost per request : 3; Responsibility: Manage change; Key Activities: Version data/code/models, canary/shadow/A-B, rollback, retrain : 4; Responsibility: Collaborate; Key Activities: Work with data, infrastructure, and product stakeholders

Must remember:

- Model engineering = making models usable, reliable, scalable in production
- Four responsibilities: services, constraints, change management, collaboration
- Lifecycle focus: deployment → monitoring → retraining (continuous loop)
- Five constraints: latency (P95/P99), throughput, cost, reliability, compliance
- Four failure modes: skew, drift, silent failures, infra issues
- MLOps = DevOps + Data + Models; version, pipeline, CI/CD, monitor
- Lab 1: venv → Jupyter → Hugging Face → inference pipeline → packaging path

Exam trap: Treating Module 1 as "just introduction" - these concepts recur in every subsequent module Memorizing definitions without understanding the notebook-to-production gap

Week 1 Exam Lens

Likely questions

- Define and explain The Seven-Stage ML Lifecycle; include the mechanism and one limitation.
- Compare The Seven-Stage ML Lifecycle with Deployment, Monitoring, and Retraining: The Production Loop and state when each is preferred.
- Apply Where This Course Fits in the ML Lifecycle to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Mistakes to avoid: Treating high offline accuracy as sufficient for production readiness - deployment, monitoring, and operations are separate skill sets Equating "trained a good model" with "shipped ML to production" - integration, monitoring, and operations are separate work Blaming the model when the real blocker is missing infrastructure - great metrics do not auto-deploy

Week 2: Inference Patterns and Performance Metrics

Week roadmap: Model Inference in Production: Module Overview → Definition of Model Inference → Latency, Throughput, and Cost: The Inference Metrics Triangle → Inference Patterns: Setup and Mental Model → Batch Inference: Definition and Architecture → Batch Inference: Offline Use Cases and Trade-offs → 12 more topics.

Coverage: 18 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

MLME-W02-T01 Model Inference in Production: Module Overview

Tier C

Definition and why it matters: Training gets most of the academic spotlight, but production ML systems spend the vast majority of their lifetime serving predictions, not fitting parameters. Module 2 zooms in on that serving phase: what happens when a trained model is asked for a prediction, how we measure it, and how we choose among different calling patterns

Core explanation: By the end of this module, you should be able to:

Must remember:

- Production models are services, not static files - they receive requests and return predictions reliably
- Inference dominates the model lifecycle in both call volume and operational cost
- Three core metrics: latency (how long), throughput (how many per second), cost (resources per prediction)
- Three calling patterns: batch (scheduled bulk scoring), online (synchronous request-response), streaming (continuous event processing)
- Same model function f(x), different invocation patterns based on business requirements
- Module 2 adds structure around calling patterns and measurement, building on Module 1's deployment foundations

Formula / decision rule: hat(y) = f(x); f(x)

Exam trap: Trap: "Inference is just calling model.predict()." - Inference includes the full request path: validation, feature engineering, post-processing, serialization, and network overhead Trap: "Training is the expensive part." - At scale, inference cost often dominates because it runs continuously across millions or billions of calls

MLME-W02-T02 Definition of Model Inference

Tier A

Definition and why it matters: Every ML system has two distinct phases:

Core explanation: Phase: Training; Purpose: Learn parameters from historical data; Input: Labelled (or unlabelled) training set; Output: Trained model weights Phase: Inference; Purpose: Apply the trained model to new, unseen data; Input: Fresh features at serving time; Output: Predictions, scores, embeddings Concept flow: Training Phase: Learn f from historical data - Trained Model f - Inference: Churn scoring - Inference: Search ranking - Inference: Spam classification - Inference: Recommendations Concept flow: JSON / row / event - Caller - Service - Model Step: 1. Receive input; What Happens: Accept raw data in the caller's format; Examples: JSON over HTTP, Parquet row, Kafka event Step: 2. Validate & parse; What Happens: Check required fields, types, value ranges; convert to internal objects/tensors; Examples: Reject malformed requests early (400 errors) Training builds the function. Think of it as defining f such that predictions are useful

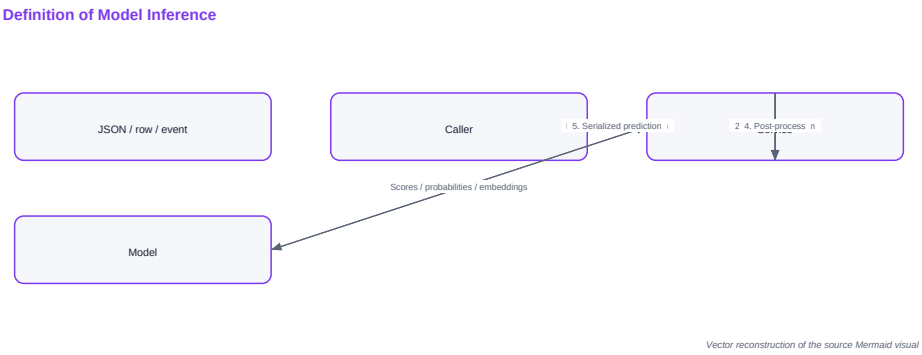
Must remember:

- Training learns f from historical data; inference calls f on new data continuously
- Inference dominates the model lifecycle in call volume and operational cost
- Every inference call follows: receive → validate → transform → predict → post-process → return
- The same six-step pipeline applies to batch, online, and streaming - only scale and timing differ
- Feature preprocessing at inference must exactly match training to avoid silent accuracy loss
- prediction = f(input features) is the core mental model for all serving patterns

Formula / decision rule: prediction = f(input features); from historical data; **inference** calls

Example or contrast: Company / System: Netflix; Inference Action: Rank content for a user's homepage; Pattern: Online (user waiting) + batch (precomputed candidates)

Exam trap: Trap: "Inference = forward pass only." - The forward pass is one step in a six-step pipeline; network, validation, and feature prep often dominate latency Trap: "Batch and online inference use different models." - They typically use the same model; only the calling pattern and metric priorities differ



Concept map: Definition of Model Inference

MLME-W02-T03 Latency, Throughput, and Cost: The Inference Metrics Triangle

Tier A

Definition and why it matters: Inference is not a one-off event. It runs continuously - for every page view, transaction, and overnight batch job. The metrics we choose to measure inference directly determine:

Core explanation: User experience - Does the app feel snappy or laggy? Metric: Average (mean); Definition: Mean across many requests; When It Matters: General system health Metric: P50 (median); Definition: 50% of requests are faster; When It Matters: Typical user experience Metric: P95; Definition: 95% of requests are faster; When It Matters: Product SLA targets Scalability - Can the system keep up as traffic and data grow? Metric: P99; Definition: 99% of requests are faster; When It Matters: Worst-case user experience Affordability - Can we run the model at the required frequency without blowing the cloud budget? Three metric families govern all inference design decisions: latency, throughput, and cost Latency is the elapsed time from when a request arrives until the prediction is ready to return. It includes everything: Validation and feature preparation Model forward pass Concept flow: Average looks great: 50 ms - But P99 = 2000 ms - Users feel the tail - Broken flows, timeouts,: abandonment

Must remember:

- Three core inference metrics: latency, throughput, cost - they drive all serving design
- P95/P99 latency matters more than average for user-facing systems; users feel the tail
- Throughput = predictions per unit of time; measure in RPS (online), rows/sec (batch), events/sec (streaming)
- Utilisation reveals whether capacity is wasted (idle) or dangerously saturated (maxed out)
- Every prediction burns compute, memory, storage, and bandwidth - cost scales with call volume
- Latency, throughput, and cost form a trade-off triangle with no universal optimum
- Different inference patterns (batch/online/streaming) are different navigations of this triangle

Exam trap: Trap: Optimizing average latency when the SLA specifies P95/P99 - averages hide tail problems Trap: "High throughput automatically means low latency." - Batching improves throughput but can increase per-item latency

MLME-W02-T04 Inference Patterns: Setup and Mental Model

Tier A

Definition and why it matters: A trained model is fundamentally a function:

Core explanation: prediction = f(input features) Dimension: Frequency; What Varies: How often we invoke f; Examples: Once a month, once a minute, thousands per second Dimension: Batch size; What Varies: How many items per call; Examples: Single record, 10,000-row chunk, continuous stream Dimension: Urgency; What Varies: Who is waiting; Examples: Nobody (batch), user in UI (online), downstream pipeline (streaming) Concept flow: Large static dataset - Scheduled job - Write predictions to warehouse - Single request - Synchronous response - Caller blocked until answer - Continuous event stream - Long-running pipeline - Alerts / features / downstream services Pattern: Batch; Optimized For: Throughput, total job time; Primary Metrics: Rows/sec, job completion time; Caller Behavior: No one waiting per row Pattern: Online; Optimized For: Low per-request latency; Primary Metrics: P95/P99 latency, error rate; Caller Behavior: User or system actively waiting

Must remember:

- Model inference is prediction = f(input features) - the function stays constant
- Three patterns: batch (scheduled bulk), online (synchronous request-response), streaming (continuous pipeline)
- Pattern choice depends on who is waiting, freshness requirements, and data volume/frequency
- Batch optimizes throughput; online optimizes P95/P99 latency; streaming optimizes event-to-action latency and sustained throughput
- Real systems often use hybrid architectures combining patterns
- The metrics triangle (latency, throughput, cost) provides a consistent framework for comparing all three patterns

Formula / decision rule: prediction = f(input features); $\hat{y} = f(x)$

Example or contrast: Stays the Same: Model weights and architecture; Changes by Pattern: Invocation frequency

Exam trap: Trap: "We need real-time, so we must use online inference." - Streaming may be better for continuous event reaction without a blocking caller Trap: Defaulting to online APIs for everything - batch is simpler, cheaper, and often sufficient when freshness tolerance is hours or days

MLME-W02-T05 Batch Inference: Definition and Architecture

Tier A

Definition and why it matters: Batch inference means: Take a large set of inputs (thousands, millions, sometimes billions of rows)

Core explanation: Batch inference answers the question: "When do I not need real-time predictions, and how can I take advantage of that?" Sometimes you have a large list of items to score and you are happy as long as predictions are ready by a deadline - say, before business hours tomorrow. No human sits waiting on each individual prediction Property: Schedule; Batch Characteristic: Periodic - once a day, hourly, every 15 minutes Property: Caller; Batch Characteristic: No human waiting per row Property: Latency concern; Batch Characteristic: Total job completion time, not per-row latency Property: Output; Batch Characteristic: Bulk write to storage for later consumption Concept flow: Data Source: CSV / Parquet / Warehouse - Preprocess &: Featureize Rows - Model Inference: Chunks / Mini-batches / Parallel - Write Predictions: DB / File / Feature Store - Job Complete: Monitor & Alert Step: 1. Read input; Action: Load bulk data snapshot; Tools / Formats: BigQuery, Snowflake, S3 Parquet, DB export

Must remember:

- Batch inference scores many items at once on a schedule; nobody waits per row
- Inputs: large static datasets; outputs: predictions written to warehouse/file/feature store
- Pipeline: read → preprocess → model inference (parallel/chunked) → write → monitor
- Success metric: total job time and throughput (rows/sec), not per-row latency
- Infrastructure: scheduler + batch job + output target + monitoring (not 24/7 API)
- Ideal when predictions can be hours or days old and the whole dataset must be ready by a deadline

Example or contrast: Aspect: Data shape; Batch: Static snapshot; Online: Single request; Streaming: Continuous stream

Exam trap: Trap: Measuring per-row latency in batch - it is irrelevant; only total job time matters Trap: Running batch jobs during peak hours - schedule off-peak for cost savings and to avoid competing with online traffic

MLME-W02-T06 Batch Inference: Offline Use Cases and Trade-offs

Tier A

Definition and why it matters: Batch inference is the workhorse of enterprise ML. It appears wherever you have lots of data but no user waiting for each individual prediction

Core explanation: If nobody cares whether an individual prediction is 5 seconds old or 5 minutes old - as long as the whole dataset is ready on time - batch inference is a strong candidate Advantage: High throughput; Why It Matters: Vectorization, batching, and parallelism across cores/machines - score millions of rows in one job Advantage: Simpler infrastructure; Why It Matters: No highly available 24/7 API with strict SLAs; a scheduled job on a cluster or beefy machine suffices Advantage: Easier rollback; Why It Matters: Discover a bug? Fix it and rerun the job - overwrite bad output with good predictions

Must remember:

- Classic batch use cases: churn scoring, credit risk, offline recommendations, marketing lists, compliance reports
- Choose batch when no one waits per row and predictions can be hours/days old
- Advantages: high throughput, simpler infra, easy rollback, supports heavier models
- Trade-offs: staleness, slower experimentation feedback loop, unsuitable for instant context-aware decisions
- Hybrid pattern: batch precomputes candidates/features; online serves fresh rankings
- Rule of thumb: if deadline matters more than per-row freshness, batch is the right tool

Example or contrast: Domain: Customer analytics; Use Case: Monthly/weekly churn predictions for all customers; Freshness Tolerance: Days Domain: Credit risk; Use Case: Portfolio-wide risk scoring for all accounts; Freshness Tolerance: Daily Domain: Recommendations; Use Case: Precompute user-item scores and candidate lists overnight; Freshness Tolerance: Hours (served online later)

Exam trap: Trade-off: Staleness; Impact: Predictions are only as fresh as the last batch run; user behavior changes during the day may not be reflected until the next run Trade-off: Longer feedback loop; Impact: Experiment cycle becomes: train - run large batch - wait for downstream metrics - adjust - repeat (days, not minutes)

MLME-W02-T07 Batch Inference: Metrics, Pros, Cons, and Architecture

Tier A

Definition and why it matters: In batch inference, the metrics priority order inverts compared to online serving:

Core explanation: Metric: Total job time; Batch Priority: Top priority; Online Priority: N/A Metric: Throughput (rows/sec); Batch Priority: Top priority; Online Priority: Important but secondary Metric: Per-row latency; Batch Priority: Low priority; Online Priority: Top priority (P95/P99) Metric: Cost; Batch Priority: Optimizable via scheduling; Online Priority: Critical at peak traffic Strategy: Off-peak scheduling; Mechanism: Run jobs at night when compute is cheaper; Benefit: Lower cloud bills "I have a million rows to score and I need them done by 7:00 a.m." Strategy: Spot/preemptible instances; Mechanism: Use interruptible VMs for fault-tolerant batch work; Benefit: Up to 70-90% cost savings

Must remember:

- Pros: High throughput via vectorization and parallelism; Cons: Predictions are stale between runs
- Pros: Simpler infrastructure (no 24/7 SLA); Cons: Slower experimentation feedback loop
- Pros: Easy rollback (rerun job to fix bad output); Cons: Cannot support instant, context-aware decisions
- Pros: Supports heavier/slower models; Cons: Not suitable when user is actively waiting
- Pros: Cost-efficient (off-peak, spot instances); Cons: Lag behind rapidly changing user behavior
- Batch optimizes throughput and total job time; per-row latency is low priority
- Cost savings via off-peak scheduling, spot instances, and tolerating longer wall-clock times

- Architecture: data source → scheduler → batch pipeline → output target → alerting
- Pros: high throughput, simple infra, easy rollback, supports heavy models
- Cons: staleness, slow feedback loops, unsuitable for real-time user-facing decisions
- Batch = backstage heavy lifting; online = on-stage performance with strict latency SLOs

Exam trap: Trap: Applying online latency SLOs to batch jobs - per-row latency is irrelevant in batch Trap: "Batch is always cheaper." - It is cheaper per prediction at scale, but a poorly designed batch job (no parallelism, wrong instance type) can still be expensive

MLME-W02-T08 Online Inference: The Request-Response Pattern

Tier A

Definition and why it matters: Online (synchronous) inference means: A single request arrives at the service (typically HTTP or gRPC)

Core explanation: Batch inference processes many items on a schedule with no one waiting per row. Online inference flips that entirely: One request arrives Concept flow: click, checkout, search - User - App - LB - MS - Model One response must return immediately Step: Receive payload; Action: JSON over HTTP or protobuf over gRPC; Latency Impact: Network RTT The caller is blocked until the prediction arrives Step: Validate input; Action: Check fields, types, ranges; reject bad requests early; Latency Impact: Microseconds The mindset shifts from "finish the whole job by a deadline" to "respond to this single request within milliseconds." Step: Feature transform; Action: Encoding, normalization, embeddings - same as training; Latency Impact: Often significant Step: Forward pass; Action: Compute logits, probabilities, scores; Latency Impact: Model-dependent Step: Post-process; Action: Thresholds, top-k, label mapping; Latency Impact: Microseconds

Must remember:

- Online inference = one request in, one response out; caller is blocked until prediction arrives
- Transport: HTTP/JSON or gRPC/protobuf; caller can be UI, mobile app, or backend service
- Same inference pipeline as batch (validate → transform → predict → post-process) but per-request with tight latency budget
- Architecture includes load balancer, replicas, feature store, cache, and monitoring
- Total latency = sum of all pipeline stages; P95/P99 targets apply to the entire path, not just the model
- Mindset shift: from "finish job by deadline" to "respond to this request now"

Formula / decision rule: Total Latency = t_(network) + t_(validate) + t_(features) + t_(model) + t_(postprocess)

Exam trap: Trap: "Online inference only means REST APIs." - gRPC, internal service-to-service calls, and even synchronous batch-of-one are all online patterns Trap: Optimizing only model inference time - feature store lookups and network hops often dominate total latency

MLME-W02-T09 Online Inference: Interactive Use Cases and Metrics

Tier A

Definition and why it matters: Online inference is required when a live interaction cannot proceed until the prediction arrives. The common theme: something important is blocked, waiting for the model

Core explanation: In online inference, model latency directly affects: Business Metric: Page load time; Latency Impact: Every ms of inference adds to perceived load Business Metric: App responsiveness; Latency Impact: Slow predictions make the app feel "broken" Business Metric: Conversion / abandonment; Latency Impact: Users drop out of checkout funnels that feel sluggish Business Metric: Fraud prevention effectiveness; Latency Impact: Payment provider timeouts if fraud check is too slow Use Case: Show recommendations; Latency Target: < 100-200 ms Use Case: Complete fraud check; Latency Target: Before payment provider timeout (~2-5 s hard limit, but UX breaks well before)

Must remember:

- Online inference is required when a live interaction blocks on the prediction
- Use cases: search ranking, real-time recommendations, fraud checks, dynamic personalization
- Model latency directly impacts page load, responsiveness, conversion, and abandonment
- Key metrics: P95/P99 latency and error rate are product-level requirements
- Benefits: fresh context, per-request personalization, faster training feedback loops
- Cost is driven by peak traffic - every ms at peak is multiplied across thousands of requests

Example or contrast: Domain: Search & ranking; Scenario: User types a query; results must be ranked in real time; Why Online?: User staring at search box Domain: Recommendations; Scenario: Homepage or product page shows personalized suggestions; Why Online?: Page load blocked on ranking Domain: Fraud / risk; Scenario: Payment, login, or password reset checked before proceeding; Why Online?: Transaction blocked until decision

Exam trap: Trap: Reporting average latency when the SLA specifies P95 - always match the metric to the requirement Trap: "100 ms model inference = 100 ms user experience." - Network, feature lookup, and serialization add to total latency

MLME-W02-T10 Online Inference: Latency, Reliability, and Production Techniques

Tier A

Definition and why it matters: Online inference delivers fresh, personalized predictions but demands strict engineering discipline. The advantages come with real costs in latency requirements, scaling complexity, and operational fragility

Core explanation: Advantage: Fresh predictions; Description: Latest session data, recent actions, geolocation, device type - context absent from yesterday's batch snapshot Advantage: Per-request personalization; Description: Two users on the same

page simultaneously see completely different rankings or recommendations Advantage: Tighter feedback loop; Description: Log each prediction with user interactions; feed outcomes back into training faster than batch cycles allow Challenge: Strict latency requirements; Impact: P95/P99 must stay within target - "usually fast" is not enough Challenge: Scaling complexity; Impact: Traffic spikes from campaigns, launches, time-of-day patterns, bots Challenge: More moving parts; Impact: Load balancers, caches, feature stores, external dependencies - any failure is user-visible immediately Challenge: Model complexity limits; Impact: Heavy models may be too slow for reliable real-time serving Concept flow: Traffic Spike - Slow Dependency: Feature Store / DB / API - Higher Latency - Retries Multiply Load - Timeouts Cascade - Cascading Failure: Everything times out

Must remember:

- Online advantages: fresh context, per-request personalization, faster training feedback
- Online challenges: strict P95/P99 SLOs, scaling complexity, cascading failure risk
- Key production techniques: caching, auto-scaling, circuit breakers/timeouts, rate limiting, hybrid batch+online
- Hybrid pattern: batch precomputes heavy work; online does light real-time ranking
- Batch = backstage; online = on-stage - same model, different engineering constraints
- MLOps is essential: robust service design, not just model accuracy

Formula / decision rule: f(input)

Example or contrast: Dimension: Mental model; Batch: "10M rows scored by tomorrow morning"; Online: "User clicked - page must load now"

Exam trap: Trap: "Circuit breakers are only for microservices." - They are essential around feature stores, embedding services, and any inference dependency Trap: Auto-scaling alone solves everything - scaling takes time (cold starts); pre-warming and capacity buffers are still needed

MLME-W02-T11 Streaming Inference: Architecture and Pattern Comparison

Tier A

Definition and why it matters: Streaming inference occurs when: Data arrives as a continuous stream of events (not a static file, not a single API request)

Core explanation: Batch processes a static dataset on a schedule. Online handles one request at a time with a synchronous response. Streaming inference introduces a third model: data arrives as a continuous flow of events, and the model processes them as they happen There is no clear job start or job end. The pipeline is always running Concept flow: Event Sources: Apps / Sensors / Logs / Clicks - Stream Transport: Kafka / Kinesis / Pub/Sub - Stream Processing: Flink / Spark Streaming - Model Step: Score each event / mini-batch - Sinks: Alerts / DB / Feature Store / Dashboard Component: Event source; Role: Where events originate; Examples: Applications, IoT sensors, click streams, server logs Component: Stream transport; Role: Durable, ordered event delivery; Examples: Apache Kafka, AWS Kinesis, Google Pub/Sub Component: Stream processing; Role: Transform, join, aggregate events; Examples: Apache Flink, Spark Structured Streaming Component: Model step; Role: Call model on each event or mini-batch; Examples: Embedded inference within processing job

Must remember:

- Streaming inference processes continuous event flows through a long-running pipeline
- Architecture: event source → stream transport → processing layer → model step → sinks
- No job start/end; pipeline runs 24/7; outputs go to alerts, features, dashboards - not direct user responses
- Differs from batch (static snapshot, scheduled) and online (single request, synchronous response)
- Introduces stateful concepts: windowing, watermarks, checkpoints
- Choose streaming when you have a continuous event stream and need near-real-time reaction without a blocking caller

Example or contrast: Some scenarios could use either pattern:

Exam trap: Trap: "Streaming = online but faster." - Streaming has no blocking caller; it is an always-on pipeline, not request-response Trap: Using streaming when hourly batch suffices - streaming adds significant operational complexity

MLME-W02-T12 Streaming Inference: Use Cases, Metrics, and Trade-offs

Tier A

Definition and why it matters: Streaming inference is the right tool when events arrive continuously and you need to react quickly as the stream evolves - not in response to a specific user request, but as part of an always-on monitoring and reaction system

Core explanation: General inference metrics (latency, throughput, cost) apply, but streaming adds domain-specific measurements: Metric: Event-to-action latency; Definition: Time from event occurrence to downstream action triggered by model prediction; Why It Matters: End-to-end reaction speed Metric: Sustained throughput; Definition: Events per second the pipeline handles over long periods (not just spikes); Why It Matters: Can the pipeline keep up 24/7? Metric: Lag / backlog; Definition: How far behind processing is from the event source; Why It Matters: Growing lag = falling behind

Must remember:

- Streaming use cases: fraud detection, clickstream analytics, IoT sensors, log/security analytics
- Key metrics: event-to-action latency, sustained throughput, lag/backpressure, 24/7 cost
- Lag growing = pipeline falling behind; backpressure = system overloaded
- Benefits: near-real-time reaction, continuous behavioral view, temporal sequence context
- Costs: higher complexity, 24/7 operations, harder debugging, restart/replay challenges
- Use streaming when events demand immediate reaction - not just because it sounds modern

Formula / decision rule: Event-to-Action Latency = t_(action) - t_(event occurrence)

Example or contrast: Domain: Fraud / anomaly detection; Stream: Transaction, login, or system metric events; Model Action: Score each event for suspiciousness; Downstream Effect: Trigger alert or automated block Domain: Clickstream analytics; Stream: Page views, clicks, scrolls, interactions; Model Action: Detect patterns, segment users, compute real-time features; Downstream Effect: Personalization, A/B test assignment Domain: IoT / sensor data; Stream: Device readings from machines, vehicles, sensors; Model Action: Predict failure, anomaly, or state transition; Downstream Effect: Maintenance alert, autonomous action

Exam trap: Trap: "Streaming is always better than batch because it is real-time." - Use streaming only when the problem genuinely demands it; daily batch may suffice Trap: Ignoring lag monitoring - growing lag silently degrades prediction freshness

MLME-W02-T13 Choosing the Right Inference Pattern: Decision Guide

Tier A

Definition and why it matters: Pattern selection is not about technology fashion - it is about matching who is waiting, how fresh predictions must be, and how much data flows how often

Core explanation: Concept flow: Need sub-second response: to a specific user action? - Online Inference: Request-Response API - Continuous stream of events: needing quick reaction? - Streaming Inference: Event Pipeline - Millions of rows on a schedule: with no one waiting per row? - Batch Inference: Scheduled Job Signal: User clicked a button; system called an API; flow cannot proceed without prediction; Pattern: Online Signal: Payment checkout blocked until fraud decision; Pattern: Online Signal: Search results must rank before page renders; Pattern: Online Signal: Score all customers weekly for a marketing campaign; Pattern: Batch

Must remember:

- Question: Who is waiting?; Answer - Pattern: User/system blocked - Online
- Question: How fresh?; Answer - Pattern: Hours/days acceptable, bulk data - Batch
- Question: Data shape?; Answer - Pattern: Continuous events, near-real-time reaction - Streaming
- Think of it as three axes:
- Concept flow: Who is waiting? - How fresh must: predictions be? - How much data,: how often? - Pattern Choice
- Three decision questions: sub-second user response? scheduled bulk? continuous event stream?
- Online: caller blocked, synchronous request-response
- Batch: millions of rows, deadline-driven, no one waiting per row
- Streaming: continuous events, near-real-time reaction, no direct caller
- Decision axes: who is waiting, freshness requirements, data volume/frequency
- Use streaming only when the problem genuinely demands it - not by default
- Hybrid patterns (batch precompute + online serve) are common in production

Exam trap: Trap: Defaulting to online APIs by habit - batch is simpler and cheaper when freshness tolerance is hours or days Trap: Choosing streaming for problems that only need hourly batch - operational cost is 10-100x higher

MLME-W02-T14 Inference Patterns: Metrics Mapping and Scenario Guide

Tier A

Definition and why it matters: The same ML model can sit underneath any serving pattern. What changes is which metric you optimize and which engineering decisions follow

Core explanation: Pattern: Batch; Optimize For: Throughput, total job time; De-prioritize: Per-row latency; Cost Strategy: Off-peak scheduling, spot instances Pattern: Online; Optimize For: P95/P99 latency, error rate; Cost Strategy: Right-sized replicas, caching, auto-scaling Pattern: Streaming; Optimize For: Event-to-action latency, sustained throughput; Cost Strategy: Efficient 24/7 workers, minimize lag Concept flow: Same ML Model f(x - Batch: Throughput + Job Time - Online: P95/P99 Latency - Streaming: Event-to-Action + Lag Metric: Total job time; Target Example: $<$ 4 hours for 10M rows; Alert On: Job exceeds SLA deadline Metric: Throughput; Target Example: 1,000 rows/sec; Alert On: Throughput drops below baseline Metric: Job success rate; Target Example: 99.9%; Alert On: Job failure Metric: Per-row latency; Target Example: Not monitored Metric: P95 latency; Target Example: $<$ 200 ms; Alert On: P95 exceeds SLO Metric: P99 latency; Target Example: $<$ 500 ms; Alert On: P99 exceeds SLO Metric: Error rate; Target Example: $<$ 0.1%; Alert On: Error rate spike Metric: Throughput (RPS); Target Example: Handle 5K RPS at peak; Alert On: Saturation approaching limit

Must remember:

- Each pattern optimizes different metrics: batch (throughput/job time), online (P95/P99), streaming (event-to-action/lag)
- Scenario mapping: weekly churn = batch, payment fraud = online, login monitoring = streaming
- Don't default to online - match pattern to business needs
- Same model f(x), different serving wrapper and metric priorities
- Lab measures batch (total time, rows/sec) vs online (P95 latency, RPS) to make trade-offs tangible

Formula / decision rule: prediction = f(input features); f(x)

Example or contrast: Aspect: Business need; Detail: Marketing team decides who to contact for retention campaign

Exam trap: The hands-on lab builds both a batch client and an online client against the same model API: Client: Batch; Metrics Measured: Total time, rows per second

MLME-W02-T15 Lab: Model Selection and Single-Request Inference

Tier A

Definition and why it matters: This lab moves from a basic model (DistilGPT-2) to a more capable instruction-tuned model, then establishes the foundation for measuring inference performance. The goal is to understand how model architecture affects loading and prediction, and to time a single inference call before scaling to bulk scoring

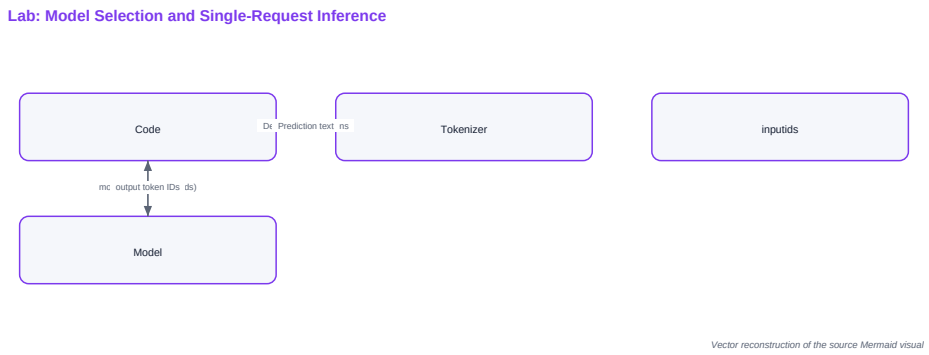
Core explanation: Factor: Parameter count; Impact on Inference: Larger models need more memory and compute per prediction Factor: Architecture type; Impact on Inference: Determines which Hugging Face AutoModel class to use Factor: Instruction tuning; Impact on Inference: Models trained on tasks (translation, classification) follow prompts rather than just predicting next token Concept flow: Code - Tokenizer - inputids - Model Flan-T5 Small (~77M parameters) is instruction fine-tuned - trained not just on raw text but on tasks with explicit instructions Column: Input text; Content: Instruction-prefixed prompts The model learns to follow instructions rather than only predict the next word Column: Task type; Content: Sentiment, translation, classification, etc

Must remember:

- Model selection must balance capability with hardware constraints (Flan-T5 Small: 77M params, runs on CPU)
- Causal LM (GPT family) uses AutoModelForCausalLM; Seq2Seq (T5/Flan-T5) uses AutoModelForSeq2SeqLM
- Flan-T5 is instruction-tuned - follows task prompts like "translate" or "classify sentiment"
- Single-request inference: tokenize → generate → decode; ~60-70 ms on CPU
- Lab scales to 1,000-row CSV for bulk scoring experiments
- Always measure end-to-end latency including tokenization and decoding

Example or contrast: Model: GPT-OSS 120B; Parameters: ~120 billion; Approximate Size: ~240 GB; Runnable Locally?: No - requires cloud GPU cluster Model: GPT-OSS 20B; Parameters: ~20 billion; Approximate Size: ~40 GB; Runnable Locally?: No - requires multi-GPU setup Model: Flan-T5 Small; Parameters: ~77 million; Approximate Size: ~300 MB; Runnable Locally?: Yes - runs on CPU

Exam trap: Trap: Using AutoModelForCausalLM for T5/Flan-T5 models - must use AutoModelForSeq2SeqLM Trap: Assuming larger models are always better for inference labs - they may not fit local hardware



Concept map: Lab: Model Selection and Single-Request Inference

MLME-W02-T16 Lab: Sequential vs Batch Inference Performance

Tier A

Definition and why it matters: Process 1,000 instruction-prefixed prompts from a CSV file and compare sequential inference (one input at a time) against batched inference (multiple inputs per forward pass). Measure total time, average per-input latency, and throughput (inputs per second) across different batch sizes

Core explanation: A CSV with 1,000 rows across four task types: Task Type: Sentiment classification; Example Prompt: classify the sentiment of this review: Task Type: Translation; Example Prompt: translate English to French: Task Type: Question answering; Example Prompt: What type of machine learning is Task Type: Summarization; Example Prompt: summarize: Metric: Total inference time (1,000 inputs); Value: ~61 seconds Metric: Average time per input; Value: ~0.06 seconds (60 ms) Metric: Throughput; Value: ~16.25 inputs/second Benefit: Parallelism; Mechanism: GPU/CPU processes multiple inputs in one matrix operation Benefit: Hardware utilization; Mechanism: Better use of vectorized compute units Process one input at a time in a loop: Benefit: Amortized overhead; Mechanism: Model loading and setup costs spread across batch Concept flow: Short: 5 tokens - Padded: 12 tokens - Long: 12 tokens - 12 tokens (no padding - Batch Tensor: 2 x 12

Must remember:

- Sequential baseline: 1,000 inputs in ~61 sec, ~16.25 inputs/sec on CPU
- Batch inference groups multiple inputs per forward pass for better hardware utilization
- Padding required for variable-length text; adds compute overhead
- Small batches (size 2) can be slower than sequential on CPU due to padding
- Large batches (16, 64) amortize padding cost and achieve higher throughput (21-26 inputs/sec)
- Optimal batch size is data-driven: start small, increase, measure, stop at plateau or memory limit
- Throughput = inputs / total time; the key metric for comparing inference strategies

Formula / decision rule: Throughput = (Number of inputs)/(Total time) = (1000)/(61) ~= 16.25 inputs/sec

Example or contrast: Strategy: Sequential; Batch Size: 1; Total Time: 61 sec; Throughput (inputs/sec): 16.25; vs Baseline: Baseline

Exam trap: Factor: Padding operations; Small Batch (2): 500; Large Batch (64): 16 Factor: Padding cost per input; Small Batch (2): High; Large Batch (64): Low

MLME-W02-T17

Lab Summary: Interpreting Inference Metrics and Trade-offs

Tier A

Definition and why it matters: The lab experiments processed thousands of inputs using different batching strategies and measured throughput in inputs per second. The results provide a concrete foundation for understanding when batching helps and when it hurts

Core explanation: Processing multiple inputs together in a single forward pass, rather than one at a time Term: Batch size; Meaning: Number of inputs processed per forward pass Term: Sequential; Meaning: Batch size = 1; one input per pass Term: Padding; Meaning: Adding tokens to shorter inputs so all inputs in a batch have equal length With batch size 2: Each batch of 2 requires padding shorter input to match longer one On CPU, padding overhead outweighs parallelism gains Result: 81 seconds vs 61 seconds sequential - 33% slower Concept flow: Start with small batch size - Measure throughput - Increase batch size - Throughput still: improving? - Memory limit hit? - Use previous batch size Sequential processing avoids padding entirely because each input is processed independently Step: 1; Action: Start with small batch size (2-4) With batch sizes 16 and 64: Step: 2; Action: Measure throughput (inputs/sec)

Must remember:

- Strategy: Sequential; Batch Size: 1; Throughput (inputs/sec): 16.25; Relative Performance: Baseline
- Batching trade-off: parallelism benefit vs padding cost
- Sequential baseline: 16.25 inputs/sec; batch size 2 was worse (12.3); batch size 64 was best (26.0)
- Small batches fail because padding overhead outweighs parallelism on CPU
- Large batches win by amortizing padding and setup costs across many inputs
- Choose batch size empirically: start small, increase, measure, stop at plateau or memory limit
- Throughput (inputs/sec) is the key metric for comparing inference strategies
- These principles ensure efficient and scalable model deployment

Formula / decision rule: Cost per input = (Padding overhead + Setup cost)/(Batch size)

Exam trap: Concept flow: Parallelism - Hardware Utilization - Padding Overhead - Memory Usage - Net Performance - Benefit - Cost Side: Benefit; Effect: Increased parallelism, improved hardware utilization

MLME-W02-T18

Module 2 Summary: Model Inference Patterns

Tier A

Definition and why it matters: Module 2 focused on the serving side of the ML lifecycle: what happens when a trained model is asked for a prediction, how we measure performance, and how to choose among three inference patterns

Core explanation: Concept: Inference; Detail: Using a trained model to produce output on new, real-world data Concept: vs Training; Detail: Training learns parameters occasionally; inference runs continuously Concept: Production form; Detail: A running service that receives requests, runs the model, returns predictions Concept: Core equation; Detail: prediction = f(input features) Concept flow: Receive Input - Validate & Parse - Feature Transform - Model Forward Pass - Post-process - Return Prediction Metric: Latency; Definition: Time from request to prediction; Key Insight: P95/P99 matter more than average for user-facing systems Metric: Throughput; Definition: Predictions per unit of time; Key Insight: Reveals scaling capacity Metric: Cost; Definition: Compute, memory, storage, bandwidth per prediction; Key Insight: Compounds at scale into real cloud bills Pattern: Batch; When: Large dataset, scheduled, no one waiting per row; Primary Metrics: Throughput, total job time; Infrastructure: Scheduled job (Airflow, cron)

Must remember:

- Inference = applying trained model to new data via a running service; dominates the model lifecycle
- Three metrics: latency (P95/P99 for online), throughput (rows/sec or RPS), cost (per-prediction resources)
- Three patterns: batch (scheduled bulk), online (synchronous request-response), streaming (continuous events)
- Decision: who is waiting? how fresh? how much data how often?
- Online needs: caching, auto-scaling, circuit breakers, hybrid architectures
- Lab proved: batch size matters - measure throughput empirically; padding overhead can make small batches slower
- Unified framework: same three metrics compare all patterns; same model f(x), different serving wrappers

Formula / decision rule: prediction = f(input features); f(x)

Exam trap: Trap: Treating inference as just model.predict() - the full pipeline includes validation, feature prep, and post-processing Trap: Using average latency when SLAs specify P95/P99

Week 2 Exam Lens

Likely questions

- Define and explain Definition of Model Inference; include the mechanism and one limitation.
- Compare Definition of Model Inference with Latency, Throughput, and Cost: The Inference Metrics Triangle and state when each is preferred.
- Apply Inference Patterns: Setup and Mental Model to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: hat(y) = f(x); f(x); prediction = f(input features); from historical data; **inference** calls; Total Latency = t_(network) + t_(validate) + t_(features) + t_(model) + t_(postprocess); f(input); Event-to-Action Latency = t_(action) - t_(event occurrence); Throughput = (Number of inputs)/(Total time) = (1000)/(61) ~= 16.25 inputs/sec

Mistakes to avoid: Trap: "Inference is just calling model.predict()." - Inference includes the full request path: validation, feature engineering, post-processing, serialization, and network overhead Trap: "Inference = forward pass only." - The forward pass is one step in a six-step pipeline; network, validation, and feature prep often dominate latency Trap: Optimizing average latency when the SLA specifies P95/P99 - averages hide tail problems

Week 3: Serving, APIs, Containers, and Rollouts

Week roadmap: Model Serving Patterns and Containerization: Module Overview → Defining Model Serving: Artefact vs Service → Core Responsibilities of the Model Serving Layer → Monolithic Model Serving Architecture → Microservice Model Serving Architecture → Serverless Model Serving Architecture → 11 more topics.

Coverage: 17 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

MLME-W03-T01 Model Serving Patterns and Containerization: Module Overview

Tier C

Definition and why it matters: In earlier modules, the focus was on the client side of inference - how batch scripts score many rows, how online clients send one request and wait for one response, and how metrics like latency and throughput define success. The mental model was straightforward: send JSON to a predict function, receive predictions back

Core explanation: This module flips the camera around. The central question becomes: what lives behind the predict endpoint? Where does the trained model actually reside, and how does it handle real traffic in production?

Must remember:

- Module 3 shifts focus from the client (Module 2) to the serving layer behind the predict endpoint
- Serving answers: where the model lives, how it handles requests, and how it integrates with production infrastructure
- Four topics: definition/responsibilities, architectures, APIs, deployment/operations
- Inference patterns (batch/online/streaming) and serving architectures (monolith/microservice/serverless) are two layers of the same picture
- Labs build a FastAPI + Docker service that embodies these concepts in code
- The serving layer is what turns a trained model artefact into a measurable production system

Exam trap: Confusing inference pattern with serving architecture - batch/online/streaming describes how clients call the model; monolith/microservice/serverless describes where the model lives in the system. They are orthogonal layers Assuming serving is just model.predict() - serving orchestrates validation, feature transformation, inference, response formatting, and operational concerns

MLME-W03-T02 Defining Model Serving: Artefact vs Service

Tier A

Definition and why it matters: Training produces a serialised artefact - model.pkl, model.pt, or similar - that sits passively on disk. Production requires something fundamentally different: a long-lived service that loads the model, listens for incoming requests, runs inference on each request, and returns structured responses to callers

Core explanation: Model serving is not "we have a file." It is "we have a reliable service that takes real traffic, answers prediction calls, and behaves like a proper production component." Concept flow: Client - Server - predict - Model Step: Receive; What Happens: Accept HTTP/JSON, gRPC, or similar; Why It Matters: Entry point for all traffic Step: Validate & parse; What Happens: Check types, required fields, ranges; convert raw JSON to structured objects; Why It Matters: Prevents garbage input and training-serving skew

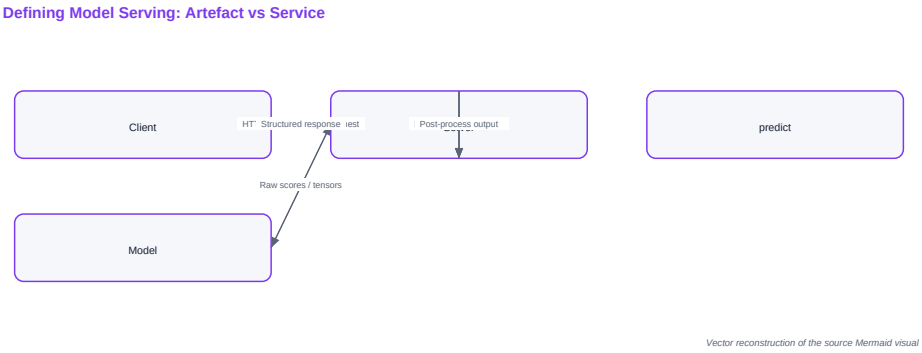
Must remember:

- Model serving = long-lived service that loads a model, handles requests, and returns predictions
- Artefact = passive file; service = active process with endpoints and operational integration
- Each request flows through: receive → validate → transform → infer → post-process → respond
- Serving orchestrates the full pipeline, not just model.predict()
- Runtime can be FastAPI, Flask, gRPC, serverless, or specialised engines - the pattern is the same
- Production serving integrates with monitoring, logging, and client contracts from day one

Formula / decision rule: $p > 0.85$

Example or contrast: Aspect: Form; Model Artefact: Serialised file on disk (.pkl, .pt, .onnx); Model Service: Running process or container Aspect: State; Model Artefact: Passive - waits to be loaded; Model Service: Active - listens for requests Aspect: Interface; Model Artefact: None; Model Service: Exposes endpoints (POST /predict, GET /health)

Exam trap: Equating artefact with service - having model.pkl on S3 is not model serving; serving requires a running process with an API Skipping preprocessing in the serving path - calling predict on raw client JSON without the training-time transformations causes silent accuracy degradation (training-serving skew)



Concept map: Defining Model Serving: Artefact vs Service

MLME-W03-T03 Core Responsibilities of the Model Serving Layer

Tier A

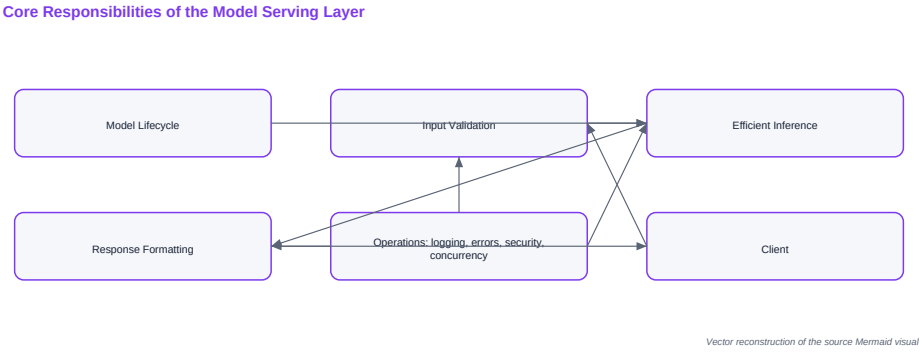
Definition and why it matters: A production model service is not a thin wrapper around predict. It is a production-grade component with five overlapping responsibility domains: model lifecycle management, input validation, efficient inference, response formatting, and operational concerns. Most real-world engineering effort goes into the operational layer, not the predict call itself

Core explanation: The serving layer owns the model's life inside the running process: Load at startup (or lazily on first request) - deserialize the artefact into memory Concept flow: Service Startup - Load Model into Memory - Service Ready - Request 1 - reuse model - Request 2 - reuse model - Request N - reuse model Check: Required fields present; Purpose: Prevent partial inputs from reaching the model Check: Correct types (string, number, enum, array); Purpose: Catch client bugs early Check: Value ranges and constraints; Purpose: Block out-of-distribution garbage Version management - activate v1, v2, or roll back to a previous version Check: Graceful error handling; Purpose: Return HTTP 400 with clear messages, not stack traces Resource fit - ensure the model fits within available CPU, GPU, and memory budgets Critical anti-pattern: loading the model inside the request handler on every call. Deserializing a large model can take seconds to minutes - doing this per request destroys latency and wastes CPU

Must remember:

- Five responsibility domains: model lifecycle, input validation, efficient inference, response formatting, operations
- Load model once at startup; never per request
- Schema enforcement (Pydantic, protobuf, JSON Schema) prevents garbage input and training-serving skew
- Inference must respect latency (P95/P99), throughput, and stability under real traffic
- Responses need stable schemas with predictions plus optional metadata (version, confidence, request ID)
- Operational concerns (concurrency, errors, logging, security) dominate real production effort
- The serving layer is predict under real-world constraints, not predict in isolation

Exam trap: Per-request model loading - the single most common serving anti-pattern; always load once at startup Ignoring schema enforcement - without Pydantic/JSON Schema, training-serving skew and silent failures are inevitable



Concept map: Core Responsibilities of the Model Serving Layer

MLME-W03-T04 Monolithic Model Serving Architecture

Tier A

Definition and why it matters: In a monolithic setup, everything lives together in one application: Concept flow: Web UI / Dashboard - Business Logic - Database - Model Code + Artefact - User

Core explanation: Once you understand what the serving layer does (load model, validate, infer, format, log), the next question is architectural: how do you organise that serving layer inside a real system? Three patterns dominate - monolith, microservice, and serverless. This note covers the simplest: the monolith Monoliths get a bad reputation, but they are often exactly what you want early on Advantage: Simple deployment; Explanation: Build one artefact, deploy one artefact Advantage: Low infrastructure overhead; Explanation: No extra services, configs, or deployment pipelines Advantage: Straightforward local development; Explanation: Run one process; test end-to-end locally Advantage: Fast iteration; Explanation: Speed

matters more than perfect architecture for small teams Good fit for: Small internal tools and dashboards Proof-of-concept ML features

Must remember:

- Monolith = model embedded inside the main application, same process, no separate service
- Pros: simple deployment, low overhead, fast iteration, easy local dev
- Cons: cannot scale model independently, coupled deployments, tech stack lock-in, large blast radius
- Good for: internal tools, POCs, low-traffic ML features
- Move to microservices when: model is heavy/GPU-hungry, traffic grows, teams need independent evolution
- Even in a monolith, the full serving pipeline (validate → transform → infer → format) still applies

Example or contrast: Dimension: Deployment units; Monolith: 1; Microservice: 2+; Serverless: Function per invocation Dimension: Scale model independently; Monolith: No; Microservice: Yes; Serverless: Automatic but limited Dimension: Operational complexity; Monolith: Low; Microservice: Medium-High; Serverless: Low (managed)

Exam trap: As models and traffic grow, monoliths hit real walls: Pain Point: Coupled scaling; Consequence: Model needs GPUs - you scale the entire app, including parts that do not need GPUs

MLME-W03-T05 Microservice Model Serving Architecture

Tier A

Definition and why it matters: In a microservice architecture, the model lives in its own separate service - its own process or container, exposing a dedicated API (typically POST /predict over HTTP or gRPC). Other parts of the system (frontend, backend, batch jobs) call this service as a remote dependency

Core explanation: The model is no longer a function inside a big app. It becomes its own product-like service with a clear contract Concept flow: Frontend - Backend API - Batch Job - Model Microservice: POST /predict - Loaded Model Scale the model service up or down without touching the frontend or backend. If the model needs more CPUs, GPUs, or memory, adjust its replica count independently Example: an e-commerce recommendation engine runs 20 model-service replicas on GPU nodes while the Django backend stays at 4 CPU-only instances The model service can run Python with specific ML libraries, use GPUs, and maintain its own dependency tree. The rest of the system can be in Go, Java, or Node.js - they communicate over HTTP/gRPC, not shared memory A team owns the model API, its request/response schema, and its SLOs (latency, uptime). Other teams only need to know: "call POST /predict with this JSON, get this JSON back."

Must remember:

- Model microservice = separate process/container with its own POST /predict API
- Benefits: independent scaling, tech stack freedom, clear ownership, independent deploy/rollback
- Costs: more infra, API versioning, network overhead, distributed tracing needs
- Right when: model is critical, traffic is high, independent iteration is needed
- The service contract (schema + SLOs) is the foundation that makes microservices work
- Common in production: model service behind API gateway, called by backend and batch jobs alike

Exam trap: Trade-off: Deployment complexity; Detail: More services to deploy, configure, and monitor Trade-off: API design discipline; Detail: Other teams depend on your predict API; breaking changes are costly

MLME-W03-T06 Serverless Model Serving Architecture

Tier A

Definition and why it matters: Serverless (Function-as-a-Service, FaaS) packages model code as a function that a cloud provider manages. The provider handles underlying servers, scaling, and infrastructure - you only write the inference logic

Core explanation: The function can be triggered by HTTP requests, queue messages, scheduled events, or other cloud events Concept flow: HTTP Request - Serverless Function: Model + Code - Queue Message - Scheduled Event - Cloud Provider: AWS Lambda / GCP Cloud Functions - Prediction Response Advantage: Built-in autoscaling; Detail: Traffic spikes - more function instances; traffic drops - instances spin down Advantage: Pay per use; Detail: Cost-effective for low or bursty traffic; no idle server charges Advantage: Zero server management; Detail: Cloud provider handles VMs, patching, and capacity Serverless is attractive when: Traffic is spiky or unpredictable - Black Friday surges, campaign launches Volume is low or medium - you do not want to pay for idle servers Models are simple or lightweight - small sklearn models, rule-based scorers You want automatic scale-up and scale-down with pay-per-use pricing Real-world example: a marketing team deploys a lead-scoring function on AWS Lambda. It runs 200 times per day normally, spikes to 50,000 during a webinar. Lambda spins up instances automatically; cost scales with actual invocations, not provisioned capacity

Must remember:

- Serverless = model packaged as a cloud-managed function (Lambda, Cloud Functions)
- Pros: auto-scaling, pay-per-use, zero server management
- Cons: cold starts, time/memory limits, packaging constraints, statelessness
- Best for: spiky traffic, lightweight models, prototypes, event-driven ML
- Poor for: latency-critical APIs, large GPU models, sustained high throughput
- Mitigate cold starts with provisioned concurrency and minimum warm instances

Exam trap: Constraint: Cold start; Impact on ML: Function idle for a while - first invocation is noticeably slower (model load + container init) Constraint: Time limits; Impact on ML: Per-invocation timeout (e.g., 15 min on Lambda) - heavy or long-running inference may fail

MLME-W03-T07 Design Decisions in Model Serving Systems

Tier A

Definition and why it matters: Choosing a serving architecture is an engineering trade-off, not a correctness question. The right choice depends on how critical the model is, what traffic looks like, and how the organisation is structured. This note synthesises the three architectures and connects them to inference patterns from Module 2

Core explanation: Dimension: Build & deploy; Monolith: One app, one artefact; Microservice: Separate service with own pipeline; Serverless: Function package to cloud Dimension: Scale model independently; Monolith: No; Microservice: Yes; Serverless: Automatic (with limits) Dimension: Coupling; Monolith: Model and app tightly coupled; Microservice: Model is isolated with API contract; Serverless: Model is ephemeral per invocation Dimension: Operational complexity; Monolith: Low; Microservice: Medium-High; Serverless: Low (provider-managed) Dimension: Cost at low traffic; Monolith: Fixed (always-on); Microservice: Fixed (replicas); Serverless: Near zero (pay per use) Dimension: Cold start risk; Monolith: None; Microservice: Low (if min replicas set); Serverless: High Dimension: Best for; Monolith: Internal tools, POCs, experiments; Microservice: High-traffic critical model APIs; Serverless: Spiky, lightweight, event-driven Concept flow: Traffic level? - Monolith - Need independent scaling? - Microservice - Serverless Inference Pattern: Batch; Typical Serving Architecture: Scheduled job or container in a cluster; may call a model microservice to score data; Communication Style: Async jobs, queues

Must remember:

- Three architectures: monolith (simple, coupled), microservice (independent, complex), serverless (auto-scale, constrained)
- No universal right answer - depends on criticality, traffic, team structure, and SLOs
- Inference patterns (batch/online/streaming) and serving architectures are two orthogonal layers
- Batch → jobs/queues; online → microservice or serverless; streaming → pipeline consumers
- Lab Docker container works as monolith or microservice depending on deployment context
- Serving responsibilities stay constant; architecture changes how you build, deploy, and operate

Exam trap: Treating architecture choice as permanent - teams commonly start monolith and migrate to microservice as traffic grows Confusing inference pattern with architecture - batch inference can use a microservice or a monolith; they are independent choices

MLME-W03-T08 REST APIs for Machine Learning Serving

Tier A

Definition and why it matters: Every served model is exposed through an API. The design of that API directly impacts developer experience, performance, service contract stability, and how easily other teams integrate. A well-designed API is the foundation for connecting your model to the rest of the organisation - not a minor technical detail

Core explanation: REST (Representational State Transfer) uses standard HTTP verbs (GET, POST) and sends input/output in the request/response body, typically formatted as JSON Concept flow: Client - API Reason: Universal language support; Detail: Callable from Python, JavaScript, Java, Go, curl, browsers Reason: Human-readable payloads; Detail: Easy to inspect, log, and debug Reason: Excellent tooling; Detail: curl, Postman, Bruno, Swagger/OpenAPI auto-documentation Reason: Low onboarding friction; Detail: Any developer who knows web APIs can call your model The dominant ML serving pattern: Step: 1; Client Side: Build JSON with feature fields Step: 2; Client Side: POST /predict with Content-Type: application/json; Server Side: Receive and parse JSON Step: 3; Client Side: Wait for response; Server Side: Validate - transform - infer - format

Must remember:

- REST = HTTP verbs + JSON payloads; the default ML serving API style
- Dominant pattern: POST /predict with JSON features in, JSON predictions out
- Pros: universal, human-readable, excellent tooling, low friction
- Cons: verbose payloads, no compile-time typing, inefficient at extreme scale
- Best default for prototypes, external APIs, and moderate-traffic production
- FastAPI + Pydantic + OpenAPI gives schema enforcement and auto-documentation
- Graduate to gRPC when throughput, payload size, or internal service mesh demands it

Exam trap: Assuming REST means no schema enforcement - Pydantic + OpenAPI provides strong runtime validation even with REST Using GET for predictions with sensitive features - features in URL query strings leak in logs; always use POST with JSON body

MLME-W03-T09 gRPC for Machine Learning Serving

Tier A

Definition and why it matters: REST with JSON is the right default for most ML APIs. But in larger, performance-sensitive systems - many internal microservices, strict latency SLOs, high throughput - teams adopt gRPC (Google Remote Procedure Call)

Core explanation: gRPC is a high-performance, contract-first framework that uses Protocol Buffers (protobuf) instead of JSON Concept flow: .proto Schema - Code Generation - Python Client - Go Client - Java Client - gRPC Model Service Advantage: Binary encoding; Detail: Payloads are much smaller; serialization/deserialization is much faster than JSON Advantage: Strong typing; Detail: Field changes caught at compile time, not discovered at runtime in production Advantage: High throughput; Detail: Efficient for service-to-service communication at scale Advantage: Multi-language codegen; Detail: One schema, clients in every language your org uses Instead of free-form JSON, request and response formats are defined in a .proto file - a strongly typed schema. Client and server both generate code from this single contract Aspect: Payload size; REST/JSON: Larger (text); gRPC/Protobuf: ~3-10x smaller (binary) Aspect: Serialization speed; REST/JSON: Slower; gRPC/Protobuf: Faster

Must remember:

- gRPC = high-performance, contract-first RPC using Protocol Buffers (binary, not JSON)
- proto schema generates typed clients in multiple languages; changes caught at compile time
- Advantages: smaller payloads, faster serialization, strong typing, high throughput
- Best for: internal microservices, real-time systems, large distributed architectures
- REST at the edge, gRPC inside - the common large-system hybrid pattern
- Start with REST; adopt gRPC when scale, payload size, or service mesh complexity demands it

Example or contrast: Criterion: Audience; REST: External clients, partners, browsers; gRPC: Internal service-to-service

Exam trap: gRPC for external/public APIs - browsers do not natively support gRPC; you need gRPC-Web or a REST gateway Skipping .proto versioning - breaking proto changes require careful backward-compatible field numbering

MLME-W03-T10 Synchronous vs Asynchronous API Calls for ML Serving

Tier A

Definition and why it matters: API protocol (REST vs gRPC) is one design choice. Another orthogonal dimension is when the client receives the result - synchronously (blocking) or asynchronously (non-blocking). Both sync and async can be implemented with REST, gRPC, queues, or any protocol

Core explanation: In a synchronous call, the client sends a request and blocks until the model returns a prediction. Nothing else happens in that workflow until the response arrives or times out Concept flow: Client - Model Use Case: UI fetching real-time recommendations; Why Sync: User is staring at the screen waiting Use Case: Mobile app getting sentiment score; Why Sync: Screen update depends on the result Use Case: Payment backend checking fraud before approving; Why Sync: Transaction cannot proceed without the score Concept flow: Client - Queue - Worker - Store Works well when: Inference is fast (typically under 100-300 ms) The prediction has direct impact on user experience or business decisions The caller needs the answer right now Use Case: Scoring millions of rows in a churn model; Why Async: Hours of processing; no user waiting

Must remember:

- Sync = client blocks until prediction arrives; async = client submits job and moves on
- Sync for: fast inference (< 300 ms), real-time UX, immediate business decisions
- Async for: heavy/slow inference, peak smoothing, offline/near-real-time use cases
- Batch → async queues; online → sync REST/gRPC; streaming → event-driven pipelines
- Same model can be served both ways depending on consumption pattern
- Course default: sync POST /predict with JSON - simple, debuggable, production-viable

Example or contrast: Dimension: Client behaviour; Synchronous: Blocks until response; Asynchronous: Submits and moves on

Exam trap: Sync for long-running inference - blocking a client for minutes ties up connections and violates UX; use async Async for real-time fraud detection - the transaction cannot wait for a queue worker; sync is mandatory

MLME-W03-T11 Single-Instance Deployment of Model Services

Tier B

Definition and why it matters: Once you have a POST /predict endpoint (FastAPI, gRPC, or other), the next questions are operational: where does this service run, how do you update it safely, and how do you scale with traffic? This note covers the simplest deployment pattern - a single instance - and the build-package-run pipeline that gets you there

Core explanation: The simplest production setup: one VM or one container running your model API Concept flow: Single Container / VM: FastAPI + Model - Client - model.pkl in memory Listens on a port (e.g., 8000), directly or behind a load balancer / reverse proxy Step: Build; What Happens: Commit code - CI runs tests, linters, health checks; Tools: GitHub Actions, GitLab CI, Jenkins

Must remember:

- Simplest deployment: one VM or container running the model API on a single port
- Build-Package-Run pipeline: CI tests → Docker image (tagged) → run container with config
- Docker image = portable, reproducible deployment unit (code + model + dependencies)
- Configuration via environment variables, config files, and secrets managers
- Production rollouts need controlled strategies - not "deploy and hope."
- Lab pattern (FastAPI in Docker on localhost) is single-instance deployment in practice

Exam trap: Skipping CI before packaging - deploying untested images to production is the most common deployment failure mode Not tagging images with versions - latest tag makes rollbacks impossible; always use semantic versioning

MLME-W03-T12 Blue-Green and Canary Deployment for Model Services

Tier B

Definition and why it matters: Deploying a new model version to production is risky - bugs, worse live performance, latency regressions. Rollout strategies let you test with real traffic in a controlled way before committing fully, and roll back quickly if something goes wrong

Core explanation: Run two complete environments side by side: Concept flow: Load Balancer - Blue Environment: v1 - stable - Green Environment: v2 - new Advantage: Fast, simple rollback; Detail: Switch traffic back to blue in seconds Advantage: Parallel testing; Detail: New version runs on real infrastructure before full exposure Blue = current stable version (serving all production traffic)

Must remember:

- Blue-green: two environments side by side; switch all traffic at once; instant rollback

- Canary: route small % to new version; ramp gradually; stop or rollback on problems
- Canary = safety/reliability question; A/B test = business impact question
- Watch error rates, P95/P99 latency, and model-specific quality metrics during canary
- Blue-green costs 2x temporarily; canary costs minimal extra infrastructure
- Production systems often combine both strategies for different release types

Example or contrast: Both involve splitting traffic, but they serve different purposes:

Exam trap: Confusing canary with A/B test - canary asks "is it safe?"; A/B asks "is it better?" Ramping canary without monitoring - increasing traffic without watching error rates and latency defeats the purpose

MLME-W03-T13 Autoscaling for Model Services

Tier A

Definition and why it matters: Real systems see daytime peaks, nighttime lulls, weekend patterns, campaign spikes, and seasonal surges. Fixed capacity creates a dilemma:

Core explanation: Too small → timeouts, errors, unhappy users during spikes Signal: CPU usage; Scale Up When: Average CPU stays too high across instances; Scale Down When: CPU very low for extended period Signal: Request rate (RPS/QPS); Scale Up When: Requests per second exceed per-instance capacity; Scale Down When: Sustained low request rate Signal: Queue length; Scale Up When: Async job queue grows - system falling behind; Scale Down When: Queue consistently empty Signal: Latency (P95/P99); Scale Up When: Latency rises above target SLO; Scale Down When: Latency well below target with excess capacity Signal: GPU utilisation; Scale Up When: GPU saturated on ML workloads; Scale Down When: GPU idle Signal: Memory usage / OOM events; Scale Up When: Memory pressure or out-of-memory kills; Scale Down When: Memory comfortably below limits Too large → paying for pods that sit mostly idle Concept flow: Metrics Collector - Decision Engine - Scale Up - Scale Down - Model Service Instances Autoscaling automatically adjusts the number of running instances based on load and performance signals, keeping you within SLOs without burning money

Must remember:

- Autoscaling adjusts instance count based on load signals to balance SLOs and cost
- Signals: CPU, RPS, queue length, P95/P99 latency, GPU utilisation, memory
- ML-specific: model load time (cold starts), GPU vs CPU profiles, batch vs online preferences
- Set min replicas (warm instances), max replicas (cost cap), combined signals
- Full lifecycle: train → build image → staging → canary/blue-green → monitor → ramp/rollback
- Model engineers own the entire deploy-monitor-scale loop, not just training

Exam trap: Scaling on CPU alone for GPU models - GPU utilisation is the relevant signal, not CPU Min replicas = 0 for latency-critical services - every scale-from-zero event triggers a cold start with model load time

MLME-W03-T14 Building a FastAPI Model Service

Tier A

Definition and why it matters: This note walks through the anatomy of a FastAPI-based model serving service - the same pattern used in hands-on labs and widely adopted in production ML systems. The goal is to understand API structure, model loading strategy, and the request lifecycle inside the predict endpoint

Core explanation: A typical ML serving project layout: Why: Performance; Detail: Loading a large model takes seconds to minutes; doing it per request destroys latency Why: Efficiency; Detail: One loaded instance is reused across all incoming requests In production, trainmodel.py is often replaced by a model download script (e.g., pulling a pre-trained model from Hugging Face). The serving application loads whatever artefact exists in the model/ directory Why: Robustness; Detail: Graceful handling if model file is missing (service starts but predict returns error) Concept flow: Application Startup - joblib.load('model.pkl - Model in Memory - Request 1 - model.predict - Request 2 - model.predict - Request N - model.predict

Must remember:

- FastAPI app with Pydantic models = schema-enforced ML API with auto-generated docs
- Model loaded once at startup via joblib.load(); reused across all requests
- Three endpoints: health check (GET /), predict (POST /predict), model info (GET /model-info)
- Predict flow: receive → validate (Pydantic) → transform (NumPy) → infer → respond (JSON)
- Run with uvicorn app:app; test with Bruno, curl, or /docs UI
- This pattern scales from lab prototype to production microservice with Docker and orchestration

Example or contrast: Code pattern: from fastapi import FastAPI; app = FastAPI(; title="ML Model Service",; description="Serves predictions from a trained model";) app is the root object - all endpoints attach to it FastAPI auto-generates interactive documentation at /docs (Swagger UI)

Exam trap: Loading model inside predict handler - the 1 serving anti-pattern; always load at startup Skipping Pydantic validation - raw dict access bypasses type checking and schema enforcement

MLME-W03-T15 Containerizing the ML Service with Docker

Tier A

Definition and why it matters: A Docker image is a portable, reproducible deployment unit that bundles your code, model artefact, and all dependencies. The same image runs identically on your laptop, a staging VM, a Kubernetes cluster, or a serverless container platform. Containerization is the bridge between "it works on my machine" and "it works in production."

Core explanation: Code pattern: FROM python:3.11-slim; WORKDIR /app; COPY requirements.txt; RUN pip install -r requirements.txt; COPY model.pkl app.py ./; EXPOSE 80; CMD ["uvicorn", "app:app", "–host", "0.0.0.0", "–port", "80"]

Instruction: FROM python:3.11-slim; Purpose: Base image - slim variant keeps size small and attack surface low Instruction: WORKDIR /app; Purpose: Set working directory inside container Instruction: COPY requirements.txt + RUN pip install; Purpose: Install dependencies first (layer caching) Instruction: COPY model.pkl app.py; Purpose: Copy application code and model artefact Instruction: EXPOSE 80; Purpose: Document which port the service listens on Instruction: CMD uvicorn; Purpose: Command to start the service when container runs Concept flow: Layer 1: FROM python:3.11-slim - Layer 2: COPY requirements.txt + pip install - Layer 3: COPY app.py + model.pkl - Layer 4: CMD uvicorn

Must remember:

- Docker image = portable unit bundling code, model, and dependencies
- Dockerfile: slim base → install deps (cached layer) → copy app + model → expose port → CMD uvicorn
- Layer caching: copy requirements.txt and pip install before application code
- Build: docker build -t name:version . from the correct directory
- Run: docker run -d -p host:container –name name image:tag
- Bind to 0.0.0.0 inside container for external accessibility
- Same image deploys to laptop, VM, Kubernetes, ECS, Cloud Run, and more

Exam trap: Wrong build context - running docker build from the wrong directory causes "file not found" errors; always cd to the directory containing the Dockerfile Binding to 127.0.0.1 inside container - service unreachable from host; must use 0.0.0.0

MLME-W03-T16 Testing the Containerized Model Service Locally

Tier A

Definition and why it matters: Building a Docker image is only half the deployment story. Before pushing to any remote environment, you must verify that the containerised service works correctly on your local machine - health checks pass, predictions return sensible values, and the service fails gracefully when stopped

Core explanation: With the container running (docker run -d -p 8000:80 –name ml-service ml-serving-api:v1), the service is accessible at http://localhost:8000 on the host machine Concept flow: Bruno / curl / Browser - FastAPI + Model Endpoint: /; Method: GET; Expected Result: 200 OK - {"message": "API is running"} Endpoint: /model-info; Method: GET; Expected Result: 200 OK - model metadata JSON Endpoint: /predict; Method: POST; Request Body: {"feature1": 1.5, "feature2": 2.3}; Expected Result: 200 OK - {"prediction": } Endpoint: /nonexistent; Method: GET; Expected Result: 404 Not Found Important: stop any locally running uvicorn process first. Only the container should be serving requests - otherwise port conflicts and ambiguous test results occur Confirms the container is running and the FastAPI process is healthy. In production, load balancers and Kubernetes liveness probes call this endpoint continuously Returns static model metadata. Verifies routing works and the service can respond to non-predict endpoints Validation: Health check returns 200; What It Confirms: Container started, uvicorn running, process healthy

Must remember:

- Test containerised service at localhost:8000 (mapped from container port 80)
- Test matrix: health (GET /), model info, predict (POST /predict), negative (404)
- Change predict inputs to verify model inference is live, not static
- Invalid inputs should return 422 (Pydantic validation)
- Stop container after testing; verify endpoints are dead
- Full pipeline: train → FastAPI wrap → Docker image → local test → production deploy
- Docker image = immutable artefact guaranteeing identical behaviour everywhere

Exam trap: Testing against local uvicorn instead of container - always stop the dev server before testing the containerised version Not testing invalid inputs - only testing happy path misses Pydantic validation verification

MLME-W03-T17 Module 3 Summary: Model Serving Patterns and Containerization

Tier A

Definition and why it matters: Model serving transforms a passive model artefact into a reliable, measurable production system. This module covered the full stack - from definition and responsibilities through architectures, APIs, deployment, and hands-on containerization

Core explanation: Concept: Model artefact; Definition: Passive serialised file (.pkl, .pt) on disk Concept: Model service; Definition: Long-lived running process that loads the model, listens for requests, runs inference, returns responses Concept: Model serving; Definition: The act of bringing the inert artefact to life as a production callable component : 1; Responsibility: Model lifecycle; Key Point: Load once at startup; manage versions; fit within resources : 2; Responsibility: Input validation; Key Point: Schema enforcement (Pydantic, protobuf); prevent training-serving skew : 3; Responsibility: Efficient inference; Key Point: Batching, correct transforms, concurrency; respect P95/P99 latency : 4; Responsibility: Response formatting; Key Point: Stable JSON schemas with predictions + metadata : 5; Responsibility: Operations; Key Point: Concurrency, error handling, logging, observability, security Architecture: Monolith; Best For: POCs, internal tools, low traffic; Key Trade-off: Simple but cannot scale model independently Architecture: Microservice; Best For: Critical, high-traffic model APIs; Key Trade-off: Independent scaling/deploy but more complexity Each request flows through: receive → validate → transform → infer → post-process → respond

Must remember:

- Model serving = long-lived service orchestrating the full predict pipeline under production constraints
- Five responsibilities: lifecycle, validation, inference, response formatting, operations
- Three architectures: monolith (simple), microservice (scalable), serverless (bursty)
- APIs: REST default; gRPC for internal scale; sync for online; async for batch

- Deployment: Docker image → single instance → blue-green/canary rollouts → autoscaling
- Lab: FastAPI + Pydantic + Docker + local testing = complete serving pipeline
- Model engineers own train → deploy → monitor → scale, not just training
- Inference patterns and serving architectures are two layers of one system design

Exam trap: Confusing inference pattern with serving architecture - they are orthogonal layers Treating model file as model service - artefact != service

Week 3 Exam Lens

Likely questions

- Define and explain Defining Model Serving: Artefact vs Service; include the mechanism and one limitation.
- Compare Defining Model Serving: Artefact vs Service with Core Responsibilities of the Model Serving Layer and state when each is preferred.
- Apply Monolithic Model Serving Architecture to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: $p > 0.85$

Mistakes to avoid: Confusing inference pattern with serving architecture - batch/online/streaming describes how clients call the model; monolith/microservice/serverless describes where the model lives in the system. They are orthogonal layers Equating artefact with service - having model.pkl on S3 is not model serving; serving requires a running process with an API Per-request model loading - the single most common serving anti-pattern; always load once at startup

Week 4: Deployment Pipelines and MLOps Foundations

Week roadmap: Deployment Pipelines and MLOps Basics - Module Introduction → Manual Workflow Pain, Pipeline Definition, and Benefits → Classic CI/CD in Software Engineering → ML-Specific Differences from Classic CI/CD → The Hybrid Picture: Code CI/CD + ML Pipeline → ML Artefacts: What Pipelines Move and Track → 10 more topics.

Coverage: 16 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

MLME-W04-T01 Deployment Pipelines and MLOps Basics - Module Introduction

Tier C

Definition and why it matters: An ML deployment pipeline is a sequence of automated steps that transforms raw inputs into a deployed, production-ready model artefact or service

Core explanation: A model that trains successfully in a Jupyter notebook is not the same as a model running reliably in production. Notebooks excel at exploration - quick experiments, visualisations, and iterative tuning - but they are poor at repeatability, auditability, and team-scale deployment Production ML systems must answer questions that notebooks rarely capture:

Must remember:

- Production ML requires more than notebook success - it needs repeatability, auditability, and team-scale processes
- An ML deployment pipeline automates: data prep → train → evaluate → package → deploy
- Each stage consumes and produces artefacts (data, models, metrics, containers)
- Pipelines deliver repeatability, auditability, speed, and fewer manual errors
- MLOps automates the full ML lifecycle; pipelines are its execution engine
- Shift from notebook-first to pipeline-first: notebooks explore; pipelines ship
- Real systems must answer "which model is in production?" and "how was it built?" - pipelines make that possible

Exam trap: Trap: "A notebook with good metrics is production-ready." - Metrics in a notebook do not imply traceability, reproducibility, or automated deployment Trap: Confusing packaging (Docker image) with the full pipeline (data through deploy). Packaging is one stage, not the whole system

MLME-W04-T02 Manual Workflow Pain, Pipeline Definition, and Benefits

Tier A

Definition and why it matters: Concept flow: Train in Notebook - Manually save model.pkl - Copy file to server/repo - Hand-edit config/code - Production - unknown model version?

Core explanation: Problem: Steps forgotten or done out of order; Consequence: Wrong model or stale config in production Problem: Model saved but config not updated; Consequence: Serving code loads old weights Problem: No central record of runs; Consequence: Cannot identify which model is live Problem: Metrics in screenshots or local logs; Consequence: Cannot explain performance changes Business risk: In regulated domains (finance, healthcare), you cannot demonstrate how a model was trained. In fast-moving product teams, you cannot roll back confidently when behaviour shifts Without structure, teams typically lack: A record of which data trained the model Concept flow: Data Preparation - Train - Evaluate - Package - Deploy Hyperparameters and code versions tied to each run Stage: Data preparation; Consumes: Raw data, configs; Produces: Clean, versioned dataset (train/val/test splits)

Must remember:

- Manual notebook workflows hide steps in one person's head and break under team scale
- Typical manual flow: train → save → copy → hand-edit config - fragile and untraceable
- Lack of tracking means you cannot explain production behaviour changes or pass audits
- A deployment pipeline automates data prep → train → evaluate → package → deploy with defined artefact I/O
- Pipelines deliver repeatability, auditability, speed, and fewer human errors
- Pipelines are the automation backbone of MLOps (monitor → retrain loops plug in here)
- Mindset shift: notebook explores ideas; pipeline ships models to production

Example or contrast: You train a model in a notebook, get solid metrics, and produce nice plots. Behind the scenes, the real process looks like: Run cell 3, then cell 7 Export a file manually

Exam trap: Trap: "Manual deployment is fine for small teams." - It breaks as soon as multiple people and multiple models are involved Trap: Saving model.pkl without updating serving config - a classic cause of "wrong model in production."

MLME-W04-T03 Classic CI/CD in Software Engineering

Tier B

Definition and why it matters: Before understanding MLOps pipelines, you need a clear picture of classic CI/CD from software engineering. ML deployment pipelines extend this foundation - they do not replace it. Copy-pasting a standard software CI/CD pipeline and expecting it to cover all ML risks is a common and costly mistake

Core explanation: Continuous Integration means developers push code frequently, and automated builds and tests run on every change

Must remember:

- CI: frequent commits + automated build/test on every change; catch problems early
- CD: promote tested artefacts to staging/production in small, low-risk releases
- Classic pipeline: checkout → build → test → package → deploy

- Traditional CI/CD produces one main artefact (binary/container) from code
- System behaviour in classic software is determined by code version
- Classic CI/CD is essential but insufficient alone for ML - data and models are additional first-class artefacts
- ML MLOps builds a hybrid: classic CI/CD for software + ML pipeline for data and models

Example or contrast: Dimension: Primary artefact; Classic CI/CD: Code / container; ML Pipeline (extends CI/CD): Code + data + model + metrics

Exam trap: Trap: "CI and CD are the same thing." - CI integrates and tests; CD delivers/deploys. They are related but distinct Trap: Assuming ML teams can skip classic CI because they have training pipelines - serving code, APIs, and infra still need linting, unit tests, and container builds

MLME-W04-T04 ML-Specific Differences from Classic CI/CD

Tier A

Definition and why it matters: A traditional software system's behaviour is determined largely by which code version you deploy. A machine learning system's behaviour depends on at least three factors:

Core explanation: Code - feature logic, model architecture, serving API, preprocessing Data and labels - training dataset version, label definitions, data distribution at training time Model parameters and config - hyperparameters, thresholds, preprocessing choices If CI/CD only examines code, you miss two huge risk surfaces: data issues and model issues. ML pipelines must treat data and models as first-class citizens When you ship a new model, you implicitly ship: Concept flow: Code Version - ML Release - Data Snapshot - Model Weights - Production Behaviour A particular training dataset version A particular label definition and labelling process A snapshot of the data distribution at training time Artefact Type: Model files; Examples: pkl, .pt, .onnx, SavedModel If data changes - a feature disappears, distribution shifts (P(X) drifts), labels are redefined - model behaviour changes even when code is identical Artefact Type: Evaluation metrics; Examples: AUC, accuracy, F1, business KPIs

Must remember:

- ML behaviour depends on code, data/labels, and model parameters - not code alone
- Data must be versioned, schema-checked, and drift-monitored as part of every release
- Models are artefacts: weights, metrics, and reports must be stored and versioned
- ML gate question: "Is this model good enough?" not just "Did tests pass?"
- ML-specific checks: schema, distribution, holdout validation vs baseline
- Classic CI (lint, unit tests, container build) still required for all software components
- MLOps = classic CI/CD + ML-specific layer for data and model governance

Formula / decision rule: P(X)

Exam trap: Trap: "Green CI means safe to deploy the model." - Code tests passing does not mean the model beats baseline or passes fairness checks Trap: Versioning only code, not training data - identical code + different data = different model behaviour

MLME-W04-T05 The Hybrid Picture: Code CI/CD + ML Pipeline

Tier A

Definition and why it matters: In practice, production MLOps runs two coordinated flows:

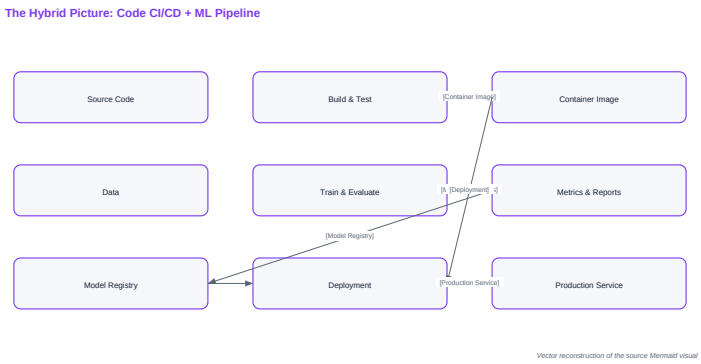
Core explanation: Code CI/CD - source code through build and test → containers and infrastructure Concept flow: Source Code - Build & Test - Container Image - Data - Train & Evaluate - Metrics & Reports - Model Registry - Deployment - Production Service ML pipeline - data through training and evaluation → candidate models and metrics → model registry At deployment time, you combine infrastructure/code from CI/CD with the chosen model from the ML pipeline MLOps orchestrates both flows so code, data, and model advance in a controlled, automated way The code CI pipeline runs: Lint, unit tests, integration tests Build container image Optional: fast smoke training on sample data to catch obvious breakages The ML pipeline runs: Concept flow: Code Change - Code CI/CD - Data Change / Retrain - ML Pipeline - Code OK? - Model OK? - BOTH - Deploy - Block Promotion Data quality checks Flow: Code CI/CD; Trigger: Git push, PR; Primary Outputs: Container, infra config; Validates: Code correctness, integration Full training and evaluation

Must remember:

- Production MLOps uses a hybrid: code CI/CD + ML pipeline, orchestrated together
- Code CI/CD: build, test, containerise software components
- ML pipeline: data quality → train → evaluate → register candidate models
- Deployment combines container/infrastructure from CI/CD with chosen model from registry
- Code changes trigger CI; data/retrain triggers ML pipeline - different triggers, coordinated gates
- Deploy only when both code and model pass their respective checks
- Traditional CI/CD remains essential; ML pipeline adds data/model versioning and quality gates

Example or contrast: A payments company maintains:

Exam trap: Trap: Running only code CI and assuming ML is covered - model quality and data drift require the ML pipeline Trap: Running only ML retraining without code CI - serving bugs and API regressions still reach production



Concept map: The Hybrid Picture: Code CI/CD + ML Pipeline

MLME-W04-T06 ML Artefacts: What Pipelines Move and Track

Tier A

Definition and why it matters: In ML pipelines, an artefact is anything a stage consumes or produces. Tracking artefacts - knowing what they are, where they live, and how they relate - is the heart of MLOps

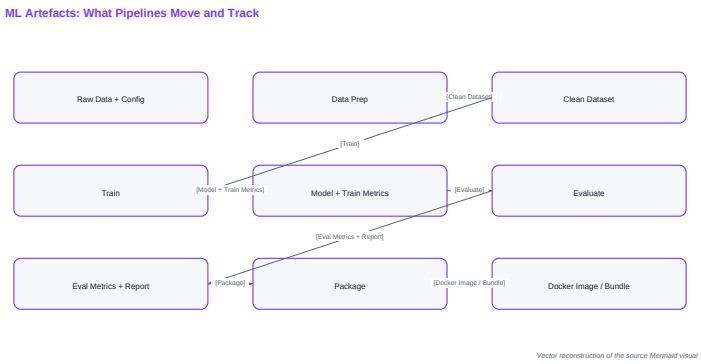
Core explanation: Without artefact discipline, you cannot debug production issues, pass audits, or reproduce experiments Item: Training scripts; Examples: train.py, feature engineering modules Item: Serving code; Examples: FastAPI handlers, batch inference jobs Item: Config files; Examples: YAML/JSON hyperparameters, data paths, model output paths Item: Raw extracts; Examples: Warehouse dumps, streaming batch exports Item: Processed datasets; Examples: Cleaned, feature-engineered data Item: Splits; Examples: Train, validation, test partitions Type: Scalar metrics; Examples: AUC, accuracy, RMSE, F1 Type: Curves; Examples: Loss curves, ROC plots Configs are critical pipeline inputs - changing learning rate or data path without versioning breaks reproducibility Type: Reports; Examples: Confusion matrices, calibration plots, fairness PDFs Concept flow: Raw Data + Config - Data Prep - Clean Dataset - Train - Model + Train Metrics - Evaluate - Eval Metrics + Report - Package - Docker Image / Bundle Stage: Data prep; Inputs: Raw data, config; Outputs: Versioned clean dataset

Must remember:

- ML artefacts include code, configs, data snapshots, models, metrics, and reports - not just code
- Each pipeline stage consumes artefacts and produces new artefacts (artefact factory pattern)
- Categories: code/config, data, models (.pkl, .pt, .onnx), metrics/reports
- Tracking artefacts = knowing what, where, and how they relate - core of MLOps
- Artefacts are versioned outside Git (registries, object storage); must link back to code commits
- Managing artefact relationships is the heart of pipeline engineering and lineage

Example or contrast: Source Code: Versioned in Git; Artefacts: Versioned in artifact stores / registries

Exam trap: Trap: Only versioning code, not model files - you cannot reproduce or roll back Trap: Hardcoding paths in scripts instead of config-driven paths - breaks portability and reproducibility



Concept map: ML Artefacts: What Pipelines Move and Track

MLME-W04-T07 Lineage and Traceability in ML Systems

Tier B

Definition and why it matters: In production ML, you constantly need answers to questions like:

Core explanation: Which data, config, and code produced this particular model? Concept flow: Code Commit abc123 - Training Run ID - Config: trainconfig.yaml - Dataset Snapshot v2024-03 - Metrics: AUC 0.91 - Model v7 - Promoted to Production Which model version is currently running in production? What changed between model v3 and v4 that caused a performance shift? Which experiments led to choosing this model? Without lineage, you are guessing. With lineage, you can inspect, compare, and roll back with confidence

Must remember:

- Lineage answers: which data/code/config produced which model, and what is in production

- Model lineage as a graph: commit + config + data + run ID → metrics → model version → stage
- Without lineage: guessing; with lineage: inspect, compare, rollback confidently
- Tools: experiment tracker (MLflow), model registry, metadata store
- Runs, parameters, metrics, and models must be recorded and linkable
- Critical for regulated industries (audit) and production debugging (root cause)
- Lineage = structured provenance relationships, not just event logs

Example or contrast: Logging: Records events as they happen; Lineage: Records relationships between artefacts

Exam trap: Trap: Logging metrics but not linking them to model files - incomplete lineage Trap: Promoting models without recording which evaluation report justified it

MLME-W04-T08 Reproducibility in Machine Learning Systems

Tier B

Definition and why it matters: Reproducibility means: if you run the same pipeline with the same code, config, and data snapshot, you get the same model - or one effectively equivalent within acceptable randomness (e.g., stochastic training with fixed seed)

Core explanation: This is fundamentally different from "it worked once in my notebook." Use Case: Debugging; Why Reproducibility Is Essential: Replay exact training when production performance drops Use Case: Compliance / audit; Why Reproducibility Is Essential: Prove how a model was created in regulated industries Use Case: Collaboration; Why Reproducibility Is Essential: New teammate reruns your pipeline and extends your work confidently Use Case: Scientific rigour; Why Reproducibility Is Essential: Results should be repeatable, not one-off accidents

Must remember:

- Reproducibility: same code + config + data snapshot → equivalent model (within tolerance)
- Critical for debugging, compliance, collaboration, and scientific validity
- Achieved via pipeline (encoded steps) + tracking (recorded metadata per run)
- Capture: commit hash, data version, config, environment, outputs
- Pipeline defines what to do; tracker records what was done - together enables replay
- Accept minor metric tolerance for GPU/stochastic non-determinism
- MLflow pattern: startrun → logparams → logmetrics → logmodel

Exam trap: Trap: "Reproducibility = same notebook cell order" - notebooks are not reproducible deployment units Trap: Using mutable paths like data/latest.csv - always version data snapshots

MLME-W04-T09 CI/CD for Machine Learning - Introduction

Tier B

Definition and why it matters: CI in ML is not full data science - it catches catastrophic breakages fast Layer: Standard software; Examples: Lint, format, unit tests, integration tests

Core explanation: Previous topics established: Deployment pipelines - automated data → train → evaluate → package → deploy Concept flow: Code / Config Change - CI Checks - Pass? - CD Promotion - Block Merge / Deploy - Staging - Production ML vs classic CI/CD - data and models as first-class artefacts; hybrid architecture Artefacts, lineage, reproducibility - what pipelines move and how to track it

Must remember:

- CI/CD for ML turns abstract pipeline concepts into concrete verification and promotion workflows
- CI: automated checks on every change (push/PR) - catch issues early
- CD: promote validated artefacts to staging/production safely
- ML CI checks: code quality, ML unit tests, data schema, smoke training
- ML CD promotes: specific model version + metrics + code/config - not just latest image
- Design question: what is verified on every change vs what is required for promotion?
- Lab repos demonstrate this with MLflow + CI workflow files

Exam trap: Trap: Equating CI with "running full training" - CI uses fast smoke runs; full training is CD/scheduled pipeline territory Trap: CD as "deploy latest Docker image" without specifying model version - must promote model + code together

MLME-W04-T10 CI for Machine Learning - What Gets Tested

Tier B

Definition and why it matters: ML projects are still software projects. These components need classic CI regardless of model complexity:

Core explanation: Feature engineering code Concept flow: Checkout Repo - Install Dependencies - Lint / Format - Unit Tests - Integration Tests - Pass / Fail Check: Linting / formatting; Purpose: Code style consistency (flake8, black, ruff) Check: Unit tests; Purpose: Core logic correctness Check: Integration tests; Purpose: Components work together (API starts, health check responds) Serving APIs and batch jobs Utility modules, config parsers Test Type: Feature functions; What It Validates: Output shape, dtype, no NaN; Example Failure Caught: Refactor breaks column count

Must remember:

- ML CI starts with standard software CI: lint, unit tests, integration tests
- ML-specific unit tests: feature shape, preprocessing, model load/predict on dummy data
- Data validation in CI: schema, null rates, bounds on small sample - catch breaking data changes

- Smoke training: tiny data, few epochs - verifies end-to-end pipeline health, not model quality
- CI fails fast to block bad merges before expensive training or production deploy
- Three layers: software CI → ML unit tests + data validation → smoke training

Example or contrast: A data scientist opens a PR changing feature engineering:

Exam trap: Trap: Running full training in CI - too slow; use smoke training instead Trap: Skipping ML unit tests because "we evaluate in training" - fast shape/load tests catch regressions earlier

MLME-W04-T11 CD for Machine Learning - What Gets Deployed

Tier A

Definition and why it matters: In machine learning, Continuous Delivery/Deployment is not simply deploying the newest container image. It is promoting a specific model version along with its full context

Core explanation: Component: Model artefact; Why It Must Travel Together: Weights / serialised model file Component: Metrics; Why It Must Travel Together: AUC, accuracy, business KPIs on validation data Component: Code version; Why It Must Travel Together: Training and serving logic that produced the model Component: Config; Why It Must Travel Together: Hyperparameters, thresholds, preprocessing settings Concept flow: Training Completes - Promotion Rules - Register in Registry - Staging - Archive / Reject - Integration + Canary Tests - Update Stage - Production Rule: Minimum metric threshold; Example: AUC = 0.85, error rate below 2% Rule: Beat current baseline; Example: New model must improve key KPIs vs production model Rule: Data / fairness checks; Example: No large performance drop on critical user segments; no bias violations Rule: Integration health; Example: Serving tests pass in staging environment CD question: Which model + code combination do we trust enough to promote to the next environment? Feature: Version storage; Description: Multiple versions per model name

Must remember:

- ML CD promotes a specific model version + metrics + code + config - not just latest image
- Promotion rules: metric thresholds, beat baseline, fairness checks, integration tests
- Model registry tracks versions, metrics, lineage, and stages (Staging → Production)
- Deployed unit = container (serving code) + model artefact + config
- CD flow: pass rules → register staging → canary/integration → promote to production
- Mindset: training success != deployment eligibility
- Canary/blue-green strategies reduce risk when swapping models in production

Formula / decision rule: >= 0.85

Example or contrast: Phase: CI; Responsibility: Verify code works; smoke training doesn't crash; data schema OK

Exam trap: Trap: Promoting every model that finishes training - promotion rules exist for a reason Trap: Deploying new model with old serving code (or vice versa) - versions must be paired

MLME-W04-T12 CI/CD for Machine Learning - Summary

Tier B

Definition and why it matters: ML CI operates on three layers:

Core explanation: Linting and formatting Concept flow: CI on Every Change - Software Tests - Data Validation - Smoke Training Element: Model version; Description: Known weights with registry ID Unit tests for core logic Element: Metrics; Description: Validation performance that justified promotion

Must remember:

- ML CI tests: code (lint/unit/integration), data schema on samples, smoke training for pipeline health
- ML CD deploys: specific model version + metrics + code/config as containerised service
- CI answers "is it broken?"; CD answers "is it good enough to ship?"
- Good ML CI/CD documents what was checked, approved, and deployed
- Lab repo: training script + MLflow + CI YAML = mini end-to-end pattern
- Production adds staging, canary, scheduled retrain, and monitoring on top of this foundation
- Hybrid: code CI builds container; ML pipeline produces model; CD pairs them for deployment

Example or contrast: Dimension: Trigger; CI: Every push / PR; CD: Model passes promotion gates Dimension: Speed; CI: Fast (minutes); CD: Slower (full eval, staging, canary)

Exam trap: Trap: Summarising ML CI/CD as "just GitHub Actions" - the workflow file is implementation; the concepts are layered verification and gated promotion Trap: CI covers everything - full model evaluation and baseline comparison belong to CD / ML pipeline, not PR-time CI

MLME-W04-T13 Repository Structure for MLOps Pipelines

Tier A

Definition and why it matters: A well-structured ML repository is not arbitrary organisation - it is a blueprint that enables experiment tracking, reproducibility, and CI/CD automation. The layout separates concerns so pipelines know exactly what to call, where artefacts live, and which configs drive each run

Core explanation: Repository = blueprint defining where everything goes Concept flow: Project Root - .github/ - CI workflows - configs/ - YAML parameters - data/ - sample datasets - models/ - local artefact output - scripts/ - entry points - src/ - core ML logic - README.md - onboarding Directory: github/; Responsibility: CI/CD workflow definitions (GitHub Actions ci.yml) Directory: configs/; Responsibility: YAML configs: hyperparameters, data paths, model output paths Directory: data/;

Responsibility: Small sample data for local dev and CI (not full production datasets) Directory: models/; Responsibility: Local artefact output (e.g., model.pkl); placeholder for registry in production Directory: scripts/; Responsibility: Top-level entry points (train.py) called by pipelines Config files = instructions for each run Directory: src/; Responsibility: Core ML logic: data loading, training, evaluation utilities

Must remember:

- MLOps repo layout: .github/, configs/, data/, models/, scripts/, src/, README.md
- configs/ drives reproducibility - hyperparameters and paths externalised from code
- scripts/ = entry points; src/ = reusable ML logic (pipeline step implementations)
- data/ holds samples locally; production data lives in warehouse/lake
- models/ is local artefact placeholder; production uses registry + object storage
- Config + train.py pairing is the standard config-driven training pattern
- Structure enables CI smoke tests and deployment pipelines to call the same entry point

Example or contrast: Location: scripts/train.py; Role: Orchestrator: load config - load data - train - log - save; Called By: Pipeline, CI smoke test, manual CLI

Exam trap: Trap: Hardcoding data/trainingdata.csv in Python instead of config - breaks reproducibility and environment portability Trap: Putting all logic in scripts/train.py with no src/ separation - untestable, unmaintainable at scale

MLME-W04-T14 MLflow Experiment Tracking - Training Script Instrumentation

Tier B

Definition and why it matters: Experiment tracking turns a one-off training script into a fully tracked experiment with complete lineage. MLflow is the tool used to make artefact and lineage concepts tangible: every run gets a unique ID with linked parameters, metrics, and model files

Core explanation: Every instrumented training job follows four steps inside an mlflow.startrun() context: Concept flow: mlflow.startrun - logparams - Train Model - logmetrics - logmodel - Run Complete - Full Lineage Step: 1. Start run; API: mlflow.startrun(); What Gets Captured: Unique run ID; groups all logged data Step: 2. Log parameters; API: mlflow.logparams(); What Gets Captured: Hyperparameters from config (learning rate, epochs, paths)

Must remember:

- MLflow pattern: startrun → logparams → train → logmetrics → logmodel
- startrun() groups all logged data under a unique run ID
- Parameters from config are searchable for experiment comparison
- logmodel creates the core lineage link: run → model artefact
- Run training via python scripts/train.py –config configs/trainconfig.yaml
- Inspect runs with mlflow ui - parameters, metrics, artifacts per run
- Local mlruns/ for dev; remote tracking URI for team/production use

Example or contrast: Mode: Local; Storage: /mlruns/ directory; Use Case: Development, labs

Exam trap: Trap: Logging metrics outside startrun() context - data not associated with any run Trap: logparams with nested dicts or lists - MLflow params must be flat strings; use logdict or flatten

MLME-W04-T15 CI Workflow for ML Repositories - GitHub Actions

Tier B

Definition and why it matters: A CI workflow defined in YAML (e.g., .github/workflows/ci.yml) acts as an automated gatekeeper for ML deployment. It ensures code is always in a good state before merging to main - running the same checks a careful reviewer would, but on every push and pull request

Core explanation: Concept flow: Push to main / PR to main - Job: build - Checkout repo - Setup Python - Install requirements.txt - Lint checks - Unit tests - pytest - Smoke training run - All pass? Event: Push to main; When It Fires: Direct commits to protected branch Event: Pull request to main; When It Fires: Every PR targeting main - primary gate for code review

Must remember:

- CI YAML in .github/workflows/ defines automated jobs on push/PR to main
- Job steps: checkout → setup Python → install deps → lint → test → smoke training
- Fresh Ubuntu runner; requirements.txt ensures environment parity with local dev
- Any failing step blocks merge - CI is the automated gatekeeper
- ML-specific addition: smoke training run catches integration errors early
- PR UI shows per-step pass/fail - core CI feedback loop
- CI (fast, sample data) != full ML pipeline (slow, full data, model promotion)

Exam trap: Trap: CI workflow only on push, not PR - PRs merge without checks Trap: No requirements.txt pin - CI passes but production fails on different package versions

MLME-W04-T16 Module 4 Summary - Deployment Pipelines and MLOps Basics

Tier A

Definition and why it matters: This module moves from one-off notebook heroics to systematic, repeatable model shipping. Every topic plugs into one core concept: the ML deployment pipeline

Core explanation: Concept flow: Topic 1: Pipeline Definition - Topic 2: ML vs Classic CI/CD - Topic 3: Artefacts & Lineage - Topic 4: CI/CD for ML - Lab: Repo + MLflow + CI Benefit: Repeatability; Impact: Same inputs - same outputs

Benefit: Auditability; Impact: Record of what ran, when, with what Notebook-only workflows are fragile: Benefit: Speed; Impact: Faster idea-to-production after initial setup Manual steps live in one person's head Benefit: Fewer errors; Impact: No forgotten config updates or manual copy-paste No clear record of which model runs in production Metrics scattered across logs and screenshots Non-reproducible models fail audit and debugging needs An ML deployment pipeline is a sequence of automated steps: Data Prep - Train - Evaluate - Package - Deploy Concept flow: Code CI/CD - Deployment - ML Pipeline - Production

Must remember:

- Notebook workflows fail at team scale - pipelines automate data → deploy with traceable artefact I/O
- MLOps pipelines extend (not replace) classic CI/CD; hybrid model coordinates code CI + ML pipeline
- ML behaviour depends on code, data, and model parameters - all are first-class release artefacts
- Lineage links commit + config + data + run → model; reproducibility requires pipeline + tracking
- ML CI: lint, tests, data schema, smoke training on every PR; ML CD: gated promotion of model + code bundle
- Lab pattern: structured repo + config-driven train.py + MLflow + GitHub Actions CI
- Core mindset: pipeline-first, not notebook-first; training success != deployment eligibility

Formula / decision rule: Data Prep → Train → Evaluate → Package → Deploy; commit + config + data snapshot + run ID → metrics → model vN

Exam trap: Trap: Treating any single topic as the whole module - everything connects through the pipeline concept Trap: "MLOps = MLflow" - MLflow is one tool; MLOps is the full practice of automated, traceable ML lifecycle

Week 4 Exam Lens

Likely questions

- Define and explain Manual Workflow Pain, Pipeline Definition, and Benefits; include the mechanism and one limitation.
- Compare Manual Workflow Pain, Pipeline Definition, and Benefits with ML-Specific Differences from Classic CI/CD and state when each is preferred.
- Apply The Hybrid Picture: Code CI/CD + ML Pipeline to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: P(X); >= 0.85; Data Prep → Train → Evaluate → Package → Deploy; commit + config + data snapshot + run ID → metrics → model vN

Mistakes to avoid: Trap: "A notebook with good metrics is production-ready." - Metrics in a notebook do not imply traceability, reproducibility, or automated deployment Trap: "Manual deployment is fine for small teams." - It breaks as soon as multiple people and multiple models are involved Trap: "CI and CD are the same thing." - CI integrates and tests; CD delivers/deploys. They are related but distinct

Week 5: Monitoring, Observability, Drift, and Alerts

Week roadmap: ML Model Monitoring: Module Introduction → Why Monitoring ML Models Is Different → What to Monitor: The Three-Layer Framework → System, Data, and Feature Metrics in Depth → Model Performance Metrics and Production Logging → Types of Drift in Production ML Systems → 8 more topics.

Coverage: 14 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

MLME-W05-T01 ML Model Monitoring: Module Introduction

Tier C

Definition and why it matters: Deploying a machine learning model is not the finish line - it is the starting point of a new operational phase. Earlier work in model engineering focused on getting models into production: exposing them as REST APIs, packaging them in containers, wiring them into CI/CD pipelines, and promoting versions safely. That work answers how to ship a model

Core explanation: Monitoring answers a different question: once a model is live, how do we know it is still doing the right thing?

Must remember:

- Deployment (APIs, containers, CI/CD) gets models into production; monitoring keeps them correct in production
- The core shift: from "is the service up?" to "are inputs sane, predictions accurate, and business KPIs healthy?"
- Silent degradation is the primary ML-specific risk - fast, error-free wrong answers
- Monitoring spans three layers: system health, data health, prediction/business health
- Real production systems need observability from day one, not after the first major incident
- This module progresses from concepts through drift/alerting to hands-on instrumentation and dashboards

Exam trap: "Deployed = done" - Deployment is the beginning of operational risk, not the end of engineering work Conflating uptime with model health - A service returning HTTP 200 does not imply correct predictions

MLME-W05-T02 Why Monitoring ML Models Is Different

Tier A

Definition and why it matters: Traditional software monitoring asks: Is the system healthy enough to serve requests? Machine learning monitoring must additionally ask: Are the predictions still correct, fair, and useful?

Core explanation: A model service can be perfectly healthy from an infrastructure perspective - HTTP 200 responses, low latency, near-zero error rates - while being actively wrong from an ML perspective because input data drifted, labels changed, or the feature-label relationship shifted Concept flow: CPU / Memory - Error Rate - Latency P95/P99 - Requests/sec - Data Quality & Drift - Prediction Metrics - Segment Fairness - Business KPIs - Incoming Request Models learn patterns from a fixed training snapshot. Production data is a live stream that changes continuously Missing values - Sudden spike in nulls breaks imputation assumptions Schema drift - Numeric fields become strings; new columns appear; expected columns vanish Task Type: Classification; Metrics to Track: Accuracy, precision, recall, F1, AUC Out-of-range values - Negative wages, transaction amounts orders of magnitude above training max

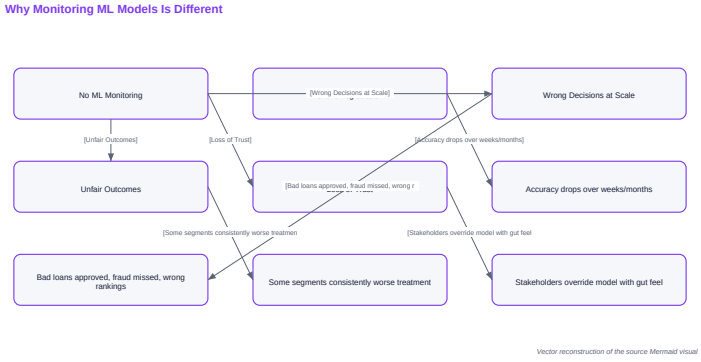
Must remember:

- ML services need traditional monitoring and data/prediction monitoring
- "Service is up" does not mean "model is good" - silent degradation is the defining ML risk
- Data layer: quality (missing, schema, range) + drift (distribution shift, new segments)
- Prediction layer: ML metrics over time, business KPIs, fairness across segments
- Unmonitored models cause silent degradation, scaled wrong decisions, unfair outcomes, and stakeholder distrust
- Both infrastructure health and ML-specific signals must be green for true production health

Formula / decision rule: R^2

Example or contrast: For any web service or API, standard observability covers: Category: Infrastructure; Typical Metrics: CPU, memory, disk, network; Alert Trigger Examples: CPU 85% for 10 min Category: Service health; Typical Metrics: HTTP 4xx/5xx rates, timeouts; Alert Trigger Examples: Error rate 1%

Exam trap: "Green dashboards = healthy model" - The most dangerous ML failure mode produces no HTTP errors Monitoring only latency and errors - Misses drift, calibration shift, and segment unfairness



Vector reconstruction of the source Mermaid visual

Concept map: Why Monitoring ML Models Is Different

MLME-W05-T03 What to Monitor: The Three-Layer Framework

Tier A

Definition and why it matters: Once we accept that ML monitoring differs from traditional app monitoring, the next question is concrete: what exactly should we measure and log?

Core explanation: The answer is organised into three layers. Each layer catches failure modes the others miss. Monitoring only one layer creates blind spots that lead to production incidents Concept flow: Layer 1: System Health: Is the service running? - Layer 2: Data Health: Are inputs sane and stable? - Layer 3: Prediction & Business Health: Are outputs useful, accurate, fair? - Silent ML Failure - Good data, bad predictions - OOM before metric update Metric: Latency (P95, P99); Why It Matters: Average hides tail latency; P99 reveals worst user experience; Example Threshold: P95 $<$ 100 ms Metric: Error rates (4xx, 5xx); Why It Matters: Client bugs vs. server failures; Example Threshold: 5xx $<$ 0.5% Metric: Connection timeouts; Why It Matters: Network or overload issues; Example Threshold: $<$ 0.1% of requests Metric: Traffic (RPS per endpoint); Why It Matters: Load spikes, upstream outages; Example Threshold: Alert on 50% drop Metric: CPU / memory usage; Why It Matters: Resource exhaustion under load; Example Threshold: CPU $<$ 80% sustained Metric: Container restart count; Why It Matters: Crash loops, OOM kills; Example Threshold: 3 restarts/hour These are the classic metrics for any web service. ML inference endpoints need them just as much as a payment API or search service

Must remember:

- ML monitoring uses three layers: system health, data health, prediction/business health
- Layer 1 (system): latency P95/P99, errors, traffic, CPU/memory - same as any API
- Layer 2 (data): schema, quality, cardinality, drift (mean/std, PSI), volume anomalies
- Layer 3 (predictions): ML metrics on delayed labels, calibration, segments, business KPIs, fairness
- Monitoring only one layer leaves critical blind spots
- Structured per-prediction logging underpins all metric computation
- Use the three-layer checklist when onboarding any new production model

Formula / decision rule: R^2

Exam trap: Single-layer monitoring - Each layer catches different failure modes; all three are required Using mean latency only - P95/P99 reveal tail experiences that averages hide

MLME-W05-T04 System, Data, and Feature Metrics in Depth

Tier B

Definition and why it matters: Every inference request passes through three observability lenses. System metrics ask whether the plumbing works. Data metrics ask whether the input makes sense. Prediction metrics ask whether the output still delivers value. This note details the concrete metrics at each layer and how they interconnect

Core explanation: Do not rely on mean latency alone. Use percentile distributions: P95 - 95% of requests complete faster than this value; represents typical worst-case experience Code Class: 4xx; Meaning: Bad client requests (malformed features, auth); Action: Fix client or API contract Code Class: 5xx; Meaning: Server failures (model crash, OOM); Action: Page on-call immediately Code Class: Timeouts; Meaning: Overload or dependency failure; Action: Scale or optimise

Must remember:

- System metrics: P95/P99 latency, 4xx/5xx errors, RPS, CPU/memory, restarts
- Data metrics: schema, missing rates, min/max, cardinality, drift (mean/std, PSI), volume
- Prediction metrics: task-appropriate ML metrics on delayed labels, calibration, thresholds
- Always compute segment-level breakdowns - global averages hide local failures
- Business KPIs and fairness gaps connect model behaviour to real impact
- Structured per-prediction logging enables all retrospective metric computation
- Use the five-row checklist when putting any new model into production

Exam trap: Mean latency as sole SLA - Always pair with P95/P99 for user-experience accuracy No training baseline stored - Drift detection requires a reference distribution from training or a stable period

MLME-W05-T05 Model Performance Metrics and Production Logging

Tier A

Definition and why it matters: Every monitoring metric discussed in the three-layer framework - system health, data drift, model accuracy, business KPIs - depends on one foundation: structured, per-prediction logging

Core explanation: Without logs that capture inputs, outputs, metadata, and (eventually) ground truth, metrics cannot be computed retrospectively by time window, segment, or model version. Logging is not an afterthought; it is the data pipeline for observability Category: Request metadata; Fields: Timestamp, model version, endpoint, request ID; Purpose: Traceability, version comparison Category: Input; Fields: Feature values (privacy-safe), segment tags (country, product); Purpose: Drift detection, segment analysis Category: Output; Fields: Raw score, final decision after thresholds/rules; Purpose: Performance and calibration tracking Category: Ground truth; Fields: Actual outcome (when available); Purpose: Delayed metric computation Category: System; Fields: Per-request latency; Purpose: Tail latency analysis Log every prediction, or a statistically representative sample (e.g., 10% stratified by segment): Concept flow: Inference Request - Structured Log Entry - Metadata: time, version, ID - Input features + segments - Score + decision - Latency - Ground Truth arrives later - Metric Aggregation Engine - System Dashboard In production, raw PII may not be loggable. Use:

Must remember:

- All monitoring metrics depend on structured per-prediction logging
- Log: metadata, inputs (privacy-safe), outputs, latency; backfill ground truth when available
- Compute metrics by time window, segment, and model version
- Task-appropriate ML metrics: AUC/F1 (classification), RMSE (regression), NDCG (ranking)
- Monitor calibration and decision thresholds, not just ranking metrics
- Segment breakdowns are mandatory - global metrics hide local failures
- Use the five-category checklist when onboarding new production models

Formula / decision rule: R^2

Example or contrast: A fraud detection service on Cloud Run logs each /score call to Cloud Logging with structured JSON:

Exam trap: Print statements instead of structured logs - Unsearchable, no severity levels, cannot be ingested by ELK/Prometheus Logging without model version - Cannot compare performance across deployments or rollbacks

MLME-W05-T06 Types of Drift in Production ML Systems

Tier A

Definition and why it matters: A model is a frozen snapshot of patterns learned at training time. Production is a live, changing environment. Drift is what happens when the world changes but the model's code and weights stay the same Concept flow: Training Snapshot - Deployed Model - Live Production Data - Drift Detected - Response - Adjust threshold - Fix data pipeline - Retrain model

Core explanation: People use "drift" loosely. In production ML, three distinct types matter - each requires different detection methods and responses Type: Covariate drift; What Changes: Input feature distributions; Detection Signal: PSI, KS test, mean/std shift; Typical Response: Investigate population or pipeline change Type: Label drift; What Changes: Target variable distribution; Detection Signal: Class ratio shift, base rate change; Typical Response: Recalibrate thresholds, update business rules Type: Concept drift; What Changes: Feature-label relationship; Detection Signal: Performance degradation despite stable inputs; Typical Response: Retrain on fresh data

Must remember:

- Drift = world changes, model does not; primary cause of silent production degradation
- Covariate drift: P(X) shifts - detect via PSI, histograms, mean/std comparison
- Label drift: P(Y) shifts - affects precision/recall at fixed thresholds
- Concept drift: P(Y|X) shifts - shows as performance drop; needs ground truth to detect
- Covariate drift is an early warning; concept drift is the most insidious
- Drift triggers investigate → decide (threshold fix, data fix, or retrain), not automatic retrain
- All three types are related but require different detection methods and responses

Formula / decision rule: P(X); P(Y)

Example or contrast: A credit model trained on Region A users (mean age 35, mean income INR 6L) is deployed to Region B (mean age 24, mean income INR 3.4L). The model code is unchanged; the API is healthy. Covariate drift on age and income signals the model operates on an unfamiliar population

Exam trap: Treating all drift as covariate drift - Label and concept drift require different responses Retraining immediately on every PSI alert - May be seasonality or a pipeline bug fixable without retraining

MLME-W05-T07 Drift Detection Methods and Alert Design

Tier A

Definition and why it matters: Drift detection does not require exotic machine learning. The most effective production systems combine simple descriptive statistics with a few well-understood statistical tests. The goal is useful, explainable signals - not perfect statistics

Core explanation: For every important feature, track over rolling time windows: Statistic: Mean; What It Catches: Central tendency shift Statistic: Standard deviation; What It Catches: Spread/volatility change Statistic: Min / max; What It Catches: Out-of-range values Statistic: Missing rate; What It Catches: Pipeline or source degradation Statistic: Histogram; What It Catches: Full distribution shape change Concept flow: Reference Distribution: Training / Stable Period - Compare - Recent Production Window: Last 7 days - Shift Threshold? - Drift Alert - Continue Monitoring Define a reference distribution (training data or a known-stable production period) PSI Value: < 0.1; Interpretation: No significant shift Compare against a recent window (last day, week, or month) PSI Value: 0.1 - 0.2; Interpretation: Minor shift - monitor closely Flag features whose distributions shifted beyond a threshold PSI Value: 0.2; Interpretation: Major shift - investigate, likely need action A single-number stability score comparing binned distributions

Must remember:

- Start drift detection with mean, std, min/max, missing rates, and histograms
- PSI: single stability score; 0.2 major shift
- KS test for continuous features; chi-square for categorical features
- Compare reference (training/stable) vs. recent production window
- Alert on sustained, significant, multi-feature drift - not every tiny bump
- Group related signals; route to correct team; link to runbooks
- Drift alerts trigger investigation, not automatic retraining

Formula / decision rule: PSI = sum (A_i - E_i) * ln(A_i / E_i); A_i

Example or contrast: Method: Mean / std comparison; Feature Type: Numeric; Output: Per-feature delta; Explainability: Very high; Compute Cost: Very low Method: Histogram overlay; Feature Type: Any; Output: Visual; Explainability: High; Compute Cost: Low

Exam trap: PSI 0.2 always means retrain - Investigate first; may be fixable upstream or expected seasonality Single-feature alerts in isolation - Correlated features drifting together signal systemic change

MLME-W05-T08 Responding to Drift: Detection, Investigation, and Action Tier A

Definition and why it matters: Detecting drift is only the first step. The operational value comes from a structured response workflow that converts signals into correct actions - whether that means adjusting a threshold, fixing a data pipeline, or retraining and redeploying

Core explanation: Concept flow: Detect Drift - Investigate Root Cause - What changed? - Adjust threshold / business rules - Fix upstream data - Retrain on fresh data - Document & continue monitoring - Deploy new version Question: Is this a real business change?; Distinguishes: Market expansion vs. bug Question: Is this a data pipeline bug?; Distinguishes: Fixable without retraining Question: Is this expected seasonality?; Distinguishes: Holiday spike vs. true drift Drift can appear in: Question: Is this isolated to one segment?; Distinguishes: Localised issue vs. global shift

Must remember:

- Method: Basic stats (mean, std, min, max); Best For: Quick per-feature screening
- Method: Histogram comparison; Best For: Visual distribution shape
- Method: PSI; Best For: Single-score stability for critical features
- Method: KS test; Best For: Continuous distribution comparison
- Method: Chi-square; Best For: Categorical frequency comparison
- Always compare training data (or stable reference period) against a recent production window (last week/month)
- Drift response: detect → investigate → decide (threshold, data fix, or retrain)
- Drift is a signal for investigation, not an automatic retrain command
- Three drift types: covariate (P(X)), label (P(Y)), concept (P(Y|X))
- Detection: basic stats, histograms, PSI, KS, chi-square vs. training reference
- Alert on sustained, significant, grouped signals with clear ownership and runbooks
- Confirmed concept/covariate shifts feed into automated retraining pipelines
- Avoid alert fatigue: severity tiers, sustained breaches, grouped incidents

Formula / decision rule: P(X); P(Y)

Example or contrast: Situation: Fraud rate shifts from 1% (training) to 5% (production) after a coordinated attack

Exam trap: "Drift detected = retrain now" - Always investigate root cause first Ignoring seasonality - Holiday traffic patterns mimic drift; document expected cycles

MLME-W05-T09 From Metrics to Action: Designing an ML Monitoring Workflow Tier A

Definition and why it matters: Collecting metrics is necessary but not sufficient. A monitoring setup must organise signals so humans can see issues, triage them, and fix them. This note covers objectives, the observability pillars (logs, metrics, traces), SLOs, and the workflow that connects signals to action

Core explanation: Concept flow: Monitoring Objectives - Service Level Objectives - Dashboards - Pull - Alerts - Push - Runbooks - Action: Fix / Retrain / Rollback Production ML systems rely on the same observability foundation as any distributed service, extended with ML-specific signals What: Discrete event records - one per inference request Properties: High cardinality, rich context, immutable, searchable ML use: Per-prediction feature values, scores, decisions, model version, segment tags. Enables retrospective drift and performance analysis Stack examples: ELK (Elasticsearch, Logstash, Kibana), AWS Cloud Watch Logs, GCP Cloud Logging, Fluentd → object storage Pillar: Logs; Best For: Forensics, drift analysis, audit; ML-Specific Signal: Feature values, ground truth backfill What: Numeric time-series aggregations - counters, gauges, histograms Pillar: Metrics; Best For: Dashboards, alerting, trends; ML-Specific Signal: PSI, AUC, error rate, latency percentiles Properties: Low cardinality, efficient storage, ideal for alerting Pillar: Traces; Best For: Latency debugging across services; ML-Specific Signal: Feature store vs. inference bottleneck ML use: P99 latency, error rate, PSI per feature, rolling AUC, prediction volume, business KPIs Stack examples: Prometheus + Grafana, Datadog, CloudWatch Metrics, Azure Monitor

Must remember:

- Observability pillars: logs (events), metrics (time-series), traces (request paths) - all needed for ML
- Start with monitoring objectives → translate to 3-5 SLOs per model
- One dashboard per model: system + data + prediction health; scannable in 30 seconds
- Alerts are push monitoring tied to SLOs; dashboards are pull monitoring
- Severity tiers: info (log), warning (Slack), critical (pager)
- Assign ownership: data eng (pipelines), ML eng (drift/performance), platform (infra)
- Runbooks accelerate triage; monitoring signals feed retraining pipelines

Exam trap: Metrics without objectives - Collecting everything but alerting on nothing meaningful Dashboards without exception highlighting - Operators cannot triage in 30 seconds

MLME-W05-T10 Dashboards, Alerts, and the Typical ML Monitoring Stack

Tier A

Definition and why it matters: Knowing what to monitor is step one. Step two is building the infrastructure - dashboards people actually use, alerts that fire at the right time, and a technology stack that scales from a single model service to an enterprise ML platform

Core explanation: A well-designed model dashboard has three zones: Concept flow: Zone 1: System: Traffic - Zone 2: Data: PSI Top Features - Zone 3: Predictions: AUC vs Baseline - 30-Second Scan - Fine - Deeper Investigation Rule: Highlight exceptions (red/amber); Rationale: Operators spot problems without reading every number Rule: Compare to baseline; Rationale: Absolute values lack context; deltas reveal drift Rule: Show segment breakdowns; Rationale: Global metrics hide local failures Rule: 30-second scan test; Rationale: If triage takes longer, simplify the dashboard Metric: P95 latency; Condition: 200 ms; Duration: 15 min sustained; Severity: Warning Metric: Error rate; Condition: 1%; Duration: 5 min sustained; Severity: Critical Metric: PSI (critical feature); Condition: 0.2; Duration: 4 hours sustained; Severity: Critical Metric: AUC; Condition: 10% below baseline; Duration: On 7-day labelled window; Severity: Warning Dashboards = pull monitoring (humans look when needed) Alerts = push monitoring (system notifies proactively) Sustained breach - Not a single spike Hysteresis - Metric must return inside band before alert resolves

Must remember:

- One primary dashboard per model: system + data + prediction zones; 30-second scan
- Alerts tied to SLOs: sustained breach, severity tiers, grouped incidents
- Ownership: data eng (pipelines), ML eng (drift/performance), platform (infra)
- Runbooks: business meaning, causes, checks, next actions
- Typical stack: Prometheus + Grafana + ELK + Alertmanager (+ OpenTelemetry)
- Separate alert config from code (YAML); GitOps for threshold management
- Dashboards = pull; alerts = push - both required

Example or contrast: A company runs 8 ML models on Kubernetes:

Exam trap: Wall-of-numbers dashboards - No exception highlighting; fails the 30-second scan test Pager for every warning - Reserve pagers for critical; use Slack for warnings

MLME-W05-T11 Closing the Loop: From Monitoring to Model Updates

Tier A

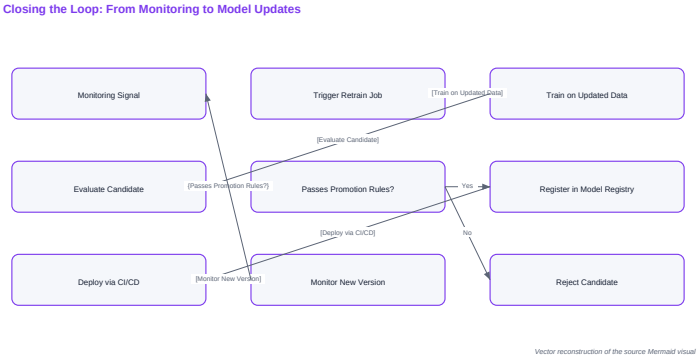
Definition and why it matters: A monitoring system that only produces dashboards is incomplete. The ultimate purpose is to close the loop - converting detected signals into model updates, threshold adjustments, or data fixes that restore production quality

Core explanation: Concept flow: Continuous Monitoring - Signal? - Investigation Ticket - Root Cause - Fix Upstream Data - Adjust Threshold / Rules - Retrain Pipeline - Evaluate vs Current Model - Deploy New Version Signal: Repeated drift on top features (PSI 0.2 sustained); Likely Cause: Population shift or pipeline change; Urgency: High Signal: Persistent AUC/recall drop on fresh labelled data; Likely Cause: Concept drift; Urgency: High Signal: Segment performance gap widening; Likely Cause: Fairness or localised drift; Urgency: Medium-High Signal: Sudden missing value spike; Likely Cause: Pipeline failure; Urgency: Critical (immediate) Signal: Business KPI divergence; Likely Cause: Model no longer aligned with goals; Urgency: Medium Concept flow: Monitoring Signal - Trigger Retrain Job - Train on Updated Data - Evaluate Candidate - Passes Promotion Rules? - Register in Model Registry - Deploy via CI/CD - Monitor New Version - Reject Candidate After triage, one of three paths typically emerges: Data issue - Fix upstream ETL, schema, or feature engineering. No model change needed Threshold / business logic adjustment - Label drift changed the optimal operating point. Recalibrate without retraining

Must remember:

- Monitoring must close the loop: detect → investigate → fix, adjust, or retrain
- Repeated drift or persistent performance drops should create investigation tickets
- Three outcomes: data fix, threshold adjustment, or retrain-and-deploy
- Retraining pipeline: updated data → evaluate → register → deploy → monitor again
- Promotion rules prevent worse candidates from reaching production
- Monitoring signals should automatically feed retraining triggers
- Model registry and CI/CD are prerequisites for a closed MLOps loop

Exam trap: Monitoring without a retraining path - Detection without remediation accumulates technical debt Retraining without evaluation gates - New model may be worse; always compare to current



Concept map: Closing the Loop: From Monitoring to Model Updates

MLME-W05-T12 ML Monitoring Lab: Four Production Failure Scenarios

Tier A

Definition and why it matters: Concept flow: Part 0: Four Failure Scenarios: (Conceptual motivation - Part 1: System + Data Metrics: (From production logs - Part 2: PSI + Alerts: (Automated monitoring - Part 3: Full Observability Stack: (Instrumentation Part: Part 0; Focus: Why traditional monitoring fails; Data Source: Synthetic scenario tables

Core explanation: Parts 1-3 build the monitoring stack that catches these scenarios: Part 1: Structured logging, system metrics (P95/P99 latency), data metrics (mean/std comparison, missing value checks) Part 2: PSI calculation, model performance on ground truth, automated alerting via config-driven thresholds Part 3: Instrument model service with Prometheus-compatible metrics endpoints and dashboard integration In production, connect the same patterns to real log aggregation (ELK), metric collection (Prometheus), and tracing (OpenTelemetry/Datadog)

Must remember:

- Lab uses simulated scenarios for reproducibility; patterns scale to production
- Scenario 1: silent degradation - green infra, collapsing accuracy
- Scenario 2: covariate drift - population shift, feature means change dramatically
- Scenario 3: feedback loop - model output biases future training data (catalogue coverage drops)
- Scenario 4: label drift - fraud rate 1% → 5%, precision tanks at fixed threshold
- Traditional monitoring catches infra issues only; misses all four ML failure modes
- Parts 1-3 build logging, PSI, alerts, and instrumentation to catch these scenarios

Example or contrast: Hands-on monitoring labs use simulated data and scenarios for two reasons: portability (every student gets identical, reproducible conditions) and controlled demonstration (specific failure modes can be injected without waiting for real production drift) Week: Week 1; System Metrics: Latency stable, error rate ~0%, pipeline stable; ML Metrics: Accuracy 92% The patterns and code built in the lab scale directly to production by connecting the same techniques to real ML service logs, metric systems, and distributed tracing infrastructure (Prometheus, Datadog, ELK, OpenTelemetry)

Exam trap: "Lab uses synthetic data so patterns don't apply" - Patterns scale directly; only the data source changes Monitoring only Scenario 1 (accuracy drop) - Covariate drift, feedback loops, and label shift require different signals

MLME-W05-T13 Instrumenting Model Services: Metrics, PSI, and Automated Alerting

Tier A

Definition and why it matters: This note covers the hands-on implementation of ML monitoring: structured logging, system and data metric computation from production logs, PSI-based drift scoring, model performance evaluation, and config-driven automated alerting

Core explanation: Each row represents one inference request: Column: timestamp; Purpose: Time-window filtering Column: latencyms; Purpose: System metric computation Column: statuscode; Purpose: Error rate calculation Column: Input features; Purpose: Drift detection Column: prediction; Purpose: Model output tracking Column: groundtruth; Purpose: Delayed performance metrics Property: Timestamps; print(): Manual; Structured logging: Automatic Property: Severity levels; print(): None; Structured logging: INFO, WARNING, ERROR In production: thousands to millions of rows per day streaming into log aggregation (ELK, CloudWatch, Kinesis) Property: Searchable; print(): No; Structured logging: Yes (ELK, Cloud Watch) Stores reference statistics from training data: Property: Machine-readable; print(): No; Structured logging: Yes (JSON formatting) Numeric features: mean, standard deviation Property: File + console output; print(): Manual; Structured logging: Built-in handlers Categorical features: frequency distributions This is the reference distribution for all drift comparisons. In production, save actual training distributions at model registration time Replace print() with Python's logging module: Concept flow: Production Logs - Latency Stats - Mean - P95 - P99 - Error Rate - Threshold? Check: Central tendency; Numeric Feature: Mean vs. training mean; Categorical Feature: Top category frequencies

Must remember:

- Production monitoring starts with structured logs + stored training baselines
- System metrics: P95/P99 latency, error rate - log as machine-readable entries
- Data metrics: compare production mean/std and category frequencies to training stats
- PSI: 0.2 major; lab example PSI 3.19 = catastrophic
- AlertManager: config-driven thresholds (YAML), evaluate → alert → export → route
- Perfect AUC with catastrophic drift and errors = evaluation integrity problem, not success
- Check: data leakage, selection bias, sample size, label correctness
- Prometheus /metrics endpoint + Grafana = production dashboard pattern

Formula / decision rule: PSI = sum (A_i - E_i) * ln(A_i / E_i)

Exam trap: Perfect AUC with high PSI and error rate - Classic exam trap; check evaluation integrity and sample size PSI 0.2 = always retrain - Investigate first; may be pipeline bug or seasonality

MLME-W05-T14 Module Summary: ML Model Monitoring and Observability

Tier C

Definition and why it matters: Monitoring is not optional for production machine learning. A mature ML system requires observability across three layers - and only when all three are healthy can we confidently say the service is up, inputs are sane, and predictions are still good and fair

Core explanation: Concept flow: Layer 1: System Health: CPU, memory, errors, latency - Layer 2: Data Health: Schema, distributions, drift, quality - Layer 3: Prediction Health: ML metrics, fairness, business KPIs - All Green? - Reliable Long-Running System - Silent or Loud Failure

Must remember:

- Production ML requires three monitoring layers: system, data, prediction/business
- ML monitoring differs because failures can be silent - no HTTP errors, wrong predictions
- Three drift types: covariate (P(X)), label (P(Y)), concept (P(Y|X))
- Observability pillars: logs, metrics, traces - extended with ML-specific signals
- Workflow: objectives → SLOs → dashboards + alerts → runbooks → ownership → closed loop
- PSI thresholds: 0.2 major
- Perfect metrics on drifted data with errors = evaluation integrity failure
- Monitoring transforms one-off deployment into a reliable, self-correcting ML system

Formula / decision rule: P(X); P(Y)

Exam trap: "Service is up = model is good" - The most common and dangerous misconception Single-layer monitoring - Each layer catches different failures; all three required

Week 5 Exam Lens

Likely questions

- Define and explain Why Monitoring ML Models Is Different; include the mechanism and one limitation.
- Compare Why Monitoring ML Models Is Different with What to Monitor: The Three-Layer Framework and state when each is preferred.
- Apply Model Performance Metrics and Production Logging to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: R^2; P(X); P(Y); PSI = sum (A_i - E_i) * ln(A_i / E_i); A_i

Mistakes to avoid: "Deployed = done" - Deployment is the beginning of operational risk, not the end of engineering work "Green dashboards = healthy model" - The most dangerous ML failure mode produces no HTTP errors Single-layer monitoring - Each layer catches different failure modes; all three are required

Week 6: Continuous Training and Safe Promotion

Week roadmap: When and Why to Retrain ML Models → Static Models and the Retraining Loop → Retraining Triggers: Drift, Performance, and Policy → Continuous Training: Scheduled vs Event-Driven Retraining → Designing a Retraining and Promotion Pipeline → Evaluating and Selecting the Right Model: Champion vs Challenger → 11 more topics.

Coverage: 17 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

MLME-W06-T01 When and Why to Retrain ML Models

Tier B

Definition and why it matters: Model deployment is not the end of engineering work - it is the start of operational life. Monitoring (drift detection, performance metrics, business KPIs, alerts) produces signals. Retraining is the response loop that turns those signals into an updated model when the current one is no longer fit for purpose

Core explanation: The central question this module addresses:

Must remember:

- Monitoring produces signals; retraining is the controlled response when the current model is misaligned with reality
- Always ask: drift alert → new model, or threshold fix / data bug / business rule change?
- Four-gate checklist: persistent change, alternatives ruled out, sufficient fresh data, pipeline ready
- Retraining is a continuous loop (deploy → monitor → detect → retrain → deploy), not a one-time upgrade
- This module connects monitoring (Week 5) to pipeline design, evaluation methods, governance, and hands-on MLflow workflows
- High-impact domains (credit, fraud, recommendations) require systematic evaluation before promotion
- Human judgement remains essential for risk, fairness, and policy-driven retrains

Example or contrast: A fintech credit-scoring model processes thousands of loan applications per hour. Each prediction has direct financial impact:

Exam trap: Treating every drift alert as a retrain trigger - drift is a warning sign, not an automatic button; investigation comes first Retraining without fresh labels - retraining on unlabelled or stale data produces no meaningful improvement

MLME-W06-T02 Static Models and the Retraining Loop

Tier A

Definition and why it matters: Not every fluctuation warrants retraining. Meaningful change means a sustained shift that investigation confirms is real - not noise, not a temporary spike, not a logging bug Examples of meaningful change:

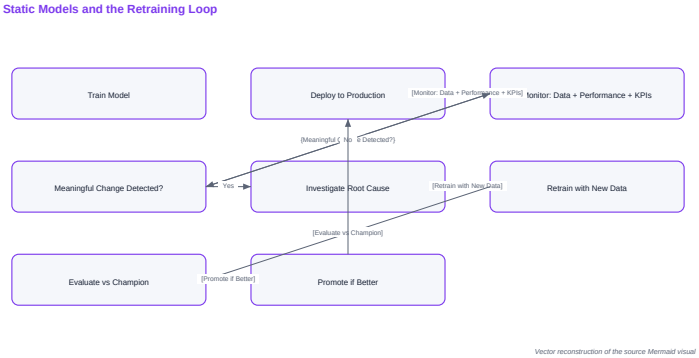
Core explanation: A static model is trained on a fixed dataset, deployed to production, and left unchanged. This approach works in stable, low-stakes environments - but fails in most real-world ML systems where data, users, and business context evolve continuously Intuition: A static model is a photograph of the world at training time. Production is a live video. Over time, the photograph becomes increasingly inaccurate Concept flow: Train Model - Deploy to Production - Monitor: Data + Performance + KPIs - Meaningful Change Detected? - Investigate Root Cause - Retrain with New Data - Evaluate vs Champion - Promote if Better Phase: Deploy; What Happens: Model serves predictions; Failure Mode if Skipped: N/A - starting point Phase: Monitor; What Happens: Track drift, metrics, KPIs; Failure Mode if Skipped: Silent degradation Phase: Detect; What Happens: Alerts on meaningful change; Failure Mode if Skipped: Problems discovered by users

Must remember:

- Static models degrade silently as the world changes; monitoring exposes this, retraining addresses it
- The production lifecycle is a loop: deploy → monitor → detect → investigate → retrain → evaluate → redeploy
- "Meaningful change" requires persistence and investigation - not every alert triggers retraining
- Cheaper fixes (thresholds, rules, pipeline repairs) should be ruled out first
- Real failures include data drift, adversarial adaptation, and structural product misalignment
- Retraining aligns the model with the current data regime and business intent

Example or contrast: Before entering the retraining loop, consider cheaper alternatives:

Exam trap: "The model still runs, so it's fine" - uptime does not equal prediction quality Treating retraining as a one-time Version 2.0 project - it is an ongoing operational loop



Concept map: Static Models and the Retraining Loop

MLME-W06-T03 Retraining Triggers: Drift, Performance, and Policy

Tier A

Definition and why it matters: Retraining is expensive and risky. A rushed retrain can produce a worse model than the current champion. Triggers define when to enter the retraining loop - they are hypotheses that must be investigated, not automatic commands

Core explanation: Concept flow: Data Drift & Quality - Performance Degradation - Policy & Product Changes - Investigate - Confirmed Real Change? - Monitor / Fix Pipeline / Tune Threshold - Consider Retraining Signal: High drift scores on key features; Example: Income distribution shifted after economic event; First Response: Investigate: business change or pipeline bug? Signal: New segments/categories never seen in training; Example: New product tier, new country code; First Response: Check if model handles unknowns gracefully Signal: Persistent data quality issues; Example: Missing-value rate jumped from 2% to 18%; First Response: Fix pipeline before retraining

Must remember:

- Three major retraining triggers: data drift/quality, performance degradation, policy/product changes
- All triggers require investigation before action - drift alerts are not automatic retrain commands
- Performance degradation on fresh labels is the strongest signal, after ruling out evaluation and business changes
- Policy retrains align with intent and compliance, not just metric optimisation
- Cheaper alternatives (threshold tuning, pipeline fixes) should be considered first
- Compare like-with-like when evaluating metric changes over time

Formula / decision rule: P(X); P(Y|X)

Example or contrast: A recommendation model shows stable offline AUC but rising bounce rates. Investigation reveals a new mobile UI changed how users interact with recommendations - offline metrics miss this behavioural shift. Policy trigger: product team launches a new category. Retraining incorporates new item embeddings and interaction patterns, aligning the model with current product structure rather than chasing a metric that no longer reflects user experience

Exam trap: "High PSI → retrain immediately" - drift requires investigation; pipeline bugs mimic drift Retraining when threshold tuning suffices - a 2% AUC drop may be fixed by moving a decision threshold

MLME-W06-T04 Continuous Training: Scheduled vs Event-Driven Retraining

Tier A

Definition and why it matters: Continuous training (also called continuous learning or automated retraining) is the practice of periodically or reactively updating a production model with newer data - rather than treating training as a one-off research activity Intuition: Just as CI/CD continuously integrates and deploys code, continuous training continuously refreshes the model artefact that code serves

Core explanation: Concept flow: Weekly / Monthly / Quarterly - Fixed Calendar Trigger - Drift Threshold Crossed - Retrain Pipeline - SLO Breach / Policy Change - Baseline Schedule - Prevent Staleness - Event Triggers Pattern: Scheduled; Trigger: Calendar (weekly, monthly, quarter); Best For: Steady data flow, reliable labels, low volatility; Risk: Model may be stale between cycles Pattern: Event-driven; Trigger: Drift alert, SLO breach, major product change; Best For: Dynamic environments, high-stakes models; Risk: Alert fatigue; false triggers Pattern: Hybrid; Trigger: Schedule + events; Best For: Most production systems; Risk: Requires tuning both layers

Must remember:

- Continuous training keeps models fresh via scheduled, event-driven, or hybrid retraining patterns
- Scheduled retraining is simple and prevents staleness; event-driven responds to acute signals
- Hybrid (baseline schedule + event triggers) is the production best practice
- Retraining has real costs: compute, engineering time, and risk of deploying a worse model
- Event triggers should often include human approval for high-impact models
- Match retraining frequency to label availability and business volatility

Example or contrast: A fraud team runs weekly scheduled retrains on the last 4 weeks of labelled transactions (baseline freshness). Additionally, an event trigger fires when precision drops below 85% for 3 consecutive days on a rolling evaluation set - initiating an expedited retrain with human approval. During a known product launch (new payment method), a manual policy trigger overrides the schedule to retrain immediately with feature engineering updates

Exam trap: Retraining is not free: Cost Type: Compute; Description: GPU/CPU hours for training multiple candidates

MLME-W06-T05 Designing a Retraining and Promotion Pipeline

Tier A

Definition and why it matters: Knowing that retraining is warranted is only half the problem. Production MLOps requires a structured, repeatable pipeline that produces candidate models, compares them to the current champion, and promotes winners through controlled stages - with full auditability at every step

Core explanation: Intuition: Ad-hoc retraining (a data scientist runs a notebook and emails a pickle file) does not scale. A pipeline turns retraining into an engineering process with predictable inputs, outputs, and gates Concept flow: Data Snapshot - Feature Engineering - Train Candidates - Evaluate vs Champion - Passes Rules? - Register in Model Registry - Archive & Log - Staging / Canary - Production Promotion Stage: 1. Data & Features; Purpose: Reproducible training dataset; Key Output: Snapshot + metadata (time window, sources, hashes) Stage: 2. Train Candidates; Purpose: Explore hyperparameters / architectures; Key Output: Logged runs with code version, configs, metrics Stage: 3. Evaluate & Select; Purpose: Governance gate vs champion; Key Output: Promotion decision with multi-metric evidence Stage: 4. Register; Purpose: First-class artefact with lineage; Key Output: Versioned model entry in registry Stage: 5. Promote; Purpose: Safe rollout to production;

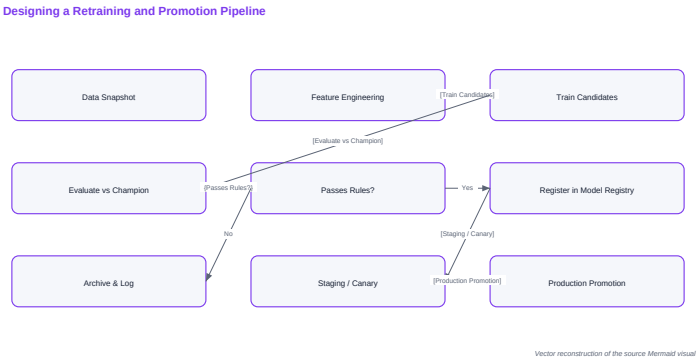
Key Output: Staging - canary/shadow - full production Everything is logged so any model can be audited: exactly which data, code, and config produced this model

Must remember:

- Retraining pipeline: snapshot data → build features → train candidates → evaluate → register → promote
- Stage 1 treats training data as a first-class artefact with time window, sources, and hashes
- Stage 2 trains multiple candidates with full logging of code, config, data, and metrics
- Config-driven pipelines make retraining repeatable - change config, not code
- MLflow (or similar) organises runs and links candidates to the same data snapshot
- Full logging enables audit, reproducibility, and rollback

Example or contrast: A fintech team refreshes their credit model quarterly (scheduled) and on drift alerts (event-driven):

Exam trap: Hardcoding data paths in training scripts - breaks reproducibility and auditability Training without production feature logic - train-serve skew invalidates offline evaluation



Concept map: Designing a Retraining and Promotion Pipeline

MLME-W06-T06 Evaluating and Selecting the Right Model: Champion vs Challenger Tier A

Definition and why it matters: Training candidate models is straightforward; choosing which one replaces production is where governance, risk management, and business alignment converge. Stage 3 of the retraining pipeline compares every challenger against the current champion using a rigorous, predefined evaluation protocol

Core explanation: Intuition: Without explicit promotion rules, model selection becomes subjective - driven by whoever ran the notebook last, not by evidence Concept flow: Load Champion - Same Evaluation Protocol - Load Challenger(s - Held-Out Validation Set - Multiple Time Slices - Compute Metrics - Promotion Rules Pass? - Register Winner - Archive & Document Metric Category: Statistical ML metrics; Examples: Accuracy, AUC, RMSE, F1; Why It Matters: Core predictive quality Metric Category: Business KPIs; Examples: Expected profit, fraud loss, conversion rate; Why It Matters: Ties model to dollars-and-cents impact Metric Category: Segment-level metrics; Examples: Performance by demographic, region, product; Why It Matters: Fairness and equity checks Metric Category: Multi-time-slice; Examples: Metrics across rolling weekly windows; Why It Matters: Detects instability over time Concept flow: Challenger AUC: 0.91 - Rules - Champion AUC: 0.89 - Promote - Reject Every candidate must be evaluated with the same protocol as the champion - same holdout set, same time slices, same metric definitions. Changing the evaluation between models invalidates comparison

Must remember:

- Stage 3 compares challengers to champion on held-out data, multiple time slices, ML metrics, business KPIs, and segment fairness
- Promotion rules are predefined: minimum metric delta, no KPI/fairness degradation
- Failed candidates are logged and archived; only rule-passing models proceed
- Model registry stores version, lineage (data, code, config), metrics, and environment tags
- Champion/challenger pattern: production stage = champion; explicit version = challenger
- Business metrics (expected profit) often matter more than pure statistical accuracy

Formula / decision rule: delta; Expected Profit = sum_i (payoff(hat(y)_i, y_i))

Example or contrast: Role: Champion; Definition: Current production model serving live traffic; Registry Stage: Production stage

Exam trap: Promoting on a single metric - AUC improvement with profit degradation is a net loss Different evaluation sets for champion vs challenger - invalidates head-to-head comparison

MLME-W06-T07 Promotion, Deployment, and the Continuous Retraining Loop Tier A

Definition and why it matters: Selecting a winning candidate is not the finish line. Promotion is the controlled process of moving a validated model through staging, canary/shadow testing, and full production - while keeping the previous champion available for immediate rollback

Core explanation: Intuition: Deploying directly from offline evaluation to 100% traffic is like merging code to main without CI - it works until it doesn't, and then recovery is painful Concept flow: Offline Evaluation - Staging Environment - Smoke

Tests / Traffic Replay - Canary or Shadow in Production - Metrics OK? - Full Production Promotion - Rollback to Champion - Keep Old Model Available Stage: Staging; Purpose: Run smoke tests, replay recorded traffic; User Impact: None - isolated environment Stage: Canary; Purpose: Route small % of live traffic to new model; User Impact: Partial - subset of users see new model Stage: Shadow (dark launch); Purpose: New model runs in parallel; only champion output served; User Impact: None - predictions logged but not used Stage: Full production; Purpose: New model serves all traffic; User Impact: All users Concept flow: Monitor Production Model - Signals? - Trigger Retraining - Retraining Pipeline - Snapshot - Train - Evaluate - Register - Promote: Staging - Canary - Production

Must remember:

- Stage 5 promotes models through staging → canary/shadow → full production, keeping old champion for rollback
- Shadow testing uses real inputs with zero user impact; canary exposes a subset of users
- Continuous retraining is a closed loop: monitor → signal → pipeline → promote → monitor
- Triggers may be automatic or human-approved; judgement matters for risk and fairness
- Promotion-phase monitoring validates behaviour under real traffic, not just service health
- Tested rollback turns incidents from crises into manageable events

Example or contrast: Decision: Data snapshot creation; Typical Automation Level: Fully automated

Exam trap: Promoting directly from offline eval to 100% traffic - skips real-world validation under live distributions Deleting the old champion after promotion - eliminates rollback capability

MLME-W06-T08 Offline Evaluation and Backtesting

Tier A

Definition and why it matters: Retraining produces candidate models - but how do we know a challenger is truly better before exposing users to it? Offline evaluation is the first layer of evidence: fast, cheap, and zero user risk - but with important limitations

Core explanation: Offline evaluation encompasses everything done without touching live traffic: Concept flow: Historical Data - Train / Val / Test Splits - Compute ML Metrics - Backtest: Replay Decisions - Simulated Business Outcomes - Candidate Ranking Advantage: Fast and cheap; Limitation: Assumes past resembles future Advantage: Try many candidates in parallel; Limitation: Cannot capture user behaviour changes from new model Advantage: Zero risk to real users; Limitation: Misses UX interactions and downstream side effects Advantage: Reproducible and auditable; Limitation: Depends on quality of logged features and labels Train/validation/test splits Time-based splits (train on past, test on future) Split Type: Random; When to Use: IID data, no temporal structure; Trap: Leaks future information into training Backtesting on historical data Split Type: Time-based; When to Use: Sequential data (logs, transactions); Trap: Test set must be strictly after train set

Must remember:

- Offline evaluation: train/val/test splits, time-based splits, cross-validation - fast, cheap, zero user risk
- Offline eval is necessary but not sufficient; it assumes the past looks like the future
- Backtesting replays historical data through candidate models to simulate counterfactual business outcomes
- Backtest quality depends on logged features, labels, and awareness of behavioural feedback limitations
- Use time-based splits for sequential production data; random splits leak future information
- Combine statistical metrics (AUC, RMSE) with business metrics (profit, fraud savings) for ranking

Formula / decision rule:) | False Declines (; 2.1M |

Example or contrast: Offline evaluation should compute both:

Exam trap: "Best offline AUC → deploy" - offline metrics miss user behaviour feedback and business impact Random splits on temporal data - inflates offline metrics via data leakage

MLME-W06-T09 Live Environment Validation: Shadow Testing and A/B Testing

Tier A

Definition and why it matters: Offline evaluation and backtesting filter candidates efficiently - but they cannot observe how users react to new model decisions. Live environment validation closes this gap using real production inputs (shadow testing) and real user outcomes (A/B testing)

Core explanation: Concept flow: Offline Eval - Backtest - Shadow Testing - A/B Test - Full Rollout Concept flow: Production Request - Champion Model - Challenger Model - User Response - Log Predictions Only - Offline Analysis Benefit: Real production input distributions; Tradeoff: Extra compute cost (2x inference) Benefit: Zero user risk; Tradeoff: Increased log volume and storage Each layer adds fidelity and cost. Not every model needs all four - match the safety gear to the change's impact Benefit: Detects train-serve skew; Tradeoff: Privacy and data retention considerations Mechanism: Live traffic arrives as usual. Both the champion and challenger models receive the same input, but only the champion's output is served to the user. The challenger's predictions are logged for offline analysis

Must remember:

- Shadow testing: both models see same input; only champion output served; challenger logged for analysis
- Shadow uses real production inputs with zero user risk; costs extra compute and logging
- A/B testing splits traffic; measures real business outcomes (revenue, conversion, fraud, churn)
- A/B requires sufficient traffic, duration, a primary metric, guardrails, and no overlapping experiments
- Typical progression: offline → backtest → shadow → A/B → full rollout
- Match evaluation depth to model impact - not every change needs all four stages

Example or contrast: Method: Offline eval; User Risk: None; Cost: Low; Captures User Behaviour: No; Best For: Early candidate filtering

Exam trap: Skipping shadow and going straight to A/B - wastes traffic on models with obvious offline/shadow failures Underpowered A/B tests - declaring winner on 2 days of data with 100 users is not statistically valid

MLME-W06-T10 Choosing Evaluation Methods and Connecting Them to Promotion Tier A

Definition and why it matters: Not every model change warrants the full evaluation stack. The art of MLOps is selecting the right depth of validation for each change's risk profile - then connecting that evidence to a governed promotion decision

Core explanation: Concept flow: Model Impact Level? - Offline Eval Only - Offline + Backtest + Shadow - Offline + Backtest + Shadow + A/B - Promote with Simple Rules Technique: Backtest; Cost: Cheap, fast; Risk to Users: None; When to Use: Early exploration; narrowing many candidates; low-risk internal models Technique: Shadow testing; Cost: Medium (2x compute); Risk to Users: None; When to Use: Medium-to-high impact; validate on live inputs before user exposure Technique: A/B test; Cost: High (traffic, time); Risk to Users: Partial (test group); When to Use: High-impact decisions; need proof of business metric improvement Technique: Full rollout; Cost: Deployment cost; Risk to Users: All users; When to Use: After combined evidence justifies promotion Stage Transition: Offline - Backtest; Gate Question: Does candidate beat champion on historical replay? Stage Transition: Backtest - Shadow; Gate Question: Are there fewer than 3 promising candidates remaining?

Must remember:

- Match evaluation depth to model impact: backtest (cheap) → shadow (live inputs) → A/B (real users)
- Common pattern: backtest → shadow → A/B → full rollout; not all stages always required
- Promotion decisions synthesise offline metrics, backtest, shadow, A/B, and risk/fairness evidence
- High-impact models require combined evidence; low-impact models may need offline comparison only
- Evaluation is layered - each stage filters candidates before the next, more expensive stage
- Predefined promotion rules per model tier remove ad hoc, emotion-driven decisions

Formula / decision rule: delta

Example or contrast: An e-commerce dynamic pricing model affects revenue directly:

Exam trap: Using A/B tests for every tiny model tweak - wasteful; reserve for high-impact changes Promoting on offline metrics alone for high-impact models - misses user behaviour and business proof

MLME-W06-T11 Governance and Safety: Why Approvals Matter Tier B

Definition and why it matters: As models and teams scale, ad-hoc deployment practices create systemic risk:

Core explanation: Silent model changes in production - no one knows a new model went live Concept flow: Governance - Accountability - Traceability - Controlled Risk - Model Owner Defined - Approval for Major Changes - Lineage: Data + Code + Config - Deployment History + Approvers - Review for High-Impact Promotions Pillar: Accountability; Definition: Known owner for each model; clear approvers for major changes; Production Mechanism: Model owner field in registry; PR-based promotion Pillar: Traceability; Definition: Reconstruct what happened and why; Production Mechanism: Lineage tracking; audit logs; registry history

Must remember:

- Governance provides accountability (owners, approvers), traceability (lineage, audit trail), and controlled risk
- Model promotion must be a visible, reviewable change - PR, ticket, or formal approval workflow
- Three pillars: who owns it, can we reconstruct what happened, do big changes get proper review
- Organisational roles: data science (train/evaluate), ML engineering (deploy/rollback), business (metrics), compliance (fairness)
- Governance enables faster iteration by building trust in deployment safety
- Non-negotiable in regulated and high-impact domains (finance, healthcare, hiring)

Example or contrast: A fintech promotes a new credit risk model:

Exam trap: "Governance slows us down" - untested deployments cause incidents that slow teams far more than review gates No model owner assigned - incidents have no responsible party for response

MLME-W06-T12 Traceability, Audit Trails, and Rollback Mechanisms Tier A

Definition and why it matters: Governance requires two operational capabilities: looking backward (what model was live, trained on what data, approved by whom) and acting forward (rolling back to the last known-good model in seconds, not hours)

Core explanation: Concept flow: Version + Owner + Stage - Training Lineage - Deployment History - Data Snapshot ID / Hash - Code Git Commit - Config File Version - Staging Transition + Timestamp - Production Transition + Approver - Rollback Events Audit Field: Version & stage; Content: v7, Production; Used For: Identify current live model Audit Field: Owner; Content: Team / individual; Used For: Incident response contact Audit Field: Training lineage; Content: Data snapshot, code commit, config; Used For: Reproduce or explain model behaviour Audit Field: Evaluation summary; Content: Metrics at promotion time; Used For: Justify why this model was chosen Audit Field: Deployment history; Content: Stage transitions with timestamps and approvers; Used For: Timeline reconstruction Audit Field: Rollback history; Content:

When and why rollback occurred; Used For: Post-incident analysis Lineage answers: "Exactly which ingredients produced this model?" Model v7 = f(Data Snapshot D3, Code Commit abc123, Config trainv2.yaml)

Must remember:

- Audit trail: registry entry with version, owner, stage, training lineage, deployment history, approvers
- Lineage links model to data snapshot, code commit, and config - essential for root cause analysis
- Rollback: keep previous version, pin version/stage in config, test rollback in deployment playbook
- Registry-driven rollback: change stage, restart service - no code or image rebuild needed
- Assume production failures will happen; optimise for recovery speed, not just prevention
- Full traceability supports external audits and compliance in regulated domains

Formula / decision rule: Model v7 = f(Data Snapshot D3, Code Commit abc123, Config train_v2.yaml)

Example or contrast: Approach: modelversion: latest; Risk: Unpredictable - next registry update silently changes production model

Exam trap: Deleting old model versions - eliminates rollback capability entirely Using "latest" in production config - silent, untraceable model changes

MLME-W06-T13 Guardrails and the Governed Promotion Workflow

Tier A

Definition and why it matters: Even well-evaluated models can fail in production - due to edge cases, adversarial inputs, upstream data corruption, or unforeseen interactions. Guardrails are hard limits and safety checks that contain damage when model behaviour deviates from expectations

Core explanation: Mindset: Don't rely on the model always behaving. Add rails so that when it doesn't, the system stays within safe bounds Concept flow: Model Prediction - Output Sanity Checks - Rate Limiting - Policy Constraints - Kill Switch / Feature Flag - Safe Output to User Guardrail: Output sanity checks; Purpose: Reject impossible predictions; Example: Probabilities outside [0,1]; credit scores below 0 or above 1000 Guardrail: Rate limiting; Purpose: Prevent traffic spikes from overwhelming model or downstream systems; Example: Max 1000 predictions/sec per client Guardrail: Kill switches / feature flags; Purpose: Instantly disable a risky model or feature; Example: Flip config flag to route all traffic to fallback rule Guardrail: Policy constraints; Purpose: Technical enforcement of regulatory rules; Example: Block use of protected attributes (age, race) in feature vector Guardrail: Input validation; Purpose: Reject malformed or out-of-schema requests; Example: Schema check before inference; return 400, not garbage prediction Guardrail: Fallback models / rules; Purpose: Default behaviour when model fails; Example: Simple heuristic if model service is down or output fails sanity check

Must remember:

- Guardrails: output sanity checks, rate limiting, kill switches, policy constraints, input validation, fallbacks
- Mindset: assume models will misbehave; contain damage with external safety rails
- Full governed lifecycle: monitor → retrain → evaluate → approve → controlled rollout → guardrails → rollback → monitor
- Kill switches enable instant model disable via config flag without code deployment
- Policy constraints must be technically enforced, not just documented
- Governed MLOps transforms ML from research project to reliable, auditable operational capability

Formula / decision rule: [0,1]; [300, 850]

Example or contrast: A credit risk service enforces:

Exam trap: No output sanity checks - garbage predictions reach users silently Guardrails only in documentation, not code - policy constraints must be technically enforced

MLME-W06-T14 Config-Driven Retraining Pipeline: Training Script and MLflow Integration

Tier A

Definition and why it matters: A production retraining pipeline is config-driven: training scripts are generic; behaviour is controlled entirely by configuration files. This pattern makes retraining repeatable, auditable, and safe - the same script produces different model versions by swapping configs, not code

Core explanation: Concept flow: trainconfigv1.yaml - train.py - trainconfigv2.yaml - Load Data from Config Path - Feature Engineering - Train Model - MLflow Logging - MLflow Register Model - Registry: Version 1 Config Change: Different data.path; Effect: Retrain on newer snapshot Config Change: Different hyperparameters; Effect: New candidate with same data Config Change: Different model type; Effect: Architecture experiment Training data selection is controlled by config, not hardcoded paths: Logged Item: Model parameters; Purpose: Hyperparameters for reproducibility Logged Item: Training metrics; Purpose: RMSE, AUC, loss curves Logged Item: Config file (as artefact); Purpose: Exact config used for this run Why separate config files? Each file is a permanent record of which data powered which training run - critical for audit and reproducibility Logged Item: Model artefact; Purpose: Serialized model file The training script (train.py) is designed to be: Logged Item: Tags; Purpose: Data version, experiment name

Must remember:

- Retraining pipeline is config-driven: same train.py, different config files for data/hyperparameters
- Config files record which data powered which run - essential for audit and reproducibility
- MLflow logs params, metrics, config artefact, and model; registermodel creates official registry version

- Simulating scheduled retrain: run pipeline with v1 config (champion), then v2 config (challenger)
- MLflow UI shows runs, registered versions, and enables head-to-head comparison
- Separate what changes (config) from what stays stable (training/evaluation/serving code)

Exam trap: Hardcoding data paths in train.py - breaks config-driven pattern and auditability Logging model without registering - experiment runs are not promotion-ready candidates

MLME-W06-T15 Champion vs Challenger Evaluation and Automated Promotion

Tier A

Definition and why it matters: After training produces registry versions, the pipeline must answer one question with evidence: Is the challenger actually better than the production champion? The evaluation-and-promote stage loads both models, computes statistical and business metrics on the same holdout data, and applies predefined promotion rules

Core explanation: Code pattern: Conceptual pattern; champion = loadmodel(modelname="creditriskmodel", stage="Production"); challenger = loadmodel(modelname="creditriskmodel", version=2) Category: Statistical; Metric: MSE, RMSE, AUC, F1; Purpose: Predictive quality comparison Category: Business; Metric: Expected profit; Purpose: Financial impact of model decisions Both models run on the identical holdout evaluation dataset - never seen during training of either model Condition: Statistical metric improves; Rationale: Model predicts better Condition: Business metric improves; Rationale: Better predictions translate to financial gain Condition: Both must pass; Rationale: Prevents promoting on statistical gain with business loss Promotion rules are predefined and automated:

Must remember:

- Champion loaded by Production stage; challenger by explicit version number
- Both evaluated on same unseen holdout data with statistical (MSE) and business (profit) metrics
- Promotion rule: challenger must beat champion on both metric types (with optional tolerance delta)
- Automated promotion: challenger → Production, old champion → Archived
- Pattern is production-ready; labs simplify data size and skip shadow/A/B stages
- Business metrics tie model decisions to financial outcomes - essential for high-impact models

Formula / decision rule: Profit = sum_i payoff(hat(y)_i, y_i); +P

Example or contrast: Concept flow: Model Registry - Champion: Production Stage - Challenger: Explicit Version - Same Holdout Evaluation Set - Compute Metrics - Promotion Rules - Challenger - Production - Archive Challenger - Archive Old Champion Role: Champion; How Loaded: Lookup by Production stage tag; Registry Stage: Production Role: Challenger; How Loaded: Explicit version number (e.g., v2); Registry Stage: None or Staging

Exam trap: Evaluating on different datasets - invalidates head-to-head comparison Promoting on MSE alone without business metric - statistically better but financially worse

MLME-W06-T16 Registry-Driven Serving, Promotion Updates, and Rollback

Tier A

Definition and why it matters: Promoting a model in the registry is only half the deployment story. The serving service must load the newly promoted model - and when problems arise, roll back to the previous champion without code changes or image rebuilds

Core explanation: Anti-pattern: Hardcoded model file path in serving code Concept flow: Environment Variables - Serving Service - MLflow Registry - Load Production Model - Predict Endpoint Environment Variable: MODELNAME; Default: creditriskmodel; Effect: Which registered model to load Environment Variable: MODELSTAGE; Default: Production; Effect: Which stage (Production, Staging) Environment Variable: MLFLOWTRACKINGURI; Default: local URI; Effect: Where to find registry Production pattern: Load model from registry by name and stage Flexibility without code changes: Load staging model → set MODELSTAGE=Staging Load specific version → use version URI instead of stage Concept flow: Detect Problem with v2 - Registry: v1 - Production - Registry: v2 - Staging/Archived - Restart Serving Service - Service Loads v1 - Verify via Health/Predict Endpoints Rollback → change registry stage, restart service After promotion (v2 → Production), start the service and verify: Startup logs show: Loaded model version 2, stage: Production Health endpoint reports modelversion: 2 Prediction endpoint returns predictions with correct version metadata

Must remember:

- Serving service loads model from MLflow registry by name + stage (via environment variables)
- No hardcoded paths - promote and rollback by changing registry stage, not code
- Rollback: v1 → Production, v2 → Staging, restart service - under 60 seconds
- End-to-end loop: retrain → register → evaluate → promote → serve → monitor → rollback/retrain
- Registry-driven deployment separates model lifecycle from application code lifecycle
- This architecture is what separates companies that dabble in ML from those that scale it

Example or contrast: v2 is in production. Fraud rates spike. Customer complaints increase. Investigation confirms v2 is the cause. Recovery needed in under 60 seconds

Exam trap: A credit model preprocessing thousands of applications per hour: Every prediction has direct financial impact

MLME-W06-T17 Module Summary: Retraining, Evaluation, and Governed Model Promotion

Tier C

Definition and why it matters: This module completes the operational ML lifecycle by connecting monitoring signals (Week 5) to controlled model refresh - when to retrain, how to retrain safely, how to evaluate candidates, and how to promote or roll back with full governance

Core explanation: Static models degrade as the world changes. Production ML is a continuous loop: deploy → monitor → detect → investigate → retrain → evaluate → redeploy Trigger: Data drift & quality; Signals: PSI spikes, new categories, missing-value spikes; First Step: Investigate: business change or pipeline bug?

Must remember:

- Retraining is a continuous loop triggered by drift, performance degradation, or policy changes - after investigation
- Four-gate checklist before retraining: persistent change, alternatives ruled out, fresh data, pipeline ready
- Pipeline: snapshot → train candidates → evaluate vs champion → register → promote through staging/canary
- Config-driven, MLflow-integrated training makes retraining repeatable and auditable
- Evaluation layers: offline → backtest → shadow → A/B; match depth to model impact
- Promotion requires combined evidence: statistical metrics, business KPIs, fairness, risk review
- Governance: accountability (owners/approvers), traceability (lineage/audit), controlled risk (guardrails/rollback)
- Registry-driven serving enables promotion and rollback without code changes - under 60 seconds
- Champion/challenger with automated promotion rules removes emotion from model selection
- MLOps transforms ML from research project into reliable, revenue-generating operational capability

Formula / decision rule: delta

Example or contrast: Pattern: Scheduled; Mechanism: Calendar (weekly/monthly/quarterly); Best For: Stable data, reliable labels

Exam trap: Drift alert = automatic retrain - investigation always comes first Offline AUC alone justifies promotion - high-impact models need layered evaluation

Week 6 Exam Lens

Likely questions

- Define and explain Static Models and the Retraining Loop; include the mechanism and one limitation.
- Compare Static Models and the Retraining Loop with Retraining Triggers: Drift, Performance, and Policy and state when each is preferred.
- Apply Continuous Training: Scheduled vs Event-Driven Retraining to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: P(X); P(Y|X); delta; Expected Profit = sum_i (payoff(hat(y)_i, y_i));) | False Declines (; 2.1M |; Model v7 = f(Data Snapshot D3, Code Commit abc123, Config train_v2.yaml); [0,1]

Mistakes to avoid: Treating every drift alert as a retrain trigger - drift is a warning sign, not an automatic button; investigation comes first "The model still runs, so it's fine" - uptime does not equal prediction quality "High PSI → retrain immediately" - drift requires investigation; pipeline bugs mimic drift

Week 7: Model Standardization and Optimization

Week roadmap: Model Standardisation and Optimisation in Production → The Research-Production Deployment Gap → Production Optimisation Goals: Portability, Speed, and Footprint → Three Levers for Production Model Optimisation → Standard Model Formats: Foundations and ONNX → TensorFlow Lite: Mobile and Edge Deployment → 13 more topics.

Coverage: 19 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

MLME-W07-T01 Model Standardisation and Optimisation in Production

Tier B

Definition and why it matters: By the time a model reaches production, teams already know how to expose it as an API, containerise it, monitor it, and roll out new versions. The next bottleneck is rarely accuracy - it is operational fit:

Core explanation: Can the model run fast enough to meet latency SLAs? Can it run cheap enough per prediction?

Must remember:

- Production ML adds speed, cost, and hardware constraints on top of accuracy
- Research environments (FP32, single GPU, loose latency) differ sharply from serving environments
- Three goals: portability, latency/throughput, footprint
- Three levers: standard formats, compression, optimised runtimes
- The module systematically attacks the research-to-production gap
- Baseline measurement precedes any optimisation work

Example or contrast: Dimension: Precision; Research / Training: FP32 weights, full precision; Production / Serving: May need INT8/FP16 for speed Dimension: Hardware; Research / Training: Single powerful GPU; Production / Serving: CPUs, shared GPUs, phones, IoT Dimension: Driver; Research / Training: Notebooks, ad-hoc scripts; Production / Serving: Always-on services under load

Exam trap: Trap: Assuming a high-accuracy model is production-ready - accuracy is necessary but not sufficient; latency, cost, and hardware compatibility are separate gates Trap: Optimising before measuring - without baseline size and latency numbers, improvements cannot be quantified

MLME-W07-T02 The Research-Production Deployment Gap

Tier B

Definition and why it matters: A model can achieve state-of-the-art validation accuracy and still be undeployable. Production teams routinely face a familiar situation: the model is impressive on paper, but it is too big, too slow, or too awkward to integrate into the serving stack

Core explanation: Model standardisation and optimisation exist to close the gap between how models are built in research and what production environments demand Concept flow: FP32 model - Single GPU - Notebook workflow - CPU / GPU / Edge - P95 latency SLA - Cost per prediction - Research - Production Large FP32 models on a single powerful GPU

Must remember:

- Production constraints: P95/P99 latency, throughput, cost-per-prediction, diverse hardware
- Research uses FP32, single GPU, loose latency tolerance
- Three pain buckets: portability, latency/throughput, footprint
- Accurate models often fail on size, speed, or deployability
- Standardisation + optimisation close the research-production gap
- Footprint affects disk, memory, power, and edge feasibility

Example or contrast: A ResNet-class image classifier trained in PyTorch at ~45 MB FP32 may score 92% top-1 accuracy. In production:

Exam trap: Trap: Focusing only on average latency - P95/P99 tail latency is what breaks SLAs and user trust Trap: Assuming GPU training environment equals serving environment - many production workloads run CPU-only or on constrained edge hardware

MLME-W07-T03 Production Optimisation Goals: Portability, Speed, and Footprint

Tier B

Definition and why it matters: Every model optimisation decision in production maps to one or more of three objectives. Understanding them separately - and how they interact - is the foundation for choosing formats, compression techniques, and runtimes

Core explanation: Intuition: Training and deployment are often done by different teams with different toolchains. Portability means the training team can use any framework while the deployment team can run the model on CPUs, GPUs, or edge devices without rewriting it Concept flow: PyTorch - ONNX format - TensorFlow - CPU runtime - GPU runtime - Edge runtime Property: Framework-agnostic graph; Benefit: One export, many runtimes Property: Shared inspection tools; Benefit: Graph analysis, validation

Must remember:

- Portability: train in any framework, deploy on any hardware via standard formats
- Latency: per-prediction time; P95/P99 tails matter most for SLAs
- Throughput: req/s per instance; drives server count and cost
- Footprint: disk, memory/VRAM, and power - especially critical for edge

- Identical-accuracy models can differ 3-5x in inference cost
- Standard formats, compression, and runtimes map to these three goals

Exam trap: Trap: Optimising latency without measuring P95/P99 - average latency can hide severe tail problems Trap: Assuming portability implies speed - ONNX export improves deployability; speed gains require runtime optimisation and/or compression

MLME-W07-T04 Three Levers for Production Model Optimisation

Tier B

Definition and why it matters: Closing the research-production gap requires coordinated use of three engineering levers. Each targets different constraints; together they form the standard MLOps optimisation pipeline

Core explanation: Purpose: Portability - train once, deploy widely Format: ONNX; Ecosystem: Cross-framework; Best for: Cloud servers, general CPU/GPU Format: TF Lite; Ecosystem: TensorFlow; Best for: Mobile, Android, iOS, IoT Format: OpenVINO; Ecosystem: Intel; Best for: Intel CPUs, integrated GPUs, accelerators Concept flow: Any framework - Standard format - ONNX Runtime - TF Lite runtime - OpenVINO runtime Technique: Quantisation; What it does: Fewer bits for weights/activations (FP32 - FP16 - INT8); Primary gain: Size, memory bandwidth, speed

Must remember:

- Three levers: standard formats, compression, optimised runtimes
- Formats → portability; compression → size/speed; runtimes → hardware-efficient execution
- ONNX = cross-platform default; TF Lite = mobile; OpenVINO = Intel-first
- Quantisation, pruning, distillation shrink models with accuracy trade-offs
- ONNX Runtime, TensorRT, XLA optimise graph execution on target hardware
- Always measure size, latency, and accuracy before and after changes

Exam trap: Trap: Using only one lever - format export without runtime tuning or compression often yields minimal gains Trap: Expecting compression to fix portability - quantisation shrinks the model but does not replace the need for a deployable format

MLME-W07-T05 Standard Model Formats: Foundations and ONNX

Tier A

Definition and why it matters: Conceptually, a model format is a serialised blueprint plus parameters: Concept flow: Model Format - Graph structure\nops + topology - Parameters\nweights + biases - Metadata\nshapes, names, dtypes - Runtime loads,\noptimises, executes

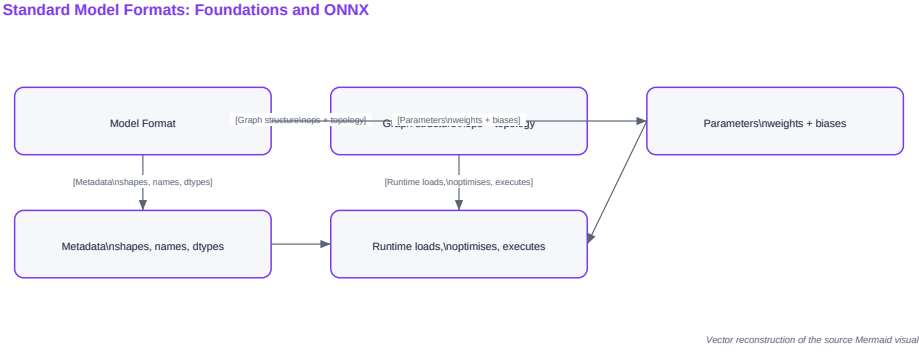
Core explanation: The guiding principle of production model engineering is decoupling training from serving. Training teams should use the framework that fits their workflow; deployment teams should run models on whatever hardware the product requires - without constant rewrites Standard model formats are the bridge Framework: PyTorch; Native artefact: pt / .pth checkpoint; Native runtime: PyTorch Framework: TensorFlow; Native artefact: SavedModel, .h5 (Keras); Native runtime: TensorFlow Framework: JAX; Native artefact: Checkpoints + Python; Native runtime: JAX / XLA Each ML framework stores and executes models in its own way: Problems when staying framework-native across environments: Different artefact formats per team Custom conversion glue for every hardware target No shared tooling for graph inspection and optimisation Fragile handoffs with subtle numerical drift Concept flow: PyTorch model - .onnx file - TensorFlow model - ONNX Runtime - TensorRT - OpenVINO converter

Must remember:

- Model format = graph + parameters + metadata (blueprint + numbers)
- Framework-native formats create portability pain across teams and hardware
- ONNX is the open, cross-framework, cross-runtime standard
- Export enables interoperability; compression and runtime tuning deliver speed/size gains
- ONNX is the practical default for cloud/server, multi-framework organisations
- Format change alone is typically lossless and size-neutral for FP32 weights

Example or contrast: Format: ONNX; Portability: Highest (cross-framework); Primary target: Cloud / server; Ecosystem lock-in: Low

Exam trap: Trap: Confusing model format with runtime - ONNX is the representation; ONNX Runtime is the engine that executes it Trap: Expecting ONNX export to reduce file size - weights are still FP32; size reduction comes from compression (quantisation), not format change



Concept map: Standard Model Formats: Foundations and ONNX

MLME-W07-T06 TensorFlow Lite: Mobile and Edge Deployment

Tier B

Definition and why it matters: TensorFlow Lite (TF Lite) is both a model format (.tflite files) and a runtime engineered specifically for mobile and embedded devices. Where ONNX targets general-purpose cross-platform serving, TF Lite optimises for constrained environments: small binary footprint, mobile CPU/NPU kernels, and quantised INT8 inference

Core explanation: Goal: Small runtime binary; How TF Lite achieves it: Minimal interpreter, no full TensorFlow stack Goal: Fast on mobile hardware; How TF Lite achieves it: Optimised kernels for ARM CPUs and NPUs

Must remember:

- TF Lite = format + runtime for mobile and embedded deployment
- Optimised for small binary, mobile CPU/NPU kernels, INT8 inference
- Natural choice when training in TensorFlow and deploying to Android/iOS/IoT
- INT8 quantisation is a first-class workflow, often yielding ~4x size reduction
- Not a replacement for ONNX in cloud/server multi-framework environments
- Always benchmark on the target device, not a development machine

Example or contrast: Dimension: Primary target; TF Lite: Mobile / embedded; ONNX: Cloud / server / general

Exam trap: Trap: Using TF Lite for cloud server deployment - it is mobile-first; ONNX Runtime or TensorRT are better server choices Trap: Benchmarking TF Lite on desktop CPU - mobile ARM performance and NPU availability differ completely

MLME-W07-T07 OpenVINO and the Standard Format Pipeline

Tier A

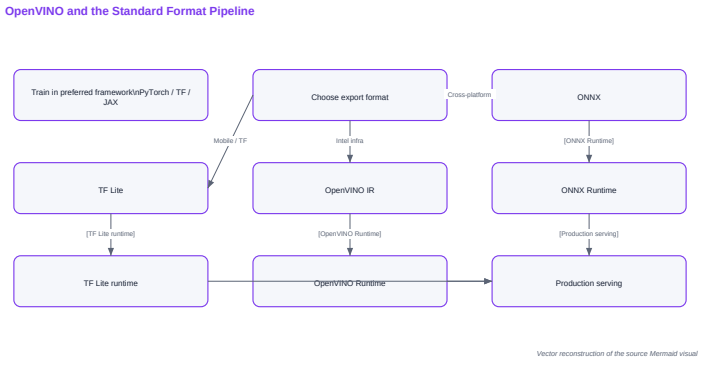
Definition and why it matters: OpenVINO (Open Visual Inference and Neural Network Optimization) is an Intel toolkit for optimising and deploying models on Intel hardware: CPUs, integrated GPUs, VPUs, and other Intel accelerators

Core explanation: Unlike ONNX (vendor-neutral) or TF Lite (mobile-first), OpenVINO targets organisations whose inference infrastructure is predominantly Intel-based servers without dedicated NVIDIA GPUs Concept flow: Source model\n ONNX or TensorFlow - OpenVINO Model Optimizer - OpenVINO IR\n.xml + .bin - OpenVINO Runtime - Intel CPU / iGPU / accelerator Scenario: Intel-only data centre; Why OpenVINO: Deep CPU kernel optimisation (AVX-512, AMX) Scenario: No NVIDIA GPU budget; Why OpenVINO: Strong CPU inference without GPU capex Scenario: Edge with Intel hardware; Why OpenVINO: Intel NUC, industrial PCs Scenario: Mixed ONNX source models; Why OpenVINO: Single Intel runtime for all exports Start from ONNX or TensorFlow model Concept flow: Train in preferred framework\nPyTorch / TF / JAX - Choose export format - ONNX - TF Lite - OpenVINO IR - ONNX Runtime - TF Lite runtime - OpenVINO Runtime - Production serving Convert to OpenVINO Intermediate Representation (IR) via Model Optimizer Run with OpenVINO Runtime on Intel machines Format: ONNX; Reach for it when: Cross-framework teams, cloud servers, CPU+GPU, general default

Must remember:

- OpenVINO optimises for Intel CPUs, iGPUs, and accelerators
- Flow: ONNX/TF → Model Optimizer → IR → OpenVINO Runtime
- Best when infra is Intel-heavy and GPU budget is limited
- Standard format = contract between training and serving teams
- ONNX = general cross-platform; TF Lite = mobile; OpenVINO = Intel-first
- Format choice enables portability; compression + runtime deliver speed/size gains

Exam trap: Trap: Choosing OpenVINO for NVIDIA GPU servers - TensorRT is the NVIDIA-specific optimiser; OpenVINO targets Intel Trap: Assuming one format per organisation - multi-format strategies are normal at scale



Concept map: OpenVINO and the Standard Format Pipeline

MLME-W07-T08 Why Model Compression Matters in Production

Tier B

Definition and why it matters: A model that scores well on a benchmark is not automatically practical in production. Compression techniques take a good model and make it affordable, fast, and deployable on real hardware - often with only a small accuracy trade-off

Core explanation: Compression is not about making bad models better; it is about making good models fit production constraints Benefit: Faster inference; Mechanism: Fewer bits, fewer ops, smaller graphs; Downstream effect: Lower latency Benefit: Higher throughput; Mechanism: Less memory bandwidth, faster arithmetic; Downstream effect: More req/s per instance

Must remember:

- Compression makes accurate models practical for production constraints
- Benefits: faster inference, higher throughput, smaller disk/memory, edge feasibility
- Three techniques: quantisation, pruning, knowledge distillation
- Compression stacks with standard formats and optimised runtimes
- Always measure size, latency, and accuracy before and after
- Trade-off is speed/size vs accuracy - find the acceptable Pareto point

Example or contrast: Compression always involves a trade-off space:

Exam trap: Trap: Compressing before establishing baseline metrics - without before/after size, latency, and accuracy numbers, trade-offs cannot be evaluated Trap: Treating compression as a substitute for a better architecture - a poorly designed large model compressed may still underperform a well-designed small model (distillation addresses this)

MLME-W07-T09 Quantisation and Pruning

Tier A

Definition and why it matters: Deep learning models are typically trained with 32-bit floating point (FP32) weights and activations. Each FP32 value occupies 4 bytes. Quantisation reduces the bit width - storing and computing with FP16, BF16, or 8-bit integers (INT8) - which shrinks the model and accelerates arithmetic on hardware that supports low-precision ops The catch: quantisation approximates continuous values with discrete levels. Some accuracy loss is possible; the art is maximising speed/size gains while keeping accuracy within acceptable limits

Core explanation: Precision: FP32; Bits: 32; Size vs FP32: 1x (baseline); Typical use: Training, high-accuracy serving Precision: FP16; Bits: 16; Size vs FP32: ~2x smaller; Typical use: GPU inference (TensorRT, mixed precision) Precision: INT8; Bits: 8; Size vs FP32: ~4x smaller; Typical use: CPU/mobile inference, edge Concept flow: FP32\n4 bytes/value - FP16\n2 bytes/value - INT8\n1 byte/value Pros: Easy to apply - no retraining; Cons: May lose more accuracy at INT8 Pros: Fast to experiment; Cons: Aggressive low-bit PTQ can degrade badly Pros: Better accuracy at same bit width; Cons: Requires additional training effort

Must remember:

- Quantisation: FP32 → FP16/INT8; ~4x size reduction at INT8; faster memory-bound inference
- PTQ: quantise after training; easy but may lose accuracy at INT8
- QAT: simulate quantisation during training; better INT8 accuracy
- Unstructured pruning: zero individual weights; needs sparse kernels for speed
- Structured pruning: remove channels; smaller dense net; standard HW friendly
- Pruning workflow: train → prune → fine-tune → measure
- Try PTQ first; escalate to QAT if accuracy insufficient

Example or contrast: Dimension: What changes; Quantisation: Bit width of values; Pruning: Number of parameters

Exam trap: Trap: Assuming PTQ always preserves accuracy - INT8 PTQ on small calibration sets can fail on out-of-distribution inputs Trap: Unstructured pruning without sparse runtime support - sparsity may not translate to speed

MLME-W07-T10 Knowledge Distillation

Tier A

Definition and why it matters: Quantisation and pruning reduce an existing model's precision or parameters. Knowledge distillation takes a different approach: design a smaller student architecture and train it to replicate the behaviour of a larger, accurate teacher model

Core explanation: The result is often a student that is both smaller/faster and more accurate than a same-size model trained only on hard labels Role: Teacher; Model: Large, accurate, already trained; Characteristics: High capacity, slow inference Role: Student; Model: Smaller, designed for constraints; Characteristics: Fewer parameters, fast inference Concept flow: Input - Teacher\nlarge, frozen - Student\nsmall, trainable - Distillation loss - Update student weights The teacher remains frozen (or is updated slowly). The student is trained to match the teacher's outputs Factor: Richer training signal; Explanation: Soft labels carry more information per sample than one-hot Traditional training: student sees input x and target class "cat" or "dog" - a one-hot distribution Factor: Regularisation effect; Explanation: Matching smooth distributions prevents overconfident student The teacher outputs a full probability distribution over all classes, e.g.:

Must remember:

- Distillation: train a small student to mimic a large teacher
- Soft labels carry inter-class similarity (dark knowledge) beyond hard one-hot targets
- Students can be smaller, faster, and more accurate than same-size models trained on hard labels alone
- Loss combines soft-target (teacher distribution) and hard-target (ground truth) terms
- Popular for edge/mobile: design student architecture for hardware constraints

- Highest training investment among compression techniques; pairs well with quantisation

Formula / decision rule: L = alpha *L_(soft)(T(z_s), T(z_t)) + (1 - alpha)* L_(hard)(y, z_s); T(*)

Example or contrast: Edge / mobile deployment: BERT-large teacher → DistilBERT student

Exam trap: Trap: Distilling from a weak teacher - student quality is bounded by teacher quality Trap: Using only hard labels for the student - this forfeits the main benefit of distillation

MLME-W07-T11 Compression Trade-offs and the MLOps Pipeline

Tier A

Definition and why it matters: No matter which technique is applied:

Core explanation: Technique: Quantisation; Ease of application: High (especially PTQ); Typical accuracy impact: Small at FP16; moderate at INT8; Speed/size gain: Large (esp. INT8); Best first step?: Yes - often first Technique: Pruning; Ease of application: Medium; Typical accuracy impact: Moderate without fine-tune; Speed/size gain: Large if structured + fine-tuned; Best first step?: When model is overparameterised Technique: Distillation; Ease of application: Low (requires re-training); Typical accuracy impact: Can improve vs small baseline; Speed/size gain: Large (new architecture); Best first step?: When architecture redesign is acceptable Concept flow: Quantisation\nquick wins - Pruning\nif still too big - Distillation\nif architecture change OK - Measure size, latency, accuracy Metric: Model size (MB); Measure before: yes; Measure after: yes Metric: Average latency; Measure before: yes; Measure after: yes Metric: P95 / P99 latency; Measure before: yes; Measure after: yes Metric: Accuracy / key business metric; Measure before: yes; Measure after: yes Start with quantisation - PTQ is easy and often yields big speed/memory gains for small accuracy drop Add structured pruning when the network is clearly larger than necessary

Must remember:

- Quantisation: best first step; easy PTQ, big INT8 gains
- Pruning: for overparameterised models; structured unstructured for HW
- Distillation: powerful but training-intensive; design student for constraints
- Always measure size, latency, accuracy before and after
- Pipeline: train → compress → export → optimised runtime → serve
- Techniques combine; change one variable at a time
- Compression + runtime optimisation address different layers of the stack

Exam trap: Trap: Applying multiple techniques simultaneously without isolating effects - cannot attribute gains or regressions Trap: Accepting accuracy drop without business context - 1% accuracy loss may be fatal for medical diagnosis but acceptable for recommendation ranking

MLME-W07-T12 Optimised Runtimes: Foundations

Tier A

Definition and why it matters: Given a model exported to a standard format (e.g. ONNX), how do we execute it as efficiently as possible on the available hardware?

Core explanation: That is the job of an optimised runtime - an inference engine that takes a computation graph and produces fast, memory-efficient predictions without changing the model's mathematical output Concept flow: ONNX / standard format graph - Optimised Runtime - Graph optimisation\nop fusion - Kernel selection\nbest impl per HW - Memory management\nbuffer reuse - Accelerator use\nSIMD, GPU, NPU, TPU - Same outputs\nlower latency Objective: Lower latency; Runtime contribution: Fused ops, tuned kernels Objective: Higher throughput; Runtime contribution: Parallel execution, memory efficiency Objective: Lower memory; Runtime contribution: Buffer reuse, in-place ops Fuse sequences of operations into single kernels. Classic example: Objective: Same accuracy; Runtime contribution: Optimisation is execution-level, not mathematical Conv - BatchNorm - ReLU Runtime: ONNX Runtime; Origin: Microsoft / ONNX ecosystem; Primary scope: General-purpose, ONNX models, cross-platform

Must remember:

- Optimised runtimes execute standard-format graphs with maximum hardware efficiency
- Four mechanisms: graph fusion, kernel selection, memory reuse, accelerator exploitation
- Outputs stay mathematically the same; latency, throughput, and memory improve
- Three focus runtimes: ONNX Runtime, TensorRT, XLA
- Trade-off 1: portability (ONNX Runtime) vs peak perf (TensorRT)
- Trade-off 2: compile-time investment vs per-request latency
- Format = what; runtime = how

Formula / decision rule: Conv → BatchNorm → ReLU

Example or contrast: Approach: ONNX + ONNX Runtime; Portability: High; Peak performance: Good, not always maximum Approach: TensorRT (NVIDIA); Portability: Low (GPU-specific); Peak performance: Highest on NVIDIA Approach: XLA (TF/JAX); Portability: Medium (framework-bound); Peak performance: High on TPU/GPU

Exam trap: Trap: Expecting runtime optimisation to fix a bad model - runtimes optimise execution, not accuracy Trap: Ignoring compile-time cost in CI/CD - TensorRT engine builds can take minutes per model version

MLME-W07-T13 ONNX Runtime, TensorRT, and XLA

Tier A

Definition and why it matters: ONNX Runtime (ORT) is a high-performance inference engine built specifically for ONNX models

Core explanation: Feature: Hardware; Detail: CPU, GPU, and accelerators via Execution Providers Feature: Execution Providers; Detail: CPUExecutionProvider, CUDAExecutionProvider, TensorrtExecutionProvider, others Feature: Graph optimisations; Detail: Built-in fusion, constant folding, layout optimisation Feature: APIs; Detail: Python, C, C++, C, Java, JavaScript Concept flow: .onnx model - ONNX Runtime - CPU EP - CUDA EP\nNVIDIA GPU - TensorRT EP\nvia TensorRT - Predictions Organisation standardises on ONNX as the deployment format Need a portable runtime across Linux, Windows, cloud, edge Concept flow: ONNX model - TensorRT builder - .engine binary - NVIDIA GPU inference Want one runtime with pluggable hardware backends Default choice for general-purpose production deployment TensorRT is NVIDIA's dedicated inference optimiser and runtime for NVIDIA GPUs Import model (often from ONNX) TensorRT performs heavy graph optimisation: layer fusion, precision calibration, kernel auto-tuning for the specific GPU model Produces a TensorRT Engine (binary artefact optimised for that GPU) Engine executes with very low latency Lowest latency and highest throughput on NVIDIA GPUs Native FP16 and INT8 quantised inference

Must remember:

- ONNX Runtime: portable, ONNX-native, execution providers for CPU/GPU/TensorRT
- TensorRT: NVIDIA GPU peak performance; builds per-GPU engine; FP16/INT8
- XLA: TF/JAX compiler; fuses ops to machine code; CPU/GPU/TPU
- ORT = general-purpose default; TensorRT = NVIDIA latency champion; XLA = TF/JAX ecosystem
- TensorRT and XLA pay compile-time cost for runtime speed
- Model format describes what; runtime describes how

Example or contrast: Dimension: Input format; ONNX Runtime: ONNX; TensorRT: ONNX, TF, others; XLA: TF/JAX graph (in-process)

Exam trap: Trap: Using TensorRT for CPU inference - TensorRT is GPU-only Trap: Expecting XLA to work with PyTorch ONNX exports - XLA is TF/JAX internal

MLME-W07-T14 Runtime Trade-offs and Deployment Fit

Tier B

Definition and why it matters: Common strategy: portable baseline → measure → selectively adopt TensorRT (or similar) for the most critical services where every millisecond counts

Core explanation: Advantage: One model format across teams; Limitation: May not reach absolute peak on every device Advantage: One runtime, multiple execution providers; Limitation: Generic kernels vs hand-tuned vendor kernels Advantage: Simpler standardisation story; Limitation: GPU speed may trail TensorRT Advantage: Best possible latency/throughput on target device; Limitation: More configuration and setup

Must remember:

- Portable: ONNX + ORT - one format, many platforms; may sacrifice peak perf
- Hardware-specific: TensorRT - best NVIDIA latency; less portable, more setup
- Compile vs runtime: TensorRT/XLA invest upfront for per-request savings
- Strategy: portable first → measure → hardware-specific for critical paths
- Full pipeline: train → compress → export → runtime
- Format = what; runtime = how; compression = how small/fast the model is

Example or contrast: Runtime style: Heavy compile (TensorRT, XLA); Upfront cost: Minutes to build engine; Per-request cost: Lowest latency; Best when: High traffic, infrequent model changes Runtime style: Light start (framework native, vanilla ORT); Upfront cost: Seconds; Per-request cost: Higher per-request; Best when: Rapid iteration, low traffic

Exam trap: Trap: Defaulting to TensorRT for all models - over-engineering if PyTorch or ORT already meets SLA on CPU Trap: Ignoring engine rebuild in deployment pipeline - TensorRT engines are not portable across GPU models

MLME-W07-T15 Lab Overview: ONNX Export and Runtime Benchmarking

Tier A

Definition and why it matters: The hands-on lab closes the gap between theory and measurement. It takes a pre-trained convolutional neural network, establishes a rigorous baseline, exports to ONNX, runs inference with ONNX Runtime, and compares metrics - demonstrating that the same logical model can have a different performance profile depending on format and runtime

Core explanation: Concept flow: Step 1: Baseline\nPyTorch ResNet-18 - Step 2: Export\ntorch.onnx.export - Step 3: Benchmark\nONNX Runtime - Step 4: Compare\nsize + latency + outputs Step: 1; Script: baseline.py; Key output: Model size, avg/P95 latency, baselinemetrics.json Step: 2; Script: exportonnx.py; Key output: Validated .onnx file, size comparison Step: 3; Script: onnxruntimebenchmark.py; Key output: ORT latency, side-by-side table Metric: File size (MB); Why it matters: Disk, download, artefact storage Metric: Average latency; Why it matters: Capacity planning Metric: P95 latency; Why it matters: Tail SLA, worst-case UX Metric: Output correctness; Why it matters: Ensure export is lossless Metric: Parameter count; Why it matters: Model complexity reference Well-known CNN architecture from torchvision ~11 million parameters - realistic but small enough for fast demos

Must remember:

- Lab: baseline PyTorch → export ONNX → benchmark ORT → compare
- Measure size, avg latency, P95 latency, and output correctness

- ResNet-18 on CPU is the reference configuration
- Fair comparison: same model, weights, input, hardware; only runtime changes
- Export is typically lossless and size-neutral for FP32
- Results teach that optimisation is empirical and context-dependent

Example or contrast: To isolate the effect of the runtime: Held constant: Model architecture; Changed: Inference engine Held constant: Weights (same checkpoint); Changed: PyTorch - ONNX Runtime

Exam trap: Trap: Skipping warm-up runs - first inference includes allocation overhead, skewing latency Trap: Comparing GPU PyTorch baseline against CPU ORT - hardware must match

MLME-W07-T16 Establishing a Baseline: Model Size and Latency

Tier A

Definition and why it matters: Before claiming any optimisation worked, engineers need a reference point. Baseline metrics answer: "How big is the model? How fast is inference out of the box?" Without this, statements like "we made it faster" are unverifiable

Core explanation: Optimisation without measurement is guesswork Concept flow: Load pretrained ResNet-18\nfrom torchvision - model.eval + CPU - Save checkpoint\nmeasure file size - Warm-up loop\nstabilise timing - 100 timed forward passes - Compute avg + P95 latency - Save baselinemetrics.json Save the model checkpoint (e.g. resnet18baseline.pt) Measure file size in megabytes via os.path.getsize Establishes whether format changes (ONNX, quantised) actually reduce storage Run forward passes on a standard CPU for reproducibility Report average and P95 latency P95 = latency value below which 95% of requests fall - better tail-SLA proxy than average alone The first few inference runs are often slower due to: Memory allocation for intermediate tensors CPU library initialisation (MKL, oneDNN)

Must remember:

- Baseline = disk size + avg/P95 latency before any optimisation
- ResNet-18 on CPU: realistic, well-known reference architecture
- Warm-up loop essential for steady-state measurements
- Save metrics to JSON for automated downstream comparison
- PyTorch CPU backend is highly optimised for standard small CNNs
- ONNX Runtime targets inference but is not universally faster
- "How much faster?" requires a measured reference point

Example or contrast: Aspect: Primary goal; PyTorch: Full ML lifecycle; ONNX Runtime: Inference only Aspect: Graph optimisation; PyTorch: Runtime-level; ONNX Runtime: Dedicated fusion passes Aspect: Input type; PyTorch: torch.Tensor; ONNX Runtime: numpy.ndarray

Exam trap: Trap: Timing without model.eval() - dropout and batch norm behave differently in train mode Trap: Using GPU for baseline but CPU for ORT comparison - invalid apples-to-oranges test

MLME-W07-T17 Exporting a CNN to ONNX Format

Tier A

Definition and why it matters: Export transforms a framework-native checkpoint into a portable, standard representation loadable by ONNX Runtime, TensorRT, OpenVINO, and other ONNX-compatible tools. The export step is about interoperability and optimisation potential - not necessarily immediate size or speed gains

Core explanation: Concept flow: Load checkpoint\nsame weights as baseline - model.eval - torch.onnx.export - onnx.checker.checkmodel - Compare file sizes Parameter: model; Role: Network in eval mode with trained weights Parameter: dummyinput; Role: Representative tensor for graph tracing Parameter: inputnames / outputnames; Role: Public API names used by all runtimes Parameter: dynamicaxes; Role: Allow variable batch size at inference time Parameter: opsetversion; Role: ONNX operator set version (compatibility) Concept flow: ONNX Graph\n49 nodes - Conv node - BatchNorm node - ReLU node - MaxPool node Artefact: PyTorch .pt; Size: ~44.67 MB Artefact: ONNX .onnx; Size: ~44.50 MB Declaring batch dimension as dynamic means the ONNX model accepts batch sizes 1, 8, 32, etc. at inference - critical for both real-time (batch=1) and batch scoring (batch=32+) without re-exporting Static analysis of the exported graph: Verifies well-formed protobuf structure PyTorch .pt: Compiled/run only by PyTorch; ONNX .onnx: ONNX Runtime, TensorRT, OpenVINO, Checks operator compatibility PyTorch .pt: Framework-locked; ONNX .onnx: Portable contract

Must remember:

- torch.onnx.export traces graph with dummy input; names define runtime API
- dynamicaxes enables variable batch size at inference
- onnx.checker.checkmodel validates graph structure and node count
- FP32 export is size-neutral and lossless - same weights, different container
- Gain is portability and multi-runtime optimisation potential, not immediate speed
- ResNet-18 ~= 49 ONNX nodes; ~11M parameters
- Next step: benchmark with ONNX Runtime using identical methodology

Exam trap: Trap: Exporting in train mode - batch norm and dropout produce wrong graph Trap: Wrong dummy input shape - graph traced for incorrect dimensions

MLME-W07-T18 ONNX Runtime Benchmarking and Interpreting Results

Tier A

Definition and why it matters: Code pattern: import onnxruntime as ort; import numpy as np; session = ort.InferenceSession(; "resnet18.onnx",; providers=["CPUExecutionProvider"], match PyTorch CPU baseline;); dummyinput = np.random.randn(1, 3, 224, 224).astype(np.float32); Warm-up

Core explanation: PyTorch: torch.Tensor input; ONNX Runtime: numpy.ndarray input PyTorch: model(tensor); ONNX Runtime: session.run(None, {name: array}) PyTorch: Implicit I/O; ONNX Runtime: Explicit inputnames from export Concept flow: numpy array - InferenceSession.run - numpy output Concept flow: Context factors - Model size\nResNet-18 = small - Hardware\nCPU only - Batch size\n1 - Optimisation level\nvanilla export - Result: PyTorch faster Lesson: Profiling is essential; Detail: Never assume a tool is faster without measuring Lesson: PyTorch is excellent for small CNN + CPU + prototyping; Detail: Do not over-engineer if already fast enough ResNet-18 has ~11M parameters and only ~49 graph operations. PyTorch's CPU backend (MKL, oneDNN) has been tuned for decades on exactly these standard CNN architectures. Overhead dominates for tiny graphs

Must remember:

- ORT inference: InferenceSession + session.run with numpy arrays and named inputs
- Match CPU provider to PyTorch CPU baseline for fair comparison
- ResNet-18 + CPU + batch=1 + vanilla ORT can be slower than PyTorch
- Reasons: small model, CPU-only, batch=1, no graph optimisations enabled
- ORT wins on large models, GPU, quantisation, cross-platform production
- Optimisation is empirical: baseline → change → measure → decide
- A negative result is valuable data, not a failed experiment

Example or contrast: Load the exported .onnx model with ONNX Runtime, run the identical latency benchmark (warm-up + 100 timed runs), load saved baseline metrics, and print a side-by-side comparison table Metric: Avg latency; PyTorch: ~8 ms; ONNX Runtime: ~17 ms A representative CPU benchmark on ResNet-18, batch size 1, vanilla export:

Exam trap: Trap: "ONNX Runtime is always faster" - disproven by this exact experiment; context matters Trap: Benchmarking without matching execution provider to baseline hardware

MLME-W07-T19

Module 7 Summary: Standardisation and Optimisation

Tier B

Definition and why it matters: A model is only truly useful in production when it is simultaneously:

Core explanation: Accurate enough for the business task Concept flow: Train in any framework - Compression\nquantize / prune / distil - Standard format\nONNX / TF Lite / OpenVINO - Optimised runtime\nORT / TensorRT / XLA - Deploy: cloud / edge / mobile - Monitor: size, latency, accuracy Goal: Portability; Pain: Framework/hardware mismatch; Primary tools: ONNX, TF Lite, OpenVINO IR Goal: Latency / throughput; Pain: SLA misses, high infra cost; Primary tools: Optimised runtimes, quantisation Goal: Footprint; Pain: Disk, RAM, power limits; Primary tools: Quantisation, pruning, distillation Format: ONNX; Best for: Cross-framework, cloud/server, general default

Must remember:

- North star: accurate + fast + cheap at production scale
- Three goals: portability, latency/throughput, footprint
- Three levers: standard formats, compression, optimised runtimes
- ONNX = cross-platform default; TF Lite = mobile; OpenVINO = Intel
- Quantise first; prune if overparameterised; distil for architecture redesign
- ORT = portable; TensorRT = NVIDIA peak; XLA = TF/JAX/TPU
- Always baseline → change → measure; optimisation is empirical
- Format = what; runtime = how; compression = how small/fast

Exam trap: Trap: Treating accuracy as the only deployment gate - latency, cost, and hardware are independent constraints Trap: Assuming ONNX Runtime always beats native framework inference - optimisation is context-dependent

Week 7 Exam Lens

Likely questions

- Define and explain Standard Model Formats: Foundations and ONNX; include the mechanism and one limitation.
- Compare Standard Model Formats: Foundations and ONNX with OpenVINO and the Standard Format Pipeline and state when each is preferred.
- Apply Quantisation and Pruning to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: L = alpha *L_(soft)(T(z_s), T(z_t)) + (1 - alpha)* L_(hard)(y, z_s); T(*); Conv → Batch Norm → ReLU

Mistakes to avoid: Trap: Assuming a high-accuracy model is production-ready - accuracy is necessary but not sufficient; latency, cost, and hardware compatibility are separate gates Trap: Focusing only on average latency - P95/P99 tail latency is what breaks SLAs and user trust Trap: Optimising latency without measuring P95/P99 - average latency can hide severe tail problems

Week 8: Accuracy, Latency, Cost, and UX Trade-offs

Week roadmap: Production ML as a Four-Way Trade-Off → The Four-Way Tug-of-War Mental Model → Accuracy, Latency, Cost, and User Experience → The Latency-Cost-UX Triangle → Why ML Services Need to Scale → Vertical Scaling, Horizontal Scaling, and Autoscaling → 11 more topics.

Coverage: 17 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

MLME-W08-T01 Production ML as a Four-Way Trade-Off

Tier B

Definition and why it matters: Model engineering through earlier modules covers improving models, scaling them, and monitoring them in production. Those skills are necessary but insufficient: a high-accuracy model that is slow, expensive, or frustrating to use can still fail as a product

Core explanation: Production ML systems are pulled in four directions simultaneously: Dimension: Accuracy; What it measures: How often the model is correct; Typical tension: Bigger models - higher latency and cost Dimension: Latency; What it measures: Time from request to prediction; Typical tension: Lower latency - more hardware or simpler models Dimension: Cost; What it measures: Memory, compute, cloud spend, engineering effort; Typical tension: Cheaper setup - slower or less accurate service

Must remember:

- Production ML is a four-way tug-of-war: accuracy, latency, cost, UX
- You rarely maximise one force without affecting the others
- UX is where accuracy, latency, cost, and reliability become visible to users
- Model engineering means finding an acceptable balance, not a single optimal number
- This module connects trade-offs to scaling, cost levers, decision frameworks, and compression workflows

Exam trap: Trap: Treating accuracy as the only success metric - offline leaderboard wins do not guarantee production success Trap: Optimising one dimension in isolation without measuring the other three

MLME-W08-T02 The Four-Way Tug-of-War Mental Model

Tier B

Definition and why it matters: Picture a tug-of-war rope. At the centre sits a knot representing the production ML system. Four teams pull from different sides: Concept flow: Accuracy\nbigger, deeper models - Production\nML System - Latency\nmilliseconds matter - Cost\ncontrol cloud bills - UX\nfast, reliable, trustworthy

Core explanation: A team ships a more complex model. Offline accuracy improves by ~3 percentage points. In production, response time doubles, page load slows, users abandon flows, and business metrics worsen despite better model metrics A team ships an extremely fast tiny model. Predictions are poor - bad recommendations, wrong fraud flags, annoying false positives. Users lose trust even though the UI feels fast

Must remember:

- Production ML sits at the centre of a four-way tug-of-war: accuracy, latency, cost, UX
- Accuracy pulls toward bigger models; latency pulls toward speed; cost pulls toward frugality; UX reflects all three
- Isolated optimisation on one axis reliably damages the others
- The goal is an acceptable balance for product and business constraints, not winning one side

Exam trap: Trap: "We improved accuracy, so the release is a success" - ignore latency/UX at your peril Trap: "Users care about accuracy" - users feel responsiveness; they rarely see ROC-AUC or F1

MLME-W08-T03 Accuracy, Latency, Cost, and User Experience

Tier A

Definition and why it matters: Accuracy measures how often the model is correct. Higher accuracy enables:

Core explanation: More reliable fraud detection Latency range: ~100-200 ms; User perception: Feels almost instant Latency range: ~500 ms - 1 s; User perception: Noticeable delay Latency range: 1-2 s; User perception: Slow, frustrating interface Better search ranking Safer or fairer automated decisions Mechanics: In many architectures, higher accuracy requires deeper networks, more parameters, and higher cost per request. Each extra percentage point of accuracy often brings disproportionate increases in latency and infrastructure spend Key question: Is this extra accuracy worth what it does to latency and cost? Latency is the elapsed time between sending a request and receiving a prediction Users never see offline metrics like AUC or F1. They feel whether the product is responsive. That is why latency is a powerful - and sometimes invisible - force in the trade-off space Cost is driven by: Number and type of machines (GPU vs CPU)

Must remember:

- Accuracy improves with model complexity but raises latency and cost per request
- Human UX is sensitive to latency: ~100-200 ms feels instant; 1 s feels slow
- Users feel responsiveness, not AUC - latency is an invisible but critical force
- Cost scales with replicas, instance type, and engineering overhead
- UX synthesises speed, correctness, reliability, and trust - it is the ultimate product metric

Example or contrast: : Accuracy; Model A: ~1-2 pp higher; Model B: Slightly lower : Speed; Model A: 3x slower; Model B: Much faster : Offline leaderboard; Model A: Looks like winner; Model B: Looks inferior

Exam trap: Trap: Choosing the offline leaderboard winner without checking P95 latency under production load Trap: Using average latency instead of P95/P99 - tail latency drives perceived slowness

MLME-W08-T04 The Latency-Cost-UX Triangle

Tier A

Definition and why it matters: Accuracy was covered separately; three operational forces form a tight triangle:

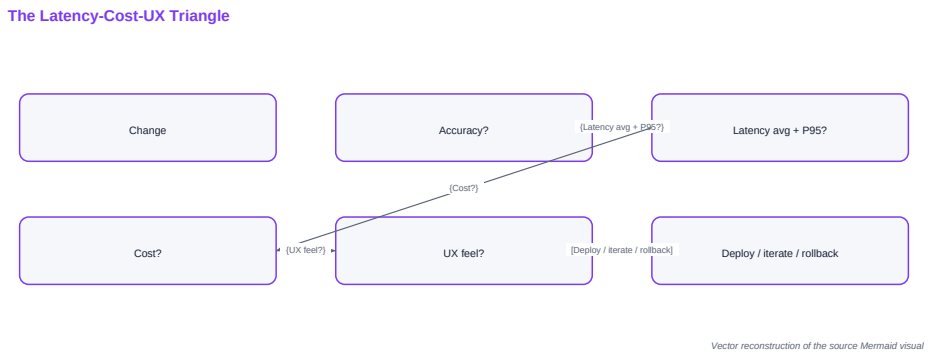
Core explanation: Concept flow: Lower Latency - Higher Cost - Higher Latency - Worse UX - Better UX - Cheaper but unstable UX Goal: Reduce latency; Typical action: Add replicas or upgrade to GPU/larger instances; Effect on cost: Increases; Effect on UX: Improves (if sized correctly) Goal: Reduce cost; Typical action: Shrink instances or reduce replica count; Effect on cost: Decreases; Effect on UX: Degrades if taken too far - jitter, timeouts, instability Question: Accuracy; What to measure: Did offline/online quality metrics go up, down, or stay flat? By how much? Question: Latency; What to measure: What happened to average and P95 latency? Question: Cost; What to measure: Impact on cost per request and monthly bills? Question: User experience; What to measure: Does the change feel snappier, more helpful, or more frustrating? Concept flow: Change - Accuracy? - Latency avg + P95? - Cost? - UX feel? - Deploy / iterate / rollback Under load, under-provisioned systems see rising latency, rising error rates, and violated SLOs - all visible as poor UX

Must remember:

- Latency, cost, and UX form a triangle: lower latency usually costs more; aggressive cost cuts raise latency and hurt UX
- Use a four-question checklist (accuracy, latency, cost, UX) for every production change
- Always track both average and P95 latency
- Inability to answer all four questions signals a blind spot in the change evaluation

Example or contrast: Example - new model version:

Exam trap: Trap: Measuring only average latency - P95/P99 drive SLA violations and user frustration Trap: Reducing cost without load testing - instability under spikes destroys UX



Concept map: The Latency-Cost-UX Triangle

MLME-W08-T05 Why ML Services Need to Scale

Tier B

Definition and why it matters: Real ML APIs do not see flat, predictable traffic. They see:

Core explanation: Spikes during marketing campaigns or major events Concept flow: Traffic increases - Instance overloaded - Latency rises - Error rates increase - SLO violations\nP95 latency, availability Pattern: Vertical scaling; Mechanism: Replace existing machine with a bigger one (more CPU, RAM, GPU); Mental model: One stronger box Daily/weekly usage patterns Pattern: Horizontal scaling; Mechanism: Run multiple copies of the service; load balancer distributes traffic; Mental model: Many identical boxes Steady growth as the product succeeds Concept flow: Small instance - Bigger instance - Load Balancer - Replica 1 - Replica 2 - Replica N

Must remember:

- ML traffic is spiky and grows over time; fixed capacity leads to rising latency and SLO violations
- Vertical scaling: bigger machine - simple but limited
- Horizontal scaling: more replicas + load balancer - standard at scale
- Autoscaling: dynamically adjusts replica count based on metrics
- Scaling keeps latency/UX stable under load; cost must still be managed deliberately

Exam trap: The four-way tug-of-war (accuracy, latency, cost, UX) sets the constraints. When traffic grows, a separate question emerges: how do we keep latency and UX under control without blowing up cost? Three classical patterns address this:

MLME-W08-T06 Vertical Scaling, Horizontal Scaling, and Autoscaling

Tier A

Definition and why it matters: Mechanism: Run the existing service on a bigger machine - more vCPUs, more memory, optionally adding a GPU where none existed before

Core explanation: Minimal application code changes Concept flow: Clients - Load Balancer - Replica 1 - Replica 2 - Replica N Architecture stays largely the same Natural first step for small teams or new products Challenge: State management; Why it matters: In-memory cache or session state on one instance must move to shared external store (Redis, DB) Challenge:

Deployment coordination; Why it matters: Rolling deploys, health checks, version consistency across replicas Challenge: Cost control; Why it matters: Unlimited replica growth - runaway cloud bills Mechanism: Run multiple replicas of the same model service. A load balancer distributes incoming requests across replicas Higher total throughput - latency stays low when many requests arrive concurrently High availability - if one replica crashes or restarts, others continue serving Backbone of most production ML services at scale Parameter: Minimum replicas; Purpose: Always-on baseline capacity

Must remember:

- Vertical scaling: bigger box - simple, limited, single point of failure
- Horizontal scaling: more replicas + load balancer - standard at scale, needs state and deploy coordination
- Autoscaling: dynamic replica count from CPU, QPS, or P95 triggers - requires min/max bounds and cooldowns
- Flapping = rapid scale in/out from over-sensitive rules - hurts cost and stability
- Good monitoring is essential for sensible autoscaling policy design

Example or contrast: Pattern: Vertical; Best for: Small/medium traffic, quick wins; Latency under load: Good until machine saturates; Availability: Single point of failure; Cost profile: Expensive at top tier

Exam trap: Pros: Easy to implement; Cons: Hard upper limit on single-machine size Pros: Manage one (or few) instances; Cons: Single point of failure - if the box dies, service is down

MLME-W08-T07 Scaling Patterns Compared: Latency, Cost, and Stability

Tier A

Definition and why it matters: Choice depends on: product stage, traffic pattern, and budget - not a one-size-fits-all answer

Core explanation: Pattern: Vertical scaling; Latency impact: Quick improvement for small-medium systems; Cost impact: Large instances can be expensive per unit; ceiling exists; Best stage / traffic: Early product, moderate traffic Pattern: Horizontal + autoscaling; Latency impact: Keeps P95 more stable as load grows; Cost impact: Grows with replica count - needs upper limits; Best stage / traffic: Spikes, high sustained traffic Concept flow: Small/medium traffic\nquick fix needed - Vertical first - Growing/spiky traffic\nHA required - Horizontal + autoscaling Often the fastest way to improve latency for small-to-medium systems Requires no architectural rewrite Lever: Spot / preemptible instances; Role: Discounted compute with interruption risk - batch and non-critical workloads Eventually hits technical limits and poor marginal economics Lever: Serverless inference; Role: Pay-per-invocation - spiky, low-volume workloads Single point of failure unless paired with redundancy patterns Lever: Batching / micro-batching; Role: Improve GPU/CPU utilisation - lower cost per request Standard for production ML at scale Absorbs traffic spikes while targeting stable P95 latency

Must remember:

- Vertical scaling: quick latency win for small systems; hits limits and cost inefficiency at scale
- Horizontal + autoscaling: standard for spikes and high traffic; stabilises P95 with proper caps
- Pattern choice depends on stage, traffic shape, and budget
- Cost levers (spot, serverless, batching) complement scaling - they do not replace it
- Scaling + cost optimisation + decision framework form the full production engineering toolkit

Exam trap: Trap: Jumping to horizontal scaling for a prototype with 10 RPS - vertical may suffice and be simpler Trap: Autoscaling without max replica limit - financial incident waiting to happen

MLME-W08-T08 Inference Cost and Spot / Preemptible Instances

Tier A

Definition and why it matters: Before optimising cost, identify the drivers:

Core explanation: Cost driver: Compute; Description: Number and type of machines (GPU vs CPU instances) - usually the largest line item Cost driver: Idle capacity; Description: Instances running but underutilised - paying for unused headroom Cost driver: Network & storage; Description: Data movement, model artefacts, logs Cost driver: Engineering time; Description: Maintenance, incident response, pipeline upkeep Model: On-demand; Commitment: None - pay as you go; Price: Highest per-hour rate; Best for: Flexible, latency-critical core services Model: Reserved / committed use; Commitment: 1-3 year commitment; Price: Discounted vs on-demand; Best for: Steady, predictable load Model: Spot / preemptible; Commitment: None - but interruptible; Price: Heavily discounted (often 60-90% off); Best for: Fault-tolerant batch and offline work Goal: Pay only for capacity actually needed and use that capacity as efficiently as possible Mechanism: Cloud provider offers spare capacity at steep discount. Provider can reclaim instances with short notice when capacity is needed elsewhere Often much cheaper than on-demand for equivalent compute

Must remember:

- Inference cost is dominated by compute, idle capacity, network/storage, and engineering overhead
- On-demand: flexible, most expensive; reserved: cheaper for steady load; spot: cheapest, interruptible
- Spot suits batch, offline scoring, training with checkpoints - not sole infrastructure for critical online APIs
- Hybrid pattern: on-demand core + spot for batch/overflow balances cost and reliability
- Cost levers must be evaluated against latency and UX, not just monthly bills

Example or contrast: Batch jobs that can restart from checkpoints Concept flow: Batch scoring - Training with checkpoints - Offline experiments - Core online API - Payment fraud check Offline scoring - nightly churn prediction, backfills

Exam trap: Low-latency online APIs that must always be available User-facing prediction paths with strict SLAs

MLME-W08-T09 Serverless Inference

Tier A

Definition and why it matters: Deploy the model as a serverless function. The cloud platform:

Core explanation: Provisions instances automatically Concept flow: Request arrives - Platform spins up\nor reuses instance - Inference - Bill: duration x memory - No traffic - Zero running cost\nafter scale-to-zero Workload shape: Spiky traffic; Why serverless works: Scales to zero between spikes - no idle cost Workload shape: Low average volume; Why serverless works: Cheaper than 24/7 dedicated instances Workload shape: Prototypes & internal tools; Why serverless works: No server management overhead Workload shape: Unpredictable demand; Why serverless works: Platform handles capacity planning Scales active instance count up and down with traffic Bills per request, execution time, and memory allocated Core idea: Pay only when code is actually running - no charge for idle servers After a period of no traffic, the first request incurs extra latency while the platform: Loads the model into memory Initialises the runtime

Must remember:

- Serverless inference: model as a function; platform scales and bills per use
- Best for spiky, low-volume, prototype, and internal workloads
- Cold starts, resource limits, and vendor lock-in are the main trade-offs
- Latency-critical online APIs need careful evaluation of cold-start impact on P95
- At high sustained QPS, dedicated services with autoscaling often outperform serverless economically

Example or contrast: Benefits: No server management; Trade-offs: Cold starts - first request after idle period is slower while platform cold-boots the function Benefits: Built-in autoscaling; Trade-offs: Memory, execution time, and package size limits Benefits: Pay-per-use economics; Trade-offs: Vendor lock-in - provider-specific config and integration

Exam trap: Trap: Using serverless for high-QPS GPU inference without checking limits - memory/time caps may block deployment Trap: Ignoring cold start in SLA planning - P95 measured only on warm requests misleads

MLME-W08-T10 Batching and Micro-Batching for Cost Efficiency

Tier A

Definition and why it matters: Instead of running the model once per incoming request, collect multiple requests and execute them as one batch

Core explanation: Fixed costs of a forward pass (kernel launch, memory setup, framework overhead) are amortised across many inputs - improving hardware utilisation and reducing cost per request Concept flow: Req 1 - Forward pass - Req 2 - Forward pass - Req 3 - Batch tensor\nshape Nx... - Req 4 - Req 5 - Single forward pass Mode: Offline batch; User waiting?: No; Latency tolerance: Hours acceptable; Typical use: Nightly churn scoring, ETL pipelines Mode: Online micro-batching; User waiting?: Yes - briefly; Latency tolerance: Milliseconds of queuing; Typical use: High-QPS GPU APIs Parameter: Max batch size; Effect if increased: Better GPU utilisation; higher per-batch latency; Effect if decreased: Lower utilisation; faster individual batches Parameter: Max wait time; Effect if increased: More requests batched; added queuing delay; Effect if decreased: Lower delay; smaller batches Pros: Much better throughput per instance; Cons: Adds queuing delay - each request may wait ms before processing Pros: Lower cost per request (especially on GPUs); Cons: Requires tuning batch size and wait window

Must remember:

- Batching amortises forward-pass overhead across many inputs - lower cost per request, higher throughput
- Offline batch: no user waiting - maximise batch size and use spot instances
- Micro-batching: brief queuing window online - tune max batch size and max wait time against SLA
- GPU APIs benefit most; tuning must protect P95 latency
- Cost levers (spot, serverless, batching) are often combined based on workload shape

Formula / decision rule: N x C x H x W

Example or contrast: Spot instances + large batches

Exam trap: Trap: Micro-batching without measuring P95 - average latency looks fine while tail blows SLA Trap: Max batch size tuned for throughput alone - wait time pushes UX over threshold

MLME-W08-T11 Reading Constraints: A Decision Framework

Tier B

Definition and why it matters: Production ML systems live in a four-dimensional space: accuracy, latency, cost, and UX. You cannot maximise all four simultaneously - you choose a balance that fits product needs and business constraints

Core explanation: This framework makes that choice systematic. The first step is not picking a scaling pattern or model format. The first step is reading the constraints Context: User waiting in real time; Inference mode: Online request-response; Latency priority: Strict P95 targets (often 100-200 ms) Context: Monthly churn score, nightly backfill; Inference mode: Batch / async; Latency priority: Throughput and cost over per-row latency For every system, answer four question groups:

Must remember:

- First step in production design: read constraints - latency/UX, accuracy/risk, cost, traffic
- Real-time user waiting → online with strict P95; no one waiting → batch/async OK
- Higher mistake cost → accuracy, monitoring, canary deployment
- Traffic shape and budget drive scaling pattern and cost lever selection
- Constraints precede model format, compression, and infrastructure choices

Exam trap: Risk tier: High stakes; Examples: Fraud on payments, medical decisions, safety/legal; Design bias: High accuracy, strong monitoring, canary rollouts, explainability Risk tier: Medium stakes; Examples: Pricing, ranking, promotions; Design bias: Mistakes hurt revenue/trust but not life-critical

MLME-W08-T12 Scenario-Based Deployment Decisions

Tier B

Definition and why it matters: Applying the constraint framework to concrete scenarios clarifies how trade-offs become architecture

Core explanation: Dimension: Accuracy; Requirement: Very high - mistakes lose money or block legitimate users Dimension: Latency; Requirement: Low - user waits for payment to complete Dimension: Availability; Requirement: High - system should rarely be down Dimension: Stakes; Requirement: Very high Concept flow: User checkout - Online inference API\nstrict SLA - Compact but strong model\ndistilled / optimised - Horizontal scaling\nautoscaling on QPS + P95 - GPU or high-end CPU - Aggressive monitoring\nlatency, drift, business metrics Dimension: Real-time user; Requirement: None - no one waiting Dimension: Time window; Requirement: Hours acceptable Dimension: Data volume; Requirement: Very large Compact but strong model - possibly distilled from a larger teacher

Must remember:

- Fraud check: online, high accuracy, horizontal autoscaling, premium hardware, heavy monitoring
- Churn prediction: batch, spot instances, throughput-focused, hours-not-ms SLA
- Mobile edge: quantised/distilled model, size/battery/latency per frame, cloud for training
- Four-step flow: user waiting → mistake risk → traffic/budget → model fit
- Architecture follows constraints - not the reverse

Exam trap: Trap: Running churn scoring as online API - wastes money; batch + spot is appropriate Trap: Deploying FP32 ResNet-18 to mobile without compression - size and battery fail

MLME-W08-T13 From Theory to Practice: Compression Lab Workflow

Tier B

Definition and why it matters: Concept flow: Apply compression\nquantisation or pruning - Benchmark before vs after\nsize, latency, accuracy - Decide deployment fit\nedge vs cloud, batch vs online - Justify trade-offs\nusing constraint framework

Core explanation: The theoretical toolkit for production ML - four-way trade-offs, scaling patterns, cost levers, and decision frameworks - becomes concrete in a compression lab workflow Apply a technique such as dynamic quantisation or pruning to an existing model (e.g. a CNN exported to ONNX from prior standardisation work) Metric: File size (MB); Why it matters: Download size, storage, RAM footprint Metric: Inference latency (avg + P95); Why it matters: User experience, fleet sizing Metric: Accuracy; Why it matters: Validate compression did not break the product

Must remember:

- Lab workflow: compress → benchmark (size, latency, accuracy) → decide deployment fit
- Apply four-way trade-off, scaling, and cost frameworks to justify choices
- Decisions cover edge vs cloud, batch vs online, and risk tier alignment
- Model engineering includes explaining why a setup fits product constraints - not just building models

Exam trap: Trap: Compressing without baseline measurements - cannot quantify trade-offs Trap: Benchmarking size/latency but skipping accuracy on real validation data

MLME-W08-T14 Dynamic Quantisation for Model Compression

Tier A

Definition and why it matters: Standardising a model to ONNX and optimising with ONNX Runtime improves portability and runtime performance. When targets still require smaller and faster models - mobile storage limits, serverless memory caps, edge battery constraints - model compression addresses the model's substance, not just its container format

Core explanation: Typical trigger: baseline model ~45 MB FP32 - too large for on-device download or tight serverless memory Effect: ~4x size reduction; Mechanism: 32 bits - 8 bits per weight Effect: Faster CPU inference; Mechanism: Integer arithmetic is typically faster than FP32 on CPUs Effect: Potential accuracy drop; Mechanism: Less precise weights - small metric degradation (must be measured) Neural network weights are stored as numbers. Standard training uses 32-bit floating point (FP32) - high precision, wide dynamic range, 4 bytes per weight Quantisation asks: do we need full precision at inference time? Can weights be represented with 8-bit integers (INT8) - whole numbers in range approximately -128 to 127? Like saving a high-resolution photo as JPEG: some fine detail is lost, but the main picture remains recognisable. The trade-off is precision for size and speed Before any compression, establish a baseline artefact:

Must remember:

- Compression targets model size/speed when ONNX export alone is insufficient
- Dynamic quantisation: FP32 weights → INT8 - ~4x smaller, faster on CPU, possible small accuracy loss
- Always establish FP32 baseline size before optimising
- ONNX Runtime quantizedynamic with QInt8 is a one-call compression path
- Valid forward pass != production-ready - benchmark size, latency, and accuracy next

Formula / decision rule: -128; 127

Example or contrast: Technique: Dynamic quantisation; What it reduces: Weight precision (FP32-INT8); Typical accuracy impact: Small - often <1 pp

Exam trap: Trap: Skipping baseline measurement - cannot report improvement percentage Trap: Assuming 4x size reduction guarantees 4x speedup - speed depends on hardware and ops still in FP32

MLME-W08-T15

Benchmarking Compression: Size, Latency, and Accuracy

Tier A

Definition and why it matters: A smaller model is useless if it is not actually faster - and dangerous if accuracy dropped unacceptably. Benchmarking quantifies the full trade-off across dimensions that drive deployment decisions

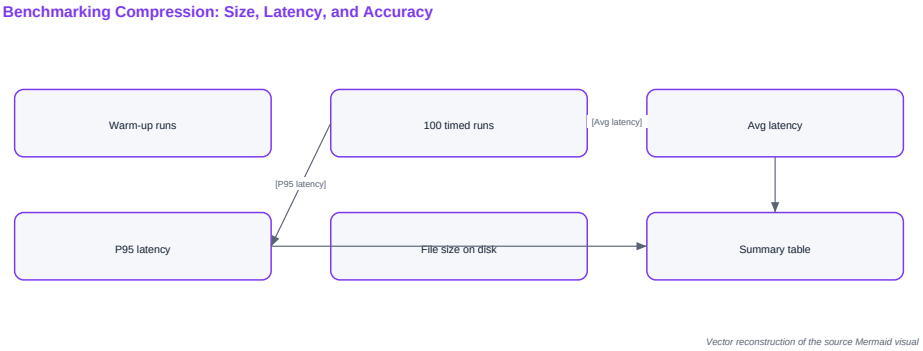
Core explanation: Head-to-head comparison of original FP32 vs compressed INT8 (or pruned) model: Dimension: File size (MB); Measurement: On-disk model artefact; Production relevance: Edge download, RAM, storage cost Dimension: Inference latency; Measurement: Average and P95 over N runs; Production relevance: UX, SLA compliance, fleet sizing Dimension: Accuracy; Measurement: Validation set metrics; Production relevance: Product quality, risk tier acceptance Concept flow: Warm-up runs - 100 timed runs - Avg latency - P95 latency - File size on disk - Summary table Run several inference iterations before timing - excludes one-time model load and cache cold effects from measured runs

Must remember:

- Model variant: FP32 baseline
- Model variant: INT8 quantised
- Size (MB): % reduction; Avg latency (ms): % change; P95 latency (ms): % change; Accuracy (val set): pp change
- All key performance information for deployment decisions in one place
- Benchmark FP32 vs compressed across size, latency (avg + P95), and accuracy
- Use warm-up + repeated runs for stable latency numbers
- Size and latency often show clear INT8 wins on CPU; accuracy requires validation data
- The decision question: is small accuracy loss acceptable for size/speed gains in your use case?
- Production compression workflow always includes accuracy on a representative validation set

Example or contrast: Metric: Size; FP32: ~44.67 MB; INT8: ~11 MB; Change: ~75% reduction (~4x) Metric: Avg latency; FP32: Higher; INT8: Lower; Change: Significant CPU speedup Metric: P95 latency; FP32: Higher; INT8: Lower; Change: Better tail performance

Exam trap: Trap: Benchmarking without warm-up - first-run load time skews averages Trap: Reporting only average latency - P95 is what SLAs and UX use



Concept map: Benchmarking Compression: Size, Latency, and Accuracy

MLME-W08-T16

Deployment Fit: Edge vs Cloud for Compressed Models

Tier B

Definition and why it matters: Choosing which model variant to deploy - and where - is not purely technical. It requires aligning model performance characteristics with business and product goals, using benchmark data and the constraint framework

Core explanation: Choose FP32 when even a tiny fraction of a percentage point in accuracy matters Domain: Medical diagnosis; Why FP32 may win: False negatives/positives have severe consequences Domain: Financial fraud detection; Why FP32 may win: Cost of mistake exceeds infrastructure cost Domain: High-stakes automated decisions; Why FP32 may win: Business value of precision outweighs cloud spend Concept flow: FP32 model - High stakes or\nGPU batch? - Deploy FP32\ncloud GPU or batch - Consider INT8

Must remember:

- FP32: max precision - high-stakes domains, GPU cloud, unconstrained batch jobs
- INT8: 4x smaller, much faster on CPU - edge/mobile, high-QPS cost-sensitive cloud APIs
- Deployment aligns model profile with business goals - not benchmark wins alone
- Always validate accuracy drop against risk tier before deploying compressed models
- Full workflow: identify need → compress → benchmark → decide → justify trade-offs

Exam trap: Trap: Deploying INT8 to fraud/medical without measuring recall/precision drop - unacceptable risk Trap: Choosing FP32 for mobile because "accuracy is king" - 45 MB model may not ship at all

MLME-W08-T17

Module Summary: Production Trade-Offs and Sustainable ML Systems

Tier C

Definition and why it matters: Production ML is framed as competing forces pulling a system in four directions:
Core explanation: Force: Accuracy; Core question: How often is the model correct?
Must remember:

- Production ML = four-way tug-of-war: accuracy, latency, cost, UX
- Scaling (vertical, horizontal, autoscaling) keeps latency stable under load
- Cost levers: spot (batch), serverless (spiky/low), micro-batching (GPU efficiency)
- Decision framework: read constraints → match architecture → fit model and compression
- Lab closes the loop: quantise → benchmark → decide edge/cloud with data-driven justification
- Sustainable production ML balances intelligence with affordability and user experience

Exam trap: Four forces interact - maximising one degrades others Four-question checklist for every production change

Week 8 Exam Lens

Likely questions

- Define and explain Accuracy, Latency, Cost, and User Experience; include the mechanism and one limitation.
- Compare Accuracy, Latency, Cost, and User Experience with The Latency-Cost-UX Triangle and state when each is preferred.
- Apply Vertical Scaling, Horizontal Scaling, and Autoscaling to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: N x C x H x W; -128; 127

Mistakes to avoid: Trap: Treating accuracy as the only success metric - offline leaderboard wins do not guarantee production success Trap: "We improved accuracy, so the release is a success" - ignore latency/UX at your peril Trap: Choosing the offline leaderboard winner without checking P95 latency under production load

Week 9: Production Features and Feature Stores

Week roadmap: Feature Engineering in Production: Module Introduction → Feature Engineering: From Notebook to Production → Training-Serving Skew: Definition, Causes, and Consequences → Offline Features: Batch Computation for Training → Online Features: Low-Latency Serving → What a Feature Store Provides → 12 more topics.

Coverage: 18 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

MLME-W09-T01 Feature Engineering in Production: Module Introduction

Tier C

Definition and why it matters: Feature engineering in a Jupyter notebook feels straightforward: load a CSV, transform columns, train a model, inspect metrics. That workflow assumes a static dataset, a single compute environment, and no serving path. Production ML breaks all three assumptions

Core explanation: The central problem this module addresses is training-serving skew - a silent failure mode where features are computed one way during training and differently during inference. Offline evaluation looks excellent; production performance collapses, with no obvious error in logs

Must remember:

- Notebook feature engineering operates on static snapshots with no serving counterpart
- Production splits feature computation into training (batch) and serving (real-time) worlds
- Training-serving skew occurs when these worlds compute features differently despite sharing names
- Skew silently degrades model quality while offline metrics remain strong
- Two core production constraints: consistency (same semantics) and performance/freshness (latency, precomputation)
- Feature stores are the architectural answer to enforcing a single source of truth
- This module covers theory (skew, offline/online, stores, governance) and labs (pandas + dict simulation)

Exam trap: "Good features = good production model" - Feature quality without consistency across training and serving is worthless Assuming notebook logic ports directly - Batch group-by on a full DataFrame is not equivalent to per-request aggregation

MLME-W09-T02 Feature Engineering: From Notebook to Production

Tier A

Definition and why it matters: Good features are necessary but not sufficient. They must be computed consistently in both training and serving. A brilliantly engineered feature that exists in two incompatible implementations is worse than a simpler feature with a single, shared definition

Core explanation: In exploratory and training workflows, feature engineering covers standard transformations: Category: Numeric scaling; Examples: Standardisation, min-max normalisation, log transforms Category: Categorical encoding; Examples: One-hot encoding, label encoding, target encoding Category: Bucketing; Examples: Age bins, income quartiles, recency tiers Category: Aggregations; Examples: Counts, sums, averages, ratios over a time window Concept flow: Large Historical Dataset - Batch Feature Computation - Millions of Rows at Once - Single Request - One User / Event - Features Needed Now - Trained Model - Training - Serving Operational context: local DataFrame, in-memory, static snapshot. No latency constraint. One code path. The model and features live in the same process This simplicity is deceptive. Teams often assume the notebook logic can be "ported" to production with minor edits. In practice, production introduces an entirely different execution model Works on large historical datasets (months or years of events)

Must remember:

- Notebook FE: scaling, encoding, bucketing, aggregations on static in-memory data
- Production introduces two worlds: batch training vs per-request serving
- Consistency requires identical feature semantics across both worlds
- Performance/freshness requires precomputation, caching, and online infrastructure for serving
- Training-serving skew silently kills model performance despite strong offline metrics
- Root cause: feature logic scattered across notebooks, SQL, and serving code
- Solution direction: single source of truth → feature pipelines → feature stores

Formula / decision rule: P_(train)(x); P_(serve)(x) != P_(train)(x)

Example or contrast: Training implementation: 30-day window, all completed transactions, computed from warehouse history

Exam trap: "We use the same feature name, so we're consistent" - Names do not guarantee semantics; windows, filters, and sources must match Copy-pasting notebook code into serving - Serving needs precomputed lookups, not batch group-by per request

MLME-W09-T03 Training-Serving Skew: Definition, Causes, and Consequences

Tier A

Definition and why it matters: Window: last 30 days of transactions Source: complete transaction history in the data warehouse

Core explanation: Training-serving skew occurs when a model encounters one distribution of features during training and a different distribution during serving - even when feature names, schemas, and logs appear correct Skew exists when P(train)(x) != P(serve)(x) Concept flow: Feature Definition - Same everywhere? - Training-Serving Skew - Great Offline Metrics - Poor Production Performance - Consistent Predictions - Different Code Paths - Different Time Windows - Different Data Sources

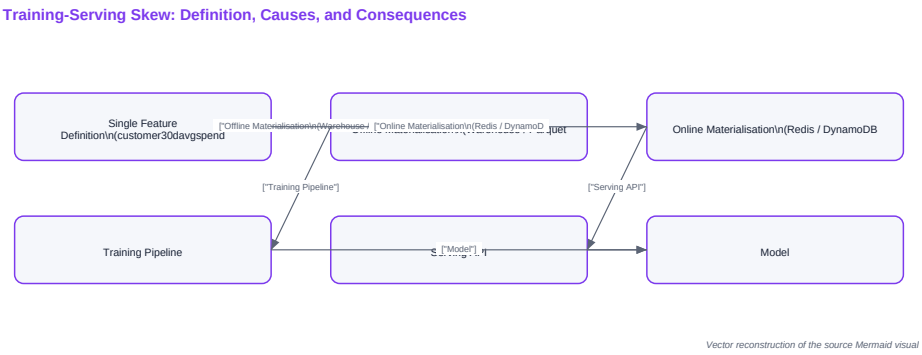
Cause: Time window mismatch; Training: 30-day aggregation; Serving: 7-day aggregation; Result: Different scale and variance Cause: Code path divergence; Training: SQL in warehouse; Serving: Reimplemented Python; Result: Subtle filter differences Cause: Data source split; Training: Batch ETL table; Serving: Live API stream; Result: Missing or extra events Cause: Filter logic; Training: Excludes refunds; Serving: Includes refunds; Result: Systematic value shift Cause: Pipeline drift; Training: Stale training job; Serving: Updated production ETL; Result: Distribution shift without retraining This is among the most dangerous and hardest-to-debug failure modes in production ML

Must remember:

- Training-serving skew: model sees different feature distributions in training vs serving
- Causes: different code, time windows, sources, filters, or unsynchronised ETL changes
- Classic trap: 30-day feature in training, 7-day feature in serving - same name, different semantics
- Consequences: silent production degradation; offline metrics remain misleadingly strong
- Root cause: fragmented, ad hoc feature logic across teams and systems
- Fix: single source of truth per feature, reused for offline and online materialisation
- Feature stores architecturally enforce this pattern at organisational scale

Formula / decision rule: Skew exists when P_(train)(x) != P_(serve)(x); 170 total) | Inactive spender (

Exam trap: Confusing skew with data drift - Drift is natural distribution change over time; skew is a pipeline implementation mismatch from day one "Features are present, so serving is fine" - Presence does not imply correctness of values



Concept map: Training-Serving Skew: Definition, Causes, and Consequences

MLME-W09-T04 Offline Features: Batch Computation for Training

Tier A

Definition and why it matters: The same feature concept operates in two distinct production contexts. Offline features serve the training and batch-scoring world - large historical datasets, heavy computation, throughput-optimised. Online features serve real-time prediction - single entities, millisecond latency, freshness-optimised

Core explanation: Understanding this split is the foundation for feature store architecture and the primary mechanism for avoiding training-serving skew at scale Concept flow: Raw Event Data\n(transactions, clicks, logs - Offline Feature Pipeline - Online Feature Pipeline - Data Lake / Warehouse\n(Parquet, BigQuery, Snowflake - Key-Value Store\n(Redis, DynamoDB - Model Training\n& Batch Scoring - Real-Time Prediction - Shared Feature Definition Offline features are computed over large historical datasets and stored in analytical systems: Data lakes (Parquet, ORC on S3/HDFS) Data warehouses (BigQuery, Snowflake, Redshift) Column: customerid; Description: Entity key Feature tables materialised by batch pipelines Column: timestamp; Description: Event time Primary use cases: Column: amount; Description: Transaction value Training ML models on historical labels Column: category; Description: Optional segmentation Batch scoring jobs (overnight churn predictions, weekly risk reports)

Must remember:

- Offline features: batch-computed, stored in data lake/warehouse, used for training and batch scoring
- Optimised for throughput; pipelines run minutes to hours on billions of rows
- Feature tables: one row per entity per asofdate with aggregated columns
- asofdate enables point-in-time correct joins with labels - critical for avoiding leakage
- Same feature concept as online features, but different storage, latency, and compute patterns
- Feature stores materialise offline features from a single canonical definition
- Example metrics: 30-day total spend, transaction count, average ticket size per customer

Formula / decision rule: training_set = features bowtie_(customer_id, as_of_date) labels; must only use data available **before**

Example or contrast: Dimension: Storage; Offline: Data lake / warehouse; Online: Key-value store / cache

Exam trap: Treating offline tables as always fresh - Batch features reflect the last pipeline run, not live state Skipping asofdate - Without a reference timestamp, point-in-time joins with labels are ambiguous or leaky

MLME-W09-T05 Online Features: Low-Latency Serving

Tier A

Definition and why it matters: Online features are computed or retrieved at request time for a single entity: Property: Latency; Requirement: Few milliseconds per lookup (often 10-20 ms total for all features)

Core explanation: During real-time prediction, a model receives one request at a time - one customer, one session, one item. It cannot wait for a Spark job or warehouse query. Online features are precomputed or cached values retrieved in milliseconds from a low-latency store The online path is where models actually live in production. Offline features build the training dataset; online features fuel every live inference call Concept flow: customerid, context - Client - API - FS - 30dspend, lastactivity, ... - Model Concept flow: Event Stream /\nBatch Updates - Feature Computation\n(shared definition - Online Store\n(Redis / DynamoDB - Serving Lookup\nby entity key Strategy: Scheduled batch push; Freshness: Minutes to hours; Complexity: Simple; nightly sync from offline table

Must remember:

- Online features: precomputed values retrieved per entity at request time in milliseconds
- Stored in low-latency key-value systems (Redis, DynamoDB, Cassandra)
- Serving flow: request → feature lookup by key → optional transforms → model inference
- Materialisation pushes computed values into the online store via batch or streaming jobs
- Optimised for latency and freshness; no heavy per-request joins
- Must share the same feature definition as the offline path to avoid skew
- Offline = throughput for training; online = latency for live prediction

Example or contrast: Dimension: Data source; Offline Features: Data lake / warehouse; Online Features: Key-value store / cache Dimension: Use case; Offline Features: Training, batch scoring; Online Features: Real-time prediction Dimension: Scale; Offline Features: Millions of rows per job; Online Features: One entity per request

Exam trap: Running batch aggregations per request - Defeats the purpose of online features; use precomputation Assuming online store equals source of truth for training - Training should use offline historical tables with point-in-time correctness

MLME-W09-T06 What a Feature Store Provides

Tier A

Definition and why it matters: A feature store is a central system that: Concept flow: Feature Definitions\n(code / YAML - Feature Store - Offline Store\n(Warehouse / Parquet - Online Store\n(Redis / DynamoDB - Registry / Catalogue - gethistoricalfeatures(- getonlinefeatures(- Search, Metadata,\nLineage, Governance - Training Pipeline

Core explanation: The feature store runs pipelines that: Read from source tables or streams Component: Historical features API; Function: Build training datasets with temporal correctness Compute feature values over historical data Component: Online features API; Function: Retrieve features for live inference Write feature tables to the data lake or warehouse Component: Registry / catalogue; Function: Searchable index of all features with metadata Support point-in-time correct joins for training datasets Component: Discovery; Function: Teams find and reuse existing features instead of reinventing API example: gethistoricalfeatures(entitydf, featurerefs, timestamp) returns feature values as they existed at each entity's event time Concept flow: Notebook SQL - Training - Different Python - Serving - Single Definition - Offline Materialisation - Online Materialisation - Training - Serving The feature store: Pushes latest feature values into a low-latency key-value store Aspect: Feature definition; Ad Hoc Pipelines: Scattered across notebooks, SQL, services; Feature Store: Central, versioned registry

Must remember:

- Feature store: central system to define features once and serve them offline and online
- Four responsibilities: definition, offline materialisation, online materialisation, serving API + catalogue
- Core promise: consistent features across training and serving → eliminates skew class
- Offline store for training/batch scoring; online store for real-time prediction
- Registry enables discovery, metadata, and reuse across teams and models
- Implementations: Feast (OSS), Tecton (managed), Hopsworks (ML platform)
- Labs simulate the pattern with pandas + Python dict + shared function

Exam trap: "Feature store = database" - It is a system for definition, materialisation, serving, and governance - not just storage Defining features only in the offline store - Online materialisation must derive from the same definition

MLME-W09-T07 Feature Store Ecosystem: Common Building Blocks

Tier A

Definition and why it matters: Declared in code or configuration. Specify: Entity key - e.g., customerid, userid, itemid

Core explanation: Feature store products differ in branding, hosting model, and integrations - but the core concepts remain remarkably stable. Understanding the shared pattern matters more than memorising any specific vendor This section identifies the universal building blocks, uses Feast as an open-source mental baseline, and positions Tecton and Hopsworks as managed/enterprise implementations of the same architecture Concept flow: 1. Feature Definitions\n(code / YAML - 2. Offline Store\n(Parquet / BigQuery / Snowflake - 3. Online Store\n(Redis / DynamoDB / Cassandra - 4. Registry / Catalogue\n(metadata, search, UI - Training & Batch Scoring - Real-Time Serving - Discovery & Reuse A feature store is a central system that: Defines features once Materialises them into offline stores (training/batch scoring) and online stores (low-latency serving) Exposes APIs plus a registry/catalogue for discovery and reuse Historical feature tables in analytical storage: Parquet files on a data lake BigQuery, Snowflake, or Redshift tables Optimised for batch reads and point-in-time joins Low-latency key-value storage for serving:

Must remember:

- All feature stores share four building blocks: definitions, offline store, online store, registry

- Feast = open-source baseline; Tecton = managed enterprise; Hopsworks = broader ML platform
- Vendor names change; core concepts (define once, materialise twice, serve via API) stay stable
- Evaluate any feature store by asking about definitions, sync, registry, and safe feature addition
- Use cases: skew prevention, feature reuse, online latency, governance
- Pattern recognition matters more than tool expertise for model engineers

Exam trap: Memorising vendor APIs instead of the pattern - Exams test concepts (offline/online, registry, materialisation), not specific SDK calls Assuming all feature stores are fully managed - Feast is self-hosted; operational burden differs significantly

MLME-W09-T08 Feast: Open-Source Feature Store Baseline

Tier A

Definition and why it matters: Features are defined in Python or YAML, specifying entities, feature views, and source tables Concept: Entity; Description: Key type for features (e.g., customerid)

Core explanation: Feast (Feature Store) is an open-source feature store widely used as a starting point and conceptual reference. If you understand the Feast pattern - feature views, entities, offline/online materialisation, serving APIs - most other feature stores feel familiar Feast is self-hosted: your team manages infrastructure, pipelines, and operations. It provides the architectural skeleton without managed pipeline orchestration Concept flow: Feature Definitions\n(Python / YAML - Feast Core - Offline Store\n(Warehouse / Data Lake - Online Store\n(Redis / etc. - Feast Registry - gethistoricalfeatures(- getonlinefeatures(- Training Pipeline - Prediction Service Offline: connects to warehouse or data lake (BigQuery, Snowflake, Parquet on S3) Online: pushes fresh values to Redis or similar key-value stores Create feature views in code or YAML: Specifies entities, time column, source table, and feature schema Run materialisation jobs that: Concept flow: Feature View Definition - Offline Materialisation - Online Materialisation - Training - Serving - Model Read from the data warehouse Compute historical feature tables

Must remember:

- Feast: open-source feature store; mental baseline for understanding all feature stores
- Core concepts: entities, feature views, feature services, data sources, registry
- Workflow: define → materialise offline → materialise online → serve via APIs
- gethistoricalfeatures() for training; getonlinefeatures() for serving
- Same feature definition drives both materialisation paths → prevents skew
- Self-hosted: flexible but requires own pipeline orchestration
- Understanding Feast pattern transfers to Tecton, Hopsworks, and custom platforms

Example or contrast: Aspect: Hosting; Feast: Self-hosted; Tecton / Hopsworks: Managed / platform Aspect: Pipeline orchestration; Feast: Bring your own (Airflow, Spark); Tecton / Hopsworks: Built-in

Exam trap: Strengths: Open-source, no vendor lock-in; Limitations: Self-hosted ops burden Strengths: Clear mental model for feature stores; Limitations: Pipeline orchestration is DIY

MLME-W09-T09 Managed Feature Platforms: Tecton and Hopsworks

Tier B

Definition and why it matters: Tecton and Hopsworks implement the identical feature store building blocks - feature definitions, offline store, online store, registry - but wrap them in managed, enterprise-oriented platforms with richer UI, pipeline automation, and deeper ecosystem integration

Core explanation: Understanding Feast provides the baseline; these platforms show how the pattern scales to multi-team organisations Capability: Pipeline management; Tecton Approach: Managed; teams define features, Tecton runs compute Capability: UI; Tecton Approach: Rich feature discovery and management dashboard Capability: Streaming; Tecton Approach: Deep integration with Kafka, Kinesis, etc

Must remember:

- Tecton: managed enterprise feature platform; runs pipelines, rich UI, deep streaming/warehouse integration
- Hopsworks: ML platform with integrated feature store, notebooks, model registry, orchestration
- Both implement the same four building blocks as Feast with more management and governance
- Differences: hosting model, UI depth, ecosystem integration, scope (feature-only vs full ML platform)
- Core pattern stable across all vendors: define → materialise offline + online → serve via API + registry
- Platform choice depends on team size, governance needs, existing infra, and real-time requirements

Example or contrast: Dimension: Hosting; Feast: Self-hosted OSS; Tecton: Managed SaaS; Hopsworks: Managed platform / self-hosted options

Exam trap: "Tecton and Feast are completely different architectures" - Same pattern; Tecton adds managed ops and enterprise UI Assuming managed = no engineering work - Feature definitions, quality, and semantics still require domain expertise

MLME-W09-T10 The Common Feature Store Pattern: Practical Takeaways

Tier A

Definition and why it matters: Stepping back from Feast, Tecton, and Hopsworks, all feature stores implement the same core workflow:

Core explanation: Concept flow: 1. Feature Definition - 2. Offline Materialisation - 3. Online Materialisation - 4. Registry / UI - Training & Batch Scoring - Real-Time Serving - Discovery & Governance Signal: Repeated training-serving skew incidents;

Why a Feature Store Helps: Enforces single definition across paths Signal: Many teams reimplementing same features; Why a Feature Store Helps: Central registry + reuse eliminates duplication Signal: Growing low-latency online feature demand; Why a Feature Store Helps: Built-in online materialisation and serving API Signal: Multi-model feature sharing; Why a Feature Store Helps: One definition serves churn, fraud, recommendation models Define features in code or configuration Signal: Compliance and access control needs; Why a Feature Store Helps: Governance layer for sensitive features Materialise to offline store for training Materialise to online store for serving Maintain a registry for discovery and management Where tools differ: self-hosted vs managed, ecosystem integrations, UI richness, governance depth. The pattern is constant Regardless of vendor, a model engineer should be able to answer:

Must remember:

- Universal pattern: define → offline materialise → online materialise → registry
- Vendor differences: hosting, integrations, UI, governance - not core architecture
- Model engineers should recognise when feature stores are needed and ask four key questions
- Organisational benefits (reuse, metadata, lineage, governance) extend beyond technical consistency
- Labs simulate the pattern with pandas + dict + shared function
- Adaptability: recognise the pattern in any tool, including custom internal platforms

Example or contrast: Differentiator: Hosting model; Examples: Self-hosted (Feast) vs fully managed (Tecton) Differentiator: Data ecosystem; Examples: BigQuery-native vs Snowflake-native vs lakehouse Differentiator: Serving ecosystem; Examples: SageMaker, Databricks, custom K8s

Exam trap: Treating vendor choice as the core learning objective - Pattern recognition and good questions matter more Assuming managed platforms eliminate feature engineering work - Definitions, semantics, and quality are always human responsibilities

MLME-W09-T11 Feature Reuse: Solving Duplication and Inconsistency

Tier B

Definition and why it matters: Technical consistency (offline/online sync) solves training-serving skew. But without a feature store, organisations face a parallel problem: the same feature is implemented multiple times by different teams, with slightly different logic each time

Core explanation: A feature store is not just infrastructure - it turns features into reusable, governed organisational assets Concept flow: Churn Team\nNotebook SQL - customer30dspend v1\n30-day, excludes refunds - Fraud Team\nSQL Script - customer30dspend v2\n30-day, includes refunds - Recommendation\nPython Code - customer30dspend v3\n28-day, different null handling - Serving Team\nMicroservice - customer30dspend v4\n7-day window (bug - Same name,\ndifferent values,\ndebugging nightmare

Must remember:

- Without feature stores: same feature reimplemented 3-4 times with subtle differences
- Symptoms: duplication, inconsistency, debugging nightmares, slow onboarding
- Feature store solution: define once, register, reuse across models and teams
- Benefits: less duplicated ETL, fewer bugs, faster new use cases, consistent semantics
- Reuse and training-serving consistency both stem from the single-definition principle
- Features become shared organisational assets, not per-project ad hoc scripts

Example or contrast: Concept: Reuse; Meaning: Multiple models consume the same feature definition

Exam trap: "Reuse means all models share all features" - Models select relevant subsets; the catalogue is shared, not monolithic Copying feature SQL into a new notebook = reuse - Reuse requires referencing the canonical definition, not duplicating code

MLME-W09-T12 Metadata and Lineage: Understanding and Tracing Features

Tier B

Definition and why it matters: A feature store is only useful if teams can find, understand, and trust features. Rich metadata transforms opaque columns in pipelines into documented, discoverable assets

Core explanation: Without metadata, teams pass "mystery columns" between pipelines - columns with no description, unknown owner, unclear freshness, and ambiguous semantics Field: Name; Description: Canonical identifier; Example: customer30dtotalspend Field: Description; Description: Human-readable explanation; Example: "Sum of completed transaction amounts in last 30 days" Field: Units; Description: Measurement unit; Example: USD, count, ratio, days Field: Owner; Description: Responsible team or individual; Example: Data Platform / Jane Smith Field: Schema; Description: Data type, allowed values; Example: Float64, non-negative, nulls not expected

Must remember:

- Metadata: name, description, units, owner, schema, freshness, quality, usage per feature
- Enables discovery, trust, onboarding, and debugging
- Lineage: upstream (sources, transformations) and downstream (models, APIs, dashboards)
- Lineage use cases: debugging breakage, impact analysis for schema changes, regulatory audits
- Metadata answers "what is this?"; lineage answers "where from and where to?"
- Feature store is the central place to capture both, automatically from definitions

Exam trap: Confusing metadata with lineage - Metadata describes a feature; lineage describes its relationships in the data graph "We have a data catalogue, so we don't need feature metadata" - Generic catalogues lack ML-specific fields (freshness SLAs, model usage, point-in-time semantics)

MLME-W09-T13 Feature Governance and Lifecycle Management

Tier A

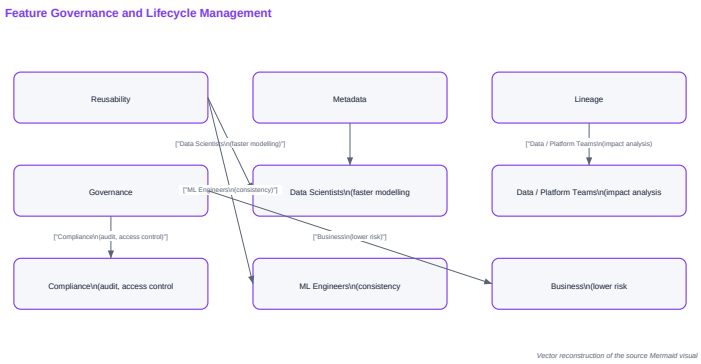
Definition and why it matters: Some features carry legal, ethical, or security risk. A feature store provides governance - policies, access controls, and audit trails - to ensure sensitive features are used safely and features evolve through a managed lifecycle

Core explanation: Category: PII; Examples: Email, phone, home address, national ID; Risk: Privacy violations, GDPR/CCPA breaches Category: Protected attributes; Examples: Race, gender, religion, age; Risk: Discrimination, regulatory bans on use in scoring Category: Financial data; Examples: Credit scores, income, account balances; Risk: PCI/financial regulation, fair lending Category: Health data; Examples: Diagnosis codes, prescriptions; Risk: HIPAA and medical privacy rules Concept flow: Feature Store Governance - Access Control\n(role-based read permissions - Usage Policies\n(feature X only in model Y - Production Blocks\n(banned features - Audit Logging\n(who accessed what, when Using these features in models without proper controls creates legal liability and ethical harm A feature store enforces governance through: Define which roles and teams can read which features Data scientists see general behavioural features; only compliance-approved roles see PII Concept flow: Experimental - Active - Deprecated - Retired Serving systems receive only features authorised for their model Stage: Experimental; Description: Under development; limited access; Example: customer60dspendv2 in testing

Must remember:

- Sensitive features: PII, protected attributes, financial, health data - require special handling
- Governance: access control, usage policies, production blocks, audit logging
- Feature lifecycle: experimental → active → deprecated → retired
- Versioning enables safe rollout, dependency tracking, and gradual migration
- Benefits span data scientists, ML engineers, platform teams, and business
- Feature stores are central to responsible AI: fairness, transparency, accountability, compliance

Exam trap: "Governance = IT security only" - Feature governance includes ML-specific policies (fairness, model-specific usage, lifecycle) No lifecycle management - Without versioning and deprecation, feature stores accumulate dead features that confuse consumers



Concept map: Feature Governance and Lifecycle Management

MLME-W09-T14 From Feature Governance to Hands-On Practice

Tier A

Definition and why it matters: The organisational layer - reusability, metadata, lineage, governance - completes the feature store value proposition. The hands-on labs translate these abstractions into a concrete, debuggable simulation using tools every ML practitioner knows: pandas and Python dictionaries

Core explanation: Concept flow: Raw Transaction Events\n(customerid, timestamp, amount - Shared Function\ncompute30dfeatures(- Offline Store\npandas DataFrame\n(batch feature table - Online Store\nPython dict\n(key-value cache - Training Pipeline - Serving Lookup - Skew Demo\n(intentional 7-day bug - Fix: import shared function

Must remember:

- Topic: Production FE; Key Ideas: Two worlds (training batch vs serving real-time); consistency and performance constraints
- Topic: Training-serving skew; Key Ideas: Different distributions in training vs serving; silent degradation
- Topic: Offline features; Key Ideas: Batch-computed, warehouse-stored, throughput-optimised
- Topic: Online features; Key Ideas: Precomputed, key-value stored, latency-optimised
- Topic: Feature store; Key Ideas: Define once, materialise offline + online, serve via API + registry
- Topic: Ecosystem; Key Ideas: Feast (OSS baseline), Tecton (managed), Hopsworks (ML platform)
- Topic: Organisational; Key Ideas: Reuse, metadata, lineage, governance, lifecycle
- Big idea: feature stores transform feature engineering from ad hoc scripts into a shared, reliable, governed platform
- Labs simulate a feature store: pandas offline table + Python dict online store + shared function
- Lab 1: build offline feature table with 30-day aggregations and asofdate
- Lab 2: encapsulate logic in compute30dfeatures(), materialise to dict, serve via lookup
- Lab 3: demonstrate skew (7-day bug) and fix (shared function) - core learning moment
- Module theory: skew, offline/online, feature stores, ecosystem, governance
- Production tools (Feast, Hopsworks) scale the lab pattern with governance and discovery

Formula / decision rule: 170 spend; online shows

Exam trap: "The lab is trivial, so feature stores are trivial" - The lab isolates the core pattern; production adds scale, streaming, governance, and multi-team coordination Using different functions for offline and online in the lab - Defeats the learning objective; always use the shared function

MLME-W09-T15 Building an Offline Feature Table in Pandas

Tier A

Definition and why it matters: Transform raw event-level transaction data into a clean, aggregated offline feature table suitable for model training. This is the batch-processing path - optimised for throughput, not latency

Core explanation: ML models cannot consume raw events directly. Starting data resembles warehouse event tables: Column: customerid; Type: string; Description: Entity key Column: timestamp; Type: datetime; Description: Event time Column: amount; Type: float; Description: Transaction value Column: category; Type: string; Description: Transaction category (optional) Feature: customer30dtotalspend; Computation: Sum of amount Feature: customer30dtxncount; Computation: Count of transactions Feature: customer30davgticket; Computation: totalspend / txncount This is event-level granularity - one row per transaction. The model needs entity-level features - one row per customer For each customer, compute behavioural features over the last 30 days: Concept flow: All Transactions - Filter: timestamp = asofdate - 30d - Group By: customerid - Aggregate: sum, count - Derive: avgticket = spend / count - Feature Table\n+ asofdate column Production feature tables must be point-in-time correct. The lab simulates this: Find the latest timestamp in the dataset → this becomes the asofdate Define the window: [asofdate - 30 days, asofdate] Filter transactions within this window Aggregate per customerid customerid: C001; customer30dtotalspend: 170.00; customer30dtxncount: 4; customer30davgticket: 42.50; asofdate: 2025-06-01 customerid: C002; customer30dtotalspend: 320.00; customer30dtxncount: 8; customer30davgticket: 40.00; asofdate: 2025-06-01

Must remember:

- Raw events (customerid, timestamp, amount) → aggregated entity-level features
- 30-day window: latest timestamp as asofdate, lookback 30 days, filter, groupby, aggregate
- Features: total spend (sum), txn count (count), avg ticket (spend/count)
- Output: one row per customer with asofdate for traceability
- Join with labels on customerid for training
- Batch processing: throughput-optimised, not latency-optimised
- This is offline materialisation - the training-side of a feature store

Formula / decision rule: training_data = feature_table bowtie_(customer_id) labels

Exam trap: Forgetting the time window filter - Aggregating all data without a window is not a "30-day" feature Missing asofdate - Without it, point-in-time joins with labels are ambiguous

MLME-W09-T16 Simulating an Online Feature Store with a Dict Cache

Tier A

Definition and why it matters: An offline feature table works for training but cannot serve real-time predictions. When a live request arrives for a single customer, running a batch groupby over the entire warehouse per request is infeasible

Core explanation: Online feature stores solve this with precomputed, key-based lookups in milliseconds Concept flow: compute30dfeatures(- For each customerid - onlinestore[id - Python Dict\n(in-memory KV store The critical move toward consistency is extracting feature computation into a reusable function - the single source of truth: Lab Implementation: Python dictionary; Production Equivalent: Redis, DynamoDB, Cassandra Lab Implementation: In-process loop; Production Equivalent: Distributed streaming/batch materialisation job Contains the exact same logic as the offline batch script Lab Implementation: Single machine; Production Equivalent: Horizontally scaled cache cluster Parameterized for a single customer at a specific asoftime Returns a dictionary of feature values This function is the single source of truth Concept flow: dict - customerid="C001 - Request - API - C001 - Store - Model

Must remember:

- Online features: precomputed, retrieved by entity key in milliseconds
- Step 1: encapsulate logic in compute30dfeatures() - single source of truth
- Step 2: materialise - iterate customers, compute, store in dict keyed by customerid
- Step 3: serve - getonlinefeatures(customerid) is O(1) dict lookup
- Python dict simulates Redis/DynamoDB; materialisation simulates streaming/batch push
- Shared function used by both offline and online paths → consistency guaranteed
- Next lab: intentionally break consistency (skew), then fix with shared function

Example or contrast: Aspect: Storage; Offline (Lab 1): pandas DataFrame; Online (Lab 2): Python dict Aspect: Compute; Offline (Lab 1): Batch groupby (all customers); Online (Lab 2): Per-customer via shared function

Exam trap: Reimplementing logic for online instead of reusing the function - The most common source of skew Running batch groupby per request - Defeats the purpose of an online store; use precomputed lookup

MLME-W09-T17 Avoiding Training-Serving Skew with a Tiny Feature Store

Tier A

Definition and why it matters: Training-serving skew is subtle, silent, and devastating. This lab creates the bug intentionally, observes its impact, and fixes it using the feature store pattern - a shared function as single source of truth

Core explanation: Load the offline feature table for customer C001 - these are the values the model was trained on: Feature: customer30dtotalspend; Offline Value (30-day): 170.0 Feature: customer30dtxncount; Offline Value (30-day): 4

Feature: customer30davgticket; Offline Value (30-day): 42.5 Feature: customer30dtotalspend; Offline (30-day): 170.0; Online Buggy (7-day): 0.0; Match?: No Feature: customer30dtxncount; Offline (30-day): 4; Online Buggy (7-day): 0; Match?: No Concept flow: Model trained on:\n30-day spend = 170\n4 transactions\nactive customer - Serving delivers:\n7-day spend = 0\n0 transactions\ninactive customer - Training-Serving Skew - Unreliable predictions\nNo errors in logs These represent the ground truth - what the model learned to expect A separate team responsible for the online prediction service reimplements the feature logic from scratch. They introduce compute7dfeaturesbug(): Mistake: 7-day window hardcoded instead of 30-day. A common, easy error under time pressure Customer C001 had transactions in the 30-day window but none in the last 7 days Feature: customer30dtotalspend; Offline (30-day): 170.0; Online Fixed (shared fn): 170.0; Match?: Yes

Must remember:

- Skew demo: offline 30-day features (spend=170) vs buggy online 7-day features (spend=0)
- Customer appears active in training, inactive in serving - predictions unreliable
- No errors in logs; skew is silent and dangerous
- Fix: import and use compute30dfeatures() - same function for both paths
- Fixed values match offline perfectly; skew eliminated
- Feature store promise: define once, materialise offline + online, architecturally prevent skew
- Production tools (Feast, Hopsworks) scale this pattern with governance and discovery

Exam trap: "The feature name is the same, so values should match" - Names do not enforce semantics; only shared logic does Fixing skew by retraining - Serving pipeline must be fixed first; retraining alone is insufficient

MLME-W09-T18

Module 9 Summary: Feature Engineering in Production

Tier A

Definition and why it matters: This module traced the journey from notebook feature engineering to production-grade feature infrastructure - covering the problems that emerge, the architectural solutions, and hands-on demonstration

Core explanation: Concept flow: Notebook FE\n(static, single path - Production Constraints\n(consistency + performance - Training-Serving Skew\n(silent degradation - Offline vs Online Features\n(dual compute patterns - Feature Store\n(define once, serve twice - Ecosystem\n(Feast, Tecton, Hopsworks - Organisation\n(reuse, metadata, lineage, governance - Labs\n(pandas + dict + shared function : Data; Training: Large historical datasets; Serving: Single entity per request : Compute; Training: Batch aggregations; Serving: Precomputed lookups

Must remember:

- Lab: Offline table (pandas); What It Demonstrates: Batch 30-day aggregation - feature table with asofdate
- Lab: Online store (dict); What It Demonstrates: Shared compute30dfeatures() - materialise to dict - O(1) lookup
- Lab: Skew demo (skew.py); What It Demonstrates: 7-day bug - values mismatch - fix with shared function - values match
- The labs prove that a shared Python function eliminates skew - production feature stores enforce the same principle at scale with governance and discovery
- Production FE has two worlds: batch training vs real-time serving
- Training-serving skew: different feature distributions; silent, dangerous, hard to debug
- Offline features: warehouse, batch, throughput; online features: KV store, lookup, latency
- Feature store: define once, materialise offline + online, serve via API + registry
- Ecosystem: Feast (OSS baseline), Tecton (managed), Hopsworks (ML platform) - same four building blocks
- Organisational: reuse, metadata, lineage, governance, lifecycle versioning
- Labs: pandas offline + dict online + shared function → skew demo and fix
- Big idea: feature stores turn ad hoc scripts into a shared, governed ML platform

Formula / decision rule: Define once → Materialise offline + Materialise online → Serve consistently

Exam trap: "Good features = good production model" - Consistency across training and serving is equally critical Confusing skew with data drift - Skew is pipeline mismatch; drift is temporal distribution change

Week 9 Exam Lens

Likely questions

- Define and explain Feature Engineering: From Notebook to Production; include the mechanism and one limitation.
- Compare Feature Engineering: From Notebook to Production with Training-Serving Skew: Definition, Causes, and Consequences and state when each is preferred.
- Apply Offline Features: Batch Computation for Training to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: P_(train)(x); P_(serve)(x) != P_(train)(x); Skew exists when P_(train)(x) != P_(serve)(x); 170 total) | Inactive spender (; training_set = features bowtie_(customer_id, as_of_date) labels; must only use data available **before**; 170 spend; online shows; training_data = feature_table bowtie_(customer_id) labels

Mistakes to avoid: "Good features = good production model" - Feature quality without consistency across training and serving is worthless "We use the same feature name, so we're consistent" - Names do not guarantee semantics; windows, filters, and sources must match Confusing skew with data drift - Drift is natural distribution change over time; skew is a pipeline implementation mismatch from day one

Week 10: Data Engineering and Real-Time ML Pipelines

Week roadmap: Data Pipelines for Machine Learning: Module Introduction → Why Machine Learning Needs Data Pipelines → ETL and ELT: Foundations of ML Data Pipelines → Pipeline Types: Batch, Micro-Batch, and Streaming → Batch Ingestion for Machine Learning → Micro-Batch Ingestion: Concept and Architecture → 12 more topics.

Coverage: 18 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

MLME-W10-T01 Data Pipelines for Machine Learning: Module Introduction

Tier C

Definition and why it matters: Production machine learning is not only about model architecture, serving, monitoring, and retraining. Every stage depends on a layer beneath it: data plumbing - the automated systems that extract, transform, load, and deliver data to training, validation, and inference

Core explanation: The governing principle is simple:

Must remember:

- Data pipelines are the circulatory system of production ML - they feed training, validation, inference, monitoring, and retraining
- Core principle: no data → no model; bad data → bad model, regardless of model sophistication
- Three ingestion modes: batch (scheduled large chunks), micro-batch (frequent small chunks), streaming (continuous events)
- Production pipelines must be automated, repeatable, and observable - notebooks are insufficient
- Pipeline failures cause delayed training, lying validation metrics, and broken production decisions
- Mode selection trades latency vs complexity vs cost - start simple, escalate when freshness requirements are clear
- This module connects data plumbing to prior work on serving, monitoring, feature stores, and retraining

Exam trap: Treating data pipelines as optional - models cannot self-heal from upstream data failures; pipeline outages cascade into monitoring and retraining blindness Confusing ingestion mode with inference mode - batch ingestion can still feed online APIs if features are precomputed; streaming ingestion is not required for every real-time use case

MLME-W10-T02 Why Machine Learning Needs Data Pipelines

Tier A

Definition and why it matters: Manually load a CSV or run a one-off SQL query

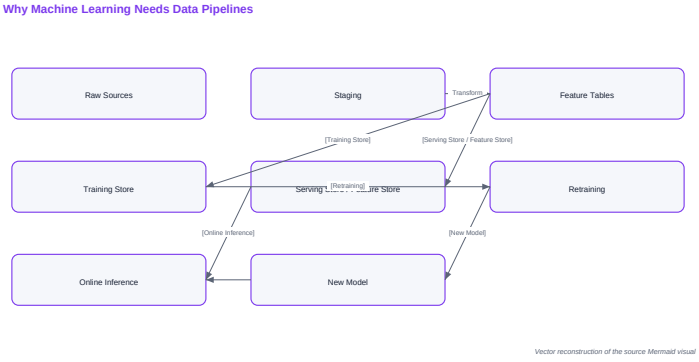
Core explanation: Ad hoc cleaning, joins, and transformations in arbitrary order Concept flow: Manual CSV Load - Ad Hoc Cleaning - Train Once - Scheduled Extract - Automated Transform - Validated Load - Train / Score / Monitor - Alert on Failure Stage: Training; What Pipelines Provide: Historical labelled data, feature tables; Consequence of Pipeline Failure: Model cannot be built or updated Run cells until output looks acceptable Stage: Validation; What Pipelines Provide: Fresh or held-out evaluation data; Consequence of Pipeline Failure: Metrics become unreliable or stale No scheduling, no alerts, no repeatability guarantees Stage: Inference; What Pipelines Provide: Live features and input schemas; Consequence of Pipeline Failure: Predictions wrong or unavailable Data must arrive every day, every hour, or every second Every step is automated, repeatable, and observable

Must remember:

- Notebooks are exploratory and manual; production pipelines are automated, scheduled, and observable
- Pipelines feed training, validation, and inference - failures cascade across the ML lifecycle
- Late data delays retraining; wrong data makes validation metrics lie and breaks production decisions
- "Run the notebook again" lacks scheduling, idempotency, state, lineage, and alerting
- The notebook-to-pipeline shift is one of the biggest transitions in operationalising ML
- Real production systems require incremental ingestion, contracts, and recovery - not one-shot scripts
- Data pipelines are infrastructure, not optional tooling around the model

Example or contrast: The gap between exploratory analysis and production ML is one of the largest shifts when turning a model into a product

Exam trap: Believing automation alone is enough - a pipeline that runs daily but ingests wrong data is worse than a manual process that catches errors Skipping observability until production breaks - without event-count baselines and freshness metrics, failures are discovered only when business KPIs drop



Vector reconstruction of the source Mermaid visual

Concept map: Why Machine Learning Needs Data Pipelines

MLME-W10-T03 ETL and ELT: Foundations of ML Data Pipelines

Tier A

Definition and why it matters: Before features, models, or serving APIs exist, raw data must move from operational systems into an analytics-ready form. Two classic patterns dominate: ETL and ELT. Both produce the clean, structured tables that downstream ML feature engineering consumes

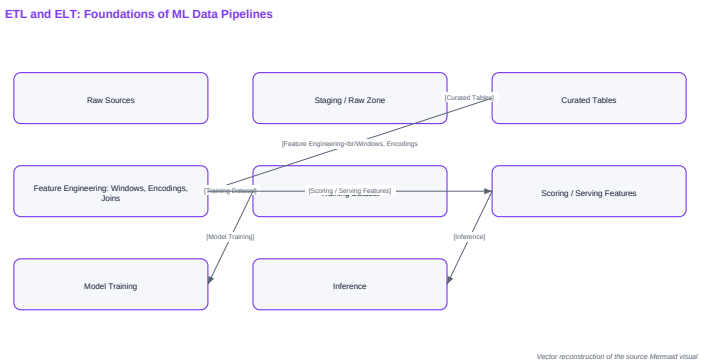
Core explanation: In ETL, transformation happens before data lands in the warehouse: Concept flow: Operational Systems: DBs, Logs, APIs - Transform Engine: Clean, Join, Aggregate - Data Warehouse / Lake: Curated Tables - ML Feature Engineering Concept flow: Source Systems - Raw Landing Zone: Warehouse / Lake - Curated Tables / Views - ML Feature Engineering Extract - pull data from source systems (databases, log files, REST APIs, message queues) Transform - clean, deduplicate, join, aggregate, enforce types and business rules

Must remember:

- ETL: extract → transform (external) → load curated tables into warehouse
- ELT: extract → load raw → transform inside warehouse (SQL/dbt)
- Both patterns produce structured tables that feed ML feature engineering, training, and scoring
- ETL suits complex heterogeneous transforms; ELT suits SQL-first teams with cheap warehouse storage
- Raw data retention in ELT enables cheap reprocessing when transform logic changes
- ML pipelines sit downstream of ETL/ELT - they do not replace data movement patterns
- Production systems often use a hybrid of ETL and ELT for different data sources
- Identical transform semantics across training and serving paths are mandatory for model reliability

Example or contrast: Dimension: Transform location; ETL: External engine (Spark, Python); ELT: Inside warehouse (SQL, dbt) Dimension: Raw data retention; ETL: Often discarded post-transform; ELT: Typically retained in landing zone Dimension: Schema enforcement; ETL: At transform time; ELT: At query/model time

Exam trap: Assuming ETL and ELT are mutually exclusive - modern platforms combine both; the question is where each transform runs Skipping the curated layer - training directly on raw landing-zone data invites schema drift and silent type errors



Concept map: ETL and ELT: Foundations of ML Data Pipelines

MLME-W10-T04 Pipeline Types: Batch, Micro-Batch, and Streaming

Tier A

Definition and why it matters: Machine learning data pipelines differ primarily in how often and how granularly data is processed. Three ingestion modes cover the spectrum from nightly retraining to sub-second fraud detection Concept flow: Daily / Hourly Job - Full Time Window - Every 1-5 min - Small Time Window - Per Event - Continuous Processing - ML System - Batch - MicroBatch

Core explanation: Definition: Process large chunks of data on a fixed schedule - once per day, once per hour, etc Operates on files or table partitions representing a complete time window Each run is a discrete, versionable batch - lineage is straightforward Jobs may take minutes to hours; that is acceptable for many ML workflows Definition: Process many small batches frequently - every 1 to 5 minutes Feels "almost streaming" but implemented as repeated small batch jobs Can use Spark Structured Streaming (micro-batch mode), Flink, Beam, or even a cron-triggered script

Must remember:

- Three ingestion modes: batch (large scheduled chunks), micro-batch (frequent small chunks), streaming (continuous per-event)
- Batch is the workhorse for retraining, offline scoring, and heavy aggregations - simple but stale between runs
- Micro-batch is the middle ground - minute-level freshness using familiar batch tools
- Streaming uses producers, topics, consumers, windows, and state for sub-second ML use cases
- Selection depends on latency, complexity, budget, and volume - not model architecture
- Start with batch, escalate to micro-batch, then streaming only with clear requirements
- All modes link to the ML lifecycle: training, validation, inference, monitoring, and retraining
- Hybrid systems commonly use different modes for different pipeline stages

Example or contrast: Use Case: Regular retraining; Example: Weekly churn model update with new labelled data Use Case: Offline scoring; Example: Nightly risk scores for all 10M users Use Case: Heavy aggregations; Example: 30-day and 90-day rolling statistics

Exam trap: Advantage: Simple mental model; Disadvantage: Data is stale between runs Advantage: Easy lineage and versioning; Disadvantage: Events from the last few hours are missing until next run

MLME-W10-T05 Batch Ingestion for Machine Learning

Tier A

Definition and why it matters: Batch ingestion processes large chunks of data on a fixed schedule - once per day at 1:00 AM, every hour for the past five hours, etc. Each job reads raw data for a specific time window, cleans and joins it, computes aggregations, and writes results into a data warehouse or feature table This is the default pattern for most ML systems. Teams typically implement batch or micro-batch pipelines long before adopting full streaming

Core explanation: Concept flow: Raw Data: Partitioned by Date - Clean & Deduplicate - Join Reference Tables: Users, Products - Compute Aggregations: Features - Write Feature Table: Warehouse / Lake - Append to Master: Training Dataset ML Activity: Regular retraining; Why Batch Works: Weekly churn model update with newly labelled data ML Activity: Offline scoring; Why Batch Works: Monthly risk scores or nightly recommendations for all users ML Activity: Heavy aggregations; Why Batch Works: 30-day and 90-day rolling statistics, complex multi-table joins

Must remember:

- Batch ingestion runs on a fixed schedule, processing a specific time window per job
- Ideal for retraining, offline scoring, and heavy aggregations where minute-level freshness is unnecessary
- Typical flow: read partitioned raw data → clean → join → aggregate → write feature table → append to master training set
- Jobs taking minutes to hours is normal for daily ML workflows
- Each run produces a versioned, lineage-friendly artifact
- Batch is the most common starting point for ML data pipelines before micro-batch or streaming
- Key limitation: staleness between scheduled runs
- Reliable batch jobs are the foundation of retraining pipelines and drift monitoring

Formula / decision rule: sum(spend)_(30d); count(clicks)_(7d)

Example or contrast: Step: Schedule; Action: Daily at 10:00 AM Step: Read; Action: Yesterday's events from partition events/2024/09/01 Step: Join; Action: Reference tables: users, products

Exam trap: Advantage: Simple mental model; Limitation: Features stale between runs Advantage: Easy versioning and lineage; Limitation: Misses recent events until next job

MLME-W10-T06 Micro-Batch Ingestion: Concept and Architecture

Tier A

Definition and why it matters: Micro-batch ingestion is still batching - but the batches are smaller and run much more frequently (every 1 to 5 minutes). Conceptually it feels like almost-streaming, but it is implemented as many small, scheduled batch jobs Concept flow: 0:00 Batch 1 - 0:05 Batch 2 - 0:10 Batch 3 - 0:15 Batch 4 - Feature DB / Cache

Core explanation: Property: Window size; Batch: Hours to days; Micro-Batch: 1-5 minutes; Streaming: Per event Property: Run frequency; Batch: Daily / hourly; Micro-Batch: Every 1-5 min; Streaming: Continuous Property: Implementation; Batch: Cron + Spark/SQL job; Micro-Batch: Spark Structured Streaming, Flink micro-batch, cron script; Streaming: Kafka + Flink/Beam Property: Latency; Batch: High; Micro-Batch: Medium (minutes); Streaming: Low (seconds) Property: Complexity; Batch: Low; Micro-Batch: Medium; Streaming: High Concept flow: Cron: Every 5 min - Read last 5 min of events - Compute features - Write to DB / Feature Store - Downstream services read latest scores Use Case: User session features; Refresh Interval: Every 2 minutes; Feature Examples: sessionduration, pagesviewedsession

Must remember:

- Micro-batch = small batches, frequent runs (every 1-5 minutes) - a compromise between batch and streaming
- Implemented via Spark Structured Streaming, Flink, Beam, or simple cron scripts
- Best for features needing minute-level freshness but not per-event updates
- Example: risk scores refreshed every 5 minutes from recent transactions
- Lower latency than batch; lower complexity than true streaming
- Trade-off: more infrastructure overhead and up to one window interval of staleness
- Reuses familiar batch tooling - a practical escalation path from daily batch
- Not equivalent to true streaming; Spark's default mode is micro-batch under the hood

Example or contrast: Spark Structured Streaming's default mode is micro-batch: it processes small chunks of data at each trigger interval. This is not the same as record-by-record processing

Exam trap: Calling micro-batch "streaming" - it is batching with a short interval; exam questions may distinguish these explicitly Choosing micro-batch when daily batch suffices - unnecessary infrastructure cost if features only change meaningfully over days

MLME-W10-T07 Comparing Batch and Micro-Batch Ingestion

Tier B

Definition and why it matters: One nightly Spark job processing 24 hours of data

Core explanation: Cluster spins up, runs for 2 hours, shuts down

Must remember:

- Batch: higher latency (hours/day), lower complexity, fewer large jobs, ideal for retraining and offline scoring
- Micro-batch: lower latency (minutes), higher complexity, many small jobs, ideal for near-real-time features
- Cost: batch = fewer big jobs; micro-batch = more runs with cumulative overhead
- Rule of thumb: start with batch; escalate to micro-batch only when freshness requirements demand it
- Batch excels at heavy aggregations and score-all jobs; micro-batch excels at periodic feature refresh
- A well-designed batch or micro-batch pipeline is already a major production win
- Lab exercise: daily CSV simulation → incremental ingestion → training hook demonstrates the batch pattern concretely
- Real systems commonly use hybrid modes across different pipeline stages

Example or contrast: Batch and micro-batch are the two most common ingestion modes for ML training and feature computation. Understanding their trade-offs is essential for architectural decisions Dimension: Latency; Batch: Hours to a full day; Micro-Batch: 1-5 minutes Dimension: Complexity; Batch: Simple - fewer moving parts; Micro-Batch: More complex - frequent runs, more monitoring

Exam trap: Escalating to micro-batch without a freshness requirement - adds cost and operational burden with no business benefit Assuming micro-batch is "free" compared to streaming - 288 daily runs require monitoring, alerting, and idempotency guarantees

MLME-W10-T08 Event Streams: Foundations of Streaming for ML

Tier A

Definition and why it matters: An event is a small record describing something that happened: Field: What; Description: Event type; Example: click, purchase, sensorreading

Core explanation: Batch and micro-batch pipelines treat data as files or table partitions - yesterday's data, last hour's data. Streaming moves to a continuous flow of events, asking: what is happening right now? Concept flow: events2024-09-01.csv - events2024-09-02.csv - click @ 10:01 - purchase @ 10:02 - click @ 10:03 Topic Name: clickevents; Events Contained: Page views, button clicks Topic Name: transactions; Events Contained: Payment approvals, refunds Topic Name: sensorreadings; Events Contained: IoT temperature, pressure data Topic Name: applicationlogs; Events Contained: Error events, latency measurements A named channel or stream of events. Topics group related events: Concept flow: Web Server - Topic: clickevents - Mobile App - Feature Job - Analytics Dashboard - Monitoring

Must remember:

- Streaming processes continuous event flows rather than files or partitions
- An event records what happened, when, who/what was involved, and details
- Topics are named channels; producers publish; consumers subscribe and process
- Architecture is many-to-many - multiple producers and consumers per topic
- Streaming asks "what is happening now?" vs batch's "what happened in this window?"
- Events power real-time ML: fraud detection, recommendations, anomaly detection, monitoring
- Event schema, timestamps, and ordering are foundational design decisions
- Topics decouple producers from consumers - a core scalability pattern

Example or contrast: Code pattern: User 123 clicked Product A at 10:01 AM; Sensor 45 reported 25C at 9:00 AM; Payment 987 was approved for USD 52 A stream is a sequence of these events ordered (or partially ordered) over time

Exam trap: Confusing event time with processing time - a click at 10:00 AM may arrive at 10:05 AM; windowed features must specify which timestamp they use Assuming global ordering - events from different users/partitions may arrive out of order; per-key ordering is the typical guarantee

MLME-W10-T09 Kafka, Spark, Flink, and Beam: Roles in Streaming ML

Tier A

Definition and why it matters: The streaming ecosystem contains many names - Kafka, Spark Streaming, Flink, Beam. The key insight is to separate event transport from event processing

Core explanation: Concept flow: Topics / Partitions / Offsets: Durable Event Log - Spark Streaming - Flink - Beam - Producers - Databases / Feature Stores / Dashboards Responsibility: Topics; Description: Named channels partitioning the event space Responsibility: Partitions; Description: Parallelism units within a topic; ordering guaranteed per partition Responsibility: Offsets; Description: Position markers - consumers track how far they have read Responsibility: Durability; Description: Events persisted to disk; configurable retention (hours to forever) Responsibility: Replay; Description: Consumers can re-read from any offset Apache Kafka primarily handles event transport and storage:

Must remember:

- Component: Kafka; Role: Durable event log, transport, replay; Analogy: Highway
- Component: Spark Streaming; Role: Micro-batch processing, large-scale transforms; Analogy: Heavy truck (batch-like trips)
- Component: Flink; Role: Low-latency record-by-record processing; Analogy: Sports car (per-event speed)
- Component: Beam; Role: Portable pipeline abstraction; Analogy: Universal adapter
- Component: Feature Store; Role: Serves computed features to training and inference; Analogy: Destination warehouse
- Kafka = event transport and durable log (topics, partitions, offsets) - the highway
- Spark, Flink, Beam = processing engines that transform events into features - the vehicles
- Three key concepts: windows (tumbling, sliding, session), aggregations (per-key statistics), state (running values)
- Windows enable ML features like clickslast10m and avgspendlasthour
- Spark suits micro-batch and unified batch/stream; Flink suits low-latency true streaming
- Beam provides portable pipeline abstraction across runtimes

- End-to-end flow: produce → Kafka → process → feature store → inference API
- Do not conflate transport (Kafka) with computation (Flink/Spark/Beam)

Formula / decision rule: count(events); sum(amount)

Exam trap: Using Kafka as a database - it is a log with retention limits, not a query engine; use a processing engine or sink for reads Confusing Spark Structured Streaming with true streaming - default mode is micro-batch; latency floor is the trigger interval

MLME-W10-T10 Streaming Machine Learning: Use Cases and Architecture

Tier A

Definition and why it matters: Streaming is not an end in itself - it earns its complexity when ML systems must react within seconds to evolving data

Core explanation: Use Case: Real-time anomaly detection; Why Streaming: Unusual traffic, suspicious transactions, system failures; Latency Requirement: Seconds Use Case: Live recommendations; Why Streaming: React to clicks and views in active session; Latency Requirement: Seconds to minutes Use Case: Dynamic pricing / bidding; Why Streaming: Respond to market data and demand signals; Latency Requirement: Sub-second to seconds Use Case: Real-time fraud checks; Why Streaming: Block fraudulent transactions before settlement; Latency Requirement: Sub-second Use Case: Observability / ML monitoring; Why Streaming: Stream predictions and metrics for live alerting; Latency Requirement: Seconds Concept flow: U - API - FS - stream-updated - M Concept flow: Events - Streaming Job - Model Inference - Predictions Topic - Alerts - Monitoring Pattern: Online inference; Trigger: HTTP/gRPC request; Output: Synchronous response; Example: Fraud API per transaction Pattern: Streaming inference; Trigger: Event arrival; Output: Async prediction/alert; Example: Anomaly alert on metric stream

Must remember:

- Streaming ML excels at anomaly detection, live recommendations, dynamic pricing, fraud, and real-time monitoring
- Two inference patterns: online (API per request) and streaming (event-driven, predict-on-stream)
- Streaming touches features, serving, and monitoring - not just one lifecycle stage
- Micro-batch suffices for minute-level freshness; full streaming for sub-second requirements
- Decision rule: start simple (batch/micro-batch), escalate to streaming only with clear latency or volume needs
- Streaming can become the backbone for real-time data serving and monitoring in ML systems
- Training typically remains batch; streaming primarily feeds inference and observability paths

Example or contrast: Criterion: Freshness need; Micro-Batch: Minute-level (1-5 min); Full Streaming: Second or sub-second

Exam trap: Adopting streaming without a latency requirement - operational cost and complexity rarely justify streaming for daily-retrained models Confusing streaming features with streaming inference - features can be stream-computed while inference remains request-driven (online API)

MLME-W10-T11 Data Freshness and Latency in Real-Time ML

Tier A

Definition and why it matters: Real-time ML systems rarely crash when data goes bad. They quietly make worse decisions - predictions based on stale features, missing events, or corrupted values degrade performance without obvious errors. Data quality must be reasoned about explicitly

Core explanation: A critical distinction: inference latency and data freshness latency are independent dimensions Type: Inference latency; Definition: Time from receiving a request to returning a prediction; Example: API responds in 50 ms Type: Data freshness latency; Definition: Time from a real-world event to that event appearing in features; Example: Transaction at 10:00 AM visible in features at 10:05 AM (5-min micro-batch) Concept flow: RW - Pipe - FS - User - API Feature Category: User profile features; Example SLA: Updated within 10 minutes; Rationale: Slow-changing attributes; micro-batch sufficient Feature Category: Fraud-related features; Example SLA: Include events from last 30 seconds; Rationale: Sub-minute freshness critical for blocking fraud

Must remember:

- Real-time ML systems fail quietly with bad data - they do not crash, they make worse decisions
- Inference latency (request → prediction) and data freshness latency (event → feature) are independent - both must be managed
- Define freshness SLAs per feature category, not just prediction latency SLAs
- Measure: average event-to-feature lag and SLA compliance percentage
- Fresher data costs more - choose SLAs per use case, not one-size-fits-all
- Match ingestion mode to freshness tier: streaming (< 30s), micro-batch (1-5 min), batch (hourly/daily)
- A fast model on stale features produces unreliable production decisions
- Freshness monitoring is as critical as model performance monitoring

Formula / decision rule: freshness_lag = t_(feature_updated) - t_(event_occurred); SLA compliance = (features meeting SLA)/(total features evaluated) x 100%

Example or contrast: Fresher data costs more:

Exam trap: Optimising inference latency while ignoring freshness - a 10 ms API on 2-hour-old features is worse than a 100 ms API on 30-second-old features for fraud One global freshness SLA - different features have different decay rates; uniform SLAs waste money or miss risks

MLME-W10-T12 Data Completeness and Correctness for ML Pipelines

Tier A

Definition and why it matters: Freshness is one dimension of data quality. Completeness (is all expected data present?) and correctness (is the data valid and accurate?) are equally critical. Issues in these dimensions rarely crash code - they silently bias features and degrade model performance

Core explanation: Issue: Missing events; Cause: Upstream system drops messages, pipeline failure, network partition; Impact on ML: Features undercount activity; model sees inactive users as inactive Issue: Duplicates; Cause: Retries, at-least-once delivery semantics; Impact on ML: Features overcount; inflated spend, click counts Issue: Out-of-order events; Cause: Network delays, distributed clocks, late arrivals; Impact on ML: Windowed features miss or double-count events Issue: Invalid values; Cause: Schema violations, upstream bugs; Impact on ML: NaN, negative amounts, impossible categories poison training and inference Concept flow: Upstream Event - Pipeline Healthy? - Missing Event: Undercount - Duplicate Event: Overcount - Out-of-Order: Wrong window - Invalid Data: NaN / negative - Biased Features - Degraded Model Decisions Semantics: At-most-once; Guarantee: No duplicates; Duplicate Risk: None; Missing Risk: Events may be lost

Must remember:

- Data quality has three dimensions: freshness, completeness, and correctness
- Common issues: missing events, duplicates (at-least-once delivery), out-of-order arrivals, invalid values
- Issues cause silent feature bias - code does not crash, model performance degrades
- Simple per-window checks: event counts, field distributions, null/invalid rates
- Compare metrics against historical baselines and alert on significant deviation
- Deduplication is mandatory under at-least-once delivery semantics
- Proactive monitoring catches issues before they poison the ML pipeline
- Data quality monitoring is a lightweight starting point - no enterprise framework required initially

Formula / decision rule: null_rate = (records with null/invalid value)/(total records)

Example or contrast: For a 5-minute micro-batch fraud pipeline:

Exam trap: Assuming pipeline success means data correctness - a job completing without error does not guarantee all events were processed correctly Ignoring at-least-once duplicates - without deduplication, retry semantics inflate feature counts silently

MLME-W10-T13 Schema Evolution and Data Contracts

Tier B

Definition and why it matters: In production systems, data structure changes over time - by design and by accident:

Core explanation: Change Type: Field added; Example: New devicetype column in click events; Risk: Downstream may ignore it (low risk) or fail if strict Change Type: Field removed; Example: legacyid column deprecated; Risk: Downstream jobs break or see nulls

Must remember:

- Schema evolution is inevitable - fields are added, removed, renamed, and retyped over time
- Unhandled changes cause runtime crashes or silent misinterpretation - both harm ML systems
- Data contracts define fields, types, allowed values, and change policies between producers and consumers
- Backward-compatible changes: add optional fields with defaults; avoid breaking removals/renames without coordination
- Schema registries enforce contracts at publish time, preventing bad data from entering the pipeline
- Data quality monitoring (freshness, counts, schema) and model monitoring (drift, performance) are complementary halves of ML safety
- ML models must be tied to a specific schema version - serving with a different schema causes silent degradation
- Root cause analysis requires checking both data quality and model monitoring signals

Example or contrast: Change: Add optional field with default; Breaking?: No; Safe Approach: Deploy directly

Exam trap: Treating schema changes as backward-compatible by default - adding a required field breaks all existing consumers No contract between producer and ML team - ML discovers schema changes when training jobs fail or metrics degrade

MLME-W10-T14 Applying Data Quality Concepts in Practice

Tier A

Definition and why it matters: Data quality concepts - freshness, completeness, correctness, schema contracts - become tangible when applied to a simple but realistic batch pipeline. A lab-style simulation demonstrates the full data-to-train flow without requiring production streaming infrastructure

Core explanation: Concept flow: Daily CSV Files: day1.csv, day2.csv, ... - Ingestion Script: Find new files - Append to Master: Training Dataset - Retrain Trigger: Volume or days threshold - Train Function: Updated Model Component: Daily CSV simulator; Purpose: Mimics new data arriving each day (day1.csv, day2.csv, ...) Component: Ingestion script; Purpose: Finds unprocessed day files, appends to master training dataset Component: Retrain trigger; Purpose: Fires based on data volume threshold or days since last retrain Component: Training function; Purpose: Consumes updated master dataset to produce a new model Concept: Freshness SLA; Batch Lab Interpretation: "Training data must include events through yesterday" Concept: Freshness lag; Batch Lab Interpretation: Days between latest event in master dataset and today

Must remember:

- Apply data quality concepts to a concrete batch pipeline: daily CSV → ingestion → master dataset → retrain trigger
- Freshness in batch context: how many days behind is the training data?

- Completeness/correctness at ingestion: row counts, null checks, schema validation before append
- Log every ingestion for lineage - which files, row counts, timestamps
- Schema contracts apply even to CSV files - reject incompatible files, do not silently accept
- Retrain triggers: volume-based, time-based, or hybrid - always gate on data quality first
- Data quality monitoring + model monitoring together enable fast root cause analysis
- Investigation order: check data first, then model, when performance degrades
- Simple lab patterns mirror production patterns - incremental ingestion, state, idempotency, quality gates

Formula / decision rule: < 0.5 x; > 3 x

Exam trap: Skipping quality checks in "simple" lab pipelines - bad habits in labs become production incidents; validate from day one Retraining on corrupted data - if a bad file is ingested, retrain trigger fires on poisoned data; quarantine before append

MLME-W10-T15 Simulating Daily Data Arrival for Pipeline Development

Tier A

Definition and why it matters: Building and testing data pipelines before touching live production requires a predictable, repeatable source of new data. A daily-file simulator mimics the production pattern where fresh data arrives on a schedule - without depending on real event streams or production databases

Core explanation: Core insight: Pipeline development should not wait for production data. Simulate the arrival pattern first, then build ingestion logic against the simulation Concept flow: Base Dataset: 5 days x 10 records - Split by Day - day1.csv - day2.csv - day3.csv - day4.csv - day5.csv - Landing Zone: data/ Parameter: Days; Value: 5; Rationale: Enough to test multi-day ingestion In production, data commonly arrives as time-partitioned files: Parameter: Records per day; Value: 10; Rationale: Small enough for inspection, large enough for aggregation Parameter: Total records; Value: 50; Rationale: Manageable for lab exercises Each file contains events for exactly one day. An ingestion pipeline scans for new files, processes them, and appends to a master dataset Field: day; Type: int; Generation Logic: Day index (1-5) Use a fixed random seed (numpy.random.defaultrng(seed=42)) to ensure identical data on every run. Reproducibility is essential for:

Must remember:

- Simulate daily data arrival before building ingestion pipelines - predictable, repeatable test data
- Generate base dataset with fixed random seed for reproducibility across runs
- Schema: day, timestamp, customerid, amount, label - mimics real event/training data
- Split by day using groupby → one CSV per day in a landing zone
- Realistic timestamps: base day + random minutes throughout the day
- Pattern mirrors production: time-partitioned files arriving on schedule
- Enables testing: incremental ingestion, idempotency, state management, retrain triggers, quality checks
- Staging environments in production use the same simulation pattern at larger scale

Exam trap: No fixed random seed - non-reproducible data makes debugging ingestion failures nearly impossible Writing DataFrame index to CSV - causes extra unnamed column that breaks downstream schema expectations

MLME-W10-T16 Incremental Ingestion: State Management and Idempotency

Tier A

Definition and why it matters: When new daily data arrives, two approaches exist:

Core explanation: Approach: Full reprocess; Description: Delete master dataset; rebuild from all daily files; Scalability: Poor - re-reads years of data for one new day Approach: Incremental ingestion; Description: Process only new files; append to master dataset; Scalability: Good - scales linearly with new data volume Concept flow: Read ALL daily files - Rebuild entire master dataset - Read state: last ingested day - Find only NEW files - Append to master dataset - Update state Property: Persistent; Benefit: Survives script restarts, machine changes Property: Externalised; Benefit: Script itself remains stateless Property: Simple; Benefit: No database required for lab/small-scale pipelines For production ML pipelines, incremental ingestion is the standard pattern. Re-reading terabytes to add one day's partition is economically and operationally unsustainable The most important concept in incremental pipelines is state management - the pipeline must know what it has already processed A simple but effective approach: a JSON state file (ingeststate.json) tracking progress:

Must remember:

- Incremental ingestion processes only new files and appends to master dataset - the production standard
- Full reprocess does not scale; avoid for any dataset beyond trivial size
- State management via externalised state file (ingeststate.json) tracks lastingestedday
- Script is stateless; state lives in persistent artifact - survives restarts and machine changes
- Flow: load state → scan for new files → filter by day last ingested → append → save → update state
- Idempotency: re-running produces the same result - essential for scheduled pipelines
- Update state only after successful save - prevents data loss on crash
- Quality checks at ingestion time catch schema and completeness issues before they enter training data

Formula / decision rule: N > last_ingested_day; O(all data)

Example or contrast: First run (state = 0):

Exam trap: Updating state before saving master dataset - crash between state update and save causes permanent data loss No state file - pipeline re-processes all files on every run; duplicates accumulate or compute is wasted

MLME-W10-T17 Training-Serving Skew: Detection and the Feature Store Fix

Tier A

Definition and why it matters: Training-serving skew occurs when features used during training differ from features provided to the model during serving - in value, computation logic, or time window. The model's production performance is worse than offline evaluation predicts, and the bug is incredibly difficult to debug because no error is thrown

Core explanation: This is not a data pipeline ingestion bug - it is a feature consistency bug that data pipelines must prevent by enforcing a single source of truth for feature logic Concept flow: Historical Data - Feature Logic A: 30-day window - Offline Features - Model Trained - Live Request - Feature Logic B: 7-day window - BUG - Online Features - Unreliable Predictions Path: Offline (training); Feature Function: compute30dayfeatures(); Window: 30 days Path: Online (serving); Feature Function: compute7dayfeaturesbug(); Window: 7 days (accidental) Feature: total30dspend; Value: 170 Feature: transactioncount30d; Value: Multiple active transactions Skew typically arises when: Feature: total7dspend; Value: 0 Two teams implement feature logic independently (training team vs serving team)

Must remember:

- Training-serving skew: features at training time differ from features at serving time - silent, devastating bug
- Most common cause: two independent implementations of the same feature logic (e.g., 30-day vs 7-day window)
- Model trains on one distribution; serving feeds another - no error thrown, production performance collapses
- Fix: single source of truth - one feature definition reused for offline and online materialisation
- Feature stores (Feast, Tecton, Hopsworks) architecturally eliminate this bug class at scale
- Detection: compare offline vs online feature values for sample entities; monitor feature distributions
- Skew != data drift - skew is a logic bug; drift is a distribution change
- Data pipelines must deliver consistent data, but feature logic consistency is a separate, equally critical concern

Exam trap: Assuming offline metrics validate serving correctness - skew is invisible to offline evaluation by definition Fixing skew with a serving-side patch - patching the online path without fixing the root cause (duplicate logic) means the next code change reintroduces skew

MLME-W10-T18 Data Pipelines for Machine Learning: Module Summary

Tier C

Definition and why it matters: This module sits at the intersection of data engineering and ML operations. Prior modules covered model serving, monitoring, feature stores, and retraining. This module addresses the layer beneath all of them: how data flows reliably into every stage of the ML lifecycle

Core explanation: Concept flow: Monitoring - Retraining - Feature Stores - Batch / Micro-Batch / Streaming Ingestion - Data Quality - Incremental Ingestion Lab

Must remember:

- Decision: Ingestion mode; Recommendation: Start batch; escalate only with clear freshness requirements
- Decision: Processing pattern; Recommendation: Incremental ingestion with state, not full reprocess
- Decision: Data quality; Recommendation: Monitor freshness, completeness, correctness from day one
- Decision: Schema management; Recommendation: Data contracts with backward-compatible change policies
- Decision: Feature consistency; Recommendation: Single source of truth (feature store); never duplicate logic
- Decision: Investigation order; Recommendation: Data quality first, then model, when performance degrades
- Data pipelines are the circulatory system - they feed training, validation, inference, monitoring, and retraining
- Core principle: no data → no model; bad data → bad model
- Three ingestion modes: batch (workhorse), micro-batch (compromise), streaming (lowest latency, highest complexity)
- ETL/ELT produce curated tables; ingestion modes control how often they refresh
- Kafka = transport; Spark/Flink/Beam = processing; windows + state = streaming features
- Data quality: freshness, completeness, correctness, schema - all cause silent degradation
- Distinguish inference latency from data freshness latency - both matter independently
- Data contracts and incremental ingestion with state are production essentials
- Data quality monitoring + model monitoring together enable root cause analysis
- Start simple (batch), escalate deliberately; a well-designed batch pipeline is already a major win
- Feature consistency (no training-serving skew) requires a single source of truth - feature stores at scale

Exam trap: Treating data pipelines as infrastructure, not ML-critical path - pipeline failure equals model failure Optimising model architecture while ignoring data plumbing - no model recovers from systematically bad input

Week 10 Exam Lens

Likely questions

- Define and explain Why Machine Learning Needs Data Pipelines; include the mechanism and one limitation.
- Compare Why Machine Learning Needs Data Pipelines with ETL and ELT: Foundations of ML Data Pipelines and state when each is preferred.
- Apply Pipeline Types: Batch, Micro-Batch, and Streaming to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: sum(spend)_(30d); count(clicks)_(7d); count(events); sum(amount); freshness_lag = t_(feature_updated) - t_(event_occurred); SLA compliance = (features meeting SLA)/(total features evaluated) x 100%; null_rate = (records with null/invalid value)/(total records); < 0.5 x

Mistakes to avoid: Treating data pipelines as optional - models cannot self-heal from upstream data failures; pipeline outages cascade into monitoring and retraining blindness Believing automation alone is enough - a pipeline that runs daily but ingests wrong data is worse than a manual process that catches errors Assuming ETL and ELT are mutually exclusive - modern platforms combine both; the question is where each transform runs

Week 11: Security, Compliance, Fairness, and Responsible AI

Week roadmap: Threats to Machine Learning Systems: What Can Go Wrong → Data and Input Threats in Machine Learning Systems → Model and Privacy Threats: Extraction, Leakage, and Over-Exposure → Why Machine Learning Systems Are Attractive Attack Targets → PII and Sensitive Attributes in Machine Learning → Data Minimisation and Anonymisation Techniques → 12 more topics.

Coverage: 18 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

MLME-W11-T01 Threats to Machine Learning Systems: What Can Go Wrong

Tier B

Definition and why it matters: Production ML work does not end at accuracy, latency, and observability. Models increasingly sit inside decision loops for high-stakes outcomes: loan approvals, fraud detection, content moderation, healthcare triage, and dynamic pricing. When these systems are attacked, abused, or misused, harm propagates across users, businesses, and regulators

Core explanation: Security and responsible use are not optional add-ons delegated to a separate security team. They are design constraints that model engineers must internalise from the first architecture sketch

Must remember:

- ML systems face threats across data, input, model/privacy, and infrastructure - use this four-bucket mental map
- Models in decision loops for money, access, and safety amplify the impact of any successful attack
- Data poisoning, label noise, and sampling skew corrupt what the model learns - not just how well it scores
- Input threats include OOD traffic, adversarial probing, and API abuse; defend with validation, rate limits, and monitoring
- Model extraction and information leakage turn outputs and internals into privacy and IP risks
- Security is woven into MLOps: deployment controls, monitoring, retraining, and feature governance all contribute
- Design defensively from the start: least privilege, minimal exposure, auditable deployments

Exam trap: Treating security as a post-deployment checklist rather than a design-time requirement Assuming high offline accuracy implies the model is safe to deploy - data poisoning and skew can produce good-looking metrics with dangerous behaviour

MLME-W11-T02 Data and Input Threats in Machine Learning Systems

Tier B

Definition and why it matters: ML systems increasingly drive real-world decisions: loan approvals, fraud checks, content moderation, healthcare triage, dynamic pricing. Attack or misuse produces unfair decisions, privacy leaks, fraud losses, regulatory fines, and reputational damage

Core explanation: Unlike rule-based systems, ML behaviour is complex and non-intuitive. Attackers can probe inputs, observe outputs, and iteratively find weaknesses. Security is therefore a core competency for model engineers, not a handoff to a separate team

Must remember:

- ML security matters because models sit in high-stakes decision loops and are harder to reason about than rule engines
- Data threats: poisoning (deliberate), label noise (accidental), skew (structural) - all corrupt learned behaviour
- Input threats: OOD inputs, adversarial probing, and API abuse break reliability at serving time
- Data quality is simultaneously a security, fairness, and accuracy concern
- Defend inputs with validation, rate limiting, monitoring, and OOD detection
- Attackers learn by observing outputs - limit query volume and log anomalies
- Always separate training-time data risks from serving-time input risks when designing mitigations

Example or contrast: Fraud scoring: Poisoned training labels that mark attacker-controlled merchants as "legitimate" cause systematic approval of fraudulent transactions

Exam trap: Assuming offline metrics on a static test set catch poisoning - poisoned examples may be in the training set, not the holdout Confusing data threats (training-time) with input threats (serving-time); fixing one does not fix the other

MLME-W11-T03 Model and Privacy Threats: Extraction, Leakage, and Over-Exposure

Tier A

Definition and why it matters: A deployed model is not a black box that only produces predictions. It is also:

Core explanation: Intellectual property - months of feature engineering, architecture tuning, and proprietary data Concept flow: Attacker - API - Target Mitigation: Rate limiting; Mechanism: Reduces query volume available for surrogate training An information channel - outputs can leak details about training data or individual records Mitigation: Output rounding / coarsening; Mechanism: Reduces fidelity of (x, hat(y)) pairs An attack surface - internals exposed through APIs, weights, or debug tooling Mitigation: Query monitoring; Mechanism: Detect systematic probing patterns The governing principle: expose only what is needed, and treat the model like source code or a production database Mitigation: Authentication and billing; Mechanism: Raises cost of large-scale querying An attacker sends a large volume of queries to a public prediction API, records (x, hat(y)) pairs, and trains a surrogate model that approximates the target Mitigation: Watermarking / model fingerprinting; Mechanism: Detect stolen surrogates (advanced)

Must remember:

- Models are sensitive assets: IP, information channel, and attack surface simultaneously
- Model extraction: query API at scale, train surrogate, analyse weaknesses offline
- Information leakage: overfitted or overly detailed outputs reveal training data patterns
- Over-sharing: weights, debug endpoints, verbose errors, and rich responses expand risk
- Principle: expose only what is needed; prefer coarse outputs and strict access controls
- Mitigations: rate limiting, output coarsening, monitoring, authentication, least-privilege artefact access
- Privacy leakage from models is a compliance issue, not just a modelling inconvenience

Formula / decision rule: (x, hat(y))

Exam trap: Models that memorise rare training examples can output high-confidence predictions that effectively encode those examples. Differential privacy and regularisation reduce this risk but trade off accuracy Equating "model leakage" only with data leakage (train-test contamination) - this topic covers model extraction and output-side privacy leakage

MLME-W11-T04 Why Machine Learning Systems Are Attractive Attack Targets

Tier B

Definition and why it matters: ML systems combine high-impact decisions with opaque behaviour and a wide attack surface. Attackers exploit the gap between what engineers can predict and what the model actually does under novel inputs. Understanding why ML is targeted shapes where to invest defensive effort

Core explanation: ML models often sit close to decisions involving money, access, safety, and reputation: Loan and credit approvals Concept flow: Data Pipelines - Feature Stores - Batch Jobs - Online APIs - Dashboards - Logs - ML System - surface - Single Weak Link

Must remember:

- ML systems are attractive targets because they control high-stakes decisions with opaque behaviour
- Attackers probe inputs and observe outputs - they do not need to understand your architecture upfront
- Attack surface spans data pipelines, feature stores, batch jobs, APIs, dashboards, and logs
- Immature deployment processes prolong exposure after vulnerabilities are found
- First-line defences: input validation, rate limiting, least privilege, monitoring, secure defaults
- ML security is an extension of MLOps discipline - automation and observability serve both reliability and security
- Design defensively from the start; do not bolt security on after launch

Example or contrast: Dimension: Behaviour predictability; Rule-based system: High - logic is explicit; ML system: Low - learned boundaries

Exam trap: Assuming security is "the infosec team's problem" after the model is trained Hardening only the prediction API while leaving the feature store and training pipeline open

MLME-W11-T05 PII and Sensitive Attributes in Machine Learning

Tier B

Definition and why it matters: Most production ML systems process personal data. Privacy is therefore not optional - it sits alongside latency, uptime, and cost as a non-functional requirement that shapes architecture, features, logging, and monitoring

Core explanation: A useful mental model: you are borrowing users' data to provide value, not owning it outright. Every design decision should answer: What data are we collecting? How long do we keep it? Who can access it, and for what purpose?

Must remember:

- Privacy is a non-functional requirement equal to latency and uptime in production ML
- PII includes direct identifiers (name, email) and quasi-identifiers (DOB, ZIP, device ID)
- Sensitive attributes (health, financial, biometric, protected characteristics) need special handling as features or labels
- PII hides inside features - user IDs, location traces, and transaction logs are highly identifying
- Quasi-identifier combinations enable re-identification without any single direct identifier
- Ask for every feature: "Could this identify or harm a person?"
- Design privacy into pipelines, logging, and feature stores from the start

Example or contrast: Aspect: Primary risk; PII: Re-identification; Sensitive attributes: Discrimination, harm, regulatory breach

Exam trap: Assuming anonymisation of user IDs is sufficient - join keys in separate tables re-identify Treating only "obvious" columns as PII while ignoring location history and transaction sequences

MLME-W11-T06 Data Minimisation and Anonymisation Techniques

Tier A

Definition and why it matters: Data minimisation means resisting the instinct to hoard data "just in case." Collect only what is required for the specific task and metric you are targeting. Less data means:

Core explanation: Smaller attack surface Question: Do we need raw identifiers?; Alternative when answer is "no": Internal surrogate ID Question: Can we use aggregated features?; Alternative when answer is "no": Age band instead of exact age Question: Can we drop data after a window?; Alternative when answer is "no": Archive or delete beyond retention period Question: Do we need full transaction history?; Alternative when answer is "no": Rolling 90-day summary statistics Simpler compliance obligations Lower risk of accidental misuse It is one of the simplest and most powerful privacy tools available to

model engineers Ask these questions during feature design: Example - credit scoring: Instead of storing full address → use city-level or postal-sector bucket Instead of exact income → use income decile Instead of email → use hashed internal customer ID with restricted mapping table

Must remember:

- Data minimisation: collect only what the task requires; reduces attack surface and compliance burden
- Masking/redaction: hide portions of sensitive fields (last four digits)
- Aggregation/bucketing: replace precise values with coarser groups
- Pseudonymisation: swap identifiers for internal IDs; mapping stored separately and restricted
- Full anonymity is hard - ML needs detail; external linkage and model memorisation undermine guarantees
- Treat anonymisation as risk reduction, not magic
- Combine minimisation, technical controls (encryption, RBAC), and organisational policies
- Balance privacy techniques against model utility - aggressive coarsening destroys predictive signal

Formula / decision rule: (lat, lon); k = 5

Exam trap: Treating pseudonymisation as equivalent to anonymisation - it is reversible with the mapping Over-bucketing features and blaming poor model performance on the algorithm rather than destroyed signal

MLME-W11-T07 Role-Based Access Control and Data Governance for ML

Tier B

Definition and why it matters: Privacy depends not only on what the system does at runtime but on how many people and services can directly touch sensitive data. Access control ensures each role receives only the permissions required for its function - the principle of least privilege

Core explanation: Different roles need different data exposure: Role: Data engineer; Typical needs: Pipeline construction, ETL; Raw PII access: Often yes - but scoped to pipeline stage Role: Model engineer; Typical needs: Feature experimentation, training; Raw PII access: Prefer pseudonymised or sampled data Role: Business analyst; Typical needs: Dashboards, aggregate reports; Raw PII access: Aggregated metrics only

Must remember:

- RBAC assigns least-privilege permissions by role - data engineer, model engineer, and analyst need different access
- Separate dev, staging, and production environments; avoid raw production data in development
- Enforce feature-level policies - some features restricted or aggregated per role and use case
- Audit logs track data access, model deployments, and config changes
- Data lineage documents feature provenance for debugging and privacy justification
- Feature governance enables seeing, controlling, and justifying data use across the ML lifecycle
- Privacy is organisational and technical - access control is as important as anonymisation

Exam trap: Granting all engineers production database access "for convenience." Using production data in Jupyter notebooks on local laptops without encryption or policy

MLME-W11-T08 Fairness and Bias: Evaluating Models Across Groups

Tier A

Definition and why it matters: For a positive class (e.g., "fraud", "approved", "disease present"): Error: False positive (FP); Definition: Predicted positive, actually negative; Example: Legitimate transaction flagged as fraud

Core explanation: A model with strong overall accuracy can still systematically harm specific groups. Fairness analysis makes these disparities visible and measurable - a prerequisite for informed human decisions about deployment, mitigation, and policy Working definition for this module: Does the model behave consistently across groups? Are some groups receiving systematically worse errors? Group: Group A; Accuracy: 92%; Population share: 70% Group: Overall; Accuracy: ~90%; Population share: 100% Deciding what to do about disparities requires domain expertise, policy, and context. The model engineer's job is to surface behaviour clearly Consider a binary classifier: The 90% overall figure feels excellent. Split by group and Group B - potentially a vulnerable population or key customer segment - consistently receives worse predictions

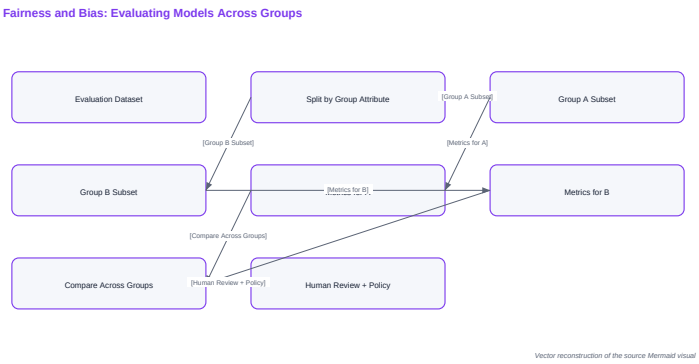
Must remember:

- High overall accuracy can mask systematic harm to specific groups
- Group comparison: same model, same metrics, different group slices - the core fairness check
- Choose a group attribute, slice evaluation data, compute metrics per group, compare
- FP and FN rates often matter more than accuracy; domain determines which error is costlier
- Check calibration: does score 0.8 mean the same thing for every group?
- Visualise disparities with bar charts, calibration plots, and tables showing deltas
- Fairness metrics surface evidence for human decisions - they do not replace policy or ethics

Formula / decision rule: FPR = (FP)/(FP + TN) (false positive rate); FNR = (FN)/(FN + TP) (false negative rate)

Example or contrast: Concept flow: Evaluation Dataset - Split by Group Attribute - Group A Subset - Group B Subset - Metrics for A - Metrics for B - Compare Across Groups - Human Review + Policy

Exam trap: Reporting only overall accuracy in stakeholder presentations Choosing group attributes without domain/legal guidance on which comparisons are meaningful



Concept map: Fairness and Bias: Evaluating Models Across Groups

MLME-W11-T09 Practical Fairness Questions and Visualisation

Tier A

Definition and why it matters: Fairness becomes actionable when vague worries ("the model might be biased") transform into specific, measurable questions with numbers and charts that non-ML stakeholders can discuss. This note catalogues the questions worth asking and how to present answers

Core explanation: Is one group systematically getting more false negatives? Example: Certain loan applicants are disproportionately denied despite meeting qualification criteria Compute FNR or recall per group A recall gap of 10 percentage points may mean one group misses 10% more legitimate positive outcomes Does model performance drop sharply for some subpopulation? Smaller language groups in NLP models Rural vs urban customers in a credit model New users vs established users in a churn predictor Look at accuracy, AUC, and error rates per slice - not just the majority group Does a predicted score of 0.8 mean roughly the same real-world probability for every group?

Must remember:

- Four key questions: FN disparity, subpopulation performance drop, calibration consistency, threshold impact
- False negative disparity often maps to denied opportunities (lending, hiring, screening)
- Calibration: score 0.8 should mean ~80% positive rate in every group
- Use bar charts, grouped error-rate bars, calibration plots, and delta tables
- Highlight gaps exceeding chosen thresholds to focus attention
- Fairness visualisation makes behaviour observable for non-ML stakeholders
- Severity of gaps is domain- and stakes-dependent - metrics inform, humans decide

Example or contrast: Question: FN disparity; Primary metric: Recall, FNR; Reveals: Denied opportunities, missed detections Question: Subpopulation drop; Primary metric: Accuracy, AUC per slice; Reveals: Model simply works worse for some groups Question: Calibration; Primary metric: Reliability diagram, Brier score; Reveals: Score interpretability across groups

Exam trap: Showing only accuracy bars without FP/FN breakdown - hides the direction of harm Using calibration plots without enough per-group samples in each bin - noisy and misleading

MLME-W11-T10 Fairness Limitations, Trade-offs, and the Model Engineer's Role

Tier A

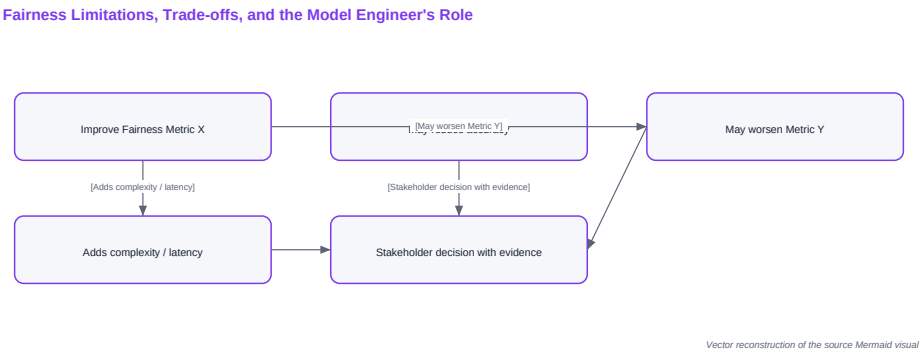
Definition and why it matters: Whether a 3% gap or a 10% gap is tolerable depends on: Domain stakes - movie recommendation vs medical diagnosis

Core explanation: Fairness is not a scalar you optimise once and forget. Different definitions capture different notions of equitable treatment - and they frequently conflict Fairness criterion: Equal accuracy; What it demands: Same error rate across groups; Example tension: May conflict with base-rate differences Fairness criterion: Equal false positive rate; What it demands: Same FP rate across groups; Example tension: Conflicts with equal recall when base rates differ Fairness criterion: Equal false negative rate; What it demands: Same FN rate across groups; Example tension: Conflicts with equal precision Fairness criterion: Equal calibration; What it demands: Same score-to-probability mapping; Example tension: Conflicts with equal FPR and FNR simultaneously Fairness criterion: Demographic parity; What it demands: Equal positive prediction rate; Example tension: May sacrifice overall accuracy significantly Impossibility result (informal): Except in trivial cases, you cannot satisfy all fairness definitions simultaneously when group base rates differ. This is not a tooling bug - it is a mathematical constraint Concept flow: Model Engineer - Domain Expert - Legal / Policy - Product - Deployment Decision

Must remember:

- No single universal fairness metric - equal accuracy, FPR, FNR, and calibration can conflict
- Improving one fairness measure may cost accuracy, latency, or complexity
- Whether a gap is acceptable depends on domain, stakes, policy, and regulation
- Model engineer role: make metrics easy, flag gaps early, document decisions, collaborate
- Fairness numbers are inputs to human decisions - not automated verdicts
- Integrate fairness checks into CI/CD and audit trails for continuous accountability
- Document which groups were tested, what gaps appeared, and what action was taken

Exam trap: Improving one fairness measure often costs: Overall accuracy - rebalancing or constraints reduce global performance



Concept map: Fairness Limitations, Trade-offs, and the Model Engineer's Role

MLME-W11-T11 Explainability: What It Is, Why It Matters, and Its Limits

Tier B

Definition and why it matters: Users denied a loan, stakeholders reviewing risk models, and regulators auditing automated decisions all ask the same question. Answering it requires tracing how data flowed through the system, which model version produced the output, and what features influenced the prediction

Core explanation: Explainability and audit trails work together as an accountability toolkit for production ML Type: Local; Question answered: Why this prediction for this specific case?; Audience: End user, support, appeals Type: Global; Question answered: What features drive behaviour in general?; Audience: Model reviewers, risk analysts, engineers

Must remember:

- Explainability makes model behaviour understandable for humans - local (one case) and global (population)
- Local: why this prediction - for users, support, appeals
- Global: what drives the model generally - for review, risk analysis, feature design
- Invest in explainability for debugging, user trust, and regulatory review
- Popular methods are approximations - they can oversimplify or mislead
- Biased data and proxy features produce polished but misleading explanations
- Use explainability with fairness checks and domain expertise - not as a substitute
- Production systems typically need both local and global explanation capabilities

Example or contrast: Dimension: Scope; Local: One prediction; Global: Entire dataset / model

Exam trap: Treating SHAP values as ground truth rather than an approximation Providing local explanations without checking global feature importance - local and global can disagree

MLME-W11-T12 Audit Trails Across the ML Pipeline

Tier A

Definition and why it matters: An audit trail is the persistent record that lets you reconstruct what happened, when, and under which configuration. For ML systems, it spans individual predictions, training runs, deployments, and rollbacks - tying data, model versions, and decisions into a coherent timeline Without audit trails, answering "which model made this decision?" requires forensic guesswork

Core explanation: For each prediction (online or batch), capture: Field category: Model metadata; What to log: Name, version, hash, config flags; Example: credit-risk-v2.1, hash:abc123 Field category: Data / feature context; What to log: Key input fields, feature store version, timestamp; Example: featuresv3.2, 2025-06-05T14:32:01Z Field category: Decision details; What to log: Prediction, score, threshold applied; Example: score: 0.73, threshold: 0.65, decision: deny Field category: Explanation summary; What to log: Optional short local explanation; Example: top drivers: income, delinquency Field category: Request context; What to log: Request ID, calling service; Example: req-8f3a, loan-origination-svc Concept flow: Data snapshot ID - Hyperparameters - Training metrics - Request metadata - Model version - Prediction + latency - Promotion approval - Rollout start / complete - Rollback events Do not log raw PII unnecessarily. Apply minimisation and access-control policies - audit logs themselves become sensitive datastores if they contain names, emails, or full feature vectors with identifying content

Must remember:

- Audit trails reconstruct what happened across the ML lifecycle
- Per-prediction logs: model version, feature version, score, threshold, request ID, optional explanation
- Training logs: data snapshot, hyperparameters, metrics (including group-wise)
- Rollout logs: promotion approval, rollout/rollback events
- Do not log raw PII - apply minimisation to audit records themselves
- Structured, append-only, versioned logs enable compliance and incident investigation
- Include fairness evaluation results in the audit trail, not just pass/fail
- A complete trail answers: which model, which data, which checks, who approved

Exam trap: Logging only predictions without model version - impossible to reproduce the decision later Storing raw PII in audit logs, creating a second sensitive datastore

MLME-W11-T13 Regulatory Expectations and Designing for Auditability

Tier B

Definition and why it matters: Specific laws vary by jurisdiction and sector (finance, healthcare, hiring, insurance). Despite differences, regulated domains share common expectations for automated decision systems:

Core explanation: Expectation: Purpose limitation; What it means in practice: Use data only for declared, consented purposes Expectation: Non-discrimination; What it means in practice: Avoid unjustified bias or harmful disparate impact Expectation: Explainability; What it means in practice: Provide understandable reasons to non-technical people Expectation: Record-keeping; What it means in practice: Maintain records of models used, evaluations performed, and approvals granted

Must remember:

- Regulated domains share expectations: purpose limitation, non-discrimination, explainability, record-keeping
- Design for auditability at design time - not after incidents
- Model versioning, structured logging, and fairness records are core requirements
- System must answer: which model, what data, what checks passed, who approved
- Write an investigation playbook before you need it
- Explainability (why) and auditability (what/when/who) complement each other
- Fairness metrics logged per model version enable regulatory and internal review
- High accuracy alone does not demonstrate responsible governance

Example or contrast: Regulator: "How do you ensure your credit model does not discriminate?"

Exam trap: Treating regulatory compliance as a legal-team-only concern - engineering design determines feasibility Building auditability after a complaint arrives - records are incomplete or lost

MLME-W11-T14 Explainability, Audit Trails, and Accountability: Integration Recap

Tier B

Definition and why it matters: Responsible production ML requires two complementary capabilities:

Core explanation: Explainability - understanding model behaviour at local and global levels Dimension: Local; Scope: Single prediction; Primary use: User-facing reasons, appeals, support Dimension: Global; Scope: Population-level patterns; Primary use: Model review, risk analysis, feature design Concept flow: Training: data version, hyperparams, metrics - Model Registry - Fairness Evaluation: group-wise metrics logged - Promotion Approval - Serving: per-prediction logs - Monitoring: drift + anomaly alerts - Audit Store Audit trails - persistent logs tying together data, model versions, decisions, and time Together they form an accountability toolkit that answers both "why?" and "what exactly happened?"

Must remember:

- Explainability (why) and audit trails (what/when/who) form the accountability toolkit
- Local explanations for individual decisions; global explanations for model review
- Audit trails span training, evaluation, promotion, serving, and monitoring
- Regulatory and organisational context demands answers for non-technical stakeholders
- Design systems to be traceable and interpretable from the start
- Lab workflow: segmented evaluation → automated fairness check → JSONL audit log
- Fairness, security, privacy, explainability, and auditability integrate into responsible production ML

Exam trap: Treating explainability and audit trails as optional "nice to haves" for regulated domains Building explainability without audit trails - cannot verify explanations against the actual model version used

MLME-W11-T15 Segmented Evaluation: Performance by Group

Tier B

Definition and why it matters: A credit scoring model reporting 95% accuracy looks excellent. But accuracy aggregated over all users can hide systematic underperformance for specific groups. A model that is "technically accurate" on average can still cause unintended harm

Core explanation: Segmented evaluation - computing metrics independently per group - is the foundational practice for any fairness analysis View: Global accuracy = 95%; What you see: One reassuring number; Risk: Hidden group disparities View: Group A accuracy = 97%; What you see: Strong performance View: Group B accuracy = 88%; What you see: Weaker performance; Risk: 9 pp gap invisible globally

Must remember:

- High global accuracy can hide poor performance for specific groups
- Segmented evaluation: compute accuracy, precision, recall, AUC per group independently
- Workflow: predict → overall baseline → loop groups → filter → compute metrics → compare
- Bar charts make disparities immediately visible
- Recall gaps often indicate denied opportunities (lending) or missed detections (fraud)
- Synthetic data with group-specific base rates is a valid way to practise fairness analysis
- Disaggregation is the first and most important step in any fairness audit

Example or contrast: Code pattern: Overall: accuracy=0.91 recall=0.85 precision=0.88 AUC=0.93; Group A: accuracy=0.94 recall=0.91 precision=0.90 AUC=0.95; Group B: accuracy=0.86 recall=0.74 precision=0.82 AUC=0.87

Exam trap: Reporting only overall metrics to stakeholders for high-impact models Forgetting to include the group column in the validation set used for evaluation

MLME-W11-T16 Automated Fairness Checks: Policy, Thresholds, and Pass/Fail Logic Tier A

Definition and why it matters: Manually inspecting a segmented metrics table and bar chart works for one model. It does not scale across dozens of models, retraining cycles, and CI/CD pipelines. The next step is encoding fairness expectations as explicit rules and thresholds that produce an automated pass or fail signal

Core explanation: Define constraints at the top of the fairness check script: Domain: High-stakes medical model; Example threshold: 1% max gap; Rationale: Patient safety Domain: Loan approval; Example threshold: 5-10% max gap; Rationale: Regulatory scrutiny, opportunity impact Domain: Movie recommendation; Example threshold: 10%+ may be acceptable; Rationale: Low individual harm Metric: Recall (1 - FNR); Measures: How well true positives are identified; Critical for: Opportunity models - loan approval, hiring (missing qualified candidates = false negative) Metric: FPR; Measures: How often negatives are incorrectly flagged positive; Critical for: Punitive models - fraud detection, content moderation (false accusations = false positive) The absolute difference in recall between any two groups must not exceed 10% The absolute difference in false positive rate (FPR) between any two groups must not exceed 10%

Must remember:

- Automate fairness by encoding policy thresholds as pass/fail rules in CI/CD
- Typical constraints: max recall difference and max FPR difference between groups
- Recall critical for opportunity models (lending); FPR critical for punitive models (fraud)
- Compute confusion matrix per group → derive FPR, recall → compute absolute deltas
- FAIL triggers human investigation - not automatic model rejection
- Thresholds must be set with product, legal, and domain experts - not by engineers alone
- Fairness check is a safety net integrated into the ML pipeline before promotion

Formula / decision rule: 1 - FNR; FPR = (FP)/(FP + TN)

Exam trap: Treating 10% as a universal threshold - always context-dependent Only checking recall and ignoring FPR (or vice versa) - domain determines which matters

MLME-W11-T17 Logging Fairness Metrics for Audit and Monitoring Tier A

Definition and why it matters: An automated fairness check that prints PASS or FAIL to the terminal works for CI/CD at the moment of execution. But a month later, when an auditor, regulator, or teammate asks "What was the fairness evaluation for model version 2.1, and why did we ship it?" - that console output is gone

Core explanation: Governance and compliance require a persistent, structured system of record Field: modelversion; Purpose: Which model was evaluated Field: dataversion; Purpose: Which dataset the evaluation ran against Field: timestamp; Purpose: When the evaluation occurred Field: groupmetrics; Purpose: Full per-group breakdown (accuracy, recall, FPR, etc.) Field: deltas; Purpose: Computed gaps between groups Field: thresholds; Purpose: Policy limits applied Field: verdict; Purpose: PASS or FAIL Field: issues; Purpose: List of violated rules (if any) Capture all context surrounding a fairness evaluation in a single structured record: Concept flow: Fairness Evaluation - Structured Record - Metadata: model, data, timestamp - Group Metrics: full breakdown - Verdict + Issues - Append to JSONL file - Future queries + trend analysis Property: Append-only; Benefit: Open file in append mode; write one new line per evaluation Property: Line-by-line processing; Benefit: Stream large files without loading everything into memory

Must remember:

- Console pass/fail is ephemeral - governance requires persistent structured logs
- A fairness audit record includes metadata, full group metrics, deltas, thresholds, and verdict
- JSONL format: one JSON object per line; append-only, streamable, language-agnostic
- Each model evaluation appends a line - building chronological fairness history
- Enables trend analysis: is fairness improving or worsening over model versions?
- Evolve to databases or governance platforms as scale demands
- Pipeline: evaluate → check → log → deploy - fairness becomes continuous, not one-time

Exam trap: Printing fairness results to stdout without persisting - no system of record Logging only verdict without per-group metrics - cannot verify or investigate later

MLME-W11-T18 Module 11 Summary: Security, Privacy, Fairness, and Accountability in Production ML Tier A

Definition and why it matters: Earlier modules focused on making models fast, reliable, and observable. This module adds the critical dimensions of security, privacy, fairness, explainability, and auditability - the foundations of responsible production ML

Core explanation: Concept flow: Poisoning - Label noise - Skewed samples - OOD inputs - Adversarial probing - API abuse - Model extraction - Information leakage - Over-sharing internals Concept: PII; Key practice: Direct and quasi-identifiers; often hidden inside features Concept: Sensitive attributes; Key practice: Health, financial, biometric, protected characteristics Concept: Data minimisation; Key practice: Collect only what the task requires Concept: Anonymisation techniques; Key practice: Masking, aggregation, pseudonymisation - risk reduction, not guarantee High-stakes decisions (money, access, safety) Concept: RBAC; Key practice: Least privilege by role; environment separation Complex, non-intuitive behaviour exploitable via probing Concept: Audit logs and lineage; Key practice: Track access, deployments, and feature provenance Broad attack surface across the full pipeline Slow recovery without mature MLOps Step: Disaggregate; Action: Compute metrics per

group, not just globally Input validation, rate limiting, least privilege, monitoring, secure defaults (debug off, minimal errors, auditable CI/CD)

Must remember:

- ML threats span data, input, model/privacy, and infrastructure - design defensively from the start
- Privacy: PII in features, minimisation, pseudonymisation, RBAC, audit logs, lineage
- Fairness: disaggregate metrics, examine FP/FN, check calibration, automate thresholds, log results
- Explainability: local (one case) + global (population); approximations, not truth machines
- Audit trails: structured, persistent, versioned - span training through serving
- Regulatory context demands traceable, interpretable systems with investigation playbooks
- Labs: segmented evaluation → fairness check → JSONL audit log
- Security and responsibility integrate with deployment, monitoring, retraining, and feature governance
- When designing any system, ask: What could go wrong, and how would we notice?

Exam trap: Treating security, privacy, and fairness as post-deployment afterthoughts Reporting only global accuracy for high-impact models

Week 11 Exam Lens

Likely questions

- Define and explain Model and Privacy Threats: Extraction, Leakage, and Over-Exposure; include the mechanism and one limitation.
- Compare Model and Privacy Threats: Extraction, Leakage, and Over-Exposure with Data Minimisation and Anonymisation Techniques and state when each is preferred.
- Apply Fairness and Bias: Evaluating Models Across Groups to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: (x, hat(y)); (lat, lon); k = 5; FPR = (FP)/(FP + TN) (false positive rate); FNR = (FN)/(FN + TP) (false negative rate); 1 - FNR; FPR = (FP)/(FP + TN)

Mistakes to avoid: Treating security as a post-deployment checklist rather than a design-time requirement Assuming offline metrics on a static test set catch poisoning - poisoned examples may be in the training set, not the holdout Models that memorise rare training examples can output high-confidence predictions that effectively encode those examples. Differential privacy and regularisation reduce this risk but trade off accuracy

Week 12: Multi-Model, Multi-Tenant, and Retrieval Systems

Week roadmap: Multi-Model Systems: Routing and Ensembles → Why Multi-Model Systems Exist → Routing Strategies: Rule-Based and Learned Routers → Fallback Models, Ensembles, and the Accuracy-Cost Trade-Off → Scaling Inference: Sharding and Replication → Caching Model Results and Embeddings → 12 more topics.

Coverage: 18 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

MLME-W12-T01 Multi-Model Systems: Routing and Ensembles

Tier A

Definition and why it matters: Early ML serving tutorials often assume a clean architecture: one endpoint, one model, one version. Production platforms rarely look like that. A mature ML system typically hosts many models simultaneously - different versions, segments, languages, and task specialists - all behind shared infrastructure

Core explanation: Intuition: A single model is like a general practitioner. A production platform is a hospital with specialists (fraud, credit risk, churn, recommendations), each tuned for a narrow domain, plus triage logic to route patients to the right expert Pattern: Routing; Definition: Select one model per request; Goal: Match request to the right expertise Pattern: Ensembles; Definition: Call multiple models on the same input, then combine outputs; Goal: Improve accuracy and robustness Concept flow: Incoming Request - Routing or Ensemble? - Model A - Model B - Model C - Model 1 - Model 2 - Model 3 - Combine: avg / vote / stack

Must remember:

- Production systems host many models - versions, segments, tasks - not one model per service
- Routing selects one model per request; ensembles call multiple models and combine outputs
- Routing strategies include rules, learned routers, and fallbacks
- Ensembles (averaging, voting, stacking) trade higher accuracy/robustness for cost and latency
- This module connects routing, scaling (shard/replicate/cache), multi-tenancy, RAG, and model registry patterns
- The central engineering question: how to match each request with the right model expertise at acceptable cost and latency

Exam trap: Trap: Routing and ensembles solve the same problem. Reality: Routing picks one expert; ensembles combine multiple opinions. They can coexist (route to an ensemble pipeline) but are distinct patterns Trap: More models always means better accuracy. Reality: Each additional model adds latency, cost, and operational complexity. The accuracy-latency-cost trade-off must be justified per use case

MLME-W12-T02 Why Multi-Model Systems Exist

Tier B

Definition and why it matters: A single-model deployment is the simplest mental model: train, export, serve. As products mature, one model cannot cover every segment, task, version, or tenant. Production platforms evolve into multi-model systems where routing and ensembles become core engineering patterns

Core explanation: Different geographies, languages, and customer tiers have different data distributions and regulatory requirements Segment: India users; Model strategy: Model trained on local transaction patterns Segment: EU users; Model strategy: Model compliant with GDPR, trained on EU data

Must remember:

- Multi-model systems arise from localization, versioning, task specialisation, and scale
- Typical production platforms run many models side by side, not one
- Routing = one model per request; ensembles = multiple models, combined output
- Canary, A/B, and champion-challenger are key multi-version patterns
- Specialist models (fraud, credit, churn, recs) outperform generalists on narrow tasks
- Both routing and ensembles aim to match requests with the right expertise for better accuracy and robustness

Example or contrast: Dimension: Complexity; Single model: Low; Multi-model: High

Exam trap: Trap: Multi-model means you need a learned router from day one. Reality: Start with simple rule-based routing (country, language, product). Learned routing is an advanced optimisation Trap: Ensembles always beat routing. Reality: Ensembles improve accuracy but multiply latency and cost. Routing is cheaper when one specialist is sufficient

MLME-W12-T03 Routing Strategies: Rule-Based and Learned Routers

Tier A

Definition and why it matters: A router sits between incoming requests and a pool of models. Its sole responsibility: given this input, which model should handle it? The answer can come from explicit business rules or from a learned model that predicts the best expert

Core explanation: The simplest and most common starting point. Rules are explicit, inspectable conditions that map request attributes to model endpoints Dimension: Country / region; Example rule: India - modelIN, Europe - modelEU, else - modelglobal Dimension: Language; Example rule: lang=hi - Hindi model, lang=en - English model Dimension: Product / tenant; Example rule: Product A - modela, Product B - modelb Dimension: Risk band / tier; Example rule: High-value customers - accurate (slow) model; low-risk segment - lightweight model Concept flow: Request - Rule-Based Router - model IN - modelEU - modelheavy - modelglobal Explainable - stakeholders can read and agree on rules Fast to implement - no training pipeline for the router itself Auditable - rules map directly to compliance and governance requirements

Must remember:

- Rule-based routing maps explicit conditions (region, language, tier) to model endpoints
- Rules are explainable and fast to build but suffer from sprawl and missed patterns
- Learned routing trains a small model to predict the best expert per input
- Learned routers appear in MoE architectures and cost-aware serving systems
- Start with rules; graduate to learned routing when complexity is justified
- Always measure per-route performance and document routing logic against SLOs

Example or contrast: Aspect: Explainability; Rule-based: High; Learned router: Low-medium

Exam trap: Trap: Learned routing is always better than rules. Reality: Rules are simpler, explainable, and sufficient for most early-stage systems. Learned routing adds complexity that must be justified Trap: MoE in LLMs and router-expert in serving are unrelated. Reality: Both use a gating mechanism to select specialists; the serving pattern is the production analogue of MoE's train-time architecture

MLME-W12-T04 Fallback Models, Ensembles, and the Accuracy-Cost Trade-Off Tier A

Definition and why it matters: The primary model will not always be the right choice. Fallback routing handles edge cases gracefully rather than failing catastrophically

Core explanation: Trigger: Low confidence / high uncertainty; Example fallback: Simpler, more conservative model Trigger: Out-of-distribution (OOD) input; Example fallback: Rule-based system or default deny Trigger: Primary service down / timeout; Example fallback: Backup model or cached response Trigger: Incident / degraded mode; Example fallback: Manual review queue or safe default Concept flow: Request - Primary Model - Confident? Available? In-distribution? - Return prediction - Fallback Model / Rule / Manual Review / Deny Intuition: Fallbacks are airbags, not performance optimisers. The goal is predictable behaviour during incidents or unusual inputs, not marginal accuracy gains Real-world example: A credit-scoring API routes OOD applicants (missing key features) to a rule-based tier assignment instead of the neural model, avoiding nonsensical scores Instead of choosing one model, call multiple models on the same input and combine their outputs Concept flow: Input - Base Model 1 - Base Model 2 - Base Model 3 - Meta-Model / Combiner - Final Prediction Average probability scores from N models: hat(p) = (1)/(N) sum(i=1)^(N) pi Best for: regression and probabilistic classifiers where calibrated scores matter

Must remember:

- Fallback routing handles uncertainty, OOD inputs, and service failures with safe alternatives
- Ensembles call multiple models and combine via averaging, voting, or stacking
- Ensembles improve accuracy and robustness when base models are diverse
- Trade-off: ensembles increase latency, cost, and operational complexity
- Start with rule-based routing; keep model count manageable; measure per-segment performance
- Document all routing, fallback, and ensemble logic against SLOs and cost budgets

Formula / decision rule: hat(p) = (1)/(N) sum_(i=1)^(N) p_i; N x

Example or contrast: Pattern: Routing; Models invoked: 1 (selected); Primary goal: Match request to best expert

Exam trap: Ensembles push the four-way production tension: Force: Accuracy; Ensemble effect: Usually increases

MLME-W12-T05 Scaling Inference: Sharding and Replication Tier A

Definition and why it matters: The simplest serving architecture - one model on one machine - works until it doesn't. As traffic grows:

Core explanation: Latency increases - requests queue behind each other Primitive: Replication; What it does: Multiple copies of the same model; Analogy: More checkout counters Primitive: Sharding; What it does: Divide traffic/data into subsets, each handled by a dedicated subset of servers; Analogy: Separate departments per region Concept flow: All Traffic - One Model Instance - Traffic - Load Balancer - Replica 1 - Replica 2 - Replica 3 - Traffic - Shard Router Timeouts multiply - clients abandon slow responses Single point of failure - one crash takes down all inference Hard QPS ceiling - one machine can only process so many queries per second Intuition: A single checkout counter works for a small store. A supermarket needs multiple counters (replication) and possibly separate departments (sharding) Run N identical copies of the same model. A load balancer distributes requests across replicas Higher throughput - N replicas handle roughly N x QPS

Must remember:

- Single-instance serving hits latency, timeout, and QPS ceilings
- Replication = multiple identical model copies behind a load balancer
- Sharding = divide traffic/data into subsets, each with dedicated resources
- Hash sharding: shard = hash(id) bmod N for even distribution
- Combine both: shards for partitioning, replicas within shards for throughput and fault tolerance
- Replication improves throughput/reliability; sharding enables scale, isolation, and specialisation

Formula / decision rule: replicas handle roughly; shard = hash(user_id) bmod N

Example or contrast: Aspect: Model copies; Replication: Identical; Sharding: May differ per shard

Exam trap: Trap: Replication and sharding are interchangeable. Reality: Replication spreads identical work; sharding partitions different work. They solve different problems and are often combined Trap: Adding replicas always fixes latency. Reality: Replication fixes throughput and availability. A single slow inference still takes the same time per request

MLME-W12-T06 Caching Model Results and Embeddings

Tier A

Definition and why it matters: Model inference - especially for large deep learning models - is expensive in both latency and compute cost. Yet many production workloads exhibit high repetition:

Core explanation: Same user asking the same question repeatedly Benefit: Latency; Impact: Cache reads are microseconds vs milliseconds for inference Benefit: Cost; Impact: Fewer GPU/CPU inference calls Benefit: Throughput; Impact: Frees model capacity for cache misses Same product page rendered thousands of times per minute Identical feature vectors scored in batch pipelines Caching lets you skip the model entirely when a recent, valid result already exists Intuition: Caching is memoisation at system scale. If you've computed f(x) recently and x hasn't changed, return the stored answer Concept flow: Query - Cache Hit? - Return Cached Result - Run Model - Store in Cache Store the final model prediction for a given input. On cache hit, return immediately - no model invocation Component: Content or ID; Why it matters: Identifies the input (text hash, user ID, item ID)

Must remember:

- Inference caching skips expensive model calls when recent results exist
- Output caching stores final predictions; embedding caching stores reusable vectors
- Cache keys must include content/ID + model version + preprocessing version
- Three invalidation strategies: TTL, version-based (new keys), event-based (data changes)
- Always trade off freshness vs performance per use case
- Caching dramatically reduces latency and cost for repeated or similar requests

Formula / decision rule: f(x); valid iff (t_(now) - t_(cached)) < TTL

Example or contrast: Use case: Static FAQ answers; Freshness need: Low; Typical TTL: Hours-days

Exam trap: Trap: Cache keys need only the input content. Reality: Must include model version and preprocessing version to avoid serving stale artefacts after upgrades Trap: Longer TTL always improves performance. Reality: Long TTLs increase staleness risk. Fraud and safety systems often cannot cache at all

MLME-W12-T07 Combining Routing, Sharding, and Caching at Scale

Tier A

Definition and why it matters: Individual patterns - routing, sharding, replication, caching - are building blocks. At scale, they compose into a layered architecture that keeps latency low, throughput high, and SLOs predictable as traffic and model count grow

Core explanation: Concept flow: Incoming Request - Cache Layer - Fast Response - Router - Shard Selection - Load Balancer - Replica 1 - Replica 2 - Replica 3 Sits closest to the client. Checks for cached outputs or embeddings before any routing or model work Cache keys include model version and preprocessing version TTL, version-based, and event-based invalidation policies apply Cache hits bypass all downstream layers - fastest path On cache miss, the router decides which shard or model to target based on: Step: 1; Component: Cache; Action: Check for existing result; return on hit Region, language, tenant Step: 2; Component: Router; Action: Select shard/model based on request attributes Risk band, product line Step: 3; Component: Shard; Action: Route to correct partition A/B experiment assignment Step: 4; Component: Load balancer; Action: Distribute across replicas within shard Each shard owns a subset of traffic or data. Shards may host:

Must remember:

- Production inference stacks compose: cache → router → shard → replicas → model
- Cache sits in front to serve repeated requests without inference
- Router selects shard/model by region, language, tenant, or experiment
- Within each shard, replicas handle throughput and fault tolerance
- Total instances = shards x replicas per shard
- This layered design keeps latency, throughput, and SLOs predictable at scale

Formula / decision rule: shards, each with; total instances = S x R

Exam trap: Trap: Add caching last as an optimisation. Reality: Cache layer design (key schema, TTL) should be planned with routing and sharding, not bolted on Trap: One global cache for all shards. Reality: Cache keys must account for shard-specific model versions and configurations

MLME-W12-T08 Integrated Architecture: Routing, Sharding, Replication, and Caching

Tier A

Definition and why it matters: Large-scale ML inference is not a single technique - it is a composed system where routing, sharding, replication, and caching each solve a distinct bottleneck. Together they form the standard blueprint for scalable model serving

Core explanation: Concept flow: Client Request - Cache Check\noutputs + embeddings - Response - Router\nregion / language / tenant - Shard - Load Balancer - Replicas - Model Inference - Cache Write Component: Router; Responsibility: Decide which shard or model variant handles this request Component: Shard; Responsibility: Partition traffic/data; may host specialised models Component: Replicas; Responsibility: Multiple instances within a shard for throughput and reliability Component: Cache; Responsibility: Store outputs and embeddings with versioned keys for fast reuse Property: Low latency; How the stack delivers it: Cache hits + replica proximity Property: High throughput; How the stack delivers it: Replication + sharding Property: Predictable SLOs; How the stack delivers it: Isolation via shards, headroom via replicas Property: Cost

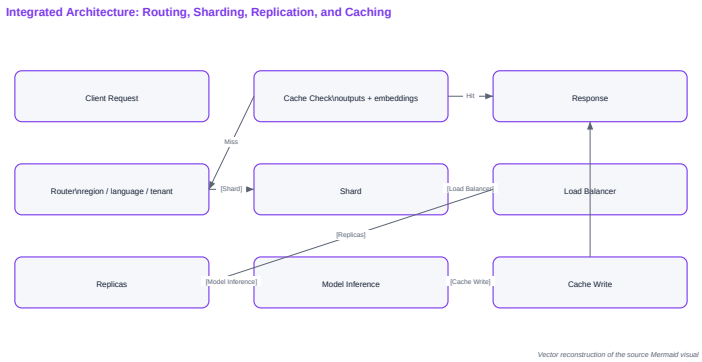
control; How the stack delivers it: Caching reduces inference calls Before any routing logic, check whether a valid cached result exists. This is the cheapest operation and eliminates downstream load entirely on hits

Must remember:

- Production ML serving composes routing + sharding + replication + caching
- Router selects shard/model; replicas handle load within each shard
- Cache sits in front with versioned keys for outputs and embeddings
- Together: low latency, high throughput, predictable SLOs, controlled cost
- Same pattern extends to multi-tenant platforms and RAG systems
- Monitor cache hit rate, shard balance, and per-replica load as first-class metrics

Example or contrast: A SaaS ML platform serving 50,000 QPS across 4 regions:

Exam trap: Trap: Caching replaces the need for replicas. Reality: Cache helps on hits; misses still need replica capacity. Both are necessary Trap: Sharding and routing are redundant. Reality: Routing is the decision logic; sharding is the infrastructure partition. Routing selects the shard



Concept map: Integrated Architecture: Routing, Sharding, Replication, and Caching

MLME-W12-T09 Multi-Tenant ML Platforms and the Noisy Neighbour Problem

Tier A

Definition and why it matters: A real ML platform rarely serves a single team. Multiple tenants - product teams, business units, or external customers - share the same infrastructure: compute, storage, networking, deployment tools, monitoring, and logging Intuition: Multi-tenancy is like an apartment building. Everyone shares the foundation, plumbing, and electricity, but each tenant has their own unit with boundaries

Core explanation: A tenant is a logical customer of the ML platform: Tenant type: Internal product team; Example: Search, recommendations, fraud detection Tenant type: Business unit; Example: Retail, enterprise, small business Tenant type: External customer; Example: SaaS ML platform subscriber Concept flow: Compute / GPU - Storage - Networking - Deploy / Monitor / Log - Tenant A: Fraud - Tenant B: Recommendations - Tenant C: External SaaS Each tenant typically has: Its own models and training pipelines

Must remember:

- A tenant is a logical customer of the ML platform (team, business unit, external client)
- Multi-tenancy shares infrastructure for efficiency but requires isolation for safety
- The noisy neighbour problem: one tenant over-consumes shared resources, degrading others
- Symptoms: latency spikes, queue buildup, timeouts, broken SLOs for unrelated tenants
- Isolation strategies span logical, resource, data, and observability layers
- Multi-tenancy connects directly to SLOs, governance, monitoring, and responsible AI

Example or contrast: Code pattern: Tenant A launches a large batch scoring job on the shared GPU cluster; GPU memory saturated; Tenant B's real-time fraud API latency jumps from 80 ms to 800 ms; Tenant B's SLO breached; Incident declared - but the root cause is another tenant's batch job

Exam trap: Trap: Multi-tenancy only applies to external SaaS products. Reality: Internal platforms with multiple product teams are multi-tenant too Trap: Noisy neighbour is a network concept only. Reality: It applies to any shared resource - GPU, memory, CPU, disk I/O, network

MLME-W12-T10 Per-Tenant SLOs and Isolation Strategy Overview

Tier A

Definition and why it matters: An SLO is a target for a specific metric over a defined window: Metric type: Latency; Example SLO: P95 inference latency < 150 ms

Core explanation: In a multi-tenant ML platform, not every tenant deserves the same guarantees. A VIP external customer paying premium rates needs tighter latency and higher availability than an internal experimental team running best-effort batch jobs Service Level Objectives (SLOs) are measurable targets for system behaviour. Per-tenant SLOs make expectations explicit and drive engineering decisions Concept flow: Per-Tenant SLOs - Capacity Planning - Isolation Strategy - Incident Prioritisation - Cost Allocation Capacity planning - VIP tenants get dedicated GPU pools; batch tenants share spot instances Isolation strategy - tighter SLOs justify stronger resource boundaries Incident response - when the cluster is under pressure,

protect high-SLO tenants first Resource: CPU; Quota example: Max 32 cores per tenant Cost allocation - premium SLOs justify premium pricing Resource: GPU; Quota example: Max 4 GPUs per tenant

Must remember:

- SLOs are measurable targets (latency, availability, error rate) per tenant
- Not all tenants get the same SLOs - VIP vs internal vs experimental tiers
- SLOs drive capacity planning, isolation design, and incident prioritisation
- Three isolation layers: logical (namespaces), resource (quotas), data (access controls)
- Principle: shared platform with clear per-tenant boundaries
- Under pressure, protect high-SLO tenants first via throttling and preemption

Example or contrast: Tenant: Tenant A (VIP external); P95 Latency: $<$ 150 ms; Availability: 99.9%; Priority: High Tenant: Tenant B (internal batch); P95 Latency: $<$ 400 ms; Availability: 99.0%; Priority: Low Tenant: Tenant C (experimental); P95 Latency: Best effort; Availability: 95.0%; Priority: Lowest

Exam trap: Trap: One global SLO covers all tenants. Reality: Different tenants have different business criticality. Per-tenant SLOs enable fair prioritisation Trap: SLOs are the same as SLAs. Reality: SLOs are internal targets; SLAs are contractual commitments (often with financial penalties)

MLME-W12-T11 Blast Radius, Resource Quotas, and Data Isolation

Tier B

Definition and why it matters: Blast radius measures how far damage propagates when something fails. In multi-tenant ML platforms, the goal is to ensure one tenant's incident does not take down the entire platform

Core explanation: Intuition: Firewalls in a building compartmentalise fire. Blast-radius controls compartmentalise failures Mechanism: Kubernetes namespace; What it isolates: Deployments, services, config maps, secrets

Must remember:

- Risk: Noisy neighbour; Mitigation: Resource quotas + priority classes
- Risk: Blast radius too large; Mitigation: Namespaces - separate clusters
- Risk: Data leak; Mitigation: Separate storage + RBAC/IAM
- Risk: Cross-tenant log exposure; Mitigation: Per-tenant log/metric segregation
- Risk: SLO breach during incident; Mitigation: Priority-based resource allocation
- Blast radius = how far an incident spreads; limit it via namespaces and separate clusters
- Resource quotas cap CPU, GPU, memory, and concurrency per tenant
- Priority classes ensure high-priority tenants get resources first under pressure
- Data isolation: separate storage + RBAC/IAM per tenant
- Logging isolation: per-tenant log streams and metric namespaces
- Multi-tenancy isolation connects to privacy, governance, compliance, and responsible AI

Exam trap: Trap: Namespaces alone prevent noisy neighbours. Reality: Namespaces isolate configuration, not resource consumption. Quotas are required Trap: Separate clusters are always necessary. Reality: Clusters are for high-risk/VIP tenants. Most internal teams are fine with namespaces + quotas on a shared cluster

MLME-W12-T12 Embeddings, Vector Similarity, and Vector Databases

Tier A

Definition and why it matters: An embedding is a dense numeric vector that represents an entity (text, image, item, user) in a way that captures semantic meaning A vector database is a specialised system designed to store embeddings and make similarity search fast and scalable

Core explanation: Many modern applications cannot rely on a model's memorised weights alone. They combine models with retrieval - looking up relevant information from external knowledge bases at inference time. Vector search and retrieval-augmented generation (RAG) are the foundational patterns Similar items → nearby vectors in embedding space Metric: Cosine similarity; Formula (intuition): cos(theta) = (a b)/(|a| |b|); Best for: Normalised vectors; direction matters, not magnitude Dissimilar items → far apart

Must remember:

- An embedding is a dense vector capturing semantic meaning; similar items are nearby in vector space
- Vector similarity search: embed query → find top-K nearest neighbours via cosine, dot product, or Euclidean distance
- Use cases: semantic search, recommendations, deduplication, clustering, anomaly detection
- A vector database stores (vector, metadata) pairs and provides fast similarity search APIs
- Vector DBs use specialised indexes (ANN) to scale to millions/billions of vectors
- Embedding model + vector database are complementary - model produces vectors, DB stores and searches them

Formula / decision rule: cos(theta) = (a b)/(|a| |b|); a b = sum_i a_i b_i

Example or contrast: Application: Semantic search; Query: User query text; Nearest neighbours: Relevant documents Application: Recommendations; Query: User embedding; Nearest neighbours: Similar users or items Application: Near-duplicate detection; Query: New document; Nearest neighbours: Existing near-copies

Exam trap: Trap: Embeddings are just feature vectors from tabular ML. Reality: Embeddings are dense semantic representations learned by neural models. Their geometry encodes meaning, not just raw features Trap: Cosine similarity and dot product always give the same ranking. Reality: They differ when vector magnitudes vary. Cosine normalises; dot product does not

MLME-W12-T13 Approximate Nearest Neighbour (ANN) Search

Tier A

Definition and why it matters: Vector databases must answer: "Given this query vector, find the top-K nearest neighbours." The exact approach - comparing the query against every stored vector - works for small collections but fails at production scale

Core explanation: Collection size: 10,000 vectors; Exact search latency: ~10 ms; Production viable?: Yes Collection size: 1 million vectors; Exact search latency: ~1-5 seconds; Production viable?: Borderline For real-time systems (search, RAG, recommendations), exact nearest neighbour (ENN) search is too slow ANN algorithms return neighbours that are good enough - not guaranteed to be the exact nearest - but do so orders of magnitude faster Use case: Real-time RAG; Recall target: 90-95%; Latency budget: $<$ 50 ms ANN: speed uparrow, exactness downarrow (slightly) Use case: Batch deduplication; Recall target: 99%+; Latency budget: Seconds acceptable Intuition: Finding the exact closest library book requires checking every shelf. ANN is like checking only the sections most likely to contain your book - you might miss the absolute closest, but you find a very good match in a fraction of the time Use case: Recommendation feed; Recall target: 85-90%; Latency budget: $<$ 20 ms Index type: HNSW (Hierarchical Navigable Small World); Mechanism: Graph-based navigation through vector space; Trade-off: Fast queries, moderate build time

Must remember:

- Exact NN search does not scale to millions/billions of vectors
- ANN trades slight accuracy loss for dramatic speed improvement
- Core trade-off: latency vs recall@K (how many true neighbours are recovered)
- ANN indexes (HNSW, IVF, LSH, PQ) use pre-computed structures to avoid brute force
- Tunable knobs: index type, probes, search depth - adjust per quality/latency requirements
- Measure recall on evaluation sets; retune when embedding models or data distributions change

Formula / decision rule: ANN: speed uparrow, exactness downarrow (slightly); Higher recall iff more probes/deeper search iff higher latency

Exam trap: The central tuning dimension for ANN systems: Term: Recall@K; Definition: Fraction of the true top-K neighbours found by ANN

MLME-W12-T14 RAG Pipelines as Multi-Model Systems

Tier A

Definition and why it matters: RAG combines search and generation. Instead of relying solely on what a language model has memorised in its weights, the system retrieves relevant documents from an external knowledge base and feeds them to the generator as context Intuition: A student taking an open-book exam (RAG) vs a closed-book exam (pure LLM). The open-book student looks up relevant passages before writing the answer

Core explanation: Concept flow: User Query - 1. Embed Query\nEmbedding Model - 2. Retrieve Top-K Documents\n Vector Database - 3. Re-rank Optional\nReranker Model - 4. Generate Answer\nLLM / Generator Model - Final Answer\ngrounded in retrieved docs An embedding model converts the user's text query into a dense vector Model role: Embedding model; Function: Text - vector; Example: text-embedding-3-small, BGE, E5 Model role: Vector database; Function: Store + search vectors; Example: Pinecone, Milvus, pgvector Model role: Reranker model; Function: Refine retrieval order; Example: Cross-encoder, Cohere Rerank Model role: Generator model; Function: Produce final text; Example: GPT-4, Llama, Mistral

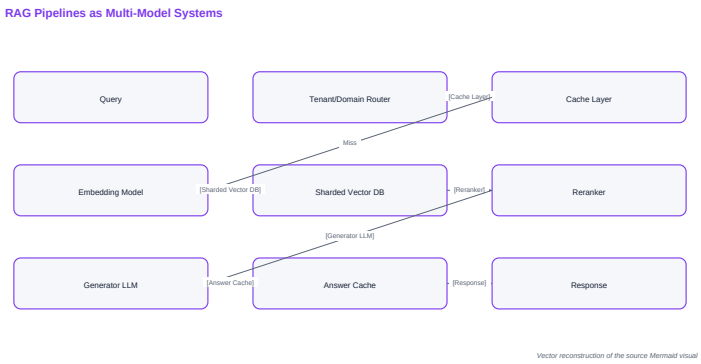
Must remember:

- RAG = embed query $\rightarrow$ retrieve top-K docs $\rightarrow$ optional rerank $\rightarrow$ generate answer with context
- RAG is multi-model by design: embedding model + vector DB + reranker + generator
- Apply sharding (per domain/tenant), caching (embeddings, retrieval, answers), and routing
- Production RAG is a multi-model, multi-tenant, sharded, cached system
- Real-world uses: Q&A, support bots, recommendations, code search, enterprise knowledge
- Production ML = models + data + retrieval infrastructure, not just a model file

Formula / decision rule: Production ML = Models + Data + Retrieval Infrastructure

Example or contrast: Application: Semantic search / Q&A over docs; RAG components: Embed + retrieve + generate

Exam trap: Trap: RAG replaces the need for a good LLM. Reality: RAG augments the LLM with external knowledge. A weak generator still produces poor answers even with perfect retrieval Trap: Retrieval quality doesn't matter if the LLM is powerful. Reality: Garbage in, garbage out. Poor retrieval $\rightarrow$ irrelevant context $\rightarrow$ hallucinated or wrong answers



Concept map: RAG Pipelines as Multi-Model Systems

MLME-W12-T15 Building a File-Based Model Registry

Tier A

Definition and why it matters: Without a registry, model artefacts accumulate in ad hoc locations:

Core explanation: Code pattern: models/modelfinal.pkl; models/modelv2.pkl; models/modelforrealthistime.pkl File: model.pkl (or .onnx, .pt, etc.); Purpose: The serialised model artefact No one knows which version is current, where files live, or how each version performed. A model registry is the single source of truth that answers: File: metrics.json; Purpose: Key performance indicators from evaluation What models do we have? Which version is which? How did each version perform during evaluation? Where is the actual model file located? A surprisingly effective registry can be built with simple file-based conventions - no dedicated tool required (though MLflow, Vertex AI Model Registry, and similar platforms add features) Concept flow: Scan models/ directory - Find subdirectories starting with 'v - For each version folder - Load metrics.json - Record model.pkl path - Add createdat timestamp - Build metadata entry - Write registry.json

Must remember:

- A model registry is the single source of truth for all trained model versions
- Convention: each version in its own folder with model.pkl + metrics.json
- Self-describing artefacts: metrics stored alongside the model file
- Registration agent scans version folders and writes registry.json catalogue
- Registry decouples model knowledge from training, promotion, and serving
- File-based registries teach core concepts applicable to MLflow, Vertex AI, and similar tools

Example or contrast: Aspect: Setup complexity; File-based registry: Minimal; MLflow / Vertex AI: Moderate

Exam trap: Trap: A registry stores model files itself. Reality: The registry stores metadata and paths to artefacts. Model files live in versioned folders Trap: Metrics can be stored only in the registry. Reality: Storing metrics.json with the model artefact makes it self-describing even without the central registry

MLME-W12-T16 Model Promotion and currentbest.json

Tier A

Definition and why it matters: Concept flow: registry.json\nAll versions + metrics - Promotion Script\nApplies policy - currentbest.json\nSingle winner - Serving Layer\nReads one file

Core explanation: Component: Registry; Knows about: All versions, all metrics; Does NOT know about: Which is "best" Component: Promotion script; Knows about: Registry + policy rules; Does NOT know about: How to serve predictions Component: Serving layer; Knows about: currentbest.json only; Does NOT know about: Training, registry, promotion logic Policy: Highest accuracy; Rule: max(accuracy) across versions Policy: Minimum threshold; Rule: Accuracy = 90% AND lowest log loss Policy: Multi-metric; Rule: Weighted score: 0.7 x accuracy + 0.3 x (1 - logloss) Policy: Canary promotion; Rule: New version must beat champion by = 1% on holdout

Must remember:

- Promotion decides which registry candidate is the current best for production
- Promotion policy is centralised in one function (e.g., highest accuracy wins)
- currentbest.json is the single file the serving layer reads - rich metadata for audit
- Decoupling: registry (catalogue) → promotion (decision) → serving (execution)
- Rollback = edit currentbest.json to previous version + restart service
- Promotion is one step in the broader staging → canary → production pipeline

Formula / decision rule: 0.7 x accuracy + 0.3 x (1 - log_loss)

Example or contrast: The model registry (registry.json) is a catalogue - it lists all available versions with their metrics and paths. But it does not answer the operational question: Which model should production use right now? Promotion is the act of evaluating registry candidates and declaring one as the current best for a specific purpose (e.g., production serving)

Exam trap: Trap: Promotion and deployment are the same thing. Reality: Promotion selects the best model; deployment loads it into the serving environment. They are sequential but distinct steps Trap: The serving layer should read registry.json and apply promotion logic. Reality: Serving reads only currentbest.json. Promotion logic stays in the promotion script

MLME-W12-T17 Dynamic Model Serving with currentbest.json

Tier A

Definition and why it matters: The full model lifecycle pipeline:

Core explanation: Code pattern: Train - Versioned folder - registry.json - Promotion - currentbest.json - Serving The final piece is a serving service that reads currentbest.json at startup and loads the designated model - without knowing anything about training or promotion logic Concept flow: Training Pipeline - registry.json - Promotion Script - currentbest.json - Serving Service - Predictions A well-designed serving application has one job: Read the configuration file (currentbest.json) Load the specified model artefact It should not know about: The training pipeline The full model registry Promotion policy logic Concept flow: Server Startup - Read currentbest.json - Extract model path + version - Load model.pkl via joblib/pickle - Store in global variable - Server accepts requests How metrics were computed Power of decoupling: Update the live model in production by changing one file and restarting - no service code changes Standard Pydantic models define the API contract: Use FastAPI lifespan events to load the model exactly once when the server starts: Concept flow: accuracy 0.91 - T - R - P - C - S - CL

Must remember:

- Serving service reads only currentbest.json - decoupled from training and promotion
- Load model once at startup via FastAPI lifespan events
- Two endpoints: metadata (GET /) and predict (POST /predict)
- Every prediction response includes modelversion for traceability
- Rollback = edit currentbest.json + restart - no code changes
- Pattern extends to multi-model, per-tenant, and A/B serving configurations

Exam trap: Trap: Load the model on every request for freshness. Reality: Model loading takes seconds. Load once at startup via lifespan events; restart to pick up new versions Trap: The serving layer should implement promotion logic. Reality: Serving reads currentbest.json only. Promotion is a separate offline step

MLME-W12-T18 Module 12 Summary: Multi-Model Systems at Scale

Tier A

Definition and why it matters: In real production, multi-model systems are the norm, not the exception. A single model behind a single endpoint is a tutorial simplification. Mature platforms host many models - different versions, segments, languages, tasks, and tenants - orchestrated by routing, scaling, isolation, and retrieval patterns

Core explanation: Production ML = Models + Data + Retrieval Infrastructure + Governance Concept flow: Module 12: Multi-Model Systems - Routing and Ensembles - Sharding and Caching - Multi-Tenancy - Vector DB and RAG - Model Registry and Promotion Strategy: Rule-based; Mechanism: Explicit if/else on region, language, tier; Best for: Day-one deployments Strategy: Learned router; Mechanism: Small model predicts best expert; Best for: Complex, high-volume routing Strategy: Fallback; Mechanism: Route to backup on uncertainty/failure; Best for: Safety and incident resilience Method: Averaging; Combination: Mean of probability scores; Cost: N x inference Method: Voting; Combination: Majority class label; Cost: N x inference Method: Stacking; Combination: Meta-model learns combination; Cost: N x inference + meta training Pattern: Replication; Purpose: Multiple identical copies for throughput and fault tolerance Pattern: Sharding; Purpose: Partition traffic/data for scale, isolation, and specialisation Pattern: Caching; Purpose: Skip inference for repeated/similar requests Trade-off: Ensembles push accuracy up but also push latency, cost, and complexity up

Must remember:

- Multi-model systems are production norm: routing, ensembles, many versions/segments/tasks
- Scale via replication (throughput), sharding (partitioning), caching (deduplication)
- Multi-tenancy requires per-tenant SLOs, quotas, namespaces, and data isolation
- Vector DBs + ANN enable semantic search; RAG combines retrieval + generation
- Model lifecycle: versioned folders → registry.json → promotion → currentbest.json → serving
- Production ML = models + data + retrieval + governance, not just a model file
- Model platform engineer: design, scale, isolate, retrieve, manage lifecycle, govern

Formula / decision rule: Production ML = Models + Data + Retrieval Infrastructure + Governance; N x

Exam trap: Speed uparrow vs Recall@K uparrow Tune index type, probes, and search depth per quality/latency requirements

Week 12 Exam Lens

Likely questions

- Define and explain Multi-Model Systems: Routing and Ensembles; include the mechanism and one limitation.
- Compare Multi-Model Systems: Routing and Ensembles with Routing Strategies: Rule-Based and Learned Routers and state when each is preferred.
- Apply Fallback Models, Ensembles, and the Accuracy-Cost Trade-Off to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: hat(p) = (1)/(N) sum_(i=1)^(N) p_i; N x; replicas handle roughly; shard = hash(user_id) bmod N; f(x); valid iff (t_(now) - t_(cached)) < TTL; shards, each with; total instances = S x R

Mistakes to avoid: Trap: Routing and ensembles solve the same problem. Reality: Routing picks one expert; ensembles combine multiple opinions. They can coexist (route to an ensemble pipeline) but are distinct patterns Trap: Multi-model means you need a learned router from day one. Reality: Start with simple rule-based routing (country, language, product). Learned routing is an advanced optimisation Trap: Learned routing is always better than rules. Reality: Rules are simpler, explainable, and sufficient for most early-stage systems. Learned routing adds complexity that must be justified

Week 13: Large-Scale ML System Design

Week roadmap: End-to-End Architecture: Recommendation Systems → Recommendation System: Problem Definition and Requirements → Recommendation Systems: The Data Layer → Recommendation Systems: Feature Store, Candidate Generation, and Ranking → Architectures for Search Ranking and Fraud Detection → Fraud Detection: Online Decisions and Delayed Labels → 10 more topics.

Coverage: 16 exact source topics; definitions and mechanisms come first, followed by formulas, examples, and traps where applicable.

MLME-W13-T01 End-to-End Architecture: Recommendation Systems

Tier A

Definition and why it matters: Recommendation systems sit at the intersection of machine learning, data engineering, and product design. A typical e-commerce homepage shows a "Recommended for You" carousel - a deceptively simple UI backed by a complex production system. The core objective is to show the right item to the right user at the right time, usually within a 100-200 ms latency budget per page load

Core explanation: This is not a single-model problem. It requires orchestrating data ingestion, feature engineering, model training, online serving, and continuous monitoring into one coherent architecture Concept flow: Data Ingestion\n(click streams, transactions - Feature Engineering\n(user + item features - Model Training\n(candidate gen + ranker - Online Serving\n(Monitoring & Feedback\n(CTR, drift, retrain Constraint: Latency; Typical Target: 100-200 ms end-to-end; Why It Matters: Users abandon slow pages; carousel must render with the rest of the UI

Must remember:

- Recommendation systems must show the right item to the right user within 100-200 ms
- They integrate data ingestion, features, training, serving, and monitoring into one loop
- Core constraints: personalisation, freshness, latency, and business rules
- Architecture splits into data, feature/serving, model, and online path layers
- This module synthesises all prior MLOps concepts into a concrete, production-grade example
- Two-stage modelling (candidate gen + ranking) is standard for scalability
- Real-world systems run continuous A/B tests and monitor both ML and system metrics
- End-to-end thinking - not isolated model training - defines ML system engineering

Exam trap: Treating recommendations as a single-model problem - production systems always use at least two stages (candidate generation + ranking); a single model cannot score millions of items in 200 ms Ignoring latency budget - offline accuracy metrics are meaningless if the online path cannot meet P95 latency targets

MLME-W13-T02 Recommendation System: Problem Definition and Requirements

Tier A

Definition and why it matters: When a user opens a homepage or product detail page, the system must render a carousel of roughly 10 recommended products. This sounds simple, but satisfying all requirements simultaneously is genuinely hard

Core explanation: Requirement: Personalisation; Description: Use individual behaviour history; Example: Past views, clicks, purchases shape recommendations Requirement: Freshness; Description: React quickly to recent activity; Example: User clicks a new category - carousel updates within minutes Requirement: Low latency; Description: No perceptible delay; Example: Entire recommendation call completes in 100-200 ms Requirement: Business constraints; Description: Honour inventory and marketing rules; Example: Only in-stock items; highlight promotions; avoid 10 near-identical products Concept flow: Click-stream logs\n(views, clicks, add-to-cart - Transaction events\n(purchases, returns - Data Lake / Warehouse\n(Parquet, ORC - User features\n(categories, recency, AOV - Item features\n(price, category, embeddings - Feature Store\n(offline + online - Candidate Generator\n(millions - hundreds - Ranking Model\n(hundreds - top N - Business Rules\n(stock, diversity, promos Diversity - prevent filter bubbles and repetitive carousels

Must remember:

- Carousel shows ~10 personalised products within 100-200 ms
- Requirements: personalisation, freshness, low latency, business constraints, diversity
- Data layer: click streams and transactions → data lake via streaming pipelines
- Feature layer: user/item features via feature store (offline training + online serving)
- Two-stage model: candidate generation (recall, scale) → ranking (precision, business trade-offs)
- Candidate gen uses heuristics, embeddings, or two-tower ANN search
- Ranking uses rich features + GBDT/neural models + business rule post-processing
- Feature store prevents training-serving skew by defining features once
- Architecture is the template for all large-scale personalisation systems

Formula / decision rule: Total latency ~= T_(feature lookup) + T_(candidate gen) + T_(ranking) + T_(post-processing)

Exam trap: Optimising ranking without candidate generation - you cannot score 10M items per request; two-stage architecture is mandatory at scale Ignoring diversity constraints - high-CTR models often recommend near-duplicates; business rules must enforce variety

MLME-W13-T03 Recommendation Systems: The Data Layer

Tier B

Definition and why it matters: The data layer is the foundation of every production ML system. No amount of model sophistication compensates for missing, delayed, or corrupt data. In a recommendation system, the data layer captures what users did and what they bought - the raw material from which all features, models, and evaluations are derived

Core explanation: Event Type: Page views; Signal Captured: Browsing intent, category interest; Use in Recommendations: Category affinity features Event Type: Clicks; Signal Captured: Active engagement; Use in Recommendations: Short-term preference signals

Must remember:

- Data layer ingests behavioural events (views, clicks) and transactional events (cart, purchases, returns)
- Events flow through streaming pipelines (Kafka/Kinesis) into data lakes/warehouses (Parquet/ORC)
- Aggregated tables (daily user/item summaries) feed feature pipelines and training jobs
- Data layer owns freshness, completeness, and schema quality contracts
- Everything upstream - features, models, serving - depends on reliable data ingestion
- Columnar formats enable efficient storage and selective reads at scale
- Online prediction logs close the feedback loop back into the data layer
- Monitoring data volume and pipeline health is a first-class operational concern

Exam trap: Skipping aggregation - models cannot train on raw click-stream events at scale; aggregated entity-time tables are required Ignoring schema evolution - adding a field without versioning breaks downstream SQL and feature definitions

MLME-W13-T04 Recommendation Systems: Feature Store, Candidate Generation, and Ranking

Tier A

Definition and why it matters: Raw data is not model-ready. The feature and serving layer transforms behavioural logs into structured feature vectors that models consume at both training time and request time. This layer also hosts the two-stage model pipeline - candidate generation followed by ranking - and orchestrates the full online request path The central design principle: define each feature once, use it everywhere, eliminating training-serving skew

Core explanation: Category: Recency / frequency; Examples: Clicks in last 7 days, days since last purchase; Computation: Sliding window aggregation Category: Category affinity; Examples: P(electronics), P(fashion) from browse history; Computation: Normalised click distribution Category: Spend patterns; Examples: Average order value, discount sensitivity; Computation: Transaction aggregations Category: Static attributes; Examples: Category, brand, price, in-stock flag; Computation: Product catalogue Category: Content features; Examples: Title/description embeddings, image embeddings; Computation: NLP/CV pipelines Category: Popularity; Examples: View count, CTR, purchase rate; Computation: Event aggregations Feature: Time of day; Why It Matters: Morning browsing differs from evening

Must remember:

- Features span user (affinity, recency, spend), item (attributes, content, popularity), and context (time, device, page)
- Feature store defines features once for offline training and online serving - prevents training-serving skew
- Candidate generation: millions → hundreds via heuristics, embeddings, or two-tower ANN search (recall-focused)
- Ranking model: hundreds → top N via GBDT/neural ranker + business rules (precision-focused)
- Online path: gateway → feature lookup → candidate gen → ranking → post-processing → UI (all under 200 ms)
- Post-processing enforces stock, diversity, and promotion rules - not optional
- Two-tower: separate fu(u) and fi(i) networks; similarity drives candidate retrieval
- FastAPI/gRPC for serving; Redis for online feature cache; FAISS for ANN retrieval

Formula / decision rule: x = [x_(user), x_(item), x_(context)]; f_u

Exam trap: Single-stage scoring at scale - scoring millions of items per request is infeasible; always use candidate generation first Feature store as optional - without it, training-serving skew is inevitable in any team larger than one engineer

MLME-W13-T05 Architectures for Search Ranking and Fraud Detection

Tier A

Definition and why it matters: Ranking systems and fraud detection systems both require data pipelines, online scoring, and monitoring - but their constraints, success metrics, and cost of errors are fundamentally different. Understanding both architectures - and what they share - is essential for ML system design

Core explanation: A user searches "running shoes" on an e-commerce site. The system must: Concept flow: Document / Item Store\n(products, metadata, embeddings - Search Index\n(BM25, inverted index - Vector Index\n(semantic embeddings - Feature Pipeline\n(CTR, quality, business scores Component: Document store; Purpose: Products, articles, or videos with metadata and precomputed embeddings Component: Lexical index; Purpose: BM25, inverted indexes for keyword matching Component: Vector index; Purpose: Semantic search via embedding similarity Retrieve potentially thousands of relevant products

Must remember:

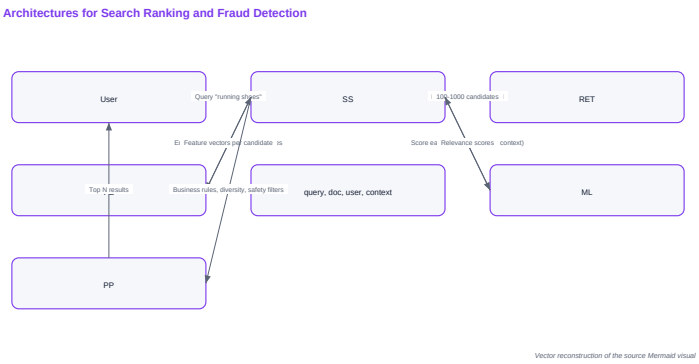
- Ranking: retrieve 100-1000 candidates → enrich with query/doc/user features → ML ranker → post-process → top N
- Ranking optimises order and relevance; success = CTR, conversion, dwell time; latency 50-150 ms
- Ranking feedback loop: log queries/clicks → train → A/B test → promote
- Fraud: assemble real-time features → risk score → threshold decision (approve/decline/step-up)
- Fraud optimises risk and correctness; errors are expensive; latency tens of ms

- Fraud uses segment-specific thresholds tuned for cost of false positives vs false negatives
- Both share: data pipelines, feature stores, online scoring, monitoring - but priorities differ radically
- Ranking experiments aggressively; fraud moves conservatively with audit requirements

Formula / decision rule: decision(s) = begin(cases) approve & if s < tau_(low) step-up & if tau_(low) <= s < tau_(high) decline & if s >= tau_(high) end(cases)

Example or contrast: Dimension: Primary goal; Search Ranking: Ordering and relevance; Fraud Detection: Risk and correctness

Exam trap: Applying ranking latency budgets to fraud - fraud decisions need sub-50 ms; ranking can tolerate 150 ms for the ranker step alone Using accuracy as the fraud metric - class imbalance means 99% accuracy can be worthless; optimise for cost-weighted error rates



Concept map: Architectures for Search Ranking and Fraud Detection

MLME-W13-T06 Fraud Detection: Online Decisions and Delayed Labels

Tier A

Definition and why it matters: Fraud detection makes real-time, one-shot decisions with incomplete information. Unlike ranking - where a bad recommendation is merely suboptimal - a fraud error has immediate financial or customer-trust consequences. The offline training side is equally challenging because ground-truth labels arrive late, sometimes weeks after the transaction

Core explanation: When a payment is initiated, the system receives: Field: Card details (tokenised); Use in Feature Assembly: Velocity checks, historical pattern Field: Transaction amount; Use in Feature Assembly: Anomaly detection vs user baseline Field: Merchant ID / category; Use in Feature Assembly: Merchant risk scoring Field: Device fingerprint; Use in Feature Assembly: New vs known device Field: IP address / geolocation; Use in Feature Assembly: Geographic anomaly detection Field: Timestamp; Use in Feature Assembly: Velocity windows (last 10 min, 1 hr, 24 hr) Concept flow: Transaction Event - Velocity Features\ncardtxncount10m - Device Features\nisnewdevice - IP Features\ngeomismatchscore - Amount Features\namountvsp95 - Fraud Model\nrisk score [0,1 - Decision Engine

Must remember:

- Fraud decisions are one-shot, real-time (tens of ms) with approve / decline / step-up outcomes
- Real-time features: velocity, device familiarity, IP risk, amount anomaly
- Thresholds tau(low) and tau(high) are segment-specific, not global
- Every decision is audited with model version, features, score, and action
- Labels arrive 30-90 days late via chargebacks and investigations
- Training requires point-in-time feature reconstruction - no future leakage
- Evaluate on cost of error (expected loss), not accuracy
- Fraud feedback loop is slower and more conservative than ranking's click-based loop

Formula / decision rule: s < tau_(low); tau_(low) <= s < tau_(high)

Example or contrast: Concept flow: Serve results - Log clicks\n(immediate - Train ranker\n(days - A/B test\n(weeks - Make decision - Log decision\n(immediate - Label arrives\n(30-90 days - Reconstruct features\n(point-in-time - Train model\n(weeks

Exam trap: Training on future information - using post-chargeback features as inputs creates leakage; always reconstruct point-in-time feature snapshots Optimising accuracy with imbalanced data - 99.9% accuracy can mean zero fraud caught; use cost-weighted metrics

MLME-W13-T07 Ranking vs Fraud Systems: Shared Platform, Different Priorities

Tier B

Definition and why it matters: Ranking and fraud detection look like different problems on the surface - one optimises click order, the other blocks stolen credit cards. Yet both run on the same ML platform primitives: data pipelines, feature stores, online scoring services, and monitoring/retraining loops

Core explanation: The toolbox is shared. The priorities and risk profiles are not Concept flow: Data Pipeline\n(raw events - tables - Feature Store\n(define once, serve twice - Online Scoring Service\n(low-latency inference - Monitoring & Retraining\n(drift, performance, alerts - Ranking System - Fraud System

Must remember:

- Ranking and fraud share: data pipelines, feature stores, online scoring, monitoring/retraining
- Both track metric drift, data drift, and model performance over time
- Ranking optimises order/relevance (CTR, conversion); bad results are annoying, not catastrophic
- Fraud optimises risk/correctness; bad decisions mean financial loss or unfair treatment
- Ranking experiments aggressively with A/B tests; fraud deploys conservatively with audit trails
- Fraud has compliance requirements; ranking does not
- The toolbox is the same; priorities and risk profiles drive architectural differences
- ML system design = walking through all five platform layers for any use case

Exam trap: Assuming ranking and fraud need different platforms - they share 80% of infrastructure; only priorities and risk profiles differ Applying ranking experimentation speed to fraud - fraud requires conservative canary rollouts with audit trails

MLME-W13-T08 The Layered ML Platform: Five-Layer Architecture

Tier A

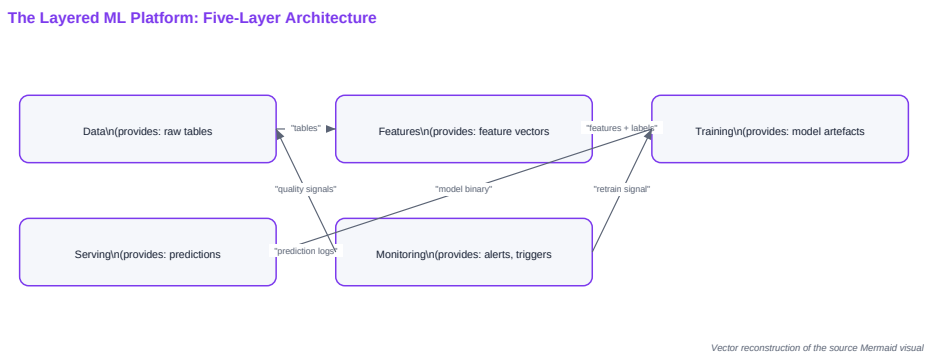
Definition and why it matters: Production ML is not a single monolith. A healthy ML platform is a set of well-defined layers that can evolve somewhat independently, as long as their interfaces stay clear. Each layer depends on the one below and exposes capabilities to the one above

Core explanation: This mental model lets you point to any part of a production system and identify which layer it belongs to - essential for design discussions, debugging, and system design interviews Concept flow: Layer 5: Monitoring & Feedback\n(drift, alerts, retrain, governance - Layer 4: Serving & Infrastructure\n(APIs, containers, scaling, routing - Layer 3: Training & Experimentation\n(pipelines, registry, promotion - Layer 2: Feature Layer\n(define once, serve twice - Layer 1: Data Layer\n(ingestion, storage, quality Layer: Data; Core Responsibility: Ingest, store, validate raw data; Key Question: Where does data come from? Is it on time and complete? Layer: Features; Core Responsibility: Transform raw data into reusable features; Key Question: Which features matter? Are they consistent in training and serving? Layer: Training & Experiments; Core Responsibility: Train, evaluate, track, promote models; Key Question: How do we choose which model goes to production?

Must remember:

- ML platform = 5 layers: Data → Features → Training → Serving → Monitoring
- Each layer depends on the one below; interfaces must stay clear
- Data layer: ingestion, storage, quality contracts (freshness, completeness, schema)
- Feature layer: define once, materialise offline + online; feature store lives here
- Training layer: pipelines, experiment tracking, model registry, promotion logic
- Serving layer: APIs, containers, deployment strategies, scaling, routing
- Monitoring layer: system/data/model metrics, alerts, retraining triggers, governance
- The closed loop: serving → logs → monitoring → retrain → new model → serving
- Layered thinking is the foundation of ML system design interviews

Exam trap: Treating the platform as a monolith - layers must have clear interfaces so they can evolve independently Skipping the data layer - no feature, model, or serving layer works without reliable data ingestion



Concept map: The Layered ML Platform: Five-Layer Architecture

MLME-W13-T09 ML Platform Layers: Serving, Monitoring, and the Closed Loop

Tier A

Definition and why it matters: This note deep-dives into Layers 4 and 5 of the five-layer ML platform - serving/infrastructure and monitoring/feedback - and shows how they connect to form the production closed loop. Layers 1-3 (data, features, training) were covered in the previous note

Core explanation: The serving layer exposes trained models as production APIs: Interface Type: Online (sync); Pattern: Request-response per prediction; Latency: Milliseconds; Example: Fraud scoring, recommendation ranking Interface Type: Batch; Pattern: Score large datasets offline; Latency: Minutes-hours; Example: Nightly churn prediction for all users Interface Type: Streaming; Pattern: Process event stream continuously; Latency: Seconds; Example: Real-time feature updates, anomaly detection Concept flow: Model Service Code\n(FastAPI app - Docker Image - Kubernetes Cluster - Pod: V3\n(3 replicas - Pod: V3\n(auto-scaled - Load Balancer Concern: Packaging; Implementation: Docker image with model

+ dependencies + API Concern: Orchestration; Implementation: Kubernetes deployments, Helm charts Concern: Rollout; Implementation: Blue-green swap or canary (5% - 25% - 100%) Concern: Rollback; Implementation: Update currentbest.json to previous version; restart Common frameworks: FastAPI (Python, REST), gRPC (high-performance RPC), TensorFlow Serving, Triton Inference Server Mechanism: Horizontal auto-scaling; Trigger: CPU 70% or QPS threshold; Action: Add pod replicas

Must remember:

- Serving layer: FastAPI/gRPC APIs, Docker/Kubernetes deployment, blue-green/canary rollouts, auto-scaling
- Model loaded from registry config (currentbest.json) - swap versions without code changes
- Runtime optimisation: ONNX, quantisation, pruning for latency/cost targets
- Monitoring layer: system metrics (latency, errors), data metrics (drift, freshness), model metrics (CTR, fairness)
- Alerts trigger: auto-scale, rollback, retrain, or block promotion
- Closed loop: serve → log → monitor → retrain → promote → serve
- Rollback = edit config file + restart; no redeployment needed
- Governance: decision logging, model lineage, fairness auditing in high-stakes settings

Exam trap: Monitoring only system metrics - data drift and model performance degradation cause silent quality loss without any 5xx errors No automated retraining trigger - manual retraining cycles are too slow; production models go stale

MLME-W13-T10

ML Platform: Module Mapping and System Design Framework

Tier B

Definition and why it matters: Every topic in the ML Model Engineering course maps to one or more layers of the five-layer platform. This note provides the cross-reference index and the system design question framework for applying layered thinking to any ML problem

Core explanation: Concept flow: Ingestion patterns - Data quality & contracts - Pipeline orchestration - Feature stores - Offline vs online features - Training-serving skew prevention - Training pipelines - Experiment tracking - Model registry & versions Layer: Data; Course Topics Covered: Ingestion patterns (batch, micro-batch, streaming); data quality monitoring; pipeline orchestration; schema contracts Layer: Features; Course Topics Covered: Feature store architecture; offline vs online materialisation; training-serving skew; feature reuse, metadata, lineage

Must remember:

- Every course topic maps to one of 5 platform layers - use this as a mental index
- System design = answer 5 questions (data, features, training, serving, monitoring) for any ML problem
- Three worked examples: recommendation, ranking, fraud - same layers, different priorities
- Paradigm shift: from single model to whole system thinking
- Governance hooks exist in every layer - not a separate concern
- Layer interfaces must stay clear so layers evolve independently
- This framework is the expected thinking in ML system design interviews
- If you can walk through all 5 layers for any use case, you are doing ML system design

Exam trap: Jumping to architecture diagrams without clarifying requirements - always start with the five questions, not boxes and arrows Designing for one use case only - the platform primitives are shared; only priorities differ

MLME-W13-T11

ML System Design: Clarifying Requirements and SLAs

Tier A

Definition and why it matters: Question: What is the product?; Why It Matters: Determines architecture pattern; Example Answers: Recommendation carousel, fraud detector, search ranker Question: Who uses it?; Why It Matters: Determines interface and SLA; Example Answers: UI-facing API, backend service, batch pipeline

Core explanation: In any system design conversation - interview, architecture review, or sprint planning - the biggest mistake is drawing boxes before understanding the problem. Requirements shape every downstream decision: which SLA targets to hit, how many instances to provision, what failure modes to plan for, and which trade-offs are acceptable Priority: Accuracy; Implication: Invest in model complexity, more features, ensemble methods Priority: Freshness; Implication: Invest in streaming pipelines, frequent retraining, online features Priority: User experience (latency); Implication: Invest in model compression, caching, auto-scaling Priority: Cost; Implication: Accept lower accuracy or higher latency to reduce infrastructure spend Failure Severity: Slightly worse homepage; System Type: Recommendation; Design Response: Graceful degradation to popular items Failure Severity: Lower search relevance; System Type: Ranking; Design Response: Fallback to BM25 baseline Failure Severity: Regulatory/financial incident; System Type: Fraud, credit; Design Response: Hard fail-safe, audit trail, immediate rollback

Must remember:

- Never start with boxes - clarify requirements, SLAs, and failure impact first
- Key questions: what product, who uses it, what does success mean, what if it fails?
- Latency SLAs: specify percentile (P95/P99) and measurement point (end-to-end vs service)
- Availability: 99% = 3.5 days downtime/yr; 99.9% = 9 hrs; 99.99% = 1 hr
- Graceful degradation: fallback models, cached results, non-personalised defaults
- SLAs directly drive architecture, capacity, and resilience decisions
- Use the 6-section requirements template for any ML system design discussion
- Failure impact severity determines how much to invest in resilience

Example or contrast: Dimension: Product; Specification: "Recommended for You" carousel on e-commerce homepage

Exam trap: Starting with architecture diagrams - always clarify requirements and SLAs first; diagrams come after Ignoring graceful degradation - interviewers specifically test whether you plan for partial failures

MLME-W13-T12 ML System Design: Traffic Estimation and Capacity Planning

Tier A

Definition and why it matters: A system that handles 100 QPS is architecturally different from one that handles 20,000 QPS. Capacity planning translates requirements and SLAs into concrete infrastructure numbers - instance counts, storage sizes, batch windows, and cost estimates. In system design interviews, you are not expected to be exact, but you must demonstrate quantitative reasoning

Core explanation: Question: Average QPS?; Why It Matters: Baseline instance count Question: Peak QPS?; Why It Matters: Burst capacity and headroom Question: When do peaks occur?; Why It Matters: Sale events, evenings, product launches Question: Geographic distribution?; Why It Matters: Multi-region requirements Question: Growth rate?; Why It Matters: Future-proofing infrastructure System Type: Internal ML API; Average QPS: 10-100; Peak QPS: 200; Notes: Small team, batch-heavy System Type: E-commerce recommendations; Average QPS: 1,000-5,000; Peak QPS: 10,000-20,000; Notes: Homepage loads per second System Type: Search ranking; Average QPS: 500-2,000; Peak QPS: 5,000-10,000; Notes: Every search query System Type: Fraud detection; Average QPS: 1,000-10,000; Peak QPS: 50,000+; Notes: Every payment transaction

Must remember:

- Capacity planning = translate SLAs into instance counts, storage, and batch windows
- Ask: average QPS, peak QPS, events/day, training data window
- Instance formula: instances = (peak QPS / QPS per instance) x headroom
- Headroom multiplier typically 2x for bursts and failover
- Storage: events/day x bytes/event x retention days compression ratio
- Batch window: sum all pipeline stages; must fit in available overnight window
- Per-instance QPS depends on model type, hardware, compression, and feature lookup time
- Multi-region adds complexity and ~1.5-2x cost; only when availability requires it
- Show quantitative reasoning in interviews - exact numbers not required, but methodology is

Formula / decision rule: daily storage = 50 x 10^6 x 500 bytes ~= 25 GB/day; 6-month storage = 25 GB x 180 days ~= 4.5 TB

Example or contrast: Given: 50M events/day, 500 bytes/event average, 6-month retention

Exam trap: Planning for average QPS only - peak traffic (4-10x average) causes outages during sales events No headroom for failover - if you need 10 instances normally, deploy 20 so losing half still meets SLA

MLME-W13-T13 ML System Design: Failure Scenarios and Resilience Strategies

Tier B

Definition and why it matters: Production ML systems fail in predictable ways. Data pipelines stall, feature computations break, model services overload, and bad deployments regress quality. Planning for these scenarios - and articulating mitigations - distinguishes a system designer from someone who only knows how to train models

Core explanation: In interviews and architecture reviews, explicitly calling out failure modes demonstrates end-to-end reliability thinking

Must remember:

- Pattern: Feature snapshot fallback; Layer: Feature; Purpose: Serve stale but valid features
- Pattern: Circuit breaker; Layer: Serving; Purpose: Prevent cascade failures
- Pattern: Canary deployment; Layer: Serving; Purpose: Safe model rollout
- Pattern: Config-based rollback; Layer: Serving; Purpose: Instant model version revert
- Pattern: Data quality alerts; Layer: Data; Purpose: Early detection of pipeline issues
- Pattern: Versioned features; Layer: Feature; Purpose: Roll back broken feature logic
- Pattern: Auto-scaling; Layer: Infrastructure; Purpose: Handle traffic spikes
- Pattern: Multi-AZ deployment; Layer: Infrastructure; Purpose: Survive zone-level failures
- Data failures: pipeline stall, partial batches, schema changes → fallback snapshots, quality alerts, versioned features
- Model service failures: outage, overload, bad deployment → cached fallback, baseline model, canary, fast rollback
- Deployment safety: canary (5% → 25% → 100%) with defined success criteria; instant rollback via config
- Infrastructure failures: auto-scaling, multi-AZ, circuit breakers, rate limiting
- Interview structure: requirements → architecture → capacity → failures → trade-offs
- Every failure needs: logs, traceability, post-incident review
- Graceful degradation hard failure for non-critical systems
- Resilience patterns: feature fallback, circuit breaker, canary, config rollback, data quality alerts

Exam trap: Accuracy vs latency vs cost vs complexity What would you validate with a prototype or A/B test?

MLME-W13-T14 MLOps Capstone: From Components to Complete System

Tier A

Definition and why it matters: Throughout the ML Model Engineering course, each topic was explored in isolation - serving patterns, feature stores, monitoring, retraining, optimisation, governance. The capstone demands a different skill: integrating every component into one coherent, production-grade system and being able to narrate that system's story end to end

Core explanation: This is the shift from knowing individual techniques to designing and operating complete ML systems Skill: End-to-end architecture; How It Is Demonstrated: Complete diagram from raw data to monitored production model Skill: Component integration; How It Is Demonstrated: Each module's artefact has a home in the repository Skill: Operational narrative; How It Is Demonstrated: Explain what the system does, how it scales, how it fails Skill: Hands-on execution; How It Is Demonstrated: Train models, promote champions, serve predictions, roll back Skill: System storytelling; How It Is Demonstrated: Present the architecture to an interviewer or review board Concept flow: data/\n(raw + processed - features/\n(shared feature logic - models/\n(V1, V2, V3, ... - services/api/\n(FastAPI + Dockerfile - pipelines/\n(training + retrain - monitoring/\n(config - docs/\n(architecture diagram - .github/workflows/\n(CI/CD

Must remember:

- Capstone integrates all course components into one production-grade system
- Repository structure mirrors the five-layer architecture
- System story: what it does, how it scales, how it fails, how it improves
- Key integration: shared features, registry → serving, monitoring → retrain, CI/CD → deploy
- Rollback = edit currentbest.json + restart - decoupled model management
- Prepare for: system design interviews, architecture reviews, production on-call
- The capstone tests system thinking, not isolated technique knowledge
- Every directory maps to a platform layer - organisation reflects architecture

Exam trap: Treating the capstone as a coding exercise - it tests system thinking and integration, not just script execution Cannot narrate the system story - knowing components without understanding how they connect fails interviews

MLME-W13-T15 MLOps Capstone: Complete System Walkthrough

Tier A

Definition and why it matters: The capstone implements a complete MLOps lifecycle - from raw data ingestion through feature engineering, model training, registry management, online serving, monitoring, and automated retraining. The architecture diagram is the single visual that ties every course module together

Core explanation: Concept flow: Click-stream Data - Transactional Data - User Profiles - Data Ingestion Pipeline\n(validate + append - Data Lake\n(master training dataset - Data Quality Checks - Feature Pipeline\n(user30davgpurchase, etc. - Offline Features\n(training - Online Features\n(real-time serving Action: Input; Detail: New CSV/Parquet files in data/raw/ Action: Validation; Detail: Schema check, null rate, volume sanity Action: Output; Detail: Appended rows in data/processed/train.parquet Action: Failure handling; Detail: Reject bad files; alert on validation failure Feature: user30davgpurchase; Computation: Mean purchase value over 30 days; Storage: Offline table + online cache Feature: numclicks7d; Computation: Click count in last 7 days; Storage: Offline table + online cache The ingestion pipeline picks up new files, validates them, and appends to the master training dataset Concept flow: Features\n(offline store - Join with\nLabels - Labels\n(from data lake - Train Model\n(new candidate - Evaluate on\nHoldout Set - Log to MLflow\n(params, metrics, artefact - Save to models/VN/\n(model.pkl + metrics.json

Must remember:

- Capstone implements the full MLOps loop: data → features → train → register → promote → serve → monitor → retrain
- Offline path: ingest → feature pipeline → training → MLflow logging → registry.json
- Online path: currentbest.json → FastAPI loads champion → predictions with online features
- Closed loop: monitoring detects drift → retrain trigger → new candidate → promote or discard
- Rollback: edit currentbest.json + restart - seconds, no code changes
- Repository structure mirrors architecture: data/, features/, models/, services/, pipelines/, monitoring/
- System is automated, auditable, and resilient
- Every course module maps to a concrete directory or script in the capstone

Exam trap: Training without registration - a model file without registry metadata cannot be promoted or rolled back Serving without reading config - hardcoding model path means every update requires code change and redeployment

MLME-W13-T16 Module 13 Summary: End-to-End ML System Architecture

Tier A

Definition and why it matters: Module 13 synthesises the entire ML Model Engineering course into production system design. It moves from isolated techniques to integrated architectures for recommendation, ranking, and fraud systems - and provides a framework for ML system design interviews

Core explanation: Concept flow: Topic 1-2\nRecSys + Ranking + Fraud\nArchitectures - Topic 3\nFive-Layer ML Platform - Topic 4\nSystem Design Lens\n(SLA, Traffic, Failures - Capstone\nComplete MLOps System - Closed Loop\n(serve - monitor - retrain Layer: Data; Components: Click streams, transactions - Kafka - data lake (Parquet) Layer: Features; Components: User affinity, item popularity, context; feature store (offline + online) Layer: Model; Components: Two-stage: candidate generation (millions - hundreds) + ranking (hundreds - top N) Layer: Online; Components: Gateway - features - candidates - rank - business rules - UI Show ~10 personalised products to the right user within 100-200 ms, using past behaviour, with business constraints (stock, promotions, diversity) Candidate generation - recall-focused; heuristics, embeddings, two-tower ANN

Must remember:

- Recommendation: two-stage (candidate gen + ranking), 100-200 ms, feature store prevents skew
- Ranking vs fraud: shared platform, different priorities (relevance vs risk) and error costs

- Five-layer platform: Data → Features → Training → Serving → Monitoring
- System design: requirements → SLAs → capacity → architecture → failures → trade-offs
- Capacity: instances = (peak QPS / per-instance QPS) x 2x headroom
- Resilience: feature fallback, canary deployment, config-based rollback
- Capstone: full MLOps loop with registry, promotion, serving, monitoring, retrain
- Rollback: edit currentbest.json + restart - decoupled, fast, auditable
- Operational challenges: freshness, multi-objective balance, latency, monitoring breadth
- Paradigm shift: from single model to whole system thinking

Formula / decision rule: instances = (peak QPS)/(QPS per instance) x headroom (2x); storage = (events/day x bytes/event x retention days)/(compression ratio)

Example or contrast: Dimension: Optimises; Ranking: Order and relevance; Fraud: Risk and correctness Dimension: Error cost; Ranking: Annoying; Fraud: Financial loss / unfair treatment Dimension: Experimentation; Ranking: Aggressive A/B; Fraud: Conservative canary

Exam trap: Single-model thinking in interviews - always present the five-layer architecture Ignoring operational challenges - freshness, multi-objective balance, and latency are as important as model accuracy

Week 13 Exam Lens

Likely questions

- Define and explain End-to-End Architecture: Recommendation Systems; include the mechanism and one limitation.
- Compare End-to-End Architecture: Recommendation Systems with Recommendation System: Problem Definition and Requirements and state when each is preferred.
- Apply Recommendation Systems: Feature Store, Candidate Generation, and Ranking to a realistic system or business scenario and justify the decision.

10-mark answer frame: Start with a precise definition; draw or state the causal flow; explain each stage and metric; add a comparison or worked example; close with trade-offs, failure modes, and the decision rule.

Formula and decision sheet: Total latency ~= T_(feature lookup) + T_(candidate gen) + T_(ranking) + T_(post-processing); x = [x_(user), x_(item), x_(context)]; f_u; decision(s) = begin(cases) approve & if s < tau_(low) step-up & if tau_(low) <= s < tau_(high) decline & if s >= tau_(high) end(cases); s < tau_(low); tau_(low) <= s < tau_(high); daily storage = 50 x 10^6 x 500 bytes ~= 25 GB/day; 6-month storage = 25 GB x 180 days ~= 4.5 TB

Mistakes to avoid: Treating recommendations as a single-model problem - production systems always use at least two stages (candidate generation + ranking); a single model cannot score millions of items in 200 ms Optimising ranking without candidate generation - you cannot score 10M items per request; two-stage architecture is mandatory at scale Skipping aggregation - models cannot train on raw click-stream events at scale; aggregated entity-time tables are required

Rapid Revision Master Sheet

Week 1: Foundations of ML Model Engineering. The Seven-Stage ML Lifecycle; Deployment, Monitoring, and Retraining: The Production Loop; Where This Course Fits in the ML Lifecycle; Production Constraints: Latency, Throughput, Cost, and Reliability; MLOps: The Operating System for Production ML.

Week 2: Inference Patterns and Performance Metrics. Definition of Model Inference; Latency, Throughput, and Cost: The Inference Metrics Triangle; Inference Patterns: Setup and Mental Model; Batch Inference: Definition and Architecture; Batch Inference: Offline Use Cases and Trade-offs.

Week 3: Serving, APIs, Containers, and Rollouts. Defining Model Serving: Artefact vs Service; Core Responsibilities of the Model Serving Layer; Monolithic Model Serving Architecture; Microservice Model Serving Architecture; Serverless Model Serving Architecture.

Week 4: Deployment Pipelines and MLOps Foundations. Manual Workflow Pain, Pipeline Definition, and Benefits; ML-Specific Differences from Classic CI/CD; The Hybrid Picture: Code CI/CD + ML Pipeline; ML Artefacts: What Pipelines Move and Track; CD for Machine Learning - What Gets Deployed.

Week 5: Monitoring, Observability, Drift, and Alerts. Why Monitoring ML Models Is Different; What to Monitor: The Three-Layer Framework; Model Performance Metrics and Production Logging; Types of Drift in Production ML Systems; Drift Detection Methods and Alert Design.

Week 6: Continuous Training and Safe Promotion. Static Models and the Retraining Loop; Retraining Triggers: Drift, Performance, and Policy; Continuous Training: Scheduled vs Event-Driven Retraining; Designing a Retraining and Promotion Pipeline; Evaluating and Selecting the Right Model: Champion vs Challenger.

Week 7: Model Standardization and Optimization. Standard Model Formats: Foundations and ONNX; OpenVINO and the Standard Format Pipeline; Quantisation and Pruning; Knowledge Distillation; Compression Trade-offs and the MLOps Pipeline.

Week 8: Accuracy, Latency, Cost, and UX Trade-offs. Accuracy, Latency, Cost, and User Experience; The Latency-Cost-UX Triangle; Vertical Scaling, Horizontal Scaling, and Autoscaling; Scaling Patterns Compared: Latency, Cost, and Stability; Inference Cost and Spot / Preemptible Instances.

Week 9: Production Features and Feature Stores. Feature Engineering: From Notebook to Production; Training-Serving Skew: Definition, Causes, and Consequences; Offline Features: Batch Computation for Training; Online Features: Low-Latency Serving; What a Feature Store Provides.

Week 10: Data Engineering and Real-Time ML Pipelines. Why Machine Learning Needs Data Pipelines; ETL and ELT: Foundations of ML Data Pipelines; Pipeline Types: Batch, Micro-Batch, and Streaming; Batch Ingestion for Machine Learning; Micro-Batch Ingestion: Concept and Architecture.

Week 11: Security, Compliance, Fairness, and Responsible AI. Model and Privacy Threats: Extraction, Leakage, and Over-Exposure; Data Minimisation and Anonymisation Techniques; Fairness and Bias: Evaluating Models Across Groups; Practical Fairness Questions and Visualisation; Fairness Limitations, Trade-offs, and the Model Engineer's Role.

Week 12: Multi-Model, Multi-Tenant, and Retrieval Systems. Multi-Model Systems: Routing and Ensembles; Routing Strategies: Rule-Based and Learned Routers; Fallback Models, Ensembles, and the Accuracy-Cost Trade-Off; Scaling Inference: Sharding and Replication; Caching Model Results and Embeddings.

Week 13: Large-Scale ML System Design. End-to-End Architecture: Recommendation Systems; Recommendation System: Problem Definition and Requirements; Recommendation Systems: Feature Store, Candidate Generation, and Ranking; Architectures for Search Ranking and Fraud Detection; Fraud Detection: Online Decisions and Delayed Labels.

Formula and Framework Recall

- hat(y) = f(x)
- prediction = f(input features)
- from historical data; **inference** calls
- Total Latency = t_(network) + t_(validate) + t_(features) + t_(model) + t_(postprocess)
- Event-to-Action Latency = t_(action) - t_(event occurrence)
- Throughput = (Number of inputs)/(Total time) = (1000)/(61) ~= 16.25 inputs/sec
- Cost per input = (Padding overhead + Setup cost)/(Batch size)
- Data Prep → Train → Evaluate → Package → Deploy
- commit + config + data snapshot + run ID → metrics → model vN
- PSI = sum (A_i - E_i) * ln(A_i / E_i)
- Expected Profit = sum_i (payoff(hat(y)_i, y_i))
-) | False Declines (
- Model v7 = f(Data Snapshot D3, Code Commit abc123, Config train_v2.yaml)
- Profit = sum_i payoff(hat(y)_i, y_i)
- L = alpha *L_(soft)(T(z_s), T(z_t)) + (1 - alpha)* L_(hard)(y, z_s)
- Conv → BatchNorm → ReLU
- P_(serve)(x) != P_(train)(x)
- Skew exists when P_(train)(x) != P_(serve)(x)
- 170 total) | Inactive spender (
- training_set = features bowtie_(customer_id, as_of_date) labels
- must only use data available **before**
- Define once → Materialise offline + Materialise online → Serve consistently
- sum(spend)_(30d)
- sum(amount)

- SLA compliance = (features meeting SLA)/(total features evaluated) x 100%
- null_rate = (records with null/invalid value)/(total records)
- FPR = (FP)/(FP + TN) (false positive rate)
- FNR = (FN)/(FN + TP) (false negative rate)
- FPR = (FP)/(FP + TN)
- hat(p) = (1)/(N) sum_(i=1)^(N) p_i
- shard = hash(user_id) bmod N
- valid iff (t_(now) - t_(cached)) < TTL

Must-Draw and Must-Compare Sheet

Use these seven structures when a question asks you to explain, compare, design, or evaluate. Draw the skeleton first, label every arrow or decision axis, then use the prose answer to explain why each element exists and what can fail.

- Closed ML lifecycle: data → train → validate → registry → deploy → monitor → retrain.
- Serving path: client → API → preprocessing → model runtime → postprocessing → response and metrics.
- Safe rollout ladder: offline test → shadow → canary/A-B → promotion → rollback.
- Drift and retraining loop: signals → detection → alert → evaluation → approval → champion replacement.
- Feature-store view: offline training store, online serving store, shared definitions, and freshness checks.
- Real-time data path: producers → Kafka/Flink/Spark → validated features → inference → monitoring.
- Large-system layers: experience, routing, feature/retrieval, model serving, and platform/governance.

Scoring method: A diagram earns marks only when the labels are explained. After drawing, state the governing assumption, one metric or formula, one realistic example, and one limitation or trade-off. For a comparison, keep the dimensions parallel: purpose, data/input, mechanism, speed/cost, failure mode, and best-fit situation.

Final 30-Minute Answer Checklist

- Define the exact term before describing tools or examples.
- State the causal mechanism in a clear sequence; label each stage.
- Write the relevant formula, define its variables, and interpret the result.
- Compare alternatives on assumptions, cost, latency, risk, or customer value as appropriate.
- Use one realistic example and explicitly connect it to the concept.
- Close with a limitation, failure mode, ethical concern, or decision rule.
- Check that the answer addresses the command word: define, explain, compare, calculate, design, or evaluate.